

UNIVERSITY of CALIFORNIA
SANTA CRUZ

**COMBINING REASON AND AUTHORITY
FOR AUTHORIZATION OF PROOF-CARRYING CODE**

A dissertation submitted in partial satisfaction of the
requirements for the degree of

DOCTOR OF PHILOSOPHY

in

COMPUTER SCIENCE

by

Nathan Whitehead

March 2008

The dissertation of Nathan Whitehead is approved:

Professor Martín Abadi, Chair

Professor Cormac Flanagan

Professor George Necula

Professor Allen Van Gelder

Lisa C. Sloan
Vice Provost and Dean of Graduate Studies

Copyright © by
Nathan Whitehead
2008

Contents

List of Figures	vii
Abstract	viii
Dedication	ix
Acknowledgments	x
1 Introduction	1
1.1 Related work	5
1.2 Organization	13
1.3 Historical development	13
2 Description of BCIC	15
2.1 Informal description	15
2.1.1 A small example	15
2.1.2 Trust annotations	19
2.1.3 Cell phone example	20
2.2 Terminology	28
2.2.1 Datalog	28
2.2.2 Binder	29
2.2.3 BCIC	30
2.3 University supercomputer example	33
2.4 Design choices	35
3 Larger examples	39
3.1 Auditing function calls	40
3.1.1 Language	40
3.1.2 Policy	42
3.1.3 Correctness	44
3.2 Trust in the λ -calculus	45
3.2.1 Language	45
3.2.2 Policy	48
3.2.3 Correctness	52

3.3	Instrumenting machine code	53
3.3.1	How instrumentation works	55
3.3.2	Example program	56
3.3.3	Static check	59
3.3.4	Dynamic safety property	61
3.3.5	Semantics of x86 instructions	61
3.3.6	Proofs	64
3.3.7	Memory safety and authorities	65
3.3.8	Policy	66
3.3.9	Comparison to Pittsfield	67
4	Analysis	69
4.1	Results on Datalog and Binder	69
4.1.1	Datalog expressiveness	69
4.1.2	Datalog decidability	71
4.1.3	Binder decidability	75
4.2	Properties of BCIC	77
4.2.1	Conservativity	77
4.2.2	Decidability	78
4.2.3	A useful restriction on policies	83
5	Proof by computation	87
5.1	Motivation	88
5.2	Reflective proofs	88
5.2.1	Decision procedures	89
5.2.2	Partial analyses	90
5.3	Simple examples	91
5.3.1	List prefix	91
5.3.2	Access control lists	93
5.3.3	Tree paths	93
5.4	Negation and revocation	94
5.5	Properties regarding reflection	95
5.5.1	Decision procedures	95
5.5.2	Partial decision procedures	96
5.6	Static analysis	96
5.6.1	Simply typed λ -calculus	97
5.6.2	Allowed free variables	100
5.7	Extending previous examples	102
5.7.1	Trust in the λ -calculus	102
5.7.2	Instrumented code	103
6	Implementation	107
6.1	The logic module	108
6.2	The cryptography module	111
6.3	The network module	111

6.3.1	Synchronization algorithm	112
6.3.2	Space	115
6.4	Certified inference engine	116
6.4.1	Datalog and Binder	117
6.4.2	Encoding BCIC	120
6.4.3	CIC as an extension	121
6.4.4	Alternate encodings	122
6.4.5	Extracted code	123
6.5	Implementation of some of the examples	123
6.6	Efficiency	124
7	Discussion	127
7.1	Experience	127
7.2	Future directions	130
7.3	Summary of contributions	132
A	Selected proof scripts	135
A.1	Decidability of BCIC	135
A.2	Extracted BCIC code	339
A.3	Auditing function calls	353
A.4	Trust in the λ -calculus	364
A.5	Proof by computation	390
	Bibliography	395

List of Figures

2.1.1 Cell phone/game device example	22
2.3.1 Example Datalog policy for a university supercomputer	33
2.3.2 Example of a Binder distributed authorization policy for a supercomputer .	34
2.3.3 An example BCIC policy for a supercomputer	35
3.3.1 Subset of allowed machine instructions	56
6.0.1 System components	109
6.0.2 A screenshot of three instances of BCIC	110
6.3.1 Synchronize	114
7.3.1 Summary of the contributions of the examples	132

Abstract

Combining Reason and Authority for Authorization of Proof-Carrying Code

by

Nathan Whitehead

Using both reason and authority as strategies for code authorization is desirable, possible, and practical. We present BCIC, a system that combines an authorization logic based on the Binder language with CIC, a logical framework able to express semantic properties of programs. BCIC is a general system for specifying and enforcing policies that rely on both reason and authority. In particular, BCIC supports extensible software systems that employ both digitally signed code and language-based security, including proof-carrying code.

We describe BCIC, establish some of its fundamental properties, and explain its use. We develop a series of examples at varying levels of detail to demonstrate our system in action and explain what types of benefits it provides. We include an example for auditing system calls, an example that includes a type system for tracking trust in the λ -calculus, and an example showing a limited form of control-flow integrity for x86 machine code.

To allow policy writers to extend the language of BCIC we also explain how proof-by-computation works in BCIC, and demonstrate how decision procedures for predicates can be embedded in policies. Such decision procedures work with utilitarian data structures and can also be used to perform automatic static analysis of programs within the policy.

To obtain high assurance that our reference monitor is correct, we have created a certified implementation of BCIC using program extraction from our Coq proofs of decidability of BCIC. Along the way we also extract certified implementations for Datalog and Binder, useful in their own right.

To my parents,
who taught me the joys of thinking and creating.

Acknowledgments

I would like to thank my advisor, Martín Abadi, for being an inexhaustible source of ideas and advice, George Necula for many fruitful discussions and encouragement as well as the example in Section 2.1.3, Allen Van Gelder for help with Prolog and advice on automatic theorem proving, and Cormac Flanagan for giving feedback on the material in this dissertation.

I would also like to thank the anonymous reviewers of CSFW 2004, LPAR 2004, and TYPES 2006 for insightful comments and corrections on paper drafts for the respective conferences. The material in this dissertation has been improved by their work. The questions and comments from other participants at the TYPES workshop in particular helped refine the direction of this work in a significant way.

Thanks to George Necula for allowing us access to the source code of the Open Verifier. Thanks to Jordan Johnson for discussing proof by computation when the idea was in a preliminary stage and for help with implementing case studies that motivated the development of the techniques described in this dissertation. Thanks to my wife, Katia Hayati, for providing feedback throughout the development of the material in this dissertation and for providing invaluable suggestions about the presentation of this dissertation. Thanks to Avik Chaudhuri, Bogdan Warinschi, Sergio Maffeis and anyone else that was in the office that I'm forgetting that I bounced ideas off of. I am also grateful to Andrei Sabelfeld and Steve Zdancewic for comments on parts of this work.

Thanks to Xavier Leroy for releasing the Cryptokit library used in the implementation, and to Adam Chlipala for releasing his Coq toolkit for program analysis.

This work was partly supported by the National Science Foundation under Grants CCR-0204162, CCR-0208800, and CCF-0524078. This work was also partly supported by a scholarship from the ARCS Foundation.

Chapter 1

Introduction

Modern software systems are often extensible, through software upgrades, third-party additions or components, or with applets. The extension techniques and policies vary significantly; so does the level of trust in the producer or distributor of the extension code. Because of this diversity, there is much interest in general mechanisms that can be used to allow system extensions, even untrusted ones, without compromising the stability and security of the host system.

Mechanisms based on digital signatures (e.g., ActiveX [Net07] and .NET [LLL⁺02]) can ensure that the untrusted code or data is endorsed by a trusted party. Digital signatures are relatively easy to use and are fairly well understood. On the other hand, digital signatures have several limitations with respect to mobile-code security. Each safety policy based on digital signatures is likely to name only a small number of principals as trusted to provide code with certain safety properties. The presence of a digital signature from somebody unknown is not very useful. Moreover, a digital signature, even from someone trusted, does not ensure the lack of mistakes in the code—it only ensures that the mistakes came from the expected principal.

Language-based security mechanisms (e.g., typed assembly language [MWCG98] and proof-carrying code [Nec97]) overcome some of these limitations of digital signatures. They are largely agnostic on the origin of the code, so can be used on code coming from

unknown principals. They are based on the semantics of the code, thus they can prevent even unintentional mistakes. However, this benefit does not come for free. Type-based mechanisms can enforce only predetermined classes of type safety policies. Proof-carrying code (PCC) is more open-ended and can in principle be used to enforce any desired semantic property. As the complexity of the property being verified increases, so does the opportunity for error and the proof generation burden. Even though this burden is entirely on the untrusted code producer who is assumed to have more intimate knowledge of the code, purely semantic mechanisms can sometimes be impractical and perhaps even inappropriate.

Since digital signatures can verify where the code is from but not what the code does, and language-based mechanisms can verify what the code does but not where it is from, it is reasonable to expect that a synergistic combination of such mechanisms would be useful. In particular, the availability of digital signatures can provide a way to control and leverage the PCC proof effort: the vast majority of the safety conditions can be proved as usual, but a few of the more complex properties (perhaps beyond the scope of previous PCC systems) can be “proved” by signature from a trusted party. The trusted party does not have to certify the entire program, but needs to sign only the validity of precise statements about a small number of points in the program. For example, these points might be where sensitive information is declassified in apparent violation of an information-flow policy (e.g., casts in Jif [Mye99]). In addition, the safety policy itself may come from signed statements made by trusted authorities.

BCIC

In this dissertation we present a system that combines a general logical framework, CIC [CH88], with an authorization logic based on Binder [DeT02]. We call our system BCIC. Terms in CIC that represent programs, properties, and proofs are embedded in the authorization logic. A code consumer constructs a policy in the logic that describes trust relationships with other principals, defines predicates declaratively, and may include axioms for reasoning. Statements made by other principals are imported into the local

policy. Reasoning in the authorization logic then determines whether a query (request) given to the resulting policy is derivable (authorized). We have designed BCIC to be as simple as possible while remaining flexible enough to support a great variety of scenarios. It is also amenable to formal analysis, as we demonstrate. In particular, we establish a conservativity result and develop a decision algorithm for reasoning about well-behaved policies.

In a computer security system with access control, the *reference monitor* decides which requests to allow (e.g. who is allowed to write to a file). Every access the system handles goes through the reference monitor, so ensuring its correctness is of paramount importance. It has been recognized since the original requirements for access control systems were developed in the 1970s [And72] that verifying the reference monitor is an important part of designing a secure access control system. To this end we have constructed our implementation of BCIC so that as much as possible is certified to be correct. If one cannot trust the implementation of the logic, it is worthless as the basis of a secure system.

We present a series of security logics of increasing expressive power leading up to BCIC. We encode each logic in CIC, develop an algorithm for deciding queries, and prove properties about the algorithm. Using Coq’s automatic program extraction mechanism we extract working implementations of the decision algorithms which satisfy the properties. This strategy yields a series of reference monitors fully certified at the source level for Datalog, Binder, Binder with a general purpose extension mechanism, and BCIC.

Further, we have implemented the components of BCIC: an interface, a cryptography module, and mechanisms for transmitting and importing statements. The implementation allowed us to develop several larger examples. These examples together with the implementation demonstrate the practicality of BCIC and validate our design.

In addition to combining statements from authorities with proofs about code, BCIC also has the ability to encode policies based on static analysis. We present a general technique for allowing extensions to BCIC using decision procedures and proof by computation (also known as proof by reflection). This technique allows BCIC to automatically

compute its own proofs of properties rather than depending on proofs being explicitly provided. Any static analysis that can be expressed as a CIC function can be embedded in BCIC security policies. Our technique also allows us to add a library of simple data structures and properties useful for writing security policies to BCIC. We present several as illustrative examples.

Examples

We present three example applications of BCIC in Chapter 3. In the first two examples we suggest an approach to code auditing using BCIC that bases auditing decisions on logical policies and tools. Specifically, we show that policies for auditing may be expressed in BCIC and evaluated by logical means. This approach thus leverages trust relations and proofs, and also allows auditing to complement them. The examples emphasize the possibility of looking at techniques for verification and analysis in the context of policy-driven systems, and the attractiveness of doing so in a logical setting.

The first example (Section 3.1) concerns operating system calls from an application extension language. With a BCIC policy, every operating system call must be authorized by an audit. A policy rule can allow entire classes of calls without separate digital signatures from an authority.

In the second example (Section 3.2), we consider an information-flow type system [SM03], specifically a type system that tracks trust (much like Perl’s taint mode [Fou], but statically) due to Ørbæk and Palsberg [ØP97]. The type system includes a form of declassification, in which expressions can be coerced to be trusted. If any program could use declassification indiscriminately, then the type system would provide no benefit. In this policy, a trusted authority must authorize each declassification.

The third example (Section 3.3) is at the machine level and demonstrates code instrumentation. Instead of statically analyzing code and deciding that it is safe, program code is rewritten to include dynamic checks. We encode the semantics of machine instructions and prove that if code has been correctly instrumented then it will satisfy a

control-flow integrity property, namely that the instruction pointer will never split an instruction in the original program. Control-flow integrity is a basic property that must be proved for the x86 architecture before proofs for most other properties can be attempted. From there we define a memory safety property that restricts memory writes of programs to being in allowed regions. The regions to which a program is allowed to write are declared by a trusted authority. This third example shows that BCIC can deal with programs and proofs at the machine code level, that combinations of static and dynamic checks of code are possible in BCIC, and that assertions can be mixed with static analysis.

1.1 Related work

We briefly review related work in three areas. The first area of related work is on the general problem of access control and authorization. This problem has been important since the invention of timesharing and networking. Our work draws on many developments in this field. The second area of related work is on the problem of proving formal statements on the computer. Again, this problem has been studied since the first programming languages were designed. The third and final work we review is on systems designed explicitly to authorize code or actions using some combination of assertions and reasoning.

Access Control

Access control is the problem of determining who may do what. Traditional access control mechanisms in computer security include mechanisms such as permission bits and groups in Unix. One way of modeling such systems is with an access control matrix [Lam71]. Rows of the matrix are labeled with principals and columns with objects. Each entry of the matrix contains a set of actions the corresponding principal may perform on the object. Storing the full matrix is usually impractical and inconvenient, so more convenient models of the access matrix are normally used. Permission bits for a file in Unix store a compressed form of several columns of the matrix, representing which users are allowed to perform

various operations on the file. Instead of storing permission bits for every principal and every file, each file has bits for the owner, group, and other principals.

Such a view of access control is appropriate for a multi-user operating system where the set of principals and possible actions is known at every stage. As computers become increasingly connected and dependent on remote resources, the access control problem becomes more distributed. The owner of a resource may not know in advance who should be allowed access to the resource; the owner may wish to depend on the judgment of someone else. Any solution must be secure against attacks, and should place as little trust in other principals as is practical. A solution should also place minimum requirements on the availability of other principals.

From this point of view, a logic for access control is desirable. Logics with access control predicates are convenient and well-defined systems for expressing the access control matrix. Logical statements are also extensible; from a small number of known facts and rules of inference, many properties of interest can be defined. The first logic for access control in distributed systems appears in Taos [ABLP93, WABL93]. This logic introduces the **says** modal operator. The actual access control mechanism of Taos uses a restricted decidable subset of the logic.

Some later work on logics for access control have built on top of Datalog [Ull88]. SD3 [Jim01] introduces quoted predicates to Datalog. Other extensions to Datalog add a **says** operator, allow some form of negation and threshold criteria [LGF03], or implement role-based access control [LMW02]. SecPAL [BGF06] is a recent example of an authorization logic based on Datalog with constraints. We base our system on Binder [DeT02], an extension to Datalog which is intuitive and easily decidable. Other choices for a distributed logic for access control would have worked as well.

Binder is an extension of Datalog that adds the **says** construct. Binder also defines a procedure whereby principals (called contexts in Binder) can make statements and export them to other contexts. A statement X signed by context U is imported as U **says** X . Binder restricts **says** so that **says** can never be nested. This means that if V signs a

statement U **says** X , the signed statement cannot be imported as V **says** U **says** X . The original signed statement X may be imported directly from U . Our system follows Binder in this respect. In practice this constraint is less restrictive than it may appear. If the statement X is digitally signed by U , then anyone may forward this message to anyone else without restriction. The result will be an imported statement of the form U **says** X .

An example of a certified reference monitor is DHARMA [CDM04]. DHARMA is a formally verified reference monitor for a distributed delegation logic. It was verified in PVS [ORS92] then automatically extracted into Lisp code. The logic of DHARMA is based on access control lists and bounded delegation, and is specific to access control using these mechanisms (e.g., one cannot define new predicates in a DHARMA policy). BCIC is a more general-purpose access control mechanism.

Mechanized Proofs

Because automated theorem proving has a long history in computer science [RV01], there are many possible mechanized systems to use for proving properties about programs. Some, particularly those used for hardware verification [BDL96, SBD02], are based on propositional logic. Propositional logic is simple, but expressing interesting properties for software can be cumbersome and lead to an exponential blowup of problem size. Satisfiability solvers can, however, have excellent performance [MMZ⁺01]. It seems that the best way to exploit this performance is to incorporate satisfiability solvers as subroutines in more full-featured systems [NO79, FLL⁺02, FJOS03].

First and higher-order predicate logics are expressive enough for most problems and form the basis for many reasoning systems (e.g. PVS [ORS92] and Isabelle [NPW02]). However, some problems are more naturally expressed in a constructive logic or modal logic rather than an ordinary classical predicate calculus (e.g. stating and proving temporal properties about events in a program). A natural approach is to develop a logical framework that can easily encode many specific logics for different applications. LF is such a framework,

and is based on dependent types [HHP93]. A previous version of our system used LF as our embedded logic.

CIC Our current embedded logic, the Calculus of Inductive Constructions (CIC) [CH88], is used by Coq [Tea, BC04a] and allows quantification over inductively defined data structures. In a theoretical sense similar deductions are possible in LF, but the rules must be added by the author of the proof rules. If the rules are not correct, the proof rules will not be sound with respect to the intended interpretation. In CIC, proof rules over inductively defined data structures are automatically generated and proven to be sound. It is simple to define inductively programs and data types, then immediately reason about properties that quantify over all programs or all values of data. Coq is a stable, widely used proof framework. Many important results from mathematics have been formalized in Coq. For instance, the four-color map coloring theorem have been proven in Coq [Gon].

Because we are interested in proof-carrying code, we must limit ourselves to reasoning systems that can provide explicit proofs. By choosing CIC, we do not limit the applicability of our system. CIC can be used as a metalogical framework to encode almost any reasoning system. In fact, many reasoning systems have already been encoded, including intuitionistic propositional satisfiability [Wei98], predicate logic and resolution methods [Hen03], and temporal logics [CG02].

Not only can CIC encode proofs that appear in the system, CIC proofs can be generated about the system. Suppose a property P that a code consumer cares about is encoded in a CIC environment in a policy. Code producers will use Coq to generate CIC proofs that their code satisfies P . BCIC will use CIC typechecking to verify the proof for the code consumer. In addition, the code consumer can also reason about the property using Coq, for example by proving that the property is correct with respect to some semantics of the programming language. This proof need not appear in the policy itself, although it may.

Proof-carrying code In proof-carrying code a verification condition generator examines a program and produces a logical statement that when true implies a desirable property of the program such as memory safety. A PCC checker is parameterized by a description of the safety conditions for certain operations (e.g., the preconditions for a memory operation or an **open** system call to be safe), and by a set of proof rules that establish the acceptable ways to construct the proofs of the safety conditions. Existing work on PCC [Nec97, App01, AF00, NS03] assumes that the untrusted proof producer and the receiving host negotiate beforehand a mutually acceptable set of proof rules. Sometimes verification condition generators or logical checkers must be trusted; foundational proof-carrying code [App01] uses proofs all the way down to the lowest level machine operations to avoid any such trust. Different underlying logics have been used for proof-carrying code, but no existing system attempts to combine unproven assertions with proof-carrying code proofs in the way BCIC does.

Code verifiers Some of the work relating to proof-carrying code has been along the lines of transmitting correct code verifiers rather than proofs themselves. Schneek and Necula present a way for untrusted verifiers to produce proofs for programs [SN02, NS03]. The Open Verifier [CCNS05] is a comprehensive framework for creating correct program verifiers. Chlipala has created a modular framework for producing correct program verifiers [Chl06]. Leroy [Ler06] and Chlipala [Chl07a, Chl07b] have both constructed certified compilers in Coq. In a sense all of this work is a shift from explicit proofs about individual pieces of code to certified tools that do not need to construct proofs along the way. Our proof-by-computation extension to BCIC is a move in the same direction. By allowing Coq code to examine programs, we open the door to policies that refer to combinations of authority, proofs, and certified static analysis.

Certified reference monitors The earliest approaches to designing and implementing secure reference monitors involved writing detailed specifications of the desired behavior of the reference monitor, then proving properties by hand about the specification. The

reference monitor would then be implemented to closely follow the specification. Stanford’s provably secure operating system follows this approach [NRL⁺75]. This method has disadvantages. The implementation may not correctly follow the specification, proofs about the specification may contain errors, and the properties that are proved might not ensure correctness. Gutmann argues that formal reasoning is the wrong tool for increasing trust in security applications [Gut04]. He cites numerous examples where code that was “formally proved correct” failed. He develops a methodology based on dynamic program assertions and informal reasoning that is designed to be easily understood by parties not acquainted with the code. We agree that formal methods are not a panacea, but we do believe that they have an important role to play in the development of correct software. With the advent of effective and reliable tools, machine checkable formal proofs about code are becoming not only possible but practical.

One technique that has proven effective at producing correct code is program extraction. By the Curry-Howard isomorphism, proofs and programs are isomorphic. Program extraction uses the isomorphism to automatically convert correct proofs into correct programs. Program extraction in Coq has many advantages. The implementation comes directly from the proof, the proofs are all machine-checked, and Coq is expressive enough to encode full correctness properties.

Extraction for Coq was developed by Paulin-Mohring [PM89] then further refined by Letouzey [Let04]. Coq and its extraction mechanism have been used to generate certified algorithms for many problems such as a compiler back-end [Ler06], binary decision diagrams, unification, and binary search trees. These and other examples are available at the Coq user contributions page: <http://coq.inria.fr/contribs/extraction-eng.html>.

Static analysis There are many existing tools for performing static analysis of code. Traditional static analysis uses heuristic methods to find possible bugs in source code, as in the original LINT tool [Joh77]. Static analysis can be more formal and precise in its analysis, eliminating entire classes of bugs and security violations in a sound way. The Java virtual machine, for example, analyzes bytecode before execution to ensure stack dis-

cipline and enforce secure calling conventions [LY97]. ESP is a static analysis tool that can enforce temporal properties such as the ordering of function calls according to a specification [DLS02]. ESC/Java [FLL⁺02] uses program annotations and automatic theorem provers to statically check for errors such as index bounds violations and null pointer references. ESC/Java is not sound, by design, so can not guarantee the absence of errors. Even so, it is not hard to imagine a more restrictive variation that is sound.

Reason and authority

Many existing systems combine formal reasoning and authority in some way, though with different purposes, characteristics, and generality. Checking the validity of an X.509 certificate chains involves a combination of trusting principals and reasoning about the transitivity of certification. Environments that execute network code often combine static typechecks of the code with digital signature checks [LY97, ECM07, LLL⁺02]. These systems can check only fixed, simple properties of the code. Proof-carrying code allows more interesting properties to be checked [Nec97, App01, AF00, NS03].

BCIC is a continuation of research into proof-carrying code. Techniques such as typed assembly language [MWCG99] and foundational proof-carrying code [App01] are the base for generating the proofs about code that BCIC uses, but a system for generating and checking proofs is not by itself an access control system for untrusted code. The goal of BCIC is to extend the idea of proof-carrying code and incorporate it into an access control framework.

Proof-carrying authentication [AF99, BSF02] is a method of doing authentication (and authorization) inspired by proof-carrying code. In proof-carrying authentication the policy is described using predicates defined in Twelf [PS99]. Requests for access to resources come paired with an explicit proof that the access should be granted according to the policy. These proofs are necessary because the policy logic is undecidable. Alpaca [LLFS⁺07] generalizes the idea of proof-carrying authentication to include cryptographic primitives, namespaces, and other elements necessary to reason about credentials from multiple sources.

Other work defines a linking logic on top of proof-carrying authentication [LA03]. The linking logic is a way to reason about composing and executing programs that are annotated with properties, where properties are asserted to hold by authorities using digital signatures. In particular, the linking logic captures the security model of .NET. Because properties are asserted only by authority, the linking logic does not capture the fundamental principal of proof-carrying code, i.e. that proofs are submitted along with code without any need for signatures. A system such as the linking logic can model proof-carrying code by making one of the authorities act as a proof checker. This special authority receives proofs and checks them, signing the property only if the proof is correct. Such an authority may even be local to the code consumer (another process on the computer).

Looking at proof-carrying code in this way is reasonable but incomplete. This scheme models proof checking but it leaves open questions such as which rules are used to check proofs, what form proofs take, and how the proof checker knows that the proofs checked correspond to the program signed. Making the proof checkers black boxes pushes these problems to a higher level without solving them.

Another approach to running untrusted code is sandboxing or containment. In this method code may be generally untrusted, but trusted to perform certain actions. The code is run in a restricted environment that allows only these actions to be performed. In a sense this approach uses dynamic checks of the actions the code wishes to perform, as opposed to static checks that a proof system might use to determine if the code is safe. By deferring checks until actions are actually requested, different types of policies become possible at the expense of performing the dynamic checks. Some properties are inherently stated as dynamic checks and are naturally enforced in that way. Some policies, such as information flow, cannot be enforced dynamically at all. In our examples we show how BCIC can be used to verify both static and dynamic properties.

1.2 Organization

In Chapter 2 we begin by introducing BCIC through some small examples and then formally define Datalog, Binder, and BCIC; we also define terminology relevant to the rest of the document in that chapter. In Chapter 3 we present three larger examples to demonstrate the system and show a variety of ways in which it can be used. Our examples include two examples at the source code level, one on auditing function calls and the other on tracking trust in the λ -calculus, and a third example at the machine code level that uses code instrumentation. In Chapter 4 we present several formal results about Datalog, Binder, and BCIC including decidability and conservativity of BCIC. In Chapter 5 we introduce the concept of proof-by-computation and explain how it works in BCIC. We also develop two examples that use proof-by-computation to show the practical advantages of the method, and update some of the previous examples to take advantage of the feature. We also prove some results about the correctness of the technique. In Chapter 6 we describe our implementation of BCIC. This includes our certified logic module obtained through extraction of our CIC proofs of decidability as well as details relating to network synchronization. In Chapter 7 we conclude with a discussion on our experiences with BCIC and thoughts about further research directions. The Appendix contains a subset of our Coq proof scripts with some explanation of how one uses Coq to prove theorems. Proof scripts, implementation code, and the full text of all the policies of the examples are available at the author’s homepage: <http://www.soe.ucsc.edu/~nwhitehe>.

1.3 Historical development

The version of BCIC presented in this dissertation subsumes our previous work. It should not be necessary to understand earlier versions to understand the version presented here.

Our previous work proposed a combination of reason and authority for authorizing code that we called BLF [WAN04]. It combined Binder with LF [HHP93] instead of CIC,

and used a different syntax than the one presented in Chapter 2. In that work we described the motivation for combining reason and authority in a security logic, presented the example from Section 2.1.3, and showed proof sketches that argued that BLF was decidable and a conservative extension of LF. BLF also included a facility for reasoning about multiple LF signatures, which the current version of BCIC does not have. We discuss some of the reasons for switching from LF to CIC and for not including multiple proof environments in Section 2.4.

After replacing LF with CIC, we implemented an early preliminary version of BCIC [WA05]. That work presented the new combination of Binder with CIC and described our implementation, including some of the details of the network synchronization algorithm. Our current implementation described in Chapter 6 is an evolution of this work.

The most important change in the implementation has been the extraction of a certified implementation of the logic module [Whi07]. By encoding a proof of decidability of BCIC into Coq, we were able to automatically extract working OCaml code that is certified to be a correct decision procedure for BCIC queries. We describe the process for the latest version of BCIC presented here in Section 6.4.

After implementing BCIC we developed and implemented some larger examples to test the system and see where improvements were needed [WJA07]. These examples are presented in Chapter 3. The examples helped us refine the system into the version presented here in Chapter 2. The examples led us to investigate several different extension mechanisms for BCIC. We decided that proof-by-computation was the best extension mechanism. Proof-by-computation is presented in Chapter 5.

Chapter 2

Description of BCIC

In this chapter we introduce our system, BCIC, first informally and then more formally. Through several small examples we hope to give the reader an understanding of how BCIC works and what types of problems it solves. In Section 2.2 we define terminology that will be used throughout this dissertation. The formal description is necessary for understanding the analysis in Chapter 4 but can otherwise be skipped on a first reading. We present a short example to concretely illustrate the definitions for Datalog, Binder, and BCIC. Finally we discuss some of the design decisions we made regarding BCIC.

2.1 Informal description

In this section we introduce BCIC through a running example that illustrates the main features of the system and shows the motivation behind its design.

2.1.1 A small example

The goal of BCIC is to allow the description of security policies that rely both on assertions from authorities and proofs of formal properties as in proof-carrying code. A *policy* is a set of formulas that expresses the trust assumptions and reasoning framework of the code consumer. A typical policy in BCIC that relies only on assertions (the Binder part of BCIC) is the following:

```

trusted(*rootkey*).
trusted(K) :- A says trusted(K), trusted(A).
safe(P) :- A says safe(P), trusted(A).

```

The policy is made up of Datalog-like clauses that describe facts and rules of inference. Variables are written in uppercase, predicate names and atoms in lower case. The difference from pure Datalog is the **says** operator, which indicates externally received statements that have been digitally signed and verified with a public key. *Principals* are identified with public keys, and represent things that make statements. For simplicity we will usually refer to principals by name; in reality policies will refer to actual public keys. As a convention we use asterisks to denote meta-variables. By exchanging signed statements and performing reasoning based on their policies, principals can perform distributed authorization. In this case the policy declares a particular root key as trusted, then allows unlimited delegation of trust. Finally, in the policy if a trusted key signs a program as being safe, it is assumed to be safe.

Proofs and formal properties are referenced in BCIC policies through the special **sat** operator. If there is a CIC proof of a theorem o , then the formula **sat**(o) will be derivable. (There are some restrictions and nuances to this description not covered here, see Section 2.2.3.) For the example policy, we can add an additional rule that says that if there is a proof of memory safety for a program P , then the program is also safe.

```

?- environment(*R*).
safe(P) :- sat(<memsafe ~P>).

```

The first line indicates that ***R*** applies, as an environment. An *environment* declares constructors for CIC terms along with proof rules; in CIC terminology, an environment is a CIC signature. We use the name *environment* to avoid confusion with digital signatures. An environment is specific to a particular programming language and proof methodology. We have adopted the syntax **?- environment(...)** within a policy to denote the global environment that will be used to interpret all CIC terms.

CIC terms represent anything described using constructors declared by an environment, including programs, properties, and proofs. CIC terms are enclosed in angle brackets. Variables inside CIC terms that are bound outside of the CIC term itself must be escaped with `~`. The remaining syntax for policies is borrowed from Prolog. Conjunction is represented using commas, disjunction with semicolons, and reverse implication (“if”) with the symbol `:-`.

For the example, the environment `*R*` may contain objects of the following types which define standard constructs for memory access:

```
mem : Set
val : Set
read : mem -> val -> val
write : mem -> val -> val -> mem
```

The predicate `memsafe` is defined in the global CIC signature paired with the policy rules; it is part of the policy. The proof of `memsafe P` for a particular program will be transmitted in the same way signed statements are transferred.

Suppose that Alice is a user who requires that every program she executes neither access memory incorrectly nor use too many resources. There may be a relatively straightforward way to prove memory safety for the programs in which she is interested, but no simple way for characterizing resource usage. Moreover, excessive resource usage may not be viewed as very harmful when there is someone to blame and perhaps bill. Accordingly, Alice may want proofs for memory safety but may trust Bob on resource usage. Alice constructs a policy that includes:

```
?- environment(*R*).
mayrun(P) :- sat(<memsafe ~P>), economical(P).
economical(P) :- bob says economical(P).
```

The first clause after the declaration of the environment, `mayrun(P) :- sat(<memsafe ~P>), economical(P)` indicates that Alice believes that she may execute a program `P` if there is a proof that `P` is memory safe and she thinks that `P` is economical. The second

clause reflects Alice’s trust in Bob. In more complex examples, other clauses may provide other ways of establishing `economical(P)`. The predicates `mayrun` and `economical` are user-defined predicates.

In turn, Bob trusts Charlie to write only economical programs, and has in his policy:

```
?- environment(*R*).
economical(P) :- charlie_says author(P, charlie).
```

where `author` is another user-defined predicate.

Suppose further that Charlie supplies a program `*prg*` that Alice wishes to execute. Charlie produces a CIC proof, `*prf*`, of memory safety of the program. In other words, in CIC, `*prf*` has type `memsafe *prg*`. Typechecking in CIC is the same as proof checking; by checking the type of `*prf*`, BCIC is verifying Charlie’s proof of memory safety. Charlie publishes his proof by asserting `proof(<*prf*>)`. The predicate `proof` does not have any built-in logical meaning; it simply serves for introducing the proof `*prf*`. Similarly, Charlie asserts `author(<*prg*>, charlie)`.

Alice, Bob, and Charlie all run BCIC servers in the background. When the servers are set up, they are given the address of an existing server. From that point, they synchronize and receive a list of all other known servers. Once connected, they occasionally choose other servers with which to synchronize. After sufficient synchronization, Alice can deduce `economical(<*prg*>)` and `charlie_says proof(<*prf*>)`. After checking the proof `*prf*`, Alice obtains `sat(<memsafe *prg*>)`. Now when Alice queries `mayrun(<*prg*>)`, she receives the answer “yes” and is prepared to run `*prg*`.

When queries are made to a policy, BCIC examines every CIC term that appears in the policy and determines its type. This is equivalent to checking proofs. When Charlie provides a proof of `<safe *prg*>` he is providing a CIC term of type `<safe *prg*>`. Once BCIC determines that the object has this type, it adds the fact `sat(<safe *prg*>)` to the database.

2.1.2 Trust annotations

To demonstrate what can be done with user-defined predicates we show how to add trust annotations to signed statements within the policy. So far we have made no distinctions about assertions from other principals: they are either believed because of some rule in the policy, or they are not. This might not be fine-grained enough for some applications. We might want to give principals varying levels of trust. The trust level in any conclusion requiring many deductions should never exceed the trust we place in any one principal on which the deduction relies.

To this end, assertions and proofs can be annotated with trust levels from a lattice (inspired by multi-level security [And01]). The final trust level will be the greatest lower bound of trust we place in the derivation steps. For example, consider the three trust levels, untrusted, semi-trusted, and trusted (with trusted on top). The policy must define how to calculate the greatest lower bound of any two trust levels. Every predicate about code is extended to include a trust annotation. Instead of `safe(P)` indicating that `P` is safe, we define `safe(P, L)` which indicates that we believe `P` is safe and the derivation that `P` is safe is trusted with trust level `L`. We accept whatever any principal says, but annotate the predicate with the trust level we assigned to the principal. If an implication relies on several hypotheses, we assign the trust level of the conclusion to the greatest lower bound of trust levels in the hypotheses.

The following example shows a policy that defines two predicates, `safe(P)` and `correct(P)`, whose conjunction yields `mayrun(P)`. The policy defines the trust lattice, the trust level of each principal, and the annotated predicates.

```
lte(untrusted, semitrusted).
lte(semitrusted, trusted).
lte(X, X).
lte(X, Z) :- lte(X, Y), lte(Y, Z).

glb(X, Y, X) :- lte(X, Y).
glb(X, Y, Y) :- lte(Y, X).
```

```

trustlevel(alice, trusted).
trustlevel(bob, semitrusted).

safe(P, L) :-
    K says safe(P),
    trustlevel(K, L).

correct(P, L) :-
    K says correct(P),
    trustlevel(K, L).

mayrun(P, L) :- safe(P, U),
                 correct(P, V),
                 glb(U, V, L).

```

2.1.3 Cell phone example

Informal description

Consider the situation in which a cell phone device, referred to as a *host*, downloads a game from an untrusted developer. The user, or *code consumer*, requires memory safety of programs run on the host. A program that violates memory safety might overwrite data such as phone numbers and appointments stored on the device. Suppose that the cell phone service also offers network services and charges a fixed price for every packet sent. A rogue program could be very expensive. Malicious programs exist that secretly use the unsuspecting user's modem to dial high-cost international phone numbers.

To protect the user, the host operating system requires all untrusted code to be memory safe and to send less than X packets per second. The host OS allows for multiple ways in which these properties can be established satisfactorily. For memory safety, the host accepts either code written in the Java Virtual Machine Language (JVML) [LY97], or code carrying proofs of memory safety with respect to a certain set of proof rules. For network usage limits, the host is not going to require proofs, since these are typically hard to construct for interesting programs. It does accept, however, certificates of safety from a

trusted principal. Alternatively, code linked with a library that performs dynamic checking to enforce resource quotas is also believed to satisfy the network quota.

The downloaded game is distributed as two modules, a *Strategy* module written in JVMML and a *Rendering* module written in native code. The developer provides a proof that *Strategy* typechecks and a proof that *Rendering* is memory safe. The JVMML code is compiled by the JVM, then linked with a library that dynamically enforces network resource limits. *Rendering* is asserted to satisfy the network resource limit by a trusted resource signer. Finally, both modules are linked with a trusted library *Gamelib* and executed on the host operating system. This situation is depicted in Figure 2.1.1.

PCC and digital signatures individually are not flexible enough to handle this example because it involves a mixture of proofs and assertions that is not known beforehand to the code consumer. This is an example of where BCIC is most useful.

We now explain the parts of the example: the environment, policy, proofs, and lemmas, and show how they fit together.

Environments

Environments define constructors for CIC terms and define the allowable proof rules. The reader unfamiliar with CIC and Coq may find it helpful to review existing descriptions of CIC and Coq [CH88, Tea, BC04a]. A simple example of an environment that defines natural numbers and proof rules for proving evenness is:

```
Inductive nat : Set :=
| 0 : nat
| S : nat -> nat.

Inductive even : nat -> Prop :=
| evenz : even 0
| evenss : forall (x : nat), even x -> even (S (S x)).
```

Terms of type `nat` include `0` and `(S (S (S 0)))`. The type `(even (S (S 0)))` represents a judgment; an object of type `(even (S (S 0)))` is a proof that 2 is even. The object `((evenss 0) evenz)` is such a proof. The subterm `(evenss 0)` has type `(even 0 ->`

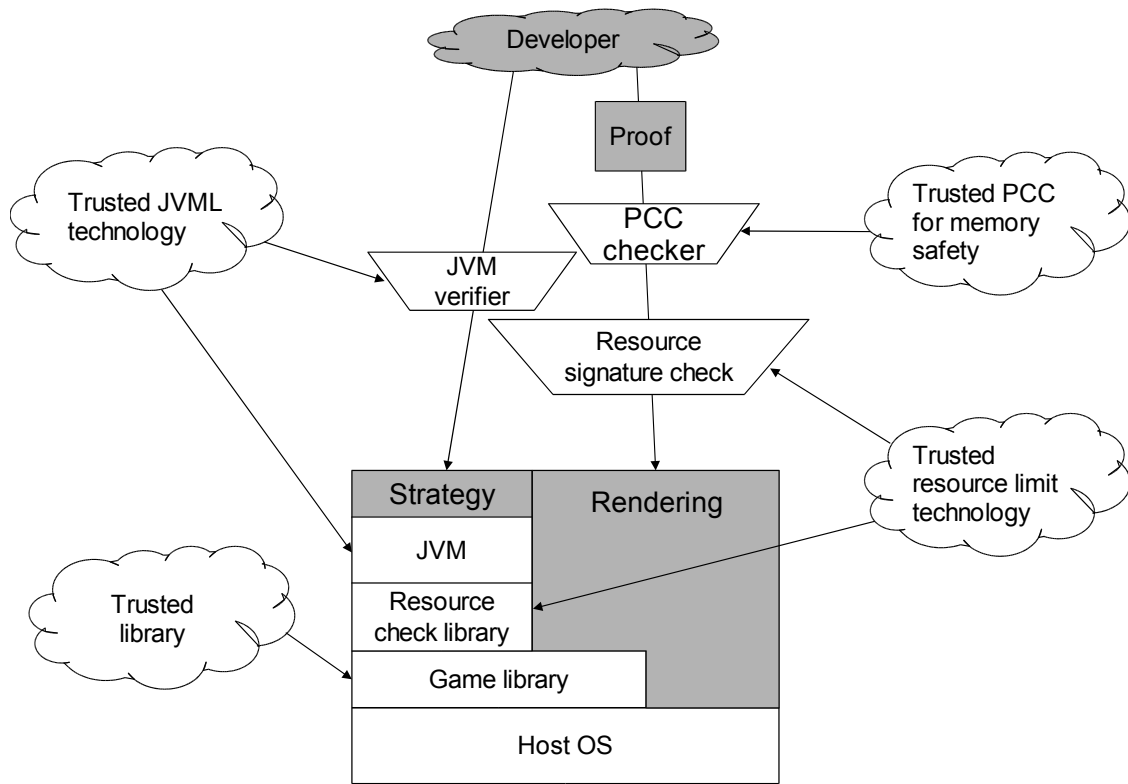


Figure 2.1.1: Cell phone/game device example

`even (S (S 0)))`, which when applied to `evenz` of type `(even 0)` yields the result `(even (S (S 0)))`. Note that an environment is a definition, neither right nor wrong on its own; the correctness of an environment (such as the one above) is dependent on the intended interpretation.

Since we use environments for encoding program properties, we must first explain how we represent programs. One approach is to require programs to be written in a type-safe programming language and then only check types at the source level and assume that executing a type-safe program is memory safe. An alternative approach is to check the actual native machine code that will run on the hardware. Code is written using a subset of the machine instructions and includes typing annotations. Programs written in such a typed assembly language (TAL) [MWCG98] can be proven safe and executed without any assumptions about an interpreter or compiler. Our example includes both situations, so our proof environment includes both sets of rules, both for proving properties about JVMML and for TAL.

Environments can be quite complicated, so we will not present a complete environment for this example. We assume that the environment `*r_tal*` describes constructors for typed assembly language programs with type `prg`, and JVMML programs with type `jprg`. To allow us to reason about linking separate program modules together, the environment also includes rules that define the predicate `link:prg->prg->type`. The predicate `link P Q R` is satisfied when `R` is the result of linking `P` and `Q`. The environment also defines predicates representing memory safety and resource quota satisfaction, `safe:prg->type` and `economical:prg->type`. Rules for deriving `safe P` are included, but there are no rules for proving `economical P`.

The environment include definitions for the predicates `typechecks:jprg->type` and `compile: jprg->prg->type`. The predicate `typechecks J` is satisfied when `J` type-checks as a JVMML program. The predicate `compile J P` is satisfied when `P` is a TAL program that corresponds to the JVMML code `J`.

Policy

The goal of our policy is to describe when we have gathered enough confidence that programs are safe to execute. In the example, to run a program we must believe that it is memory safe and that it is economical with resources. The predicate `mayrun P` indicates that program `P` may execute.

Mentions of CIC predicates and types (such as `prg` and `typechecks`) must be linked to their environment. For example, we may write:

```
?- environment(*r_tal*).  
mayrun(P) :- sat(<safe ~P>),  
             sat(<economical ~P>).
```

It is also possible to extend the policy with actual user requests to run programs. We can include:

```
run(P) :- mayrun(P), user(U),  
          U says run(P).
```

In this case a user `U` requests to run a program `P` by signing the statement `run(P)`. BCIC then imports this statement as `U says run(P)`; if BCIC can derive `run(P)` then the program is run. Many other variants are possible. For example, the `run` predicate could take many arguments indicating various parameters about which capabilities should be given to the program, which user the program should run as, etc.. Determining which keys belong to authorized users can also be performed in the Binder part of BCIC (the original Binder paper has examples [DeT02]).

JVML typechecking

Part of our policy states that we believe that if a program `P` written in JVMML typechecks, then the compiled version of that program `Q` will be safe according to the PCC definition of safety.

```
sat(<safe ~Q>) :-  
    sat(<typechecks ~P>),  
    sat(<compile ~P ~Q>).
```

This rule also illustrates that **sat** may appear in the head of a clause. Derivations of **sat** can come from proofs or from clauses. If **sat(<safe Q>)** is derived from the above rule, BCIC will act as if there were a proof object of **safe Q** even though there is not.

Trusted libraries

The principal **lib_signer** says which libraries should be trusted. We believe that any library **lib_signer** certifies as trusted is safe and satisfies the resource quota.

```
sat(<safe ~L>) :-  
    lib_signer says trusted(L).  
sat(<economical ~L>) :-  
    lib_signer says trusted(L).
```

In a more elaborate version of this example, libraries may be deemed safe only when linked with programs that satisfy certain conditions. These conditions can be expressed by predicates defined using proof rules supplied by the library provider.

There is another principal **res_signer** that we trust to sign programs or libraries that satisfy the correct resource quotas. We also trust **res_signer** to tell us of a library that dynamically enforces the quotas at run-time. We believe that any program linked with this library will satisfy the resource constraints.

```
sat(<economical ~P>) :-  
    res_signer says sat(<economical ~P>).  
sat(<economical ~R>) :-  
    sat(<link ~P ~L ~R>),  
    res_signer says dynamic_enforcer(L).
```

Practical programming languages might require more than merely linking with one library; they might also require that the library be initialized at program startup. This could be

represented by adding more predicates, but for simplicity we limit the requirements to a simple link.

Linking lemmas

Suppose one can prove that safe programs linked together remain safe, and economical programs linked together remain economical. The proofs of these facts are not included directly in our policy, but after being checked they will allow us to derive the following statements:

```
sat(<forall (p q r : prg),
    safe p -> safe q -> link p q r -> safe r>).
sat(<forall (p q r : prg),
    economical p -> economical q ->
    link p q r -> economical r>).
```

Imported statements

The principals `lib_signer` and `res_signer` generate and sign several statements which are imported into the local policy. The imported statements will appear as:

```
lib_signer says trust(p_gamelib).
res_signer says dynamic_enforcer(p_enforcer).
res_signer says sat(<economical p_render>).
```

Several actual proofs are also provided by `res_signer` and `developer`. After being checked, the single proof provided by `res_signer` yields `sat(<safe p_enforcer>)`. The proofs from `developer` let us conclude:

```
sat(<safe p_render>).
sat(<typechecks p_strategy>).
sat(<compile p_strategy p_cstrategy>).
sat(<link p_cstrategy p_enforcer p_lcstrategy>).
sat(<link p_lcstrategy p_gamelib p_linked>).
sat(<link p_linked p_render p_final>).
```

From our previously developed policy and these imported statements we can derive `mayrun(p_final)`. To show this we first consider the strategy module. Because `p_strategy` typechecks and is compiled into `p_cstrategy`, the single rule from section 2.1.3 implies `sat(<safe p_cstrategy>)`. A proof shows that `p_enforcer` is safe, so linking `p_cstrategy` with `p_enforcer` yields another program `p_lcstrategy` that is also safe (by the rules from section 2.1.3). Since `p_lcstrategy` is linked with the library `p_enforcer` from `res_signer` that dynamically enforces resource quotas, using the rules from section 2.1.3 we can derive `sat(<economical p_lcstrategy>)`. Thus, we conclude `sat(<safe p_lcstrategy>)` and `sat(<economical p_lcstrategy>)`.

The trusted library signer says that `p_gamelib` is a trusted library, so using the rules from section 2.1.3 we can conclude `sat(<safe p_gamelib>)` and also `sat(<economical p_gamelib>)`. Since the module `p_linked` is `p_lcstrategy` linked with the game library, `p_gamelib`, and we believe both `p_lcstrategy` and `p_gamelib` are safe and economical, by the linking rules in section 2.1.3 we also believe that `p_linked` is safe and economical. In addition, the developer provided a proof that `p_render` is safe, and the resource signer said it was economical, so we conclude `sat(<safe p_render>)` and `sat(<economical p_render>)`.

Since both `p_linked` and `p_render` are safe and economical, we conclude `sat(<safe p_final>)` and `sat(<economical p_final>)`. Finally we conclude `mayrun(p_final)` and the user is allowed to play the downloaded network game on the cell phone.

Summary of cell-phone example

This example shows how reason and authority can be combined in a multi-part system and how extra features can be added to the policy without changing BCIC itself through user-defined Binder predicates. For this example we have written the **policy** but have not described a CIC environment for proofs, implemented a runtime environment for executing programs, or proven anything about correctness of programs.

2.2 Terminology

We define the terms we will be using throughout the dissertation, first for Datalog, then for Binder, and finally for BCIC.

Mechanized proofs Our convention is that when we write **Theorem***, the theorem has been formalized in CIC and proved using Coq. The proof script will either appear in the Appendix or be available for download on our webpage: <http://www.soe.ucsc.edu/~nwhitehe>. If there is also a non-mechanized version of the proof available, it will always appear directly after the statement of the theorem. Sometimes we provide a short description of the method of proof for the proof script without providing an entire non-mechanized proof.

2.2.1 Datalog

BCIC is an extension of Binder[DeT02] with terms from the Calculus of Inductive Constructions [CPM90, BC04a]. Since Binder is built on top of Datalog, we start by defining Datalog.

Datalog [Ull88] is Horn logic where all function symbols have arity 0, i.e. are constants. A Datalog program contains facts and rules of inference. For the case of access control, we call Datalog programs *policies*. Queries run against the policy and either succeed or fail. Datalog is similar to Prolog, but unlike Prolog it does not have negation, function symbols, or any meta-logical constructs such as cut.

The syntax of Datalog *terms*, *atomic formulas*, and *formulas* is as follows:

$t ::=$	term:
c	<i>atom</i>
x	<i>variable</i>
$a ::=$	atomic formula:
$p(t_1, \dots, t_n)$	<i>predicate</i>
$f ::=$	formula:
$a :- a_1, \dots, a_n$	<i>clause</i>

Atoms come from an infinite set of identifiers, in our case lower case strings. *Variables* also

come from a disjoint set of identifiers; we use upper case strings for variables. Because formulas are always clauses, we generally refer to formulas as clauses. A clause is made up of a *head* and a *body*. In the definition above, a is the head and $a_1, \dots, a_n$ is the body. Clauses can have bodies that are empty (list of length zero), in which case they are called *facts*. In an atomic formula $p(t_1, \dots, t_n)$ we refer to p as the *predicate name*.

In addition we place a restriction on clauses that no variable appears in the head unless it appears somewhere in the body. This is a standard restriction that simplifies the analysis of the logic [Ull88].

We call variables that appear in the head of a clause and not the body *free variables*; they are usually written with an underscore in Prolog.

An atomic formula without variables is a *ground atomic formula*. Similarly, a clause without variables is a *ground clause*.

A *policy* is a list of clauses, and a *query* against a policy is a ground atomic formula. (We define queries this way because this is all we need; one could also define queries to allow multiple goals and to have variables). When a query q succeeds against a policy r we will write $r \vdash q$, or q is *derivable* from r . Let s be a substitution over atomic formulas that maps variables to atoms. Then $r \vdash q$ is defined by the single rule:

$$\frac{\begin{array}{l} (a :- a_1, \dots, a_n) \in r \\ r \vdash s(a_i) \text{ for } i = 1, \dots, n \end{array}}{r \vdash s(a)}$$

Datalog is a suitable match for many access control policies because it is relatively simple and expressive enough to categorize objects, represent roles of principals, and encode subgroup inclusion.

2.2.2 Binder

Binder differs syntactically from Datalog only in atomic formulas:

$a ::=$	atomic formula:
$p(t_1, \dots, t_n)$	<i>predicate</i>
$t \text{ says } p(t_1, \dots, t_n)$	<i>quoted predicate</i>

Predicates are regular Datalog atomic formulas, while *quoted predicates* are Datalog atomic formulas that are preceded by a *t* **says** The *t* is called a *principal*, but is syntactically just a term. Derivations in Binder are defined by the same rule as for Datalog.

Binder adds the notion of importing and exporting facts from one location to another. This allows policies to define mechanisms for distributed authentication and authorization. The general idea is that if Alice has a policy with the fact *f*, then this fact will be exported from Alice’s policy using public-key encryption and then imported into another context as *alice^{pk} says f*, where *alice^{pk}* is an atom that represents Alice’s public key.

Notice that **says** cannot be nested. This means that if Alice signs a statement *f* and Bob receives that statement and passes it along to Carol, Carol can only know that Alice signed *f* and/or that Bob signed *f*. Carol can never know that Bob knows that Alice signed *f*. The restriction on nesting is natural for access control applications and also convenient because it makes demonstrating decidability easier.

2.2.3 BCIC

The syntax of BCIC adds CIC terms to Binder.

<i>t</i> ::=	term:
<i>c</i>	<i>atom</i>
<i>x</i>	<i>variable</i>
< <i>o</i> >	<i>CIC term</i>

In addition to CIC terms, we must keep track of a CIC signature. In BCIC, every policy is paired with an *environment* that is a CIC signature. We do not use the term “signature” because digital signatures are used in other parts of the system with a different meaning. Policies declare an environment ***E*** with the special syntax:

```
?- environment(*E*).
```

CIC terms can represent objects, types, kinds, theorems and proofs. In addition, CIC terms can also refer to variables in the policy using an escape syntax. The syntax $\sim P$ within a CIC term references the variable *P*. Without the escape character, the syntax *P* references a variable *P* bound within the CIC environment. Variables at the policy level are

implicitly universally quantified over atoms, which includes CIC terms. Variables bound within CIC terms are explicitly quantified using **forall**, **exists**, or **fun**.

In addition to predicates and predicates quoted with **says**, in BCIC we have the **sat** predicate. The atom **sat**(t) informally means that theorem t is true, i.e. there is a CIC term with type t .

BCIC atomic formulas have the syntax:

$a ::=$	atomic formula:
$p(t_1, \dots, t_n)$	<i>predicate</i>
$t \text{ says } p(t_1, \dots, t_n)$	<i>quoted predicate</i>
sat (t)	<i>theorem</i>
$t \text{ says sat}(t)$	<i>quoted theorem</i>

It might seem somewhat odd to allow atomic formulas of the form **<cicterm> says p(...)**. In fact none of our examples use atomic formulas of this type. We only allow them because we allow CIC terms wherever atoms may appear.

In addition to the Datalog derivation rule, BCIC has a rule for deriving **sat** statements. The notation $e \vdash_{CIC} o : T$ means that in CIC environment e , the object o has type or kind T . Derivations in BCIC are of the form $(r, e) \vdash a$, which means that from policy r paired with CIC environment e , the atomic formula a is derivable.

Derivation in BCIC is defined by:

$$\frac{(a :- a_1, \dots, a_n) \in r \quad (e, r) \vdash s(a_i) \text{ for } i = 1, \dots, n}{(e, r) \vdash s(a)} \quad (2.2.1)$$

$$\frac{T, o \in U(e, r) \quad \exists o \in U(e, r). e \vdash_{CIC} o : T}{(e, r) \vdash \text{sat}(T)} \quad (2.2.2)$$

$$\frac{T, o, T' \in U(e, r) \quad (e, r) \vdash \text{sat}(T) \quad e, \alpha : T \vdash_{CIC} (\alpha o) : T'}{(e, r) \vdash \text{sat}(T')} \quad (2.2.3)$$

$$\frac{T \in U(e, r) \quad e \vdash_{CIC} \text{refl.equal true} : T}{(e, r) \vdash \text{sat}(T)} \quad (2.2.4)$$

In Formula 2.2.1 s is a substitution over atomic formulas that maps variables to atoms.

This formula represents derivation as in Datalog and Binder.

In Formulas 2.2.2, 2.2.3, and 2.2.4 the function U takes in a policy and returns a universe of CIC terms that appear in the policy or that are directly computable from the policy. Note that in Formula 2.2.2 the proof object o is restricted to being in $U(e, r)$, this universe of directly computable CIC terms. This restriction means that BCIC only considers proofs that have been imported into the policy and the consequences of those proofs. If a proof does not exist in the policy, it has no effect on the logic. Without this restriction BCIC would be immediately hopelessly undecidable. Trying to decide when proof terms exist in CIC is equivalent to trying to automatically prove arbitrarily difficult mathematical theorems.

Formula 2.2.3 allows derivations of $\text{sat}(T')$ based on the assumed existence of a proof of T . This rule allows mixing of actual proofs and facts that are assumed to be true. In the cell phone game example of Section 2.1.3, this rule would be used to derive consequences of the linking lemmas if the linking lemmas were stated directly in the policy without proofs.

As we compute U on an environment e and policy r we keep all the CIC terms in normal form. Normal form in CIC means that there are no more β , ι , δ , or η reductions to the term. Luckily CIC has normal forms for all terms [CH88]. The function U on an environment e and policy r is defined to be the minimal set of CIC terms that satisfies the following rules:

1. contains every CIC term that appears in r
2. if $u \in U(e, r)$ and $e \vdash_{CIC} u : T$ and T is in normal form but T is not **Set** or **Prop**, then $T \in U(e, r)$
3. for every $u, v \in U(e, r)$, if the CIC term $(u \ v)$ is well-typed then $(u \ v)$ must also be in $U(e, r)$
4. for every $u, v \in U(e, r)$, if u contains an escaped variable x and $u[x \rightarrow v]$ is well-typed then $u[x \rightarrow v]$ must be in $U(e, r)$

```
student(nathan).
student(avik).
faculty(martín).
admin(jordan).
user(X) :- student(X).
user(X) :- faculty(X).
user(X) :- admin(X).
mayrun(X, matlab) :- user(X).
mayrun(X, netconfig) :- admin(X).
```

Figure 2.3.1: Example Datalog policy for a university supercomputer

2.3 University supercomputer example

To illustrate the terms defined in Section 2.2, consider a University supercomputer lab with administrators who wish to control which programs may be executed by different users in order to protect the integrity of the lab. The administrators categorize users into a hierarchy of groups, then give each group permission to execute various programs. Datalog facts in the policy specify which group each user belongs to, and Datalog clauses denote the subgroup relationships.

To decide if person p should be allowed to run program g , the reference monitor must decide if `mayrun( $p$ ,  $g$ )` is derivable from the policy in Figure 2.3.1. This is not an entirely trivial decision. There are no restrictions on policies, so the subgroup relationships could describe a complicated directed graph with cycles. Deciding which groups someone belongs to is deciding graph reachability.

In the example, `nathan`, `jordan`, and `matlab` are some examples of atoms. The term `X` is a variable. Both `student(avik)` and `mayrun(X, matlab)` are atomic formulas, and `mayrun(nathan, matlab)` is a ground atomic formula. An example of a clause is `user(X) :- student(X)`. Another example of a clause is the fact `student(nathan)` which has an empty body. Figure 2.3.1 is a collection of clauses that constitutes a policy.

Continuing our computer center example, if we use Binder we can write the distributed policy shown in Figure 2.3.2 for deciding who may run which applications.

```

trusted(*rootkey*).
trusted(X) :- Y says trusted(X), trusted(Y).
registrar(X) :- Y says registrar(X), trusted(Y).
admin(X) :- Y says admin(X), trusted(Y).
student(X) :- Y says student(X), registrar(Y).
user(X) :- student(X).
user(X) :- admin(X).
mayrun(X, matlab) :- user(X).
mayrun(X, netconfig) :- user(X), Y says mayrun(X, netconfig), admin(Y).

```

Figure 2.3.2: Example of a Binder distributed authorization policy for a supercomputer

The first clause of Figure 2.3.2 defines a trusted root key from which all other trust derives. The trusted root key signs other keys which are then also trusted. The clause `trusted(X) :- Y says trusted(X), trusted(Y)` allows unbounded delegation of trust. Trusted keys determine who is the registrar, and the registrar determines who is a student. Students are users, and users may run `matlab`. For `netconfig`, the person must be a user and an administrator must explicitly authorize them.

Both `student(X)` and `Y says student(X)` are atomic formulas; `student(X)` is a bare predicate and `Y says student(X)` is a quoted predicate.

We now extend the example to BCIC. The computer lab administration just got access to a new supercomputer. For performance reasons the operating system cannot provide effective memory protection between programs. If one program crashes, it can interfere with all the results being calculated by other users. In addition, programs can dynamically declare how much processing power they need and the department will be billed accordingly. Because of these factors, the administrators require all programs that run on the supercomputer be memory safe and not request an inordinate amount of processing power without prior authorization. To enforce these requirements they construct the policy in Figure 2.3.3.

The environment `*proofrules*` defines the assembly language syntax for the supercomputer along with the predicate `memsafe` which guarantees memory safety. To run

```

?- environment(*proofrules*).
trusted(*rootkey*).
trusted(X) :- Y says trusted(X), trusted(Y).
mayrunonsupercomputer(X, Y) :- user(X), safe(Y), economical(Y).
safe(X) :- Y says safe(X), trusted(Y).
economical(X) :- Y says economical(X), trusted(Y).
safe(X) :- sat(<memsafe ~X>).

```

Figure 2.3.3: An example BCIC policy for a supercomputer

their programs on the supercomputer, principals must be authorized users and either a trusted principal must vouch that the program is safe and economical, or someone must produce a proof that the program satisfies `memsafe` and a trusted principal must vouch that it is economical.

An example CIC term is `<memsafe X>`. Terms `X` and `Y` are variables. Some example atomic formulas are `safe(X)`, `Y says economical(X)`, and `sat(<memsafe X>)`.

Summary of university supercomputer example This example shows a simple combination of reason and authority that involves proofs for one property and assertions for another, to illustrate the progression of policy languages from Datalog to Binder to BCIC. For this example we have written the **policy** but have not described a concrete CIC environment for the proofs of `memsafe`, nor have we implemented a runtime environment to run programs on a supercomputer or proven anything about the correctness of `memsafe`.

2.4 Design choices

Our goals in designing BCIC are to create a system that is practical, as simple to use as possible, and as extensible as possible while remaining logically grounded. During its development the system went through several variants that we ultimately rejected. In this section we discuss some of the design considerations that led to BCIC.

Why CIC

We choose CIC rather than some other logical framework such as LF for several reasons. The most compelling reason is that CIC allows inductive definitions of datatypes and automatically generates induction principles for these datatypes. For example, we can define natural numbers:

```
Inductive nat : Set :=  
| 0 : nat  
| S : nat -> nat.
```

To reason about natural numbers, i.e. to prove some property P about all natural numbers, we need the induction principle for natural numbers:

```
nat_ind  
  : forall P : nat -> Prop,  
    P 0 -> (forall n : nat, P n -> P (S n)) -> forall n : nat, P n
```

This induction principle `nat_ind` is calculated according to an algorithm that is part of CIC.

When representing programs, this ability to define data structures inductively and immediately use the generated induction principle is critical. It means we can always use induction as a proof strategy without worrying about creating our own induction principles. Of course CIC also has a rich type system, which allows a high degree of polymorphism and abstraction; this also helps the encoding of programs and in reasoning about programs.

Coq has good support for theorem proving in CIC. Using Coq, one can define tactics and take advantage of the library of predefined proof strategies. This makes constructing proofs of properties about programs for the examples practical, and also allows us to prove things like decidability. Another advantage of CIC is that an ever-increasing accumulation of mathematical facts have been proven using Coq. These proof scripts can often be used directly or copied and adapted to new proof developments without much difficulty. Our proof scripts make heavy use of the standard library of lemmas and theorems available in Coq.

We rely on the automatic program extraction feature of Coq to generate our certified implementation (Section 6.4). Producing a certified implementation using some other tool might be possible; in Coq the process of extracting an implementation from the proof was a single line. Because of the expressiveness of the CIC type system, the type of the extracted program guarantees that the algorithm is correct.

Multiple environments

In some earlier versions of BCIC we allowed the possibility of multiple CIC (or LF) environments [WAN04, WA05]. This allowed the examples we presented without details to be slightly more interesting, but in the end all of our more fully developed examples required only one environment. Allowing multiple environments made the policy syntax more complicated and made reasoning about the correctness of BCIC much more difficult. Each CIC term only makes sense in an environment, so every CIC term had to be paired with an environment to maintain logical consistency. In addition, multiple environments are most useful when they can prove different properties about the same data (i.e. prove different properties about code). This requires that both environments agree on the structure of the data, which requires some notion of sub-environment in BCIC. The current version of BCIC avoids these difficulties by setting a single global CIC environment for every policy.

A belief distinction

A policy writer might wish to combine reasoning based on proofs with assertions from trusted authorities, but might also wish to maintain a distinction between the conclusions based on trust and those based on actual proofs. One can imagine a separate special **believe** predicate that behaves similarly to **sat**. A proof of T might be imported as both **sat**(T) and **believe**(T), but trusting Alice about T might only result in **believe**(T). This sort of distinction could help the policy writer avoid mistakes.

In fact it is possible to construct a **believe** predicate in a policy without any modifications to BCIC. We do so in Section 3.1.2 for an example.

Axioms versus dependent proofs

When a code producer proves safety but needs some extra assumptions, these could be represented by new unproven axioms in the environment. One can imagine a version of BCIC that tracks different sections of the environment and discharges axioms by fiat of an authority or by a proof that the axiom is true, perhaps proved using other trusted axioms. Instead we model extra assumptions in proofs using dependent types and implication. A proof of T that requires axiom A is a proof of $A \rightarrow T$. CIC already has the machinery to deal with dependent types and implication, so this solution is much simpler.

Explicit proof terms

Another idea is to include proof terms in policies. Instead of `sat(T)`, the syntax would be `sat(o:T)`. Without explicit proof terms in the formulas it is sometimes difficult to tell who produces which proof in the examples. One might also want to attach trust levels to proofs as well as assertions because of the risk that proof systems themselves might be subtly inconsistent.

We decided not to mention proof terms directly in `sat` for several reasons. First, it simplified the syntax and proof rules of the logic. Also, if every logical statement about satisfaction includes a proof term, there is an issue of what proof term to use for asserted properties. It is possible to add some sort of `hole` constructor that represents a missing term of a given type [App00]. This raises questions about when the `hole` construct is allowed and generally complicates the logic. The strongest argument against including explicit proof terms in policies is that the meaning of policies should not depend on proof terms themselves, but merely on the fact that they exist or are assumed to exist.

Chapter 3

Larger examples

In this chapter we present a set of examples that demonstrate the utility and practicality of BCIC. These examples are more detailed and worked out than the earlier examples in Chapter 2. The goal of the examples in this chapter are to show how BCIC can be used in interesting ways as well as explore what types of issues arise when fleshing out the details of the system.

The first two examples we present analyze potentially unsafe programs at the source code level. The first example relates to auditing the function calls that an extension makes in a managed environment.

The second example involves tracking trust in a type system, where typecasts that escape the system must be audited.

The final example deals with code at the machine code level. In this example we show how BCIC can be used to implement a simple control-flow integrity policy for instrumented machine code.

Two of the examples presented in this chapter are extended in Section 5.7 using the proof-by-computation techniques from Chapter 5.

3.1 Auditing function calls

In this example we consider the calling behavior of programs in a managed environment of libraries. A base application may allow extensions that provide additional functionality not only to the user but also to other extensions. The extensions may come from many different sources, and accordingly they may be trusted to varying extents. By constraining calls, the security policy can selectively allow different functionality to different extensions.

3.1.1 Language

We study an interpreted extension language. Specifically, we use an untyped λ -calculus with a special `call` construct that represents operating system calls and calls to other libraries. All calls take exactly one argument, which they may ignore. In order to allow primitive data types, we also include a representation for data constructors (`constr0`, `constr1`, and `constr2`, for constructors that take zero, one, and two arguments respectively). These constructors are enough to handle all the data types that appear in our implementation, including natural numbers, pairs, and lists. Destructors have no special syntax, but are included among the calls. For example, `call fst pr` returns the first element of the pair `pr`.

The syntax of the language is defined in CIC as:

```
Inductive exp : Set :=
| var : nat -> exp
| abs : exp -> exp
| app : exp -> exp -> exp
| call : funcname -> exp -> exp
| constr0 : constrname -> exp
| constr1 : constrname -> exp -> exp
| constr2 : constrname -> exp -> exp -> exp.
```

This definition introduces a class of expressions inductively defined by cases with a type for each case. The constructor `var` represents a variable reference, `abs` represents abstraction,

`app` represents application, and `call` represents a call to a specific library function with one argument. Expressions rely on de Bruijn notation, so variables are numbered and binding occurrences of variables are unnecessary. The following example expression takes an element of data and writes it to a file:

```
(abs
  (call write
    (constr2 pair
      (call open (constr0 (string 'file.txt'))
        (var 0))))))
```

A policy can decide which calls any piece of code may execute. The policy can be expressed in terms of a parameter `audit_maycall`.

```
Parameter audit_maycall : exp -> funcname -> Prop.
```

The keyword `Parameter` in the environment declares a type to the predicate `audit_maycall` without giving any constructors; proofs will be given by assertion because there is no other way to construct proofs of `audit_maycall`. According to this type, every audit requirement mentions the entire program `exp` that is the context of the audit. Mentioning a subexpression in isolation would not always be satisfactory, and it may be dangerous as the effects of a subexpression depend on its context. The audit requirement also mentions the name of the function being called. We omit any restrictions on the arguments to the function, in order to make static reasoning easier; we assume that the callee does its own checking of arguments.

The predicate `audited_calls` indicates that a piece of code has permission to make all the calls that it could make. This predicate is defined inductively by:

```
Inductive audited_calls : exp -> exp -> Prop :=
| audited_calls_var :
  forall e n,
    audited_calls e (var n)
| audited_calls_app :
```

```

forall e e1 e2,
  audited_calls e e1 ->
  audited_calls e e2 ->
  audited_calls e (app e1 e2)
| audited_calls_abs :
forall e e1,
  audited_calls e e1 ->
  audited_calls e (abs e1)
| audited_calls_call :
forall f e e1,
  audited_calls e e1 ->
  audit_maycall e f ->
  audited_calls e (call f e1)
| audited_calls_constr0 :
forall e cn,
  audited_calls e (constr0 cn)
| audited_calls_constr1 :
forall e cn e1,
  audited_calls e e1 ->
  audited_calls e (constr1 cn e1)
| audited_calls_constr2 :
forall e cn e1 e2,
  audited_calls e e1 ->
  audited_calls e e2 ->
  audited_calls e (constr2 cn e1 e2).

```

The first argument to `audited_calls` remembers the entire program expression while the second argument represents a subterm that is being analyzed. Crucially, the case of `audited_calls_call` includes an `audit_maycall` requirement. Every `call` statement requires an audit. A code producer may construct a proof that, given the right audits, the code satisfies the predicate `audited_calls`.

3.1.2 Policy

Audit statements do not have proofs. They cannot, since there are no proof constructors for them in the CIC environment. They are provided by authorities and trusted to be true through BCIC's policies.

The policy writer can express trust in statements that have no proof using a new `believe` predicate, and rules such as:

```
believe(F) :- A says believe(F), trusted(A).
```

This rule expresses that, when a trusted authority says to believe something, the entity that evaluates the policy (in other words, the reference monitor) believes it.

In order to allow reasoning on beliefs, some additional rules are useful:

```
believe(F) :- sat(F).  
believe(Q) :- believe(P), believe(<~P -> ~Q>).
```

Here, the notation `<term>` includes a CIC term within the policy. The notation `~P` within an included CIC term indicates a free variable that is bound in the policy rule. With these rules, all formulas that have proofs are believed, and belief is closed under modus ponens. (A generalization of the second rule to dependent types, of which implication is a special case, might be attractive but is not needed for our examples.)

Using these constructs and rules, a complete policy of a code consumer may say that a program is allowed to run if it has been properly audited. The policy can specify which principals are trusted for the auditing. In addition, the policy may collect calls into groups, thus authorizing sets of calls with a single statement from a trusted authority. The complete policy may therefore look as follows:

```
trusted(alice).  
trusted(B) :- A says trusted(B), trusted(A).  
  
believe(F) :- sat(F).  
believe(Q) :- believe(P), believe(<~P -> ~Q>).  
believe(F) :- A says believe(F), trusted(A).  
  
classify(setuid, dangerous).  
classify(setgid, dangerous).  
classify(sbrk, userlowlevel).  
classify(sprintf, io).  
classify(sprintf, dangerous).
```

```

believe(<audit_maycall ~P ~F>) :-
  classify(F, C), A says allowgroup(P, C), trusted(A).

mayrun(P) :- believe(<audited_calls ~P ~P>).

```

Many variants are of course possible. In particular, a policy may let the auditing requirements depend on CIC proofs about intrinsic properties of the code, with clauses of the form `believe(<audit_maycall ...>) :- sat(...)`. Furthermore, a policy may concern not only the decision to run a program but also any requirements for checks during execution.

3.1.3 Correctness

This example is simple enough that not too much theory is needed, but there are some subtleties (in part because functions can be higher-order, and recursion is possible). We prove (in Coq) that if a piece of code passes the analysis and is executed, every actual call will have been audited (but that does not say anything about the appropriateness of particular BCIC policies such as that of Section 3.1.2). Proving this theorem requires defining the operational semantics of the language in such a way that a history of calls is kept.

We define a state as a pair that consists of a list of calls and an expression, then define the single-step reduction relation $S \rightarrow S'$ as for λ -calculus. The only nonstandard rule is for calls: $(l, \text{call } f v) \rightarrow (f :: l, O(f, v))$, where we let the expression $O(f, v)$ represent the return value of calling function f with value v . We assume that the returned values do not contain calls themselves. We obtain:

Theorem* 3.1 *For all programs e , if `audited_calls e e` is true and $([], e) \rightarrow^* (l, e')$, then for all function names f in l , `audit_maycall e f` is true.*

The proof relies on a lemma that says if a function name f appears in l then f appears in the program e . The proof of the lemma is by induction on reductions, then by case analysis of all the possible reductions. The substitutions that arise from β reductions constitute the only difficulty.

Summary of auditing function calls example

This example illustrates a fully developed framework for scripting extensions where function calls are analyzed and allowed or denied based on a security policy. For this example we have written a **policy**, defined an explicit CIC proof **environment** for proving **audited_calls**, developed a **runtime** environment for executing programs written in the language, and proven **correctness** of the **audited_calls** predicate with respect to an operational semantics of the language.

3.2 Trust in the λ -calculus

Ørbæk and Palsberg define a simple type system for a λ -calculus with trust annotations [ØP97]. In this system, one may for instance describe applets for a web server; the type system can help protect the applets from malicious inputs and the web server from malicious applets. The language includes a construct **trust** for untainting, thus supporting a form of declassification. (A dual form of declassification is turning secret data into public data.) In this example we study how to impose auditing requirements on declassifications. In Section 2.1.2 we annotated statements made by principals with trust levels. In this example the data types within a language are annotated with trust levels.

3.2.1 Language

The type system allows a small set of built-in types and function types; in addition, types include the annotations **tr** or **dis**, which indicate trust and distrust, respectively. Data from unknown outside parties, annotated with **dis**, can be treated with the proper suspicion. For instance, int^{dis} is the type of distrusted integers, while $(\text{int}^{dis} \rightarrow \text{int}^{tr})^{tr}$ is the type of trusted functions that take distrusted integer inputs and return trusted integer results.

First we define in CIC the types, annotations, and relations between types. Types are defined as **bare_type**, which become **annotated_type** when annotated by **tr** or **dis**.

Subtyping is defined in the expected way for both bare and annotated types. Auxiliary functions `trust_lte` and `join` represent the partial order of the trust annotations and give the greatest lower bound of two trust levels, respectively.

```

Inductive trust_type : Set :=
| tr : trust_type
| dis : trust_type.

Inductive bare_type : Set :=
| usertype : nat -> bare_type
| arrow : annotated_type -> annotated_type -> bare_type

with annotated_type : Set :=
| annotate : bare_type -> trust_type -> annotated_type.

Inductive trust_lte : trust_type -> trust_type -> Prop :=
| trust_lte_refl : forall (T : trust_type), trust_lte T T
| trust_lte_trdis : trust_lte tr dis.

Inductive bare_lte : bare_type -> bare_type -> Prop :=
| bare_lte_refl :
  forall (T : bare_type), bare_lte T T
| bare_lte_arrow :
  forall (X Y A B : annotated_type),
    ann_lte Y B -> ann_lte A X ->
    bare_lte (arrow X Y) (arrow A B)
with ann_lte : annotated_type -> annotated_type -> Prop :=
| ann_lte_refl :
  forall (T : annotated_type), ann_lte T T
| ann_lte_annotate :
  forall (A B : bare_type)(U V : trust_type),
    bare_lte A B -> trust_lte U V ->
    ann_lte (annotate A U) (annotate B V).

Definition join (U V : trust_type) : trust_type :=
  match U with
  | tr => V
  | dis => dis
  end.

```

Expressions in the language are similar to those of $\lambda_{<}$, the simply typed lambda calculus with subtyping, with the addition of `trust`, `distrust`, and `check` expressions. Each `trust` is tagged with an identifier so it can be referenced by an auditor. We made typecasts explicit in both origin and destination types for simplicity in the typechecker. We also included `int` and `bool` as built-in types. Here we show the definition of expressions and typechecking. As usual, typechecking is done with respect to typing assumptions in a context.

Definition `trustid` : Set := nat.

```

Inductive expr : Set :=
| const_int : nat -> expr
| const_bool : bool -> expr
| var : nat -> expr
| abs : annotated_type -> expr -> expr
| app : expr -> expr -> expr
| trust : trustid -> expr -> expr
| distrust : expr -> expr
| check : expr -> expr
| cast : expr -> annotated_type -> annotated_type -> expr.

```

```

Inductive context : Set :=
| nilctx : context
| cons : annotated_type -> context -> context.

```

```

Inductive has_type : context -> expr -> annotated_type -> Prop :=
| type_int : forall n C,
  has_type C (const_int n) (annotate (usertype 0) tr)
| type_bool : forall p C,
  has_type C (const_bool p) (annotate (usertype 1) tr)
| type_varz : forall T C,
  has_type (cons T C) (var 0) T
| type_varn : forall n T T2 C,
  has_type C (var n) T ->
  has_type (cons T2 C) (var (S n)) T
| type_abs : forall C E T T2,
  has_type (cons T C) E T2 ->
  has_type C (abs T E) (annotate (arrow T T2) tr)

```

```

| type_app : forall C E1 E2 T1 T2 U V,
  has_type C E1 (annotate (arrow T1 (annotate T2 U)) V) ->
  has_type C E2 T1 ->
  has_type C (app E1 E2) (annotate T2 (join U V))
| type_trust : forall C E T U id,
  has_type C E (annotate T U) ->
  has_type C (trust id E) (annotate T tr)
| type_distrust : forall C E T U,
  has_type C E (annotate T U) ->
  has_type C (distrust E) (annotate T dis)
| type_check : forall C E T,
  has_type C E (annotate T tr) ->
  has_type C (check E) (annotate T tr)
| type_cast : forall C E T T2,
  has_type C E T ->
  ann_lte T T2 ->
  has_type C (cast E T T2) T2.

```

3.2.2 Policy

By formalizing the type system in Coq, we can write BCIC security policies that rely on type safety according to this type system. A simple example is the following policy:

```

mayrun(C, P) :- sat(<has_type ~C ~P (annotate (usertype 0) tr)>)

```

This policy says that program P may run in context C if it typechecks and has the trusted type int^{tr} . This policy does not exclude the possibility that this type is trivially obtained by an application of `trust`, however.

The next step is to require that all applications of the `trust` operator be audited. Since the `trust` operator is basically a way to escape the type system, the value of the type system for security would be questionable at best if any program could use `trust` indiscriminately, hence the desire for auditing.

Here we define a predicate `trusts_audited` which a program satisfies when all of its `trust` expressions have been audited.

```

Inductive trusts_audited : expr -> expr -> Prop :=
| audited_int :
  forall P n,
    trusts_audited P (const_int n)
| audited_bool :
  forall P b,
    trusts_audited P (const_bool b)
| audited_var :
  forall P n,
    trusts_audited P (var n)
| audited_abs :
  forall P T E,
    trusts_audited P E ->
    trusts_audited P (abs T E)
| audited_app :
  forall P E1 E2,
    trusts_audited P E1 ->
    trusts_audited P E2 ->
    trusts_audited P (app E1 E2)
| audited_trust :
  forall P E id,
    audit P id ->
    trusts_audited P E ->
    trusts_audited P (trust id E)
| audited_distrust :
  forall P E,
    trusts_audited P E ->
    trusts_audited P (distrust E)
| audited_check :
  forall P E,
    trusts_audited P E ->
    trusts_audited P (check E)
| audited_cast :
  forall P E T1 T2,
    trusts_audited P E ->
    trusts_audited P (cast E T1 T2).

```

This predicate recursively traverses an expression while remembering the total expression. In this definition, occurrences of `trust` impose extra requirements. In particular, every occurrence of `trust` must have a corresponding `audit` statement. The `audit` predicate is

defined to be satisfied externally, in our case by assertions in the policy rather than by proofs.

Parameter `audit` : `expr -> trustid -> Prop`.

An expression `E` has all its `trusts` audited when `trusts_audited E E` holds.

As a small example, consider the context `C` that has two elements, a distrusted function `f` of type $(\text{int}^{tr} \rightarrow \text{int}^{tr})^{dis}$ and a trusted integer `x`. The expression $(\text{trust}^0 f) x$ (which is actually `app (trust 0 (var 0)) (var 1)` in our syntax but which we write as $(\text{trust}^0 f) x$ for simplicity) will pass `trusts_audited` when there is an audit statement `audit ((trust0 f) x) 0`. Suppose that the code producer `bob` provides the code $(\text{trust}^0 f) x$ and a proof of:

```

audit ((trust0 f) x) 0
->
trusts_audited ((trust0 f) x) ((trust0 f) x)

```

A trusted authority `alice` signs the statement:

```

believe(<audit ((trust0 f) x) 0>)

```

The policy of a code consumer may be:

```

trusted(alice).

believe(F) :- sat(F).
believe(Q) :- believe(P), believe(<~P -> ~Q>).
believe(F) :- A says believe(F), trusted(A).

mayrun(C, P) :-
  believe(<has_type ~C ~P (annotate (usertype 0) tr)>),
  believe(<trusts_audited ~P ~P>).

```

After importing `alice`'s statement, the code consumer will have:

```

alice says believe(<audit ((trust0 f) x) 0>)

```

After checking bob's proof, the code consumer will have:

$$\text{sat} \left(\begin{array}{l} \langle \text{audit } ((\text{trust}^0 f) x) \ 0 \\ \rightarrow \\ \text{trusts_audited } ((\text{trust}^0 f) x) \ ((\text{trust}^0 f) x) \rangle \end{array} \right)$$

With a little reasoning, the code consumer obtains:

$$\text{mayrun}(\langle (\text{int}^{tr} \rightarrow \text{int}^{tr})^{dis} :: \text{int}^{tr} :: \text{nil} \rangle, \langle (\text{trust}^0 f) x \rangle)$$

As in Section 3.1, we have much flexibility in defining policies. In particular, each audit requirement need not be discharged with an explicit assertion specific to the requirement. We can allow broader assertions. In the extreme case, once a trusted authority vouches for a program, no auditing is required. We can express this policy with a BCIC rule:

```
mayrun(C, P) :- A says vouchfor(C, P), trusted(A).
```

We can also express policies that distinguish users. First, we modify `mayrun` to include an explicit user argument: `mayrun(U, C, P)` is satisfied when user `U` is allowed to run program `P` in context `C`. Since not all users might trust the same auditors and other authorities, we also introduce predicates `believes` and `trusts` as generalizations of `believe` and `trusted`, respectively, with user arguments. Because provability is absolute, not relative to particular users, no user annotation is needed for `sat`. With these variants, we may for example write the policy:

```
trusts(bob, alice).
trusts(charlie, bob).

believes(U, F) :- sat(F).
believes(U, Q) :- believes(U, P), believes(U, <~P -> ~Q>).
believes(U, F) :- A says believe(F), trusts(U, A).

mayrun(U, C, P) :-
  believes(U, <has_type ~C ~P (annotate (usertype 0) tr)>),
  believes(U, <trusts_audited ~P ~P nil>).
```

However, some classes of policies require more advanced techniques. For instance, we may want to focus the auditing on expressions of certain types, or to exclude expressions that satisfy a given static condition established by a particular static-analysis algorithm. While static conditions can certainly be programmed as Coq predicates, once those predicates are present, corresponding proofs need to always be present for the policy to function.

Fortunately we have a way of encoding decision procedures in Coq environments in order to allow BCIC to incorporate those decisions procedures. Our approach is based on proof by computation (also known as proof by reflection). Using this technique, BCIC can integrate various static analyses. Details of our use of proofs by computation appear in Chapter 5. This example is updated to include proof by computation in Section 5.7.1.

3.2.3 Correctness

Much as in Section 3.1, the proof of correctness relies on an instrumented operational semantics for the language. In this semantics, annotations record **trust** operations. We define a state as a pair that consists of a list of trust identifiers, l , and an expression, e . We define the single-step reduction $(l, e) \rightarrow (l', e')$ mostly as usual, ignoring types and type annotations, and eliminating casts, **trusts**, **distrusts**, and **checks**. The rule for **trust** records the identifier of the trust: $(l, \text{trust}^{\text{id}} e) \rightarrow (\text{id} :: l, e)$. We obtain a correctness result that says that, when programs execute, they perform **trust** operations only as authorized:

Theorem* 3.2 *For all programs e , if $\text{trusts_audited } e$ is true and $([], e) \rightarrow^* (l, e')$, then for all identifiers id in l , $\text{audit } e \text{ id}$ is true.*

While helpful, this theorem does not aim to address the proper criteria for declassification. The study of those criteria, and the guarantees that they may offer, is an active research area [SS05].

Summary of trust in the λ -calculus example

This example illustrates a method for combining trust with a formal type system, in this case audits of `trust` statements with a type system that tracks trust. For this example we have written a **policy**, defined an explicit CIC proof **environment** for proving `has_type` and `trusts_audited`, developed a **runtime** environment for executing programs written in the language, and proven **correctness** of the `trusts_audited` predicate with respect to an operational semantics of the language that keeps track of which `trust` statements have been executed. We do not prove any results about the type system directly; they have been proven by hand elsewhere [ØP97].

3.3 Instrumenting machine code

Static methods for certifying that untrusted code behaves correctly can be powerful, but sometimes dynamic methods are more practical. Instead of checking properties before the code executes, dynamic methods check that bad things don't happen as the code runs. Even if the code is badly behaved, the checks should stop the undesirable actions.

For the case of machine code, one way of adding dynamic checks is to rewrite the code itself to include the checks. The introduced checks can then verify that jump targets are correct, memory write locations are allowed, or other properties. Hopefully the rewriting system will be correct and guarantee that the resulting program always satisfies the right properties. There are other possibilities, too, of course. Hardware might be able to block or trap certain types of instructions, allowing dynamic checks to run.

A primary concern for code instrumentation is ensuring that the checks actually get executed correctly at the right time. A malicious program should not be able to jump past a check that a memory write is to a legal location. If the added code might be skipped or subverted, then the code rewriting did not provide any more assurance about the behavior of the program.

Control-flow integrity (CFI) [ABEL08] is a way to ensure that every jump in an

execution of the program follows the control-flow graph of the program produced by the compiler. This prevents a large class of malicious attacks on the code, but requires knowledge of the control-flow graph to insert the correct dynamic checks. Software-based fault isolation (SFI) [WLAG93, ES00] is a general technique for partitioning memory between untrusted processes by inserting dynamic checks before potentially unsafe machine instructions.

Pittsfield [MM06] is an extension of SFI for x86 code. In addition to the standard bounds checks of SFI, Pittsfield rewrites the code so that every jump target is at the start of a 16-byte chunk, then adds checks for every calculated jump to ensure that the jump will be to the start of a 16-byte chunk. This prevents instruction misinterpretation due to differing instruction lengths in the x86 architecture. In addition to instrumentation code that adds the checks, Pittsfield includes a proof of correctness for instrumented code in ACL2 [McC06].

We bring the ideas of Pittsfield to BCIC policies. In our case we translate the Pittsfield method into CIC, then prove that instrumented programs never execute a split instruction. This involves encoding the semantics of a subset of x86 instructions, writing an instrumentation verifier, and constructing a series of proofs about instrumented code. We also encode a notion of memory safety as a static check on programs which constrains memory writes to approved regions. A trusted authority declares which regions of memory are writable, then the static check of memory safety along with instrumentation to ensure instructions are never split will together guarantee that memory writes are always to legal locations.

By doing this we show that BCIC can deal with programs and proofs at the machine code level. In addition, this example demonstrates a combination of static and dynamic checks of the code. This example also illustrates how a theorem can be proven once and then used in a BCIC policy to reduce the proof burden on the code producer. In this case we prove that well-instrumented code obeys a dynamic safety policy, which allows code producers to merely prove that their code is well-instrumented.

3.3.1 How instrumentation works

Our safety property is that every jump is to a valid instruction, i.e. one that appeared in the original program. The types of properties we are ultimately interested in are things like memory safety and the maintenance of invariants that indicate the program is behaving correctly. In order to prove these desirable properties we need the even more basic property of control-flow integrity, which says every jump the program makes during runtime follows a preset plan. Our safety property, that every jump is to an instruction that originally appeared in the program, is an important step towards proving control-flow integrity.

Note that control-flow integrity does not mean that the instruction pointer merely falls in a certain inclusive range of values. Because x86 instructions can be different sizes, there are values that the instruction pointer can take that place it within the range of bytes used by the program but that do not correspond to any original instruction in the program. This is why we say the dynamic safety property means that the program will never split instructions. Almost anything can happen if jumps can split instructions; in a short fragment of our example program, we show that a split instruction results in an arbitrary memory write.

To verify the safety property it is not enough to statically check the target address of every jump instruction. Some jumps are to explicit jump destinations that can be checked statically, but others are jumps to an address held in a register which could have been loaded or computed from anywhere. These jump destinations are dynamically checked.

The idea of Pittsfield is to require that every jump be to the start of a 16 byte chunk, i.e. have the lowest four bits of the address be zero. We then arrange the source program to have legitimate instructions starting at the beginning of every 16 byte chunk by inserting NOP instructions to pad the end of chunks that don't have enough space to entirely contain the next instruction. To zero out the low four bits of destination addresses, AND instructions are inserted before every calculated jump. These additional instructions are required to be in the same 16 byte chunk as the calculated jump instruction.

```
nop
inc eax
mov [addr], eax
mov eax, [addr]
mov eax, (ebx)
mov eax, (ebp)
xchg eax, ebx
xchg eax, ebp
and ebx, imm
and ebp, imm
jmp [addr]
jmp [ebx]
int imm
```

Figure 3.3.1: Subset of allowed machine instructions

With these restrictions we prove that when the program is executed, the instruction pointer only takes on values that correspond to instructions present in the original program.

3.3.2 Example program

To help see why the Pittsfield dynamic instrumentation method is necessary we present a short x86 assembly program and instrument it. In our implementation we set up operating system interrupt 10 to output a character stored in the EAX register, and interrupt 11 to exit the program and returns control to the operating system. The following program defines a subroutine that outputs “hello”, then calls it twice to output “hellohello” and then quit. The subroutine is called by storing the return instruction pointer in memory location 0x0389, then jumping directly to the start of the subroutine. The subroutine returns by loading the value at 0x0389 and jumping to it. The number before each instruction indicates the byte offset within the program of that instruction.

```
0  mov eax, 15
5  mov [0x0389], eax
10 jmp 32
15 mov eax, 30
20 jmp 32
```

```
25  intr 11
27  mov eax, 104
32  intr 10
34  mov eax, 101
39  intr 10
41  mov eax, 108
46  intr 10
48  mov eax, 108
53  intr 10
55  mov eax, 111
60  intr 10
62  mov eax, [0x0389]
67  xchng eax, ebx
68  jmp [ebx]
```

The above program is correct but somewhat dangerous. When the subroutine returns, the program jumps to a location stored in memory. A bug anywhere in the program such as a buffer overrun, an off-by-one error, or double allocating the same memory region could overwrite this memory location and cause the program to start executing arbitrary code.

As an example, suppose a bug causes memory location 0x0389 to contain 6 instead of 15 or 30 as planned. The bytes of the program itself are:

```
b8 0f 00 00 00 a3 89 03 00 00 e9 20 00 00 00 ...
```

Interpreting these bytes from the start gives the original program, but interpreting them from offset 6 gives the sequence:

```
mov [ebx], eax
add eax, eax
jmp 32
```

In other words, the program is now writing garbage to the memory location stored in register EBX, which is unused in the program and therefore could contain any value.

The rewritten program includes AND instructions and NOP padding to align jump targets on 16 byte boundaries and avoid splitting instructions.

```
0  mov eax, 16
5  mov [0x0389], eax
10 jmp 48
15 nop
16 mov eax, 32
21 mov [0x0389], eax
26 jmp 48
31 nop
32 intr 11
34 nop
35 nop
36 nop
37 nop
38 nop
39 nop
40 nop
41 nop
42 nop
43 nop
44 nop
45 nop
46 nop
47 nop
48 mov eax, 104
53 intr 10
55 mov eax, 101
60 intr 10
62 nop
63 nop
64 mov eax, 108
69 intr 10
71 mov eax, 108
76 intr 10
78 nop
79 nop
80 mov eax, 111
85 intr 10
87 mov eax, [0x0389]
92 xchng eax, ebx
93 nop
```

```
94  nop
95  nop
96  and ebx, 0xffffffff0
101 jmp [ebx]
```

Now if location 0x0389 contained an invalid return address, the AND instruction would zero out the lowest four bits. The structure of the program is such that every target at the start of a 16 byte boundary will be valid assembly instructions from the original program. The program will still misbehave because of the invalid return value, but it will not start executing misaligned instructions as before.

3.3.3 Static check

The static check verifies the following properties:

- The program only contains opcodes from a small subset of legal opcodes
- Every fixed jump is to the start of a chunk
- Before every calculated jump is an `and ebx, 0xffffffff0` instruction to zero out the low four bits of `ebx`
- No instruction straddles a chunk boundary
- Every inserted AND is in the same chunk as the jump

In our implementation we have an explicit encode function that takes a list of instructions and returns a list of machine code bytes. We define well-instrumented machine code programs as the output of encoding well-instrumented instruction lists. This allows us to define the static checks over lists of instructions rather than lists of bytes.

Note that in CIC we only define what it means to be a well-instrumented program, not how to generate well-instrumented programs from arbitrary programs. The functions for rewriting programs to include instrumentation and padding do not need to be trusted, so can be written in any convenient programming language. The rewriting functions could

be written in Coq, but Coq does not have all the datatypes and library functions of other languages. In addition, every function one writes in Coq must be proved to be terminating, which often involves a significant amount of extra work.

Verifying that the program only contains opcodes from the set of allowed opcodes comes from the way we are verifying programs. Our encoding function only knows how to handle the set of opcodes, so of course only those opcodes will appear in the program. Checking fixed jump values and ensuring that the AND instructions are in place is a simple scan through the instruction list.

To verify that no instruction straddles a chunk boundary, we use the decode function on the list of bytes representing the program. This returns a list of instruction-length pairs. We discard the instructions themselves to get a list of instruction lengths. From the list $l_0, l_1, \dots, l_n$ of instruction lengths we calculate the list $(a_0, l_0), (a_1, l_1), \dots, (a_n, l_n)$ where $a_m = \sum_{i=0}^{m-1} l_i$. Each pair represents the starting location of the instruction paired with the instructions length.

What does it mean for an instruction to straddle a chunk boundary? It means that the bytes making up an instruction appear in different chunks. Let the instruction start at position a and have length n . The first byte of the encoded instruction is at location a , the last byte is at $a + n - 1$. The instruction does not straddles a chunk boundary if $\lfloor a/16 \rfloor = \lfloor (a + n - 1)/16 \rfloor$. We encode this test in Coq and verify that every pair in the list satisfies the condition.

For the final condition that every AND instruction before a calculated jump appear in the same chunk as the jump, we create a modified version of the decode function that recognizes the AND/JMP combination as a single long instruction. Then the check that no instruction straddles a chunk boundary will also verify that no AND is separated from its JMP.

All of these checks come together in a definition for when a program is well-instrumented.

```

Definition well_instrumented (P : list instr)(L : list byte) : Prop :=
  L = encode P /\
  verify_fixed_jumps P /\
  verify_and P /\
  no_straddlers (decode_modified L).

```

3.3.4 Dynamic safety property

The dynamic safety property is that at every step of the processor during execution of the machine code that makes up the program, the instruction pointer of the processor holds an address corresponding to an instruction from the original program.

The goal of the code instrumentation is to produce programs that satisfy this dynamic safety property. We prove in CIC, following the strategies of Pittsfield, that a program that is well-instrumented must indeed satisfy the dynamic safety property. The existence of this proof allows the code producer to merely prove that their program is well-instrumented, then the code consumer will know that it will not jump to invalid instructions during execution.

3.3.5 Semantics of x86 instructions

Because the dynamic safety property refers to the execution of the hardware, we must formally model a subset of the semantics of x86 machine code. We encode the semantics of a subset of x86 opcodes as a deterministic function between states of the processor. This involves modeling memory reads and writes, register contents, and various bitwise operations. Because we need to reason about things like register overflow and the contents of particular bits in a register, we model the memory and registers using fixed length bit vectors. This in turn requires defining the semantics of operations such as AND and INC at the bit level.

Memory is represented as a data structure that keeps track of all memory writes in a list. The most recent write is placed on the head of the list.

```

Inductive memory : Set :=
| memory_empty : memory
| memory_write : word -> word -> memory -> memory.

```

Reads to memory locations are done using a linear lookup along the list, comparing addresses to find the most recent write to the address.

```

Fixpoint memory_read (M : memory)(addr : word) {struct M} : word :=
  match M with
  | memory_empty => 0
  | memory_write a v M2 =>
    if eq_word_dec addr a then v else memory_read M2 addr
  end.

```

Operations such as AND are first defined at the bit level, then generalized to lists of bits (such as words).

```

Definition bit_and (x y : bit) : bit :=
  match x with
  | b0 => match y with
    | b0 => b0
    | b1 => b0
  end
  | b1 => match y with
    | b0 => b0
    | b1 => b1
  end
end.

```

```

Fixpoint bitlist_and (a b : list bit) {struct a} : list bit :=
  match a with
  | nil => nil
  | ahd :: atl =>
    match b with
    | nil => nil
    | bhd :: btl => (bit_and ahd bhd) :: (bitlist_and atl btl)
    end
  end
end.

```

The step function from one state to the next state of the machine must also fetch and decode the current instruction. We represent instruction memory as a list of bytes separate from main memory to simplify the reasoning. In general we focus on the semantics of user-level programs rather than kernel code. Under most modern operating systems code and data are mapped into logically disjoint segments using the hardware paging mechanisms of the x86 processor, so this is a reasonable assumption. Reasoning about self-modifying code is beyond the scope of our example.

Because instructions can be different lengths, the decode function requires multiple byte comparisons to determine which instruction is present in a list of bytes. Here is one part of the decode function that checks for single byte instructions. If it cannot find a match for a single byte instruction it drops through to the next decode function that checks for instructions of length two. The decode instruction function takes a list of bytes representing machine code and returns a list of instruction-length pairs.

```

Definition decode_instr (P : list byte) : (instr * nat) :=
  match P with
  | nil => (err, 1)
  | P1 :: Pt1 => decode_level1 P1 Pt1
  end.

Definition decode_level1 (P1 : byte)(Pt1 : list byte) : (instr * nat) :=
  if eq_list_bit_dec P1 x90 then (nop, 1) else
  if eq_list_bit_dec P1 x40 then (inc_eax, 1) else
  if eq_list_bit_dec P1 x93 then (xchng_eax_ebx, 1) else
  if eq_list_bit_dec P1 x95 then (xchng_eax_ebp, 1) else
  decode_level2 P1 Pt1.

```

The step function is defined:

```

Function step (s : state) : state :=
  match s with
  | error => error
  | halted => halted
  | st L IP AX BX BP F MEM =>
    match fetch L IP with

```

```

| (ins, len) =>
  match ins with
  | err => halted
  | nop => st L (IP + len) AX BX BP F MEM
  | inc_eax => st L (IP + len) (S AX) BX BP F MEM
  | jmp J => st L (bitlist_to_nat J) AX BX BP F MEM
  | jmp_ebx => st L BX AX BX BP F MEM
  | xchng_eax_ebx => st L (IP + len) BX AX BP F MEM
  ...
  end
end
end.

```

3.3.6 Proofs

We prove, in Coq, that every well-instrumented program satisfies the dynamic safety property. To prove this, we actually prove a stronger property, namely that for every well-instrumented program, at every step of execution, the instruction pointer points to an instruction that appeared in the original program and if that instruction is a JMP, then the value stored in EBX has its low four bits set to zero.

We consider three different sets of instructions. Let S_0 be the set of offsets to all bytes in the original program, regardless of instruction. Let S_1 be the set of byte offsets in the original list of bytes making up the machine code that correspond to the starts of all the original instructions in the program. Let S_2 be the same set but without the byte offsets for calculated JMP instructions, so $S_2 \subseteq S_1 \subseteq S_0$.

First we show that the AND instruction added during instrumentation behaves correctly on the EBX register.

Lemma* 3.3 *For every state of the processor where EIP points to the instruction **and ebx**, $0xffffffff0$, after one step the low four bits of register **ebx** will all be zero.*

The next lemma says the start of every 16 byte block is a valid instruction in the original assembly program.

Lemma* 3.4 *For all $x \in S_0$, if $x \bmod 16 = 0$ then $x \in S_1$.*

We can now start reasoning about how the processor behaves after executing instructions.

Lemma* 3.5 *For every state of the processor with EIP in S_2 , after one step of the processor either EIP will be in S_2 , or EIP will be in S_1 and the instruction at the new location will be a `jmp [ebx]` instruction and the instruction at the previous EIP location was `and ebx, 0xffffffff0`.*

Now we can show that the instruction pointer of any state reachable from a well-instrumented program aligns with an instruction in the original program.

Theorem* 3.6 *Starting from any processor state with EIP in S_2 , after any number of steps the EIP in the processor state will be in S_1 .*

We name this property `no_split_instructions`.

3.3.7 Memory safety and authorities

Once we have an assurance that instructions will not be split, we can begin to examine other properties. Code consumers are interested in enforcing memory constraints on programs they run, especially if for efficiency or hardware reasons there is the possibility that a program can overwrite memory it should not.

The memory safety property we consider is allowed memory writes to contiguous regions of memory, as declared by a trusted authority. The code consumer will trust an authority to declare a list of memory regions to which the program is allowed to write. This time there will be a single static check `memory_safe` over programs that verifies that all memory locations referenced in the program are within a legal memory range. Once this static check has been passed, the code consumer will know that if instructions are never split, then the program will only write to legal memory regions.

The definition of `memory_safe` is simplified by the fact that there is only one instruction in our set that writes to memory, `mov [addr], eax`, and it writes to a fixed

location in memory. This means the static analysis only needs to check that the address constants for this instruction are within a legal memory range.

```

Definition write_addr_in_range (P0 : instr)(R0 : nat * nat) : Prop :=
  match P0 with
  | mov_addr_eax ADDR =>
    match R0 with
    | (R0lo, R0hi) =>
      (bitlist_to_nat ADDR) >= R0lo /\
      (bitlist_to_nat ADDR) < R0hi
    end
  | _ => True
  end.

```

```

Definition memory_safe1 (P : list instr)(R0 : nat * nat) : Prop :=
  forall P0, In P0 P -> write_addr_in_range P0 R0.

```

```

Definition memory_safe (P : list instr)(R : list (nat * nat)) : Prop :=
  forall R0, In R0 R -> memory_safe1 P R0.

```

It is not hard to see why `memory_safe` implies that when the program is executed according to the x86 semantics, every write will be within a legal range. The interesting fact is that proving this requires that instructions cannot be split. Recall our example of split instructions; the new spurious instruction is not guaranteed to be in the small set of legal machine instructions in the original program, and so can be a write to any memory location.

3.3.8 Policy

We can now put our definitions from the proof into an actual policy on programs. For this example the proof rules and static checks are somewhat complicated, but the policy is simple. There is a principal, Alice, who is trusted to say for every program which regions of memory can be written by the program. The reference monitor policy is to run machine language programs that have passed the memory safety check and that do not have any split instructions.

```
regions(P, R) :- alice_says_regions(P, R).
mayrun(L) :-
  regions(P, R),
  sat(<memory_safe ~P ~R>),
  sat(<no_split_instructions ~P ~L>).
```

In addition, we have a proof that well-instrumented code implies no split instructions, and well-instrumented is a decidable property. This proof is imported into the local policy as the following:

```
sat(<forall (P : list instr)(L : list byte),
  well_instrumented P L -> no_split_instructions P L>).
```

A code producer, Bob, checks that their code is indeed well-instrumented then transmits their code to the reference monitor with the proof that their code is well-instrumented. Alice signs Bob's code to certify which regions R_0 are legal for memory writes. The reference monitor will check the proofs, use the proof to deduce that the program has no split instructions, and then execute it.

3.3.9 Comparison to Pittsfield

The formalization of Pittsfield uses ACL2, based on Common Lisp. ACL2 does not have higher-order functions, and all recursion must be provably terminating. It provides a limited set of datatypes; lists and integers are used in their proof. Pittsfield has a special ERROR state that indicates an illegal operation that was not planned happened, and use non-termination for hardware trapped exceptions (guard regions of memory). We define explicit ERROR and HALTED states.

We use the same model of memory, i.e. sparsely stored with an association list of locations to values. Pittsfield proves the safety property using a static invariant computed from the program. We express the property directly using a predicate. Pittsfield does not deal with assertions or security policies. The memory safety property that Pittsfield uses must be dynamically checked because they allow indirect writes; the instrumentation

for memory also uses inserted AND instructions in the same way that computed jumps were instrumented. Also, the authors of Pittsfield added instructions incrementally to their proof development, and proof time increased dramatically quickly up to 771s for the current subset. Coq is perhaps less automatic than ACL2, but is good at explosions of cases with simple resolutions. Our Coq script takes less than 60s.

Summary of code instrumentation example

This example illustrates a combination of static and dynamic checks for machine code. In this case a static check of `well_instrumented` determines when the correct dynamic checks have been added to the code to prevent split instructions, and a static check of `memory_safe` that relies on asserted facts about which regions are legal for memory writes determines that the code does not write to illegal locations. This example shows that BCIC is capable of handling machine code as well as higher-level languages and is not limited to purely static code checks. For this example we have written a **policy**, defined an explicit CIC proof **environment** for proving `well_instrumented` and `memory_safe`, and proven **correctness** of the `well_instrumented` predicate by showing that well-instrumented programs do not split machine instructions during execution. We have not yet implemented a runtime environment for loading and executing machine code from within BCIC. We have also not proved memory safety for a set of opcodes that includes memory writes to computed locations.

Chapter 4

Analysis

In this chapter we investigate some of the theoretical properties of BCIC. We start by reviewing some results about Datalog, then see which results carry over into Binder and BCIC. We review some results about Binder and CIC individually, then examine the connection between Binder and CIC. Most of the results in this chapter have been translated and checked in Coq; see Section 6.4.

4.1 Results on Datalog and Binder

In this section we examine the expressiveness of Datalog and Binder. Although the following results are well-known, we present them here for several reasons. First, they help illustrate the behavior of Datalog and Binder and give intuition about how BCIC will behave. In addition, we formalized the following results in Coq during our formalization of BCIC in Coq. The following description thus serves as a guide to our formalization in Coq.

4.1.1 Datalog expressiveness

Datalog is a simple expressive language that is well suited as the base for writing security policies as we have seen through the examples in this dissertation. But exactly how expressive is Datalog?

Since Datalog is so similar in structure to Prolog, we can work through the standard

Prolog examples and see where Datalog works or doesn't work. A first Prolog exercise is often the ancestor relation.

```
ancestor(X, Y) :- parent(X, Y).  
ancestor(X, Z) :- parent(X, Y), ancestor(Y, Z).
```

If we know:

```
parent(myron, nathan).  
parent(myron, matthew).  
parent(norman, myron).
```

then the Prolog query `ancestor(X, nathan)` will succeed with `X=myron` and `X=norman`. This example works fine in Datalog. Datalog can represent relations between principals and programs as well as family relations.

The next simple Prolog example is list membership.

```
member(H, [H | T]).  
member(X, [H | T]) :- member(X, T).
```

The notation `[H | T]` represents a list with head *H* and tail *T*. With this definition of membership, the query `member(b, [a, b, c])` succeeds while the query `member(d, [a, b, c])` fails.

Clearly since Datalog does not have function symbols we cannot represent lists in the same way as Prolog. Is there any way to represent lists in Datalog? One method is to replace the list function symbol with predefined predicates. List membership becomes:

```
member(H, L) :- match(L, H, T).  
member(X, L) :- match(L, H, T), member(X, T).
```

In order for this to work, every possible match of every possible list that may affect a query must be precomputed and placed as a fact. Before we know what lists will be present in the query, with this encoding it is impossible to know which `match` facts to add.

In some sense the previous strategy can be used to encode any function symbols of fixed arity. Simply replace function symbols with predicates, then precompute every possible

combination. As a further example, consider addition of unary numbers. In standard Prolog:

```
add(X, 0, X).
add(X, s(Y), s(Z)) :- add(X, Y, Z).
```

Rewritten with function symbols as predicates:

```
add(X, 0, X).
add(X, SY, SZ) :- s(Y, SY), s(Z, SZ), add(X, Y, Z).
```

In order for this scheme to work for addition of two numbers x and y , there must be facts of the form $s(n, n + 1)$ for n from 0 to $x + y - 1$.

Obviously this scheme is undesirable. Is there a better way to represent lists or natural numbers that doesn't depend on adding a large number of facts that depend on the query? The answer is no. The reason is that Datalog is fundamentally finite. Datalog is fundamentally finite because for each program there exists a finite set of deductions that allows a decision procedure to determine if any query will succeed or fail, as we will see in the following theorems. Of course Datalog is not actually finite, since you can write programs like: $p(X)$, which means every atom satisfies predicate p . Variables that appear in the head of a clause and not the body are *free variables*, and are traditionally written with an underscore in Prolog.

4.1.2 Datalog decidability

It is well-known that Datalog is decidable [Ull88]. Because our formulation of BCIC depends so heavily on Datalog itself, we prove the decidability of Datalog ourselves to build up to a proof of decidability of BCIC.

Lemma* 4.1 (Finiteness of Datalog without Free Variables) *For every Datalog program with no free variables there exists a finite set of atomic formulas S such that all ground queries that succeed are in S .*

A finite number of atoms, predicate names, and predicate arities can appear in the program. Because there are no function symbols to construct new atoms, we can construct a set S of all ground atomic formulas using every possible combination of predicate name, atoms, and arities.

This finite set S does not depend on the query, but any query that succeeds must be in S . To see this, we proceed by induction on the derivation of the query that shows it succeeds. Let Q be a query that succeeds. For the base case, Q itself appears in the program and so will of course be in S . For the induction step, the query matches a head of some clause in the program under a variable assignment. We may assume by the induction hypothesis that under the variable assignment every atomic formula in the body of the clause appears in S . No free variables appear in the clause, so every variable in the head of the clause will be assigned to an atom that appeared in an atomic formula that is in S . This means the atom itself appeared in the original program by our construction of S . The head of the clause under the variable assignment thus contains only predicate names, arities and atoms that appeared in the program, so it is in S . $\square$

Theorem* 4.2 (Decidability of Datalog without Free Variables) *Queries to Datalog programs with no free variables are decidable.*

We show this by constructing a set Ω that is a subset of S containing all derivable ground atomic formulas. Deciding when a ground query is derivable then just involves checking if it appears in Ω .

To construct Ω we proceed in stages. First we let Ω_0 be the empty set. Then we define Ω_{i+1} to be the union of Ω_i and the ground atomic formulas derivable from each clause in the program assuming the facts in Ω_i . In other words, Ω_i will contain all ground atomic formulas derivable in i or less derivation steps. By the definition of Ω_i , $\Omega_i \subseteq \Omega_{i+1}$. From the previous theorem, the sequence of Ω_i is bounded by S . The sequence is bounded and non-decreasing, so it must have a fixed point Ω .

To decide if a ground query is derivable we check to see if it appears in Ω . If so, it also appears in Ω_i for some i and so is derivable in i or fewer steps. If not then the query

does not appear in any Ω_i . Every derivable query is derivable in some number of steps j , and so will appear in Ω_j . Therefore Ω consists exactly of derivable ground queries, and so our decision algorithm is sound and complete. $\square$

Programs that have free variables are still decidable but require a bit more care with the definitions of the universe of ground atomic formulas and queries. With free variables it is no longer possible to precompute a finite set of derivable queries from a Datalog program to answer queries.

Lemma 4.3 *Let D be a derivation of Q from program P . If D contains an atom α that does not appear in P , then replacing α with any other atom β yields a valid derivation.*

The proof is by induction on the depths of derivations. Suppose the lemma is true for derivations up to depth n . Let D be a derivation of size $n + 1$. The top of derivation D is a clause from P , and the head of the clause must be Q under a variable assignment s . Since α cannot appear in P , it can only appear in the head of the clause by a variable substitution. Call this variable in the substitution x . Create a new variable assignment s' that is the same as s but sends x to β instead of α . By the inductive hypothesis, replacing α with β in every sub-derivation yields valid derivations of all the atomic formulas in the body of the clause under the variable assignment. These new sub-derivations together with s' are a new valid derivation that replaces α with β . $\square$

Theorem 4.4 *Queries to Datalog programs are decidable.*

We follow the same method as in Theorem 4.2. Construct a set S that contains all possible ground atomic formulas made up of predicate names, arities, and atoms that appear in P . This time include special atoms $\alpha_1, \dots, \alpha_m$ in the set of atoms used to construct S , where the number of extra atoms m is determined by the maximum arity of any predicate that appears in the program. For simplicity, without loss of generality we can assume that the arity of the query is less than or equal to the maximum arity of predicates in the program. Only a finite number of predicate names, arities, and atoms can occur, so the set S will be finite.

Let Ω_0 be the empty set. Let Ω_{i+1} be the union of Ω_i and the ground atomic formulas derivable from each clause in the program assuming the facts in Ω_i that are made up of atoms that appear in S . Because of this restriction that the atoms appear in S , every element of Ω_i will also be in S . By the definition, $\Omega_i \subseteq \Omega_{i+1}$. Since the sequence is nondecreasing and bounded by a finite S , it must have a limit Ω .

As in Theorem 4.2, the decision algorithm will look to see if the query appears in Ω . If it does, the query is clearly derivable by the way we constructed Ω and so the decision algorithm says “yes”. If the query does not appear in Ω , check to see if any atoms of the query do not appear in P . For each atom in Q that does not appear in P , replace all copies of that atom in Q with a distinct α_i . (This step may replace nothing if there were no extra atoms.) Check to see if the query with the extra atoms replaced appears in Ω . If so, the decision algorithm answers “yes”, otherwise “no”.

First we show that this algorithm is correct when it says “yes” and there are extra atoms. Let $\beta_1, \dots, \beta_n$ be the extra atoms. Let the notation $Q[\beta_1 \rightarrow \alpha_1]$ indicate substitution. Since $Q[\beta_1 \rightarrow \alpha_1, \dots, \beta_n \rightarrow \alpha_n] \in \Omega$, there is a derivation D of $Q[\beta_1 \rightarrow \alpha_1, \dots, \beta_n \rightarrow \alpha_n]$. By applying Lemma 4.3 we see that $D[\alpha_1 \rightarrow \beta_1]$ is a valid derivation of $Q[\beta_1 \rightarrow \alpha_1, \dots, \beta_n \rightarrow \alpha_n][\alpha_1 \rightarrow \beta_1] = Q[\beta_2 \rightarrow \alpha_2, \dots, \beta_n \rightarrow \alpha_n]$. By repeated applications of Lemma 4.3, we see that $D[\alpha_1 \rightarrow \beta_1, \dots, \alpha_n \rightarrow \beta_n]$ is a valid derivation of Q . Thus the algorithm is correct when it says “yes” and there are extra atoms in the query.

Now that we know the algorithm is always correct when it says “yes” we turn to the “no” answers. Because the decision algorithm terminates, to show correctness here it suffices to show that if Q is derivable then $Q[\beta_1 \rightarrow \alpha_1, \dots, \beta_n \rightarrow \alpha_n]$ is derivable. Suppose Q is derivable by derivation D . By application of Lemma 4.3 we see that $D[\beta_1 \rightarrow \alpha_1]$ is a valid derivation of $Q[\beta_1 \rightarrow \alpha_1]$. By repeated applications of Lemma 4.3 we see that $D[\beta_1 \rightarrow \alpha_1, \dots, \beta_n \rightarrow \alpha_n]$ is a valid derivation of $Q[\beta_1 \rightarrow \alpha_1, \dots, \beta_n \rightarrow \alpha_n]$. $\square$

Datalog cannot represent arbitrary lists without adding facts into the program depending on the query. To see this, suppose there were such a program P in Datalog. Calculate S for this program. Since S is finite, the program P can only correctly answer

a finite number of facts about lists. But of course there are an infinite number of facts about lists. Pick elements x and y and a list L such that x , y , $[x \mid L]$, and $[y \mid L]$ do not appear anywhere in P . By the decision algorithm from Theorem 4.4, the queries `member(x, [x | L])` and `member(y, [x | L])` are both equivalent to `member( $\alpha_1$ ,  $\alpha_2$ )`. They must either both succeed or both fail, based on the definition of S . But this means the program doesn't calculate list membership correctly.

A similar argument shows that unary addition is also not representable directly in Datalog.

4.1.3 Binder decidability

One of the great advantages of Datalog is its decidability. Given a Datalog program and a query, there is an algorithm that decides whether the query succeeds or fails (Theorem 4.4).

Binder adds new syntax to atomic formulas for the **says** operator. Because there is no nesting of **says** and there are still no function symbols, showing the decidability of Binder is not too difficult. Note that queries occur after signed statements from outside contexts have been checked and imported; the following results do not deal with importing or exporting facts between contexts. Because allowing free variables complicates the reasoning, we now restrict policies to not have any free variables as we consider the extension from Datalog to Binder and beyond.

The most intuitive way of seeing that Binder is decidable is to translate Binder policies into pure Datalog according to the following rule:

$$\begin{aligned} t \text{ says } p(t_1, t_2, \dots, t_n) &\mapsto p(t, t_1, t_2, \dots, t_n) \\ p(t_1, t_2, \dots, t_n) &\mapsto p(\text{me}, t_1, t_2, \dots, t_n) \end{aligned}$$

The special new atom **me** is used to indicate the local context. It is not hard to see that this new Datalog policy will behave exactly like the original Binder policy. Because Datalog is decidable, Binder is also decidable.

Formalizing this translation turns out to be more work than reproving decidability

of Binder directly. The following proofs of the decidability of Binder are almost the same as the proofs for Datalog but must consider one extra case in some places.

Lemma* 4.5 (Finiteness of Binder) *For every Binder policy there exists a finite set of atomic formulas S such that all ground queries that succeed are in S .*

A finite number of atoms, predicate names, and predicate arities can appear in the program. We can construct a set S of all ground atomic formulas, both predicates and quoted predicates, using every possible combination of predicate name, atoms, and arities. The syntax of BCIC atomic formulas in Section 2.2.3 defines predicates and quoted predicates.

This finite set S does not depend on the query, but any ground query that succeeds must be in S . To see this, we proceed by induction on the derivation of the query that shows it succeeds. Let Q be a ground query that succeeds. For the base case, Q itself appears in the program and so will of course be in S . For the induction step, the query matches a head of some clause in the program under a variable assignment. We may assume by the induction hypothesis that under the variable assignment every atomic formula in the body of the clause appears in S . No free variables appear in the clause, so every variable in the head of the clause will be assigned to an atom that appeared in an atomic formula that is in S . This means the atom itself appeared in the original program by our construction of S . The head of the clause under the variable assignment thus contains only predicate names, arities, and atoms that appeared in the program, so it is in S . $\square$

Theorem* 4.6 (Decidability of Binder) *Queries to Binder policies are decidable.*

The proof is almost the same as for Theorem 4.2.

We prove the theorem by constructing a set Ω that is a subset of S containing all derivable ground atomic formulas. Deciding when a ground query is derivable then just involves checking if it appears in Ω .

To construct Ω we proceed in stages. First we let Ω_0 be the empty set. Then we define Ω_{i+1} to be the union of Ω_i and the ground atomic formulas derivable from each clause in the program assuming the facts in Ω_i . In other words, Ω_i will contain all ground

atomic formulas derivable in i or less derivation steps. By the definition of Ω_i , $\Omega_i \subseteq \Omega_{i+1}$. From Lemma 4.5, the sequence of Ω_i is bounded by S and so must have a limit Ω .

To decide if a ground query is derivable we check to see if it appears in Ω . If so, it also appears in Ω_i for some i and so is derivable in i or fewer steps. If not then the query does not appear in any Ω_i . Every derivable query is derivable in some number of steps j , and so will appear in Ω_j . Therefore Ω consists exactly of derivable ground queries, and so our decision algorithm is sound and complete. $\square$

4.2 Properties of BCIC

4.2.1 Conservativity

We now formalize the relationship between BCIC and CIC to show that BCIC is a reasonable logic. The behavior of the **sat** predicate is entirely determined by CIC; we say that BCIC is a conservative extension of CIC on the **sat** predicate. A policy writer needs assurance that **sat**(T) is derivable if and only if there actually is a CIC object of type T . If there were a subtle flaw in the derivation rules of BCIC this might not be true, and the policy writer would have no way of knowing what their policy containing **sat** meant and whether it was correct.

A policy is *conservative* if no clause has a head that is of the form **sat**(T).

The following theorem says that BCIC is conservative over CIC for conservative policies.

Theorem* 4.7 *Let r be a conservative policy with CIC environment e . For all CIC terms T , if $(r, e) \vdash \mathbf{sat}(T)$ then there exists a CIC term o such that $e \vdash_{CIC} o : T$.*

The restriction that the policy must be conservative is necessary because otherwise the policy might contain a “fact” of the form **sat**(**false**) :- . which would obviously violate conservativity of the system.

Let r be a conservative policy, e be a CIC environment, and T be a CIC term such that $(r, e) \vdash \mathbf{sat}(T)$. We prove the theorem by induction on the size of the derivation of

$\mathbf{sat}(T)$. There are three possibilities for the top of the derivation of $\mathbf{sat}(T)$: $(r, e) \vdash \mathbf{sat}(T)$ is the consequence of Formula 2.2.1, Formula 2.2.2, Formula 2.2.3, or Formula 2.2.4.

In the case of Formula 2.2.1, the policy r must have contained a clause with $\mathbf{sat}$ in the head. But the policy is conservative, so this is a contradiction. In the case of Formula 2.2.2, the derivation rule itself provides a CIC term o such that $e \vdash_{CIC} o : T$ satisfying the theorem. In the case of Formula 2.2.4, the derivation rule assures that $e \vdash_{CIC} \mathbf{refl_equal\ true} : T$, so let $o = \mathbf{refl_equal\ true}$ and the theorem is satisfied.

In the case of Formula 2.2.3 we are deriving $(r, e) \vdash \mathbf{sat}(T')$. There must be a derivation of $(e, r) \vdash \mathbf{sat}(T)$ with a smaller size than the derivation of $(e, r) \vdash \mathbf{sat}(T')$, so by the inductive hypothesis there exists a CIC term o' such that $e \vdash_{CIC} o' : T$. We also know $e, \alpha : T \vdash_{CIC} (\alpha\ o) : T'$. From these facts we can conclude the term $(o'\ o)$ has type T' in the CIC environment e , satisfying the theorem.

In every case the theorem is satisfied. $\square$

The mechanized version of this proof does not encode CIC within CIC. Instead it uses axioms about CIC that are asserted without proof. For example, the fact that typechecking in CIC is decidable is asserted as an axiom. Our formalization of BCIC in CIC would be more complete if all of CIC were explicitly formalized in CIC, but this is a large-scale effort beyond the scope of this dissertation. Bruno Barras has encoded much of CIC within CIC [BW97].

4.2.2 Decidability

An effective reasoning strategy is necessary if a system such as BCIC is to be at all practical. Since BCIC contains CIC, a system capable of representing undecidable and arbitrarily complicated logics, we must limit in some way the kinds of reasoning our system performs. After considering examples such as those in Chapter 2 and Chapter 3 we identified the requirements for a reasoning strategy. In particular, the reasoning strategy need not search for proof objects of CIC types. In the PCC framework, proof objects will be provided by code producers or other principals, and need only to be checked, not reproduced,

by the code consumer. The reasoning system should, however, be able to combine true propositions using modus ponens and instantiate universally quantified propositions with the actual objects under consideration.

To this end we develop a decision algorithm for *well-behaved* policies. Intuitively, a policy is well-behaved if repeatedly combining CIC terms that appear in a policy together results in a finite set of CIC terms. This allows us to develop a decision algorithm based on Datalog-style bottom-up evaluation that is both simple and terminating. Note that we place no restrictions on CIC environments; any reasoning system with a representation in CIC may be used in BCIC. The restriction is only on the types of CIC terms that may be mentioned explicitly in the BCIC policy.

In Chapter 2 we defined a function U that takes policies and CIC environments and extracts a set of CIC terms from the policy that is closed under various rules. We say that a policy r and CIC environment e is *well-behaved* if the set $U(e, r)$ is finite.

To better understand this restriction, consider the following simple CIC environment that defines a predicate for determining evenness of natural numbers.

```
Inductive even : nat -> Prop :=
| even0 : even 0
| evenSS : forall n, even n -> even (S (S n)).
```

The analogy is that “programs” are natural numbers, and even natural numbers are “safe” programs. A policy might refer to the following CIC terms with given types:

```
(S (S 0)) : nat
(S (S (S 0))) : nat
even (S (S 0)) : Prop
evenSS 0 even0 : even (S (S 0))
```

In this case the extracted set $U(e, r)$ is the same set of terms with the addition of **nat**, **Prop**, and **Set**, since no two terms can be applied together in a well-typed way. Suppose we mentioned the following CIC term in the policy:

```

fun (n : nat) (H : even n) => evenSS n H
  : forall n, even n -> even (S (S n))

```

The extracted set $U(e, r)$ is still finite, but it will now contain the additional following terms:

```

(fun (n : nat) (H : even n) => evenSS n H) (S (S 0))
  : even (S (S 0)) -> even (S (S (S (S 0))))
(fun (n : nat) (H : even n) => evenSS n H) (S (S (S 0)))
  : even (S (S (S 0))) -> even (S (S (S (S (S 0)))))
(fun (n : nat) (H : even n) => evenSS n H) (S (S 0)) (evenSS 0 even0)
  : even (S (S (S (S 0))))

```

Now consider adding the term:

```

fun n => S (S n) : nat -> nat

```

Now the extracted set $U(e, r)$ becomes infinite, because this function can be applied to $S (S 0)$ and then repeatedly to its own output to generate larger and larger even natural numbers.

Escaped variables in CIC terms within the policy can also be problematic. They behave the same way as functions. For example, the CIC term $S (S \sim N)$ behaves just like $\text{fun } n \Rightarrow S (S x)$ when calculating $U(e, r)$.

Our result for decidability of BCIC says that well-behaved policies are decidable.

Lemma* 4.8 *If r is a well-behaved BCIC policy and e is a CIC environment, then there exists a set S of ground atomic formulas such that for any query q with $(e, r) \vdash q$, $q \in S$.*

First calculate $U(e, r)$, which will be finite because r is well-behaved. We proceed in the same way as in Lemma 4.1 and Lemma 4.5 by defining a set S of ground atomic formulas such that every query that is derivable must be in S . A finite number of atoms, predicate names, and predicate arities can appear in the program. Let A be a set of atoms we will use to construct S . Let A consist of the finite set $U(e, r)$. Also for each term in $U(e, r)$ add the (normalized) type of the term to A . To this add all the atoms that appear in r .

Construct the set S of all ground atomic formulas, including predicates, quoted predicates, theorems, and quoted theorems, using every possible combination of atoms from A , and predicate names and arities that appear in r .

This finite set S does not depend on the query, but any ground query that succeeds must be in S . To see this, we proceed by induction on the derivation of the query that shows it succeeds.

Let q be a ground query that succeeds. There are three possibilities. The cases with Formula 2.2.2, Formula 2.2.3, and Formula 2.2.4 all require that $q = \mathbf{sat}(T)$ for some $T \in U(e, r)$. In the case of Formula 2.2.3 we rely on the fact that $(T \bullet o) \in U(e, r)$ by the definition of $U(e, r)$. By the construction of S , S will include $\mathbf{sat}(T)$.

For the base case with Formula 2.2.1, q itself appears in the program and so will of course be in S . For the induction step, the query matches a head of some clause in the program under a variable assignment. We may assume by the induction hypothesis that under the variable assignment every atomic formula in the body of the clause appears in S . No free variables appear in the clause, so every variable in the head of the clause will be assigned to an atom that appeared in an atomic formula that is in S . This means the atom itself appeared in A by our construction of S . The head of the clause under the variable assignment thus contains only atoms that appeared in A , and predicate names and arities that appeared in the program, so it is in S . $\square$

Theorem* 4.9 *If r is a well-behaved BCIC policy, e is a CIC environment, and q a query, then $(e, r) \vdash q$ is decidable.*

The proof is based on Theorem 4.6.

We prove the theorem by constructing a set Ω that is a subset of S containing all derivable ground atomic formulas. Deciding when a ground query is derivable then just involves checking if it appears in Ω .

To construct Ω we proceed in stages. First we define a finite set Ω_0 containing some facts. Examine each CIC term T in $U(e, r)$. If $e \vdash_{CIC} \mathbf{refl_equal\ true} : T$, then add $\mathbf{sat}(T)$ to Ω_0 . There are a finite number of terms to consider, and CIC typechecking

is decidable, so this only adds a finite number of entries to Ω_0 . Next examine every pair of CIC terms o, T drawn from $U(e, r)$. If $e \vdash_{CIC} o : T$ then add $\mathbf{sat}(T)$ to Ω_0 . Again, there are a finite number of pairs and typechecking is decidable, so this will also only add a finite number of elements to Ω_0 .

We define Ω_{i+1} to be the union of Ω_i and the ground atomic formulas derivable from each clause in the program assuming the facts in Ω_i . In other words, Ω_i will contain all ground atomic formulas derivable in i or less derivation steps using Formula 2.2.1 and Formula 2.2.3. This operation will terminate because it always involves examining a finite number of facts. In addition, we know that CIC typechecking terminates. By the definition of Ω_i , $\Omega_i \subseteq \Omega_{i+1}$. From the previous theorem, the sequence of Ω_i is bounded by S and so must have a limit Ω .

To decide if a ground query is derivable we check to see if it appears in Ω . To show the algorithm is sound we show that if $q \in \Omega$ then $(e, r) \vdash q$. Let $q \in \Omega$. Then $q \in \Omega_i$ for some i by the definition of Ω . For the base case, $q \in \Omega_0$. There are two possibilities. Either q was added to Ω_0 in the first stage and $e \vdash_{CIC} \mathbf{refl_equal\ true} : T$, or q was added to Ω_0 in the second stage and there are two CIC terms $o, T \in U(e, r)$ with $e \vdash_{CIC} o : T$. In the first case the derivation of q is by Formula 2.2.4. In the second case the derivation of q is by Formula 2.2.2. If $q \in \Omega_i$ for some $i > 0$, then by induction we can see that there is a valid derivation for q using facts from Ω_{i-1} .

To show the terminating decision algorithm is complete it suffices to show that if q is derivable then $q \in \Omega$. The proof is by induction on the size of the derivation of q . In fact we show that if q is derivable with a derivation of size i then $q \in \Omega_i$. For the base case, either q is a fact that appears in the policy and so appears in Ω_1 , or $q = \mathbf{sat}(T)$ and $e \vdash_{CIC} \mathbf{refl_equal\ true} : T$, or $q = \mathbf{sat}(T)$ and there is a CIC term $o \in U(e, r)$ with $e \vdash_{CIC} o : T$. In the first case $q \in \Omega_1$, in the second and third cases $q \in \Omega_0$ by the construction of Ω_0 . Since $\Omega_0 \subseteq \Omega_1$, $q \in \Omega_1$.

For the inductive step, suppose q has a derivation D using Formula 2.2.1 where D is size i . By the inductive hypothesis, $s(a_1) \in \Omega_{j_1}, \dots, s(a_n) \in \Omega_{j_n}$ with $j_1 < i, \dots, j_n < i$.

Since $\Omega_k \subseteq \Omega_i$ for $k < i$, $s(a_1) \in \Omega_i, \dots, s(a_n) \in \Omega_i$. By the construction of Ω_i , this means $q \in \Omega_i$. Now suppose q has a derivation D using Formula 2.2.3 where D is size i . By the inductive hypothesis, $\text{sat}(T) \in \Omega_{i-1}$. By the definition of Ω_i , $\text{sat}(T') \in \Omega_i$.

Thus the decision algorithm we have defined is terminating, sound, and complete.

□

4.2.3 A useful restriction on policies

To help policy writers avoid writing policies that are not well-behaved, we have defined a class of CIC terms that guarantee a policy will be well-behaved. This class is not a complete characterization of all well-behaved sets of CIC terms, but it at least covers all the examples we have tested.

First we define some terminology for CIC terms. The terms that follow are newly invented for the purpose of classifying well-behaved policies and are not standard. For the examples we use the following CIC environment:

```

Inductive nat : Set :=
| 0 : nat
| S : nat -> nat.

Inductive even : nat -> Prop :=
| even0 : even 0
| evenSS : forall (n : nat), even n -> even (S (S n)).

Definition double (n : nat) : Prop := exists (x : nat), n = 2 * x.

```

A *constant* is either a reference to an entry in the CIC signature, or the application of a constant to a constant. The term `S (S 0)` is a constant. The term `evenSS 0 even0` is a constant. A *simple type* is a type entry in the CIC signature of kind `Set`, for example `nat`. A *simple constant* is a constant of simple type. For example, `0` and `(S (S (S 0)))` are simple constants (as well as constants). A *base object* is either a simple constant or a variable. If it is a variable, the variable may be bound within the CIC term or an escaped variable within the CIC term.

A *predicate type* is a type that has kind `Prop`, or kind $T \rightarrow A$ where T is a simple type and A is a predicate type. For example, `nat→Prop` is a predicate type. A *predicate name constant* is a constant with predicate type. For example, `even` and `double` are predicate name constants. A *predicate* is an application of a predicate name constant with type $T_1 \rightarrow \dots \rightarrow T_n \rightarrow \text{Prop}$ to base objects of type $T_1, \dots, T_n$. For example, `even P` and `even (S (S 0))` are predicates.

A *property type* is either a predicate A , a type of the form `forall (x : T), P` where T is a simple type and P is a property type, or $A \rightarrow P$ where A is a predicate and P is a property type. Finally, a *property proof* is a term with a type that is a property type.

Some example property types:

```
even 0
forall (n : nat), even n
forall (n : nat), even n -> double n
```

An example types that is *not* a property type:

```
forall (n : nat), even n -> even (S (S n))
```

This example is not a property type because `even (S (S n))` is not a predicate; it is not a predicate because `S (S n)` is not a base object.

The following theorem shows how these definitions help in constructing policies that are well-behaved.

Theorem 4.10 *Let e be a CIC environment and let r be a policy. If every CIC term in r is either a simple constant, a simple type, a property type, a property proof, or `Set` or `Prop`, then r is a well-behaved policy.*

We show that $U(e, r)$ is finite, which is the definition of well-behaved policies. Recall that $U(e, r)$ consists of every CIC term in r and is closed various operations. We will show that only a finite number of new terms can be obtained from these operations.

First is taking the type of terms. The type of simple constants is a simple type. The type (kind) of a simple type is `Set`. The type of a property type is `Prop`. The type

of a property proof is a single property type. Thus taking the types of terms adds a finite number of terms to $U(e, r)$. In addition, all the terms added are simple constants, simple types, property types, property proofs, **Set**, or **Prop**.

Next consider direct term application. A simple constant applied to anything is not well-typed. A simple type applied to anything is also not well-typed. A property proof applied to a simple constant might be well-typed, in which case it is a new property type. A property proof applied to another property proof might be well-typed, in which case it is a property proof. A property proof applied to anything else is not well-typed. The kinds **Set** and **Prop** cannot be applied to anything and yield a new well-typed term. Thus closure under application only adds terms in the same classes as before, but perhaps an infinite number of them.

Suppose $U(e, r)$ were infinite. By the definition of $U(e, r)$ and the above arguments there must be an infinite chain composed of applications from CIC terms in r . We show that this cannot happen by constructing a measure on the terms in $U(e, r)$.

Let m be a function on property types P to natural numbers defined in the following recursive way. When P is a predicate A , let $m(P) = 0$. When P is the term **forall** $(x : T), P'$ where T is a simple type and P' is another property type, let $m(P) = m(P') + 1$. When $P = A \rightarrow P'$ where A is a predicate and P' is another property type, let $m(P) = m(P') + 1$.

Let m' be a function on all CIC terms in $U(e, r)$ defined in the following way. Let $m'(o) = 0$ for when $o \in \chi$ is a simple constant, simple type, **Set**, or **Prop**. Let $m'(o) = m(T)$ for property proofs, terms $o \in \chi$ with $e \vdash_{CIC} o : T$ where T is a property type. Let $m'(T) = m(T)$ for terms T that are property types.

Now consider the measure m' of terms that are added to $U(e, r)$. For application we had: property proofs applied to simple constants, and property proofs applied to other property proofs. For the first case, the only way a property proof o can be applied to a simple constant c is if the property proof has type **forall** $(x : T), P$ where T is the simple type of the simple constant. In this case, $m'(o) = m(P) + 1$ and $m'(P) = m'((oc))$,

or in other words $m'((oc)) < m'(o)$. For the second case, the only way a property proof o can be applied to another property proof o' is if $e \vdash_{CIC} o' : A$ and $o : A \rightarrow P$ where P is a property type. In this case $m'(o) = m'(P) + 1$ and $m'(P) = m'((oo'))$, and again $m'((oo')) < m'(o)$.

Thus the measure of the application of two terms is always less than the measure of the first term. The measures are always natural numbers, so there can be no infinite chain of applications. $\square$

It is not quite enough in BCIC to require that the local policy be well-behaved. Recall that statements from other contexts are imported as new facts with in the policy. If every context has a well-behaved policy, it is easy to see that after exporting and importing facts every context will still have a well-behaved policy, because exporting and importing only transfers CIC terms, it never creates new CIC terms.

A malicious principal could export statements that contain CIC terms that when imported make the local policy not well-behaved. This would be a type of denial-of-service attack; no unsafe actions would be allowed from the compromised policy, but the implementation might not be able to process any queries, safe or malicious. Our implementation does not currently have the capability to check the conditions of Theorem 4.10, but adding such a check on all incoming facts from imports would effectively stop this denial-of-service attack.

Chapter 5

Proof by computation

Up to this point we have shown how BCIC can express sophisticated distributed reasoning about authorization using assertions and explicit proofs, but there are still gaps in the types of policies we have examined. During the course of developing the examples from Chapter 3 we often found a need to define some simple data structure and refer to it in our security policies. Because BCIC is ultimately built on Datalog, all data structures and functions must be constructed as CIC terms, since Datalog does not allow function symbols.

It is of course possible to extend Datalog with various simple data structures and keep the desirable properties of the language such as decidability [LM03, JM94]. The problem is that if every practical example of the logic requires a custom extension, the benefit of using the logic as a framework is diminished. The ideal security logic is extensible at the policy level so that policy writers can write their own extensions for their particular application, while still keeping the policy language expressive and decidable. With reflective proofs, BCIC is such a policy language.

The ability to define and use decision procedures to extend BCIC is important for making BCIC practical. It allows third parties to construct libraries of decision procedures and auxiliary functions that can be used by others, creating a policy extension ecosystem that is diverse and changing as users needs change. With proof by computation, the recipient of a new predicate and decision procedure need not learn how to construct proofs of the

predicate. The decision procedure makes explicit proofs unnecessary. We have constructed a library of useful data structures and decision procedures for policy writers and present some examples from our library to illustrate proof by computation.

5.1 Motivation

To illustrate the need for reflective proofs in BCIC consider the case of list membership. List membership came up in several practical applications that we investigated, including information flow and distributed network storage permissions.

Lists cannot be represented adequately using Binder or Datalog rules alone (we prove this in Section 4.1.1). So in BCIC, lists must be represented as CIC terms. In fact, lists are defined in the Coq standard libraries, as is `In`, the list membership predicate. Thus it is possible for a policy to refer to `sat(In P L)` which will be derivable when there is a proof that `P` is in the list `L`.

The problem is the requirement for a proof. The formula `sat(In P L)` is derivable in BCIC only when there is an actual CIC proof term for `In P L` that is provided externally. For complicated proof systems and proof-carrying code, it might be reasonable to require that the code producer transmit an explicit proof about how their code satisfies the proof rules, but for list membership it does not make sense. What is needed is a way for BCIC to automatically prove properties when possible.

5.2 Reflective proofs

The solution is reflective proofs. Proof by reflection [BC04b, OG02] is a technique that elides details related to computation from proof terms, making proofs smaller. In our case we want to shrink proof terms to a single constant term in order to make proof generation entirely automatic.

Another name for reflective proofs is proof by computation, which perhaps more accurately describes what is happening. Instead of writing our security policy using `sat(In`

`P L`), we could instead define a function `f` that takes an element and a list and returns a boolean. The function would be designed to return `true` when the element was in the list and `false` otherwise. Instead of writing `sat(In P L)` we would write `sat(f P L = true)`. If the function behaves correctly, this substitution will not change the meaning of the policy.

The key point is that now the entire proof is a single computation. If the element is in the list, the proof will be simply `refl_equal true`. The constructor `refl_equal` is the single constructor for the equality predicate that demonstrates `A = A` for all `A`. It seems like `refl_equal true` just proves `true = true`. But, because the element is a member of the list, and our function `f` is correct, the term `f P L` will reduce to `true` using $\beta\delta\iota$ -reductions. The δ -reductions expand function definitions, ι -reductions expand recursive function calls, and β -reductions transform function applications into substitutions (as in the λ -calculus). By a standard CIC typing rule, if two terms or subterms are convertible (reducible to the same term) then they have the same type. This means `f P L = true` has the same type as `true = true` if `f P L` is convertible to `true`. Thus since `refl_equal true` has type `true = true`, it must have type `f P L = true`.

It is important to note that `refl_equal true` will only prove `f P L = true` when the function `f` returns `true`. If the function returns `false`, `refl_equal true` will not be a proof. This shows how the necessary computations are done at type checking time and are not explicitly expressed in the proof term, which is the essence of reflective proofs.

5.2.1 Decision procedures

We also relied on the correctness of the function `f`. If the function `f` returns values that disagree with `In`, then substituting `sat(f P L = true)` for `sat(In P L)` could change the meaning of the policy. The policy writer could prove some extra theorems in Coq to convince themselves that the function is correct, but using Coq's standard decidability notation is a more elegant way to ensure the function is correct.

Decidability is represented using the `sumbool` type, which has notation `{A}+{B}`.

Ignoring issues of definition opacity and automatic program extraction, the `sumbool` type is equivalent to $A \vee B$. Decidability of a proposition P is expressed as $\{P\} + \{\sim P\}$. Because Coq is constructive, giving a decision procedure for P of type $\{P\} + \{\sim P\}$ is the same as proving that P is decidable. Instead of making the return value of the function `bool`, we give it the type $\{\text{In } P \text{ L}\} + \{\sim \text{In } P \text{ L}\}$. The function will now return either a proof of `In P L` or a proof of its negation, `~In P L`. Since the CIC typing relation is what defines proof checking, if the function is well-typed then it must always return correct answers to whether P is in L . In fact, the Coq standard libraries provide such a decision procedure for list membership, `In_dec`.

To make use of certified decision procedures we define a utility function `decide`. Given a proposition P and a decision procedure `Pdec` for P , `decide P Pdec` can directly replace P in policies without any logical difference. However, because it uses a decision procedure for P , the proposition can now be automatically proven by reflection without the need for explicit proof terms. The definition of `decide` is:

```
Definition decide (P : Prop)(Pdec : {P}+{~P}) : Prop :=
  (match Pdec with | left Prf => true | right Prf => false end)
  = true.
```

The function works by calling the decision procedure then asserting that it returned a proof of P rather than $\sim P$. Section 5.5 contains some properties of this function. The most important property is that replacing P with `decide P Pdec` does not change the logical meaning of policies.

5.2.2 Partial analyses

Many forms of static analysis are sound but not complete. That is, they sometimes give an answer about the code; if they give an answer, it is always correct, but if they give no answer nothing is learned about the code. This type of partial reasoning can be expressed using Coq's built-in notations.

Recall that terms of the `sumbool` type in Coq, represented with the notation

$\{A\}+\{B\}$, are proofs of either A or B . Decidability of a proposition P is expressed as $\{P\}+\{\sim P\}$. The type of a procedure that can sometimes determine when P is true, but other times fails to conclude anything is simply $\{P\}+\{\text{True}\}$. If the analysis succeeds then it will prove P , otherwise it will fail and just return a trivial proof of True . Similarly, a procedure that sometimes determines when P is false is $\{\sim P\}+\{\text{True}\}$.

We can no longer use `decide` to construct automatically decidable propositions from arbitrary propositions and these types of partial decision procedures; the partial decision procedures don't have the right type. Instead we can define new utility functions as follows:

```
Definition decide_part (P : Prop) (Pdec : {P}+{True}) : Prop :=
  (match Pdec with | left Prf => true | right Prf => false end)
  = true.
```

The definition is almost identical to the definition of `decide` except for the type. Because the decision procedure is partial it will satisfy less strict properties in Section 5.5.

Some forms of static analysis might not be certified at all. Or, the analysis might be proven correct in a published paper but not in a machine checkable way. This case can be modeled using BCIC. For the case of an unproven analysis f , the policy can use `sat(f P = true)` to refer to the fact that P passed the analysis. The function f that checks code might come with a digital signature from a trusted source certifying that it is correct, as shown in the following policy that relies on Alice to sign the correct analyzer:

```
mayrun(P) :- Alice says use_analyzer(F),
             sat(<~F ~P = true>).
```

5.3 Simple examples

5.3.1 List prefix

To illustrate the idea of reflective proofs further, let us consider another example. Determining when one list of natural numbers is a prefix of another is a simple operation

of lists that is useful for policies. In this case the property is not already defined in the Coq standard libraries so we must define it. For lists of natural numbers, the list prefix predicate is defined by:

```
Inductive prefix : list nat -> list nat -> Prop :=
  | prefix_nil : forall L, prefix nil L
  | prefix_cons : forall hd tl L,
    prefix tl L -> prefix (hd :: tl) (hd :: L).
```

Once this predicate is defined, policies can include formulas such as `sat(prefix L1 L2)`. For any pair of lists, there must be an explicit proof imported into the policy that proves the first list is a prefix of the second. The proof that `1::nil` is a prefix of `1::2::3::nil` is `prefix_cons 1 nil (2 :: 3 :: nil) (prefix_nil (2 :: 3 :: nil))`.

To avoid generating these proofs, we must define a decision procedure for `prefix`. An excerpt of the proof script that defines the decision procedure is:

```
Theorem prefix_dec:
  forall L1 L2,
    {prefix L1 L2}+{~prefix L1 L2}.
Proof.
  induction L1.
  left. apply prefix_nil.
  induction L2.
  ...
Qed.
```

Now instead of referring to `sat(prefix L1 L2)` we refer to `sat(decide (prefix L1 L2) (prefix_dec L1 L2))`. The logical meaning is the same, but now instead of explicit proofs the contents of the proof will be performed as computations inside the decision procedure. If `prefix L1 L2` is true, then the proof of `decide (prefix L1 L2) (prefix_dec L1 L2)` will be exactly `refl_equal true` and can be generated automatically.

Now that we have defined some predicates and decision procedures we can use them in policies.

5.3.2 Access control lists

A good example for the usefulness of list membership is in representing access control lists. An example policy for describing access control to resources via access control lists is the following, before being converted to use decision procedures:

```
acl(R, L) :- Alice says acl(R, L).  
mayaccess(P, R) :- acl(R, L), inlist(P, L).  
inlist(P, L) :- sat(In P L).
```

The intent is that Alice declares an access control list *L* for each resource *R*. Then principal *P* may access resource *R* if they appear in the access control list *L* associated with the resource. To use the list membership decision procedure, we rewrite the rule for `inlist` to the following rule that mentions the explicit decision procedure:

```
inlist(P, L) :- sat(decide (In P L) (In_dec eq_nat_dec P L)).
```

Note that `eq_nat_dec` appears because the proof of decidability of list membership requires a proof of decidability of equality between list elements, which is also part of the standard Coq libraries.

The translation step is fairly simple. The above rule defining `inlist` is part of BCIC's policy library. Policy writers can thus use `inlist` to write policies about list membership without needing to understand how decision procedures or the `decide` function work.

5.3.3 Tree paths

List prefix and other operations are useful when describing tree structures. Considering a list as a path down a tree, list prefix indicates that one path is a subpath of another. These types of operations allow security policies to reason about access control over tree structures such as directories. In a sense Datalog and security logics based on Datalog can represent tree structures directly, for example by defining an `ancestor` relation. The problem is that this doesn't allow tree structured *data* in the policy.

Here is a simple policy that allows Alice to grant access to nodes in a tree, and includes a rule that says if a principal has access to one node, they automatically have access to all descendant nodes. If one path is a prefix of another, the first path describes an ancestor to the second.

```
mayaccess(P, L) :- Alice says mayaccess(P, L).
mayaccess(P, L2) :- mayaccess(P, L1), prefix(L1, L2).
prefix(L1, L2) :-
  sat(decide (list_prefix L1 L2) (prefix_dec L1 L2)).
```

5.4 Negation and revocation

Even though CIC is a constructive logic, negation is allowed in extensions defined by decision procedures. In constructive logics it is not generally the case that a formula is either true or false. It is the case, however, that a decision procedure will always terminate and yield either a proof or disproof, making the proposition true or false. Since decision procedures always yield an answer, we can easily define a utility function that is satisfied exactly when the proposition is *not* true:

```
Definition decide_not (P : Prop)(Pdec : {P}+{~P}) : Prop :=
  (match Pdec with | left Prf => false | right Prf => true end)
  = true.
```

To illustrate negation, consider list membership again. By using `In` and the decision procedure `In_dec` with `decide_not` we can directly express list non-membership. For security policies this is useful. Just as list membership works for access control lists, list non-membership works for revocation lists. The following policy illustrates how revocation lists can be expressed. Alice declares the revocation list; anyone not in the list may access any resource.

```
revocation(L) :- Alice says revocation(L).
mayaccess(P, R) :- revocation(L), notinlist(P, L).
notinlist(P, L) :-
  sat(decide_not (In P L) (In_dec eq_nat_dec P L)).
```


5.5 Properties regarding reflection

In this section we discuss some of the properties of BCIC specific to reflective proofs and decision procedures as an extension mechanism.

In BCIC, CIC terms are always converted to normal form before any reasoning takes place. This means that to add reflective proofs we do not have to worry about $\beta\delta\iota$ -reductions, as they will be handled by the conversion to normal form. The goal of this feature of BCIC is to make CIC terms of the form `decide P Pdec` automatically proven. The function `decide` is defined as an equality, so as we saw previously the proof of this proposition will be `refl.equal true` if the proposition is indeed true. Formula 2.2.4 enables decidable properties to be automatically proven in BCIC.

For the convenience of policy writers, we have defined a library of predefined predicates and functions to help in policy construction. These include the utility functions `decide`, `decide_not`, and `decide_part` that allow decision procedures for propositions to be declared alongside propositions, a set of useful properties about simple data structures such as lists and trees, and decision procedures for all the properties. To use this library the policy writer only has to add the definitions from the library to their CIC environment.

5.5.1 Decision procedures

First we consider properties with full decision procedures as inserted into the policy with `decide`. The most important property related to reflective proofs is that they are correct.

Theorem* 5.1 (Correct Reflective Proofs) *For all CIC environments e , propositions P , and decision procedures $Pdec$ for P , there exists a proof of `decide P Pdec` in e if and only if there exists a proof of P in e .*

It might seem slightly odd that in the previous theorem only one side of the if and only if mentions the decision procedure. The reason is that all decision procedures are identical and interchangeable as the following corollary shows.

Corollary* 5.2 (One Decision Procedure) *For all CIC environments e , propositions P , and decision procedures $Pdec$ and $Pdec'$ for P , there exists a proof of `decide` P $Pdec$ in e if and only if there exists a proof of `decide` P $Pdec'$ in e .*

In CIC, all proofs of equalities are the same, and in fact all proofs of equalities are exactly `refl_equal x`. By using some advanced proof techniques [Bar] along with John Major equality for reasoning about heterogeneous equality [McB02], this equality can be shown within Coq itself. The consequence of this fact along with the previous theorems and Theorem 4.7 is the following theorem that says BCIC will not miss any reflective proofs.

Theorem* 5.3 (No Missing Proofs) *For all CIC environments e , conservative policies r , propositions P , and decision procedures $Pdec$ for P , $(e, r) \vdash \text{sat}(\text{decide } P \text{ } Pdec)$ if and only if there exists a proof of P in e .*

5.5.2 Partial decision procedures

Consider partial decision procedures and `decide_part`. The following weaker results hold for partial decision procedures. No result equivalent to Theorem 5.2 holds because there can be many different partial decision procedures for any proposition.

Theorem* 5.4 (Partial Correctness) *For all CIC signatures Γ and terms P and $Pdec$, if there exists a proof of `decide_part` P $Pdec$ in Γ then there exists a proof of P in Γ .*

Theorem* 5.5 (Partial Policy Correctness) *For all conservative policies $r = (\Gamma, d)$ and terms P and $Pdec$, if $r \vdash \text{sat}(\text{decide_part } P \text{ } Pdec)$ then there exists a proof of P in Γ .*

5.6 Static analysis

Perhaps the most exciting application of our extension to BCIC is that it allows policy writers to specify static analyses of programs within the policy. Previously we showed how policies could refer to simple decidable properties about CIC terms in order to make policies more expressive. The decidable properties possible with reflective proofs in BCIC

are not limited to simple utility functions. In this section we present more complicated examples demonstrating security policies that rely on static analysis.

All of these forms of static analysis are possible in a proof-carrying code framework. The problem is that if every property has to have an explicit proof, there is much more effort that must be expended by the code provider to generate the proofs, not to mention more storage and bandwidth costs for the proofs to be transmitted.

It is also desirable to separate out analyzing code from the code producer because it empowers the recipient and gives them more discretion about what properties are necessary and how best to verify them. There could be many different properties that the code recipient wants code to satisfy. The code producer might know some but not all of these properties in advance. It might also be the case that the static analyzers are continually refined and upgraded over time; code produced for one specific analyzer might also satisfy newer analyzers, but if the code producer has to provide explicit proofs then the same old code and proof cannot be reused with newer static analyzers.

5.6.1 Simply typed λ -calculus

Consider an application whose plug-ins are written in a scripting language similar to the λ -calculus. It is very inconvenient for the application to recover from errors in the scripts, so the application uses a simple type system to statically verify plug-ins before executing them. Plug-ins must also be signed by keys that are certified by the main application key to help prevent malicious (but correctly typed) plug-ins from wreaking havoc.

The BCIC policy that the application defines is as follows.

```
trusted(rootkey).
trusted(B) :- A says trusted(B), trusted(A).
certified(P) :- A says certified(P), trusted(A).
typesafe(P) :- sat(decide (welltyped P) (welltyped_dec P)).
mayrun(P) :- certified(P), typesafe(P).
```

The first two rules define a trusted application key and a way for trusted keys to delegate their trust to new keys. The third rule defines how plug-ins are certified. The final rule

determines when plug-ins may be executed. They must be certified by a trusted key and also satisfy the `welltyped` predicate, defined in the CIC signature. The final rule also indicates the correct decision procedure to use through the `decide` function defined in Section 5.2.1.

The corresponding CIC signature for the policy defines the syntax of simply typed λ -calculus programs and the typing rules. Types consist of base types (enumerated with natural numbers) and arrows constructing function types.

```
Inductive type : Set :=
| base : nat -> type
| arrow : type -> type -> type.
```

Expressions are from the standard λ -calculus, but in this case lambda expressions are annotated with the type of the argument. Variable references are in de Bruijn notation.

```
Inductive expr : Set :=
| ref : nat -> expr
| lam : type -> expr -> expr
| app : expr -> expr -> expr.
```

Typing contexts `context` are simply defined as `list type`.

The typing judgment `has_type Gamma E T` indicates that in typing context `Gamma`, the expression `E` has type `T`.

```
Inductive has_type : context -> expr -> type -> Prop :=
| has_type_ref :
  forall n T C,
    nth_prop n T C ->
    has_type C (ref n) T
| has_type_lam :
  forall T C E T2,
    has_type (T :: C) E T2 ->
    has_type C (lam T E) (arrow T T2)
| has_type_app :
  forall C E1 E2 T1 T2,
    has_type C E1 (arrow T1 T2) ->
```

```

has_type C E2 T1 ->
has_type C (app E1 E2) T2.

```

Finally, `welltyped` is defined for the application to mean that an expression has base type 0 in the empty context.

```

Definition well_typed (E : expr) : Prop :=
  has_type nil E (base 0).

```

Proving that type checking is decidable (i.e. writing a correct definition for `welltyped_dec`) is not particularly hard but requires some work. Our definition is available at our homepage: <http://www.soe.ucsc.edu/~nwhitehe>. The type checking algorithm itself is simple to express. Proving that if the algorithm succeeds then it gives a valid type for the expression, i.e. soundness, is straightforward to prove. The other direction, completeness, is more difficult. Proving completeness involves reasoning about the typing rules themselves. A key lemma says that if one expression has two typing derivations then the types must be identical. One can then prove that for any typing context and expression, either there exists a type and valid typing derivation or there does not.

Now that the `welltyped` predicate, its decision procedure, and the policy using them are defined the example is complete. The implementation of BCIC is given the policy and CIC signature defining the predicate and decision procedure, then acts as a reference monitor for requests to run plug-ins.

Summary of simply-typed λ -calculus example

This example illustrates a decision procedure for a type system, in this case for the simply-typed λ -calculus. This example shows how decision procedures can be integrated into BCIC policies to avoid the need for explicit proofs by code producers.

For this example we have written a **policy**, defined an explicit CIC proof **environment** for proving `well_typed`, and defined a **decision procedure** for `well_typed`. Because of Theorem 5.1 we do not need to prove **correctness** of the decision procedure.

We have not implemented a special runtime environment for interpreting code, although Scheme by itself could be considered a runtime environment.

5.6.2 Allowed free variables

In the previous example we have demonstrated a single stand-alone property about code, namely that it type checks. While hopefully inspiring, the previous example is just the start of what can be done in BCIC. Because BCIC combines elements of authority, proof, and analysis, policies can mix and match these elements at any level.

To show how all the elements can be combined, in this example we consider programs that call library functions. For simplicity we reuse the simply typed λ -calculus from the previous example. The application policy will now say that plug-ins are safe to execute if they are certified by a trusted key and every library function they call is on a list of safe functions to call, provided by a trusted principal.

```
trusted(rootkey).
trusted(B) :- A says trusted(B), trusted(A).
certified(P) :- A says certified(P), trusted(A).
mayrun(P) :-
  certified(P),
  A says libraryfuncs(L),
  sat(decide (onlycalls P L) (onlycalls_dec P L)).
```

The definition of `onlycalls` is:

```
Inductive onlycalls : expr -> list nat -> Prop :=
| onlycalls_ref :
  forall n L,
    In n L ->
      onlycalls (ref n) L
| onlycalls_lam :
  forall E T L,
    onlycalls (lam T E) (0 :: (lift_list L))
| onlycalls_app :
  forall E1 E2 L,
    onlycalls E1 L ->
```

```

onlycalls E2 L ->
onlycalls (app E1 E2) L.

```

Similarly, `onlycalls_dec` is a proof of decidability of `onlycalls`.

```

Theorem onlycalls_dec:
  forall E L,
    {onlycalls E L}+{~onlycalls E L}.
Proof.
  intros.
  induction E.
  ...
Qed.

```

The definition of `onlycalls` and its decision procedure are placed in the global CIC signature and paired with the security policy rules. BCIC then communicates on the network and makes logical derivations to decide when code passes the security policy and may be executed. Since trusted principals declare which library functions are allowed and these statements are only known at run time, this example illustrates a mix of trust and static analysis.

Summary of allowed free variables example

This example illustrates a decision procedure for a property that is not a type system, in this case for an analysis of free variables from an allowed list. This example also illustrates how decision procedures can interact with assertions to combine static analysis and statements by authorities.

For this example we have written a **policy**, defined an explicit CIC proof **environment** for proving `onlycalls`, and defined a **decision procedure** for `onlycalls`. Because of Theorem 5.1 we do not need to prove **correctness** of the decision procedure. We have not implemented a special runtime environment for interpreting code.

5.7 Extending previous examples

The constructs from this chapter can be used to improve some of the examples from Chapter 3. Instead of requiring explicit proofs for all the properties from the examples, in some cases we can define decision procedures that can be included in the code consumer's security policy. This way the proof of the property will be computed when the policy is checked; the code producer will no longer have to construct a proof herself.

We do not update the auditing function calls example from Section 3.1 to use proof by computation. Recall that code producers generate proofs of the form `audit_maycall P F -> audited_calls P P`. The fact `audit_maycall P F` has no proof constructors; it can only be proven by assertion from the policy. To construct a decision procedure of `audited_calls` requires that `audit_maycall` be decidable. We can declare the decidability of `audit_maycall` in the environment as an axiom:

```
Axiom audit_maycall_dec :  
  forall P F, {audit_maycall P F}+{~audit_maycall P F}.
```

This allows us to prove the decidability of `audited_calls`, but is useless as a decision procedure because there is no actual decision procedure for `audit_maycall`. Our solution to this problem is redesigning the predicate. This is precisely the example from Section 5.6.2.

5.7.1 Trust in the λ -calculus

We can use the idea of proof-by-computation to reduce the proof burden on the code producer in the example for trust in the λ -calculus from Section 3.2. The typing judgment `has_type` from the example is decidable. In order to prove the decidability of `has_type` we need to build up a library of auxiliary decision procedures. Here is the decision procedure for `trust_lte`.

```
Lemma trust_lte_dec :  
  forall (T1 T2 : trust_type),  
    {trust_lte T1 T2}+{~trust_lte T1 T2}.
```

Proof.

```
intros.
induction T1; induction T2.
left. apply trust_lte_refl.
left. apply trust_lte_trdis.
right. unfold not. intro H.
inversion H.
left. apply trust_lte_refl.
```

Qed.

The final decision procedure starts like this:

Theorem has_type_dec:

```
forall (E : expr)(C : context)(A : annotated_type),
  {has_type C E A} + {~has_type C E A}.
```

Proof.

```
induction E; intros.
(* const_int case *)
left. exists (annotate (usertype 0) tr).
apply type_int.
...
```

Qed.

The rewritten policy will now contain a line like this:

```
mayrun(U, C, P) :-
  believes(U, <decide
    (has_type ~C ~P (annotate (usertype 0) tr))
    (has_type_dec ~C ~P (annotate (usertype 0) tr)))>),
  believes(U, <trusts_audited ~P ~P nil>).
```

5.7.2 Instrumented code

The example of instrumented x86 machine code from Section 3.3 is a good candidate for the use of proof-by-computation. The static check to determine whether code is instrumented correctly is much more logically done by a decision procedure on the code recipient's side than by an explicit proof.

Given P , deciding whether L is the encoded version of P is decidable; it is merely checking equality between lists of bytes. The remaining checks in `well_instrumented` are also decidable. We can then define a decision procedure for `well_instrumented`:

```

Lemma well_instrumented_dec :
  forall (P : list instr)(L : list byte),
    {well_instrumented P L}+{~well_instrumented P L}.
Proof.
  intros.
  unfold well_instrumented.
  ...
Qed.

```

Note that in the definition a value for P must be passed to the decision procedure. It would also be possible to define the decision procedure with the following type:

```

Lemma bad_well_instrumented_dec :
  forall (L : list byte),
    {exists (P : list instr), well_instrumented P L}+
    {~exists (P : list instr), well_instrumented P L}.

```

This formulation is much stronger and requires much more effort to create, and is completely unnecessary. The code producer already knows both P and L , so there is no reason the decision algorithm must disassemble the machine code to reconstruct P .

In addition, the predicate `memory_safe` is decidable. The predicate is defined as properties over all elements of a list. If a property is decidable, then the conjunction of the property over the entire list is also decidable. Checking that one number is within a range is obviously decidable, so this lets us easily define `memory_safe_dec`.

Having a decision procedure for `memory_safe` means that no-one must construct a proof of `memory_safe P R` for every given program P and list of regions R . In the policy in Section 3.3.8 it is not clear who should construct this proof. The code producer may not know exactly which regions of memory the code may access. The trusted authority that declares the memory regions may have no knowledge of individual programs. An attractive

solution is to have the code consumer perform the check using proof by computation, which is possible because of the decision procedure for `memory_safe`.

Chapter 6

Implementation

Although our system must implement the logic presented in Chapter 2 in order to support reasoning about signatures and proofs, the system is much more than a bare implementation of the logic. In such a bare implementation, for instance, signed statements may simply be logical expressions—appropriate for initial experimentation but not much else. In order to be useful, signed statements must have concrete, secure digital representations. Thus, in our system, signatures employ cryptography; they are unforgeable and tamper-evident. Furthermore, our system deals with communication over an insecure network. The network module should minimize the need for manual user intervention for synchronization. The logic interpreter should be as correct as possible; to achieve this goal, we extract the logic interpreter from a Coq proof of decidability of BCIC.

The implementation has several parts:

parser The *parser* understands the syntax of the logic and can translate between textual and abstract syntax tree representations.

logic interpreter The *logic interpreter* performs deductions in the logic from a set of given statements to produce a larger (but still finite) set of deduced statements.

cryptography module The *cryptography module* implements the necessary cryptographic operations, such as generating and checking signatures.

network module The *network module* can communicate statements over the network.

user interface The *user interface* accepts input from the user and determines which action should be taken. The user interface is a simple graphical utility that communicates with an existing daemon over a secure local socket.

supervisor The *supervisor* is in charge of coordinating the global behavior of the program. It loads existing databases of statements, decides when to communicate on the network, sign statements, draw inferences, and accept input from the user.

policy The *policy* gives rules for deciding when code should be executed, who to trust initially about what, and so forth.

database The *database* holds all known true facts from any source.

Figure 6.0.1 shows the organization of these components in the system. Boxes represent code modules, circles represent data. Figure 6.0.2 shows a screenshot of three instances of BCIC running on a single computer, each attached to a different port. Two of the instances are connected and are exchanging statements from a simple example policy. The third instance is about to join the network.

6.1 The logic module

The logic module is responsible for managing the fact database and responding to queries. Initially the fact database contains only the facts in the local policy. After synchronizing with other hosts and exchanging signed statements, the fact database will grow. The main job of the logic interpreter is finding all possible logical deductions within the database and adding them to the database.

This method of answering queries is bottom-up evaluation. The bottom-up approach has the advantage that it is simple and clearly exhaustive. In contrast, the termination of top-down inference for Datalog (and therefore for Binder) requires tabling [RSS⁺97, CW96]. Showing that a particular tabling algorithm is correct is much more difficult than

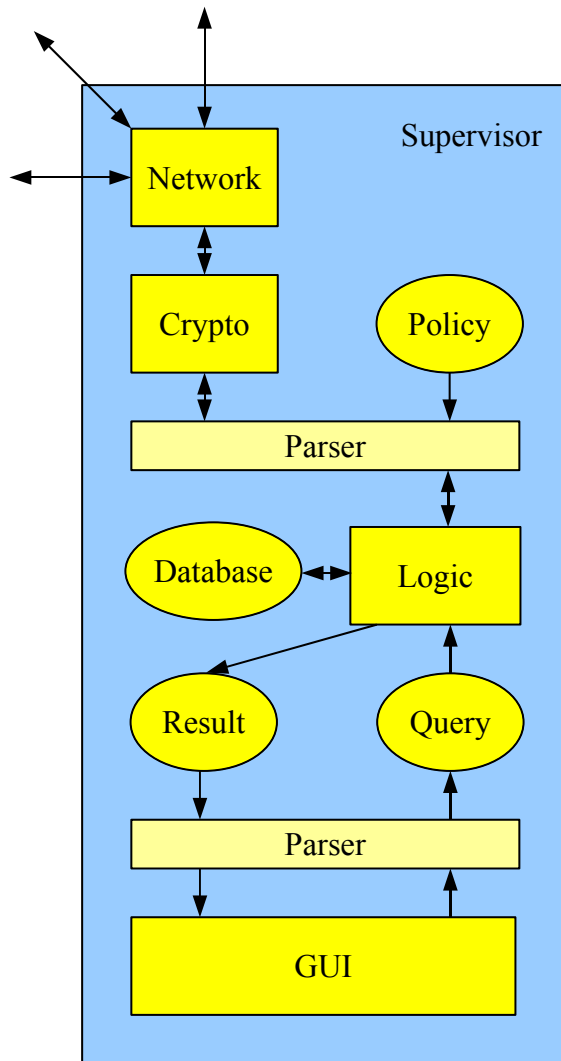


Figure 6.0.1: System components

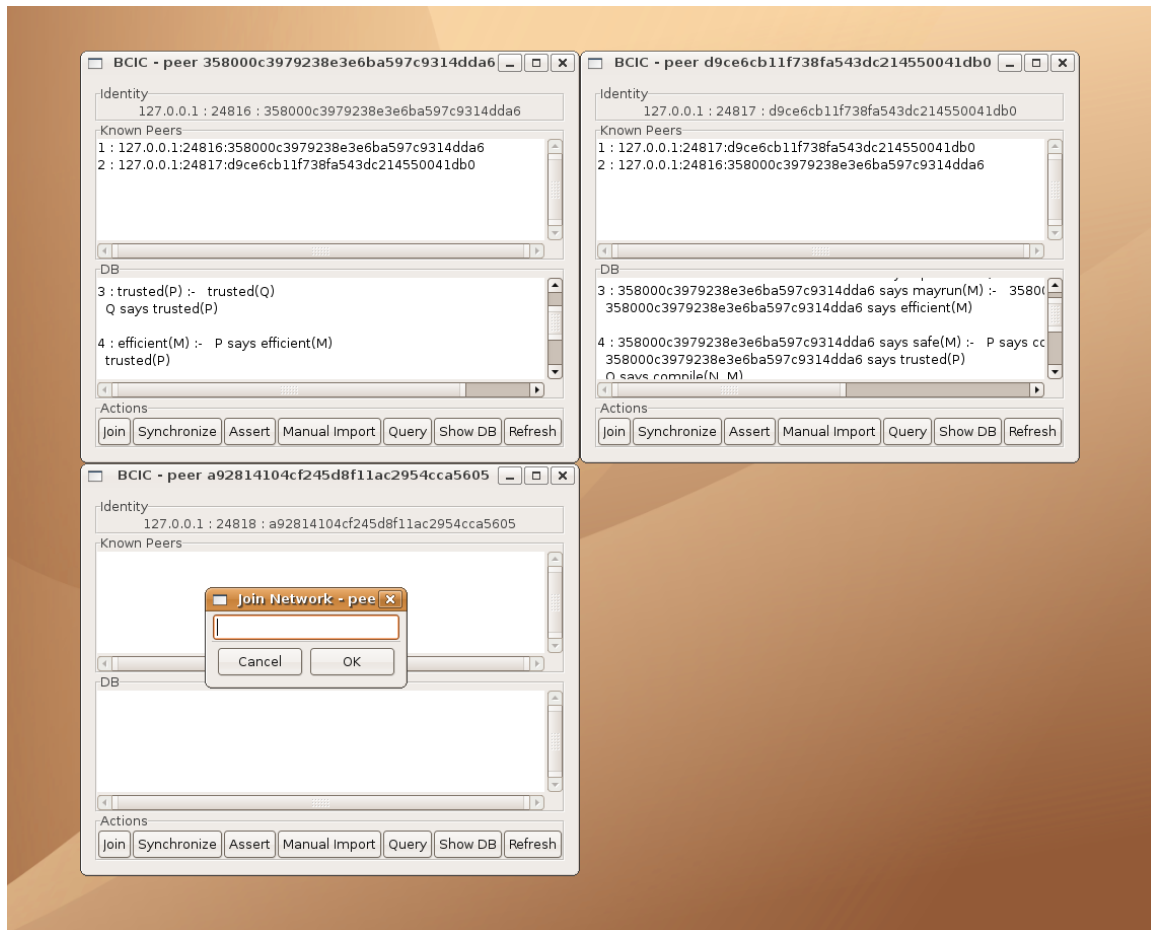


Figure 6.0.2: A screenshot of three instances of BCIC running on a single computer, each attached to a different port. Two of the instances (top half) are connected and are exchanging statements from a simple example policy. The third instance (lower left) is about to join the network.

with the bottom-up approach. For example, ensuring that all possible types of loops are correctly detected requires some care.

Although bottom-up evaluation can require more time and space than top-down evaluation, bottom-up evaluation is practical for security applications because policies do not usually require many steps of reasoning or very large databases of facts. Our most complicated example took five steps to deduce the correct conclusion.

The logic module is mostly generated by extraction from our Coq proof of decidability of BCIC, stated in English in Section 4.9. This gave us a certified implementation of the decision procedure for BCIC queries. We discuss the extraction process in more detail in Section 6.4.

6.2 The cryptography module

The cryptography module is based on Xavier Leroy’s library for OCaml, `cryptokit`, a library that provides cryptographic primitives. We use these primitives for generating keys, for signing, and for verifying signatures. We rely on RSA signatures with a key length of 1024 bits, and we apply SHA-1 hashing before signing. Each signed logical statement is accompanied by public-key information about the signer. Verifying that Alice signs statement `X` leads to an entry in the fact database, with the formula `Alice says X`. We serialize and deserialize statements using the `Marshal` standard library functions of OCaml.

When keys are not managed securely, the integrity of every signature is suspect. Therefore, following standard practices, we store secret keys in encrypted form, keyed to the hash of a passphrase supplied by the user. We use AES encryption and SHA-1 hashes for storing secret RSA keys.

6.3 The network module

The network module is only in charge of communicating signed statements between hosts, not determining their logical meaning. When new statements become available, the

logic inference module must decide how they are to be interpreted. When the logic inference algorithm adds new unquoted conclusions (that is, formulas without **says**) to the database, the cryptography module creates new signatures and stores them in the database, and the network module communicates them to other hosts.

Network communication is done using TCP/IP connections on a fixed port. Users may leave a Unix daemon running at all times waiting for connections, if they wish. When two BCIC nodes connect, they follow a protocol described in Section 6.3.1 to decide which statements are known to one and not the other. These statements are then transmitted over the connection. Connections can be scheduled to occur automatically with randomly chosen partners at regular intervals, or can be requested manually by users. Full-time servers may run the daemon and automatically communicate with one another, while individual client machines may rather connect to the nearest server sporadically at a user's request.

6.3.1 Synchronization algorithm

The most interesting aspect of the network module is the algorithm for coordinating updates. When nodes connect, they must decide who has new statements that need to be transmitted. Simply transmitting all the statements at every connection would be tremendously inefficient. A slightly more realistic possibility is a naive protocol in which each node hashes every statement in its database and communicates the entire set of hashes to the other node. Then it is easy for each node to decide which statements it must send. If n is the total number of statements known by both sides, then the naive protocol takes $O(n)$ steps per synchronization.

The naive protocol is likely not to be efficient enough in the long run. The size of the fact databases will steadily increase over time, if nothing else because expiration and revocation are rare (and they are not even modeled explicitly in the logic, although the implementation deals with them). More specifically, we may estimate the performance of the naive protocol as follows. A large library may be composed of several hundred functions. The library provider may wish to declare some functions correct by assertion and to verify

other, simpler functions. One way to do this is for the library provider to sign assertions for each of the functions separately. As new versions of library functions become available, new statements will be generated. A fairly typical Linux operating system in our lab currently uses 652 libraries and 2777 applications. If every library requires 100 statements and releases 10 major versions a year, with each version containing 10 function updates, and every application releases 10 new versions a year, then the database will initially contain 67977 statements and will increase by 92970 statements each year. After two years the naive protocol will be exchanging 5 Mb at each connection. Even if one reduces the number of statements at the expense of statement size by signing one large conjunction that contains statements for all the functions in a library release, the protocol will still be exchanging 2.7 Mb at each connection after two years.

There are many possible solutions to the synchronization problem. It is not too hard to imagine methods that record timestamps or remember which facts have already been communicated to other servers. We chose to implement a recursive divide-and-conquer protocol that does not require any extra storage outside the databases themselves. It is asymptotically efficient for small updates between large databases.

Our approach requires that every statement in every database be hashed and stored sorted by hash value. Figure 6.3.1 shows the synchronization algorithm for our protocol in pseudo-code.

Our protocol synchronizes ranges of hash values between two databases. To synchronize two entire databases, the protocol is performed over the entire range of possible hash values. To synchronize all hash values between L and H , first both participants extract all hash values in their databases between L and H . Each list is encoded and hashed, then exchanged between the participants. A special token is used to represent the hash of the empty list. If the hash values agree, then both databases are already synchronized and the protocol terminates. If one hash is nonempty and one is empty, then the protocol terminates and the participant with the nonempty list knows they must transfer the contents of the list. If both hashes are nonempty and differ, then the range of hash values is split into two

```

Input : connection with peer  $P$ , local database  $D$ , range  $[U, V]$  of hash
          values
Result : synchronize data with hash values in  $[U, V]$  with peer  $P$ 
 $S \leftarrow$  the set of hashes of data items in  $D$  in the range  $[U, V]$ 
 $h_1 \leftarrow \text{Hash}(S)$ 
Send ( $P, h_1$ )
 $h_2 \leftarrow \text{Receive}(P)$ 
if  $h_1 = h_2$  then return
if  $V - U > r_{min}$  then
    |  $M \leftarrow \lfloor \frac{U+V}{2} \rfloor$ 
    | Synchronize ( $[U, M]$ )
    | Synchronize ( $[M + 1, V]$ )
else
    | Send ( $P, S$ )
    |  $S' \leftarrow \text{Receive}(P)$ 
    | foreach  $h \in S - S'$  do
    |   | find  $x$  where  $\text{Hash}(x) = h$ 
    |   | Send ( $P, x$ )
    | end
    | foreach  $h \in S' - S$  do
    |   | add Receive ( $P$ ) to  $D$ 
    | end
end

```

Figure 6.3.1: Synchronize

equal subranges, L to M and M to H . The protocol is then applied recursively on these two subranges. (We could also use more than two subranges, of course.) If n is the total number of statements known by both sides, and m is the maximum number of statements known by one side that are not known by the other, then an update takes $O(m \log n)$ steps.

Transmitting one large batch of data is often much faster than performing several rounds of communication to determine which parts of the data should be sent. By communicating the number of statements that were hashed at each exchange, the implementation can switch to the naive protocol when the number falls beneath a threshold. In experiments we found the optimal threshold to be approximately 1200 children for connections between computers in our lab and a laptop off campus. The network module of BCIC uses the naive protocol on databases of size 1200 or smaller, and uses the recursive protocol on larger databases.

6.3.2 Space

Another important issue for transmitting statements over the network is space efficiency. The way we have formally presented BCIC requires much duplication between predicates. For example, programs are represented as CIC terms. Every statement about a particular program must explicitly mention the entire program term. This simplifies the logic and presentation of our system but is unrealistic. It is unreasonable to require that every digitally signed statement about a program include the entire text of the program.

The general solution to this problem is hash functions. Instead of signing the entire program, the principal can sign a statement that includes a hash of the program. If the recipient already knows the program, there is no need to send the program again. This immediately raises questions, such as how do peers know which terms to send and which to merely hash?

Our implementation solves this problem by adding a new datatype to the database of facts that are synchronized. Instead of just statements in the logic that are passed back and forth during synchronization, CIC terms are also synchronized using the general

purpose synchronization algorithm in Section 6.3.1. In addition, CIC terms in predicates are replaced in the network module by hashes of those terms. Both peers engage in a protocol that exchanges hash values to determine which hashes appear in one peer and not the other. The peers then send the data objects with the hashes that the other peer does not have. The end result will be that both peers have all the CIC terms and statements known to either peer.

This modification only appears in the network module. The functions that interface the network module to the rest of the implementation expand all the hashes to full values. If for some reason a predicate has a hash to a CIC term that does not appear in the database, the predicate is ignored. Well-behaved implementations will never generate and send such predicates, but malicious ones might.

With this modification there is less communication overhead between peers, but even more efficiencies could be obtained. For example, some CIC terms mention subterms that have already been mentioned elsewhere. The two terms `S (S (S (S 0)))` and `even (S (S (S (S 0))))` will be hashed to different values and thus sent as separate data objects by the implementation, but share a common subterm `S (S (S (S 0)))` that could have been sent only once. Hash consing [Ers58] is an old technique developed for Lisp that gathers common subterms for space savings. We have done some work on a similar technique for compressing CIC terms but have not finished adding this feature to the implementation.

6.4 Certified inference engine

A certified implementation of BCIC is desirable for many reasons. Because BCIC is a reference monitor, it is a good area to focus verification efforts. Any mistake in BCIC could allow malicious programs to execute. In addition, we can hardly ask that users of BCIC such as plug-in developers use proof tools to verify their code without verifying BCIC itself. We must eat our own dog food.

By formalizing BCIC in Coq we were able to prove decidability of the logic with a high degree of confidence that the proof was correct. In addition, we could then use

Coq’s automatic program extraction to extract a certified decision procedure (and therefore a certified inference engine) from the proof.

Our strategy was to start with pure Datalog and prove that it is decidable. From there we modified the proof to apply to Binder. To formalize full BCIC we first created an extension mechanism for Binder and proved decidability, then instantiated the extension appropriately to make the extended Binder be equivalent to BCIC. An advantage of this approach is that we can instantiate the same extension mechanism with a variety of other logics without rewriting all the proofs. For example, we can instantiate the extension with LF [HHP93] instead of CIC and the proofs about extended Binder still hold.

An advantage of building our certified implementation in stages was that after each stage we had a working certified implementation of the logic up to that point. Not all applications need the full complement of distributed reasoning from Binder and proofs from CIC; such applications can use whichever certified implementation is most appropriate.

6.4.1 Datalog and Binder

Formalizing Datalog and Binder in Coq was relatively straightforward. Here is the definition of the syntax of Binder, using natural numbers to name constants, variables, predicates, and principals.

```

Inductive term : Set :=
| const : nat -> term
| var : nat -> term.

Inductive atomic : Set :=
| bare : nat -> list term -> atomic
| says : nat -> nat -> list term -> atomic.

Inductive form : Set :=
| clause : atomic -> list atomic -> form.

```

After defining substitutions, the semantics of derivations for both Datalog and Binder is defined by:

```

Inductive der : list form -> atomic -> Prop :=
| modus_ponens :
  forall (LF : list form)(F : form)(S : substitution),
    In F LF ->
    forallelts
      (fun x => der LF (multisubs_atomic S x)) (body F) ->
    der LF (multisubs_atomic S (head F)).

```

The main theorem we prove about Datalog and Binder is that derivability is decidable, which is stated:

```

Theorem der_dec:
  forall (LF : list form)(A : atomic),
    no_head_vars_form_list LF ->
    {der LF A} + {~ der LF A}.

```

The condition `no_head_vars_form_list LF` expresses the restriction that variables that appear in the heads of clauses in the policy must also appear in the body of the clause.

The notation $\{A\} + \{B\}$ used in `der_dec` indicates the `sumbool` operator, which has kind $Prop \rightarrow Prop \rightarrow Set$. An object of type $\{A\} + \{B\}$ is either a proof of A or a proof of B . We use $\{A\} + \{B\}$ instead of $A \vee B$ because of the way program extraction behaves. During program extraction, elements of a proof of kind *Prop* are erased, while elements of kind *Set* are not. The expression $A \vee B$ has kind $Prop \rightarrow Prop \rightarrow Prop$, so if our theorem were stated using it, program extraction would yield an empty program. The expression $\{A\} + \{B\}$ has kind *Set* rather than *Prop*, which means that it will not be ignored during program extraction. The contents of the left and right arguments, however, are in *Prop*, so they will be ignored. What this means is that extracting from the proof of this theorem will yield an algorithm that when given a policy and a goal, will return `left` or `right` as its decision. The extracted algorithm will not remember the proof about why its answer is correct as it computes, avoiding unnecessary runtime overhead.

Section 4.1.2 shows that there is a finite universe of ground atomic formulas S such that $\forall a. ((e, r) \vdash a \implies a \in S)$. In our formalization, we define a function to calculate S and prove that every derivation must be included in it.

The next step is to decide which elements of S are in fact derivable. In Coq we define an extension function `extend : list form  $\rightarrow$  list atomic  $\rightarrow$  list atomic` that when given a policy and a set of derivable ground atomic formulas, returns the new ground atomic formulas that are derivable in one step from the inputs. We prove that `extend` is sound and complete with respect to Datalog derivations. The soundness and completeness lemmas are encoded as:

Lemma sound :

```
forall (F : form)(LF : list form)(A : atomic)(LA : list atomic),
  In F LF -> ground_atomic_list LA = true ->
  In A (extend F LA) -> forallelt (der LF) LA ->
  der LF A.
```

Lemma complete :

```
forall (F : form)(LF : list form)(LA : list atomic)
  (s : substitution),
  no_head_vars_form_list LF -> ground_atomic_list LA = true ->
  In F LF ->
  forallelt (fun x => In (multisubs_atomic s x) LA) (body F) ->
  In (multisubs_atomic s (head F)) (extend F LA).
```

The function `extend` is monotonically increasing and bounded by S , so it must have a fixed point. The fixed point Ω of `extend` is exactly the set of ground atomic formulas that are derivable from r . Soundness shows that elements in Ω are derivable, and completeness shows that every derivable element is in Ω .

Binder is decidable in almost the same way as Datalog is. The proof follows the Datalog proof almost exactly except for extra cases in atomic formulas which are trivial.

6.4.2 Encoding BCIC

Instead of directly encoding BCIC with a deep embedding of CIC into Coq and then trying to prove theorems, we instead formalized a general extension mechanism for Binder. Given an extension set, the extended version of Binder allows terms to refer to extended objects.

$t ::=$	term:
c	<i>constant</i>
x	<i>variable</i>
e	<i>extended term</i>

The extension mechanism adds generic extension predicates that represent a typing relation, substitution, and checking of equality between elements of the extended set. In addition, decision procedures for these predicates must be provided. Given these parameters, we prove that the extended version of Binder is decidable. By proving properties about Binder with the extension mechanism, we can instantiate the extension mechanism in different ways without re-proving anything about Binder. Each instantiation of the extension mechanism only requires creating a decision procedure for the extension type. We will encode BCIC by instantiating the extension mechanism with CIC terms.

For the extension mechanism, we take the extension set and other auxiliary proofs to be parameters of the extension. We do this in Coq by using structured environments. For example, the following code fragment shows how the extension set `extset` and a decision procedure for equality between elements of `extset` is defined.

```
Section ExtendedBinder.
  Variable extset : Set.
  Variable eq_extset_dec :
    forall (E1 E2 : extset), {E1 = E2}+{E1 <> E2}.
  .
  . (proofs)
  .
End ExtendedBinder.
```

The rules for derivation in extended Binder are:

```

Inductive der : list form -> atomic -> Prop :=
| modus_ponens :
  forall (LF : list form)(F : form)(S : substitution),
    In F LF ->
    forallelts atomic (fun x => der LF (multisubs_atomic S x)) (body F) ->
    der LF (multisubs_atomic S (head F))
| extension_der1 :
  forall (LF : list form)(K1 K2 : kthing),
    In K1 (extset LF) ->
    In K2 (extset LF) ->
    exttype K1 K2 ->
    der LF (extbare (kident K2))
| extension_der2 :
  forall (LF : list form)(K1 K2 K3 : kthing),
    In K1 (extset LF) ->
    In K2 (extset LF) ->
    In K3 (extset LF) ->
    extdot K1 K2 K3 ->
    der LF (extbare (kident K1)) ->
    der LF (extbare (kident K3)).

```

Besides proving the decidability of extended Binder, we prove a small result about conservativity. An extension is conservative if for all `extbare(kident K2)` derivable from the policy there exists `K1` in the extension set with `exttype K1 K2`. For policies without `extbare` in the heads of clauses, we prove all extensions are conservative. Of course if a policy is allowed to directly conclude `extbare(kident K)` as the head of a clause with no body, then the theorem would not hold. For the case of BCIC, conservativity means that by mixing Binder and CIC we do not somehow make the logic of CIC inconsistent and able to prove anything.

6.4.3 CIC as an extension

To create BCIC we instantiate the extension mechanism. The extension set is defined to be CIC terms from a fixed environment e . Given a BCIC policy r we can extract

all the CIC terms that appear in r . Extending this set to be closed under various operations (defined in Section 2.2.3) gives us $U(e, r)$, which is **extset**.

In our formalization of BCIC, we do not formalize CIC itself. Instead we assume there is an external type of CIC terms and signatures, an external typing judgment over CIC terms, an external proof that the typing judgment is decidable, and a function that extracts all the identifiers from a signature. Our instantiation of the Binder extension uses these external types and functions to model BCIC. During program extraction, places where these external items are used are converted to function calls of undefined names. Our implementation of BCIC instantiates these datatypes and functions as hand-coded OCaml routines that communicate directly with Coq running as a subprocess through Unix named pipes. It should also be possible to include the logical type-checking core of Coq directly in the implementation but we have not done so.

6.4.4 Alternate encodings

There are many possible ways to encode BCIC in Coq. An attractive option is to somehow encode the CIC part of BCIC using Coq directly. For example, we can imagine encoding BCIC terms as follows:

```
Inductive term : Type :=
| cicterm : forall (T : Type), T -> term.
```

We give **term** kind *Type* instead of *Set* because Coq requires definitions containing *Type* to be of kind *Type*. This definition can encode CIC objects of any type. It seems as though it can encode its own type, but if this is attempted Coq will give a universe inconsistency error. This is reassuring because otherwise we would have shown that Coq is inconsistent and can prove anything [Coq86].

The problem with this encoding is that program extraction is impossible. Elements of kind *Type* are removed during program extraction, so no extracted program with terms defined in this way will be able to reason about CIC terms. Another problem with this

encoding is that the CIC signature to use cannot be controlled. Choosing which signature to use for proofs is critically important when transmitting proofs between untrusted parties.

6.4.5 Extracted code

The specification of Datalog took 82 lines of Coq, Binder took 85 lines, and BCIC took 137 lines. Proving decidability took 6515 lines for Datalog, 7336 lines for Binder, and 9932 lines for BCIC. The extracted algorithms were small; Datalog was 388 lines of OCaml code, of which 50 were redundant definitions of standard data structures. Binder extracted to 487 lines, and BCIC extracted to 678 lines of OCaml.

6.5 Implementation of some of the examples

The examples we have presented in Chapter 3 are written and work in our implementation of BCIC. In order to flesh out the examples, we have created proof generators for the first two examples. Given a target program, the proof generators analyze the program and construct CIC proof terms for it. For the example of Section 3.1, proof terms establish that strings of audit requirements imply `calls_audited` properties. For the example of Section 3.2, proof terms yield not only `trusts_audited` properties but also well-typing.

Further, in order to execute the programs that have been analyzed, we have created a simple run-time environment for these two examples. The run-time environment for the first example, auditing function calls, includes several basic data types, functions for dealing with these types, and several library functions for operations such as file access. For the second example, the trust type system, the run-time is basically a typechecker and a λ -calculus interpreter.

The code samples in our implementation come from Scheme programs that do not use many Scheme language constructs. Going further, one may attempt to incorporate more of the Scheme language. For the example of Section 3.2, which requires static typing, a language such as ML may be a better match than Scheme. In this context, auditing may

also be applied to special language features that allow a program to break the type system in a potentially unsafe way (e.g., *Obj.magic* in OCaml).

6.6 Efficiency

Throughout the development of our implementation we have preferred to focus on the correctness of our implementation rather than on efficiency, with the caveat that we have always tried to make choices that avoid any gross inefficiencies. For example, the extracted implementation of the logic module uses a simple bottom-up evaluation that is easy to prove correct rather than a tricky top-down tabled resolution. However, at each stage of the computation the existing database is explicitly matched against each clause of the policy. Another way of structuring the algorithm is to compute every possible ground instantiation of every clause, then match this set using equality with the database of facts. That way would have been simpler to prove correct but would give unacceptable performance.

The extracted code was not particularly tuned for efficiency but neither was it extremely inefficient. Because the **extend** function is repeatedly evaluated to reach a fixed point during the decision procedure, the efficiency of **extend** largely determines the efficiency of the overall algorithm. We wrote the definition of **extend** by hand and then refined it as the proof development proceeded. During extraction, this version of **extend** is translated directly from Coq syntax to OCaml.

Our version of **extend** is inefficient in some ways. For example it checks for duplicate entries by linearly traversing a list rather than using a hash table. In other ways it is efficient; it matches clauses against ground atomic formulas using a simple unification algorithm and keeps track of variable bindings. A less efficient (but still correct) algorithm could expand every clause into the set of all possible ground instances of the clause before matching, yielding an exponential slowdown. Such an algorithm would have much simpler proofs but would be unacceptably slow. Although we have not done a formal analysis of the performance of our extracted code, our prototype implementation has returned results quickly for all the security policy examples we have tried.

An important aspect of efficiency is the time Coq takes to evaluate typing judgments in CIC. We know that CIC typing is decidable, and Coq for most judgments is relatively efficient, but our design makes no allowance for typing judgments that take a lot of computational power. Such judgments can arise when using proof-by-computation with inefficient computations. If a property is defined as a CIC function over programs, then checking that property amounts to evaluating the function. In CIC the function is guaranteed to terminate, but there is no guarantee of a time or space bound. Our implementation punts on this issue; if Coq takes too long to return a result for a typing judgment, we assume the answer was “no” and continue. This will always be a safe thing to do with regard to the security policy; no malicious program will be able to run because of this detail.

In our implementation the time BCIC takes to reach a decision about a query is dominated by calls to Coq for CIC typing. Because our implementation caches calls to Coq, we time examples both with and without a full cache once all network synchronization has been completed. With an empty CIC cache, queries to the examples from Chapter 3 all take more than 3 seconds and less than 10 seconds to be answered. Once the cache is full and all CIC typing judgments have been recorded, new queries take less than 1 second for all the examples.

Adding extra useless facts to the example policies does cause some slowdown after many facts have been added. For example, adding 1000 facts to the university super-computer access control example makes a query take two seconds instead of less than one second. Adding many useless CIC terms to a policy is much worse. Because BCIC tries to typecheck every combination of CIC terms and typechecking is slow, adding even a few extra CIC terms can cause a slowdown. For example, a simple example about even numbers takes 0.464s with 2 CIC terms, 2.414s with 4 terms, and 8.779s with 8 terms. Doubling the number of terms approximately quadruples the time.

Chapter 7

Discussion

7.1 Experience

During the course of our work we had the opportunity to deal with proofs about programs at many different levels. At the most obvious level, BCIC is designed to reason about proofs and so we have considered proofs as *data blocks* that are passed around and analyzed. We have also generated proofs for examples, including source code level examples such as the trust type system for λ -calculus in Section 3.2, and proofs about machine code instrumentation in Section 3.3. Finally, a significant portion of our implementation was created by automatic extraction from Coq proofs of decidability of BCIC. We have thus dealt with proofs as producers and consumers.

It is surprisingly hard to get correct and finished proofs about programs. The devil is in the details. The proofs themselves are usually mathematically trivial but require a large amount of bookkeeping. Getting the bookkeeping and auxiliary functions and lemmas stated correctly requires some amount of programming skill. Data structures must be designed correctly to give the right information in the right place. It is important that one test these functions and lemmas as they are built up, to avoid wasted effort. This shares some similarity with unit testing during programming. There is also a crucial dependence

on abstraction; knowing when and how to use abstraction is important for keeping proof complexity manageable.

The only mathematically tricky parts of the proofs we developed occurred in proofs about reflection. The tricky bits occurred around proofs on equalities. Equality in CIC is a predicate that is defined polymorphically. This makes reasoning about equality between terms of potentially different types difficult; the statement $A=B$ is not well-typed itself if A and B are not the same type. In addition, one cannot normally reason about proof objects directly in CIC because they are opaque. Using some fancy reasoning about dependent equality it is possible to do so in some cases, and in particular it is possible to reason about the equality of proof objects for reflective proofs. It should be noted that these results only need to be proven once and are specific to showing the correctness of proof by computation in BCIC.

Proving the termination of all functions defined in CIC was tedious and required some mathematical knowledge. Most simple functions we defined were obviously recursive because they were defined in a structurally recursive way. Other functions that recursed in different ways required separate proofs of well-foundedness of the relation over the inputs. These proofs were usually simple but may not have been obvious. It would have been helpful to have had a better library of existing well-founded relations and ways of combining existing well-founded relations on data structures into new compound data structures with a well-founded relation.

Our experience with the Coq extraction mechanism in creating the implementation of BCIC was entirely positive. After months of effort proving the correct theorem, a single command extracted a working complete OCaml program in seconds. We still had to connect this output with other components of our system but had little difficulty with this.

Proving anything interesting about programs is hard, and proving anything at all about programs automatically is also hard. This is not a new observation. As much as we would like to say that anyone can create proofs and plug them into a system like BCIC, we

cannot. Proof efforts must be targeted and specific. For the implementation of BCIC, we targeted the logic engine, the most obvious place to apply proof efforts.

Another consequence of working with formalized proofs is that we now have grave misgivings about traditional human-readable proofs. We no longer trust such proofs as we once did. Unformalized proofs rely on too many unwritten assumptions, leaps of logic, and unproven technical lemmas. Human-readable proofs are an effective means of communication between two well-educated mathematicians, but it is not a valid proof method. If the parties cannot rely on the honesty or abilities of each other, verifying the proof demonstrates nothing.

It is true that if a traditional proof is verified by the hand of a skilled mathematician there are unlikely to be serious errors in the proof. The errors are more likely to be simple cases that are overlooked or typographical errors that are trivial to fix. For the purposes of advancing human knowledge there is no great harm in these errors. For the purposes of verifying program correctness, these errors are all *bugs* that could affect the truth of the result, or even perhaps be exploited somehow in a program that is generated from the proof.

One might reply that machine checkable proofs are hardly worthy of trust. It is too easy to prove the wrong thing. In addition, proofs often require axioms or assumptions that might in fact be false and so invalidate the proof. Perhaps by not working out all the mind-numbing details in a proof, the author of a hand-written proof is more likely to keep the big picture in mind and not make substantive mistakes. We do not believe this is true. The hard part of generating machine checkable proofs is generally not formalizing the property to prove but in actually proving it. One can prove the wrong thing by hand, too. When machine checkable proofs require axioms or assumptions they are at least stated explicitly, unlike proofs by hand which can leave critical technical lemmas unstated and unproved.

7.2 Future directions

One of the limitations of the version of BCIC presented here is that there can only be one CIC environment in a policy. This means that all principals must agree ahead of time which CIC environment to use for proofs and theorems, then stick with it. It is not hard to imagine that even in a situation where there is general agreement on which environment to use, there are occasional changes and updates made to the commonly agreed environment. More generally, code recipients might wish to reason about properties drawn from multiple CIC environments. Currently the user must specify the environment explicitly.

It would be much nicer if BCIC could refer to multiple CIC environments in a modular way. Any given CIC term only makes sense in a particular environment, so there are many issues related to having multiple environments. This is especially true if a single CIC term must make sense in multiple environments. This could be the case, for example, for a term representing the program. It should make sense in every environment that defines a property about programs.

We have not yet formalized in Coq the restriction on policies given in Section 4.2.3, nor does the implementation check for this restriction. The implementation merely attempts the decision algorithm, computing $U(e, r)$ as if it were finite, then halts if a fixed point has not been reached in a reasonable time. An obvious improvement that we could make to BCIC would be to formalize the policy restriction and then extract this check to the implementation to prevent denial-of-service attacks by malicious principals. This would also involve encoding the computation of $U(e, r)$ to be able to specify correctness of the policy restriction. This is not a trivial task, since we would have to encode the typing rules of CIC in CIC. The Calculus of Constructions has been encoded inside CIC [Bar97]; progress has been made on encoding all of CIC within CIC but work remains to be done [BW97].

Another extension to BCIC would be to allow function symbols. As it is, BCIC allows definitions of extra data structures and operations over those data structures through the reflective proof mechanism described in Chapter 5. Another possibility is to instead allow function symbols so that data can be encoded directly in terms of the policy rather

than going through CIC. This would allow policies that look more like Prolog programs. Our solution has the advantage that the decision procedures for the data structures can be encoded nicely within the policy. If policies had function symbols, it would be much harder to determine which policies are terminating and which ones are not, especially given that statements are imported from other principals who might be malicious. Even so, allowing function symbols could be beneficial to some types of policies, particularly data intensive ones. There are ways of determining when a particular Prolog program will terminate [SS94], so it might be worthwhile investigating how to include function symbols in BCIC.

It is entirely reasonable that code producers and program analysts would want to use a different logical framework than CIC and Coq. It is possible to recast BCIC as a combination of Binder and some other logical framework, but that requires many changes and redoing all the analysis we have done for BCIC. Another possibility is to add a layer of abstraction and encode the other logical framework in CIC. Proofs in the other framework would then be proofs in a CIC environment. This encoding step might be efficient for some types of systems and inefficient for other ones; it might be worthwhile investigating how such an encoding would work and when it would be practical.

At the moment our implementation acts as a reference monitor and can report “yes” or “no” answers to queries. It cannot actually load and execute programs itself. The implementation could be extended so that it understands executable file formats and can load, analyze, and run programs itself. Ideally such a facility would be powerful enough to be useful while remaining cross-platform and simple enough to be reasonably likely to be free of bugs. This has proven to be somewhat complicated because every operating system has its own mechanisms for loading and executing code, not to mention that different processors all use different machine languages, and even the same operating system can provide many different possible runtime environments.

	Policy	CIC Env.	Proofs	Dec. Alg.	Correctness	Runtime
Trust annotations	X					
Cell-phone game	X					
University supercomputer	X					
Auditing function calls	X	X	X		X	X
Trust in the λ -calculus	X	X	X	X	X	X
Instrumentation	X	X	X	X	X	
Simply-typed λ -calculus	X	X	X	X		
Allowed free variables	X	X	X	X		

Figure 7.3.1: Summary of the contributions of the examples

7.3 Summary of contributions

We have designed a system for authorizing code based on a combination of proofs and assertions. Our system combines a flexible Binder-like policy logic with the capability to talk about CIC terms which can represent theorems, proofs, data, functions, and decision procedures. Our system can reason about proof-carrying code by checking proofs that code producers provide; in addition it can use reflection to allow decidable properties to be automatically proved. We have implemented the system; we can test fully developed examples and see how they work. We have developed several large examples to help refine the system and demonstrate its utility. Figure 7.3.1 shows a summary of our contributions for each example. We have presented several results about our system, including its decidability which we use to produce a certified logic module for the implementation. In addition along the way we produce certified implementations of Datalog and Binder.

Appendix

Appendix A

Selected proof scripts

A.1 Decidability of BCIC

This section includes the complete proof script for decidability of BCIC. We include this proof script to illustrate what types of Coq commands are needed to define a policy logic and prove its decidability. The structure of the proof scripts for Datalog, Binder, and BCIC are substantially similar.

The basic structure of the proof is as follows. The main result is decidability, which needs three parts: soundness, completeness, and termination. The algorithm itself involves repeated matchings and extending a finite database of ground atomic formulas that are derivable from the input. The soundness strand is reasonably straightforward, it just requires lots of work to build up the complete result. Completeness is a little more tricky, it involves many more technical lemmas. In a sense soundness looks at the algorithm “going forwards”, and completeness looks at it “going backwards”. Soundness and completeness follow the algorithm itself closely, building up in stages, first matching terms, then atoms, then clauses. Termination is different. Termination involves defining supplementary functions that extract all the terms, arities, and predicates in the input, and then generating all possible ground atomic formulas using these. There is a proof that everything the algorithm produces must be in this set. A separate strand proves that the output of the algorithm is

always ground, using many lemmas about free variables. Finally there are lemmas about how the database grows bigger monotonically, then by the bound of the universe of possible ground atoms it must reach a fixpoint.

The proof script is organized with initial definitions of the syntax at the start. It then defines some auxiliary functions and predicates and proves lemmas about them. Next a section defines substitution and proves properties about substitution that will be needed later. The matching algorithm is then defined. The matching algorithm matches clauses against lists of ground atomic formulas to find substitutions that make every element of the clause's body appear in the database. The matching algorithm is used to create the extension function that extends the database of facts by one step of reasoning.

Facts are then proved about matching, including soundness and completeness. Next properties and lemmas relating to free variables are defined and proved in order to deduce that the head of a matching clause is always ground. The proof script then defines predicates for determining when atoms, arities, and predicate names appear in a program. These are then used to create functions that extract every atom, arity, and predicate name that appears in the policy. Functions that construct every possible ground atomic formula using these extracted elements are defined and proved to be complete, corresponding to Lemma 4.8.

The extension function is then used in the definition of a decision function. Facts about repeated application of the extension function are proved, in particular termination based on a lemma that says the sequence of databases is nondecreasing and bounded. The soundness, completeness, and termination of the decision function come together to show that the decision function is indeed a correct decision procedure, so BCIC is decidable. Thus the script proves Theorem 4.9.

```

Require Import List.
Require Import ListSet.
Require Import Bool.
Require Import Le.
Require Import Lt.
Require Import Plus.
Require Import Minus.
Require Import Compare_dec.

(*****)
(* Main Binder data structures *)
(*****)

(* We can decide when 2 numbers are equal, this is nicer definition than stdlib
*)
Definition eq_nat_dec :
  forall (x y:nat),
    {x=y}+{x<>y}.
  intros x y; decide equality x y.
Defined.

Section ExtendedBinder.
  Variable kthing : Set.
  (* Type judgment of external things *)
  Variable extttype : kthing -> kthing -> Prop.
  (* Dot judgment of external things *)
  Variable extdot : kthing -> kthing -> kthing -> Prop.
  (* We need to be able to check when things match, decide equality of external
  things *)
  Variable eq_kthing_dec : forall (K1 K2 : kthing), {K1 = K2} + {K1 <> K2}.

  Inductive term : Set :=
    | ident : nat -> term
    | kident : kthing -> term
    | var : nat -> term.

  Inductive atomic : Set :=
    | bare : nat -> list term -> atomic
    | says : nat -> nat -> list term -> atomic
    | extbare : term -> atomic
    | extsays : nat -> term -> atomic.

  Inductive form : Set :=
    | clause : atomic -> list atomic -> form.

  Variable extset : list form -> list kthing.

  Definition extract_kthing_term (T : term) : list kthing :=
    match T with
    | ident i => nil
    | kident k => k :: nil

```

```

    | var x => nil
end.

Fixpoint extract_kthing_term_list (LT : list term) : list kthing :=
  match LT with
  | nil => nil
  | H :: T => extract_kthing_term H ++ extract_kthing_term_list T
  end.

Definition extract_kthing_atomic (A : atomic) : list kthing :=
  match A with
  | bare p lt => extract_kthing_term_list lt
  | says w p lt => extract_kthing_term_list lt
  | extbare k => extract_kthing_term k
  | extsays w k => extract_kthing_term k
  end.

Fixpoint extract_kthing_list_atomic (LA : list atomic){struct LA} : list kthing :=
  match LA with
  | nil => nil
  | hd :: tl =>
    (extract_kthing_atomic hd) ++ (extract_kthing_list_atomic tl)
  end.

Definition extract_kthing_form (F : form) : list kthing :=
  match F with
  | clause H B =>
    (extract_kthing_atomic H) ++ (extract_kthing_list_atomic B)
  end.

Fixpoint extract_kthing_list_form (LF : list form){struct LF} : list kthing :=
  match LF with
  | nil => nil
  | hd :: tl =>
    (extract_kthing_form hd) ++ (extract_kthing_list_form tl)
  end.

(* Extract head/body of formula clause *)
Definition head (F:form) : atomic :=
  match F with
  | clause H B => H
  end.

Definition body (F:form) : list atomic :=
  match F with
  | clause H B => B
  end.

(* Define propositions over lists *)
Definition foralllet (A : Set)(f : A -> Prop)(l : list A) :=

```

```

    (forall x:A, In x l -> f x).
Definition existselt (A : Set)(f : A -> Prop)(l : list A) :=
  (exists x, In x l /\ f x).

(*****)
(* Substitution *)
(*****)

(* Lift a function over terms to a function over atomic formulas *)
Definition lift_term (F : term -> term)(A : atomic) : atomic :=
  match A with
  | bare p lt =>
    bare p (map F lt)
  | says w p lt =>
    says w p (map F lt)
  | extbare k =>
    extbare (F k)
  | extsays w k =>
    extsays w (F k)
  end.

(* Lift functions from atomics to formulas *)
Definition lift_atomic (F : atomic -> atomic)(FM : form) : form :=
  match FM with
  | clause A LA =>
    clause (F A) (map F LA)
  end.

(* We only substitute in identifiers, never variables *)
Inductive rval : Set :=
  | rvident : nat -> rval
  | rvkident : kthing -> rval.

Definition subs_term (sb : nat*rval)(T1 : term) : term :=
  match sb with
  | (x, ri) =>
    match T1 with
    | ident y => T1
    | kident k => T1
    | var y => if eq_nat_dec x y then
      match ri with
      | rvident i => ident i
      | rvkident k => kident k
      end
    else T1
    end
  end.

(* Define substitution over atomics and formulas and lists of those *)
Definition subs_atomic (sb : nat*rval)(A : atomic) : atomic :=

```

```

    lift_term (subs_term sb) A.
Definition subs_form (sb : nat*rval)(F : form) : form :=
    lift_atomic (subs_atomic sb) F.
Definition subs_term_list (sb : nat*rval)(LT : list term) : list term :=
    map (subs_term sb) LT.
Definition subs_atomic_list (sb : nat*rval)(LA : list atomic) : list atomic :=
    map (subs_atomic sb) LA.

(* A substitution is a list of single substitutions *)
Definition substitution := list (nat*rval).

Definition multisubs_term (sbs : substitution)(T : term) : term :=
    fold_left (fun t => fun sb => subs_term sb t) sbs T.
Definition multisubs_atomic (sbs : substitution)(A : atomic) : atomic :=
    lift_term (multisubs_term sbs) A.
Definition multisubs_form (sbs : substitution)(F : form) : form :=
    lift_atomic (multisubs_atomic sbs) F.
Definition multisubs_term_list (sbs : substitution)(LT : list term) : list term :=
    map (multisubs_term sbs) LT.
Definition multisubs_atomic_list (sbs : substitution)(LA : list atomic) : list atomic :=
    map (multisubs_atomic sbs) LA.

(*****
(* Derivability *)
*****)

(* Derivability (this is the semantic meaning of datalog) *)
Inductive der : list form -> atomic -> Prop :=
| modus_ponens :
    forall (LF : list form)(F : form)(S : substitution),
      In F LF ->
      forallelts atomic (fun x => der LF (multisubs_atomic S x)) (body F) ->
      der LF (multisubs_atomic S (head F))
| extension_der1 :
    forall (LF : list form)(K1 K2 : kthing),
      In K1 (extset LF) ->
      In K2 (extset LF) ->
      exttype K1 K2 ->
      der LF (extbare (kident K2))
| extension_der2 :
    forall (LF : list form)(K1 K2 K3 : kthing),
      In K1 (extset LF) ->
      In K2 (extset LF) ->
      In K3 (extset LF) ->
      extdot K1 K2 K3 ->
      der LF (extbare (kident K1)) ->
      der LF (extbare (kident K3)).

Variable exttype_dec : forall (K1 K2 : kthing), {exttype K1 K2}+{~exttype K1 K

```

```

2}.
Variable extdot_dec : forall (K1 K2 K3 : kthing), {extdot K1 K2 K3}+{~extdot K
1 K2 K3}.

(*****)
(* Utility sorts of functions *)
(*****)

(* Deciding equality between terms, atoms, etc. *)
Definition eq_term_dec (A1 A2 : term) : {A1=A2}+{A1<>A2}.
  intros A1 A2.
  decide equality A1 A2; apply eq_nat_dec.
Defined.

Definition eq_atomic_dec (A1 A2 : atomic) : {A1=A2}+{A1<>A2}.
  intros A1 A2. decide equality A1 A2.
  decide equality 1 10. apply eq_term_dec. apply eq_nat_dec.
  decide equality 1 10. apply eq_term_dec. apply eq_nat_dec.
  apply eq_nat_dec.
  apply eq_term_dec.
  apply eq_term_dec.
  apply eq_nat_dec.
Defined.

Definition eq_atomic_list_dec (A1 A2 : list atomic) : {A1=A2}+{A1<>A2}.
  intros A1 A2. decide equality A1 A2.
  apply eq_atomic_dec.
Defined.

(* Tack on a substitution to every one of a list of substitutions. *)
(* Given list s and list of lists ls, append s to every list in ls *)
Definition map_append (A : Set)(s : list A)(ls : list (list A)) :=
  map (fun x => s ++ x) ls.
Implicit Arguments map_append [A].

(* Flatten a list of lists into list *)
Fixpoint flatten (A : Set)(lls : list (list A)) {struct lls} : list A :=
  match lls with
  | nil => nil
  | hd :: tl => hd ++ (flatten A tl)
  end.
Implicit Arguments flatten [A].

(* Iterating function application *)
Fixpoint iter (A:Set)(n:nat)(F:A->A)(x:A){struct n} : A :=
  match n with
  | 0 => x
  | S p => F (iter A p F x)
  end.
Implicit Arguments iter [A].

```

```

Lemma forall_dec :
  forall (A:Set)(P:A->Prop)(L:list A),
    (forall (x:A), {P x}+{~P x}) ->
      {forallelts A P L}+{~forallelts A P L}.

```

Proof.

```

  intros A P L H.
  induction L.
  left. unfold forallelts.
  intros x H2.
  simpl in H2; contradiction.
  elim IHL; intro IHL2; clear IHL.
  assert ({P a}+{~ P a}).
  apply H.
  elim H0; intro H1; clear H0.
  left. unfold forallelts.
  simpl. intro x.
  intro H2.
  elim H2; intro H3; clear H2.
  rewrite <- H3. assumption.
  unfold forallelts in IHL2.
  apply IHL2. assumption.
  right. unfold not.
  intro H2.
  unfold forallelts in H2.
  assert (In a (a :: L)).
  simpl. auto.
  assert (P a).
  apply H2. assumption.
  contradiction.
  right.
  unfold not.
  intro H2.
  unfold forallelts in H2.
  unfold forallelts in IHL2.
  apply IHL2.
  intros x H3.
  apply H2.
  simpl. right; assumption.

```

Qed.

(* Concept of ground *)

```

Definition ground_term (T : term) : bool :=
  match T with
  | ident i => true
  | kident k => true
  | var i => false
  end.

```

```

Fixpoint forall_bool (A : Set)(F : A -> bool)(L : list A) {struct L} : bool :=
  match L with

```



```

    | nil => true
    | hd :: tl => andb (F hd) (forall_bool A F tl)
end.
Implicit Arguments forall_bool [A].

Lemma forall_bool_in :
  forall (A : Set)(L : list A)(F : A -> bool)(x : A),
    forall_bool F L = true ->
      In x L ->
        F x = true.
Proof.
  intros A L F x H1 H2.
  induction L.
  simpl in H2. contradiction.
  simpl in H2.
  simpl in H1.
  elim H2; intro H3; clear H2.
  rewrite <- H3.
  induction (F a). trivial. simpl in H1. discriminate.
  apply IHL.
  induction (forall_bool F L). trivial.
  induction (F a). simpl in H1. discriminate.
  simpl in H1. discriminate. assumption.
Qed.

Lemma in_forall_bool :
  forall (A : Set)(P : A -> bool)(L : list A),
    (forall x, In x L -> P x = true) ->
      forall_bool P L = true.
Proof.
  intros.
  induction L.
  simpl. trivial.
  assert (P a = true).
  apply H. simpl. left; trivial.
  simpl.
  induction (P a).
  simpl.
  apply IHL.
  intros.
  apply H.
  simpl. right; assumption.
  discriminate.
Qed.

Fixpoint exists_bool (A : Set)(F : A -> bool)(L : list A) {struct L} : bool :=
  match L with
  | nil => false
  | hd :: tl => orb (F hd) (exists_bool A F tl)
  end.
Implicit Arguments exists_bool [A].

```

```

Definition ground_term_list (LT : list term) : bool :=
  forall_bool ground_term LT.

Definition ground_atomic (A : atomic) : bool :=
  match A with
  | bare P LT => ground_term_list LT
  | says W P LT => ground_term_list LT
  | extbare K => ground_term K
  | extsays W K => ground_term K
  end.

Definition ground_atomic_list (LA : list atomic) : bool :=
  forall_bool ground_atomic LA.

Definition var_occurs_term (x : nat)(T : term) : bool :=
  match T with
  | ident i => false
  | kident i => false
  | var i => if eq_nat_dec i x then true else false
  end.

Definition var_occurs_term_list (x : nat)(LT : list term) : bool :=
  exists_bool (var_occurs_term x) LT.

Definition var_occurs_atomic (x : nat)(A : atomic) : bool :=
  match A with
  | bare p LT => var_occurs_term_list x LT
  | says w p LT => var_occurs_term_list x LT
  | extbare K => var_occurs_term x K
  | extsays W K => var_occurs_term x K
  end.

Definition var_occurs_atomic_list (x : nat)(LA : list atomic) : bool :=
  exists_bool (var_occurs_atomic x) LA.

(*****)
(* Matching *)
(*****)

(* Matching is like unification, but just one way *)
(* match is reserved, use match_ *)
(* Output is list of ways to match, each match is a list of substitutions. *)
(* Subs are assignments for variables in T1 to make it like T2.
   Subs never put in new variables. *)

Definition match_term (T1 T2 : term) : option substitution :=
  match T1 with
  | ident i =>
    match T2 with
    | ident j => if eq_nat_dec i j then Some nil else None

```

```

      | kident j => None
      | var j => None
    end
  | kident i =>
    match T2 with
    | ident j => None
    | kident j => if eq_kthing_dec i j then Some nil else None
    | var j => None
    end
  | var i =>
    match T2 with
    | ident j => Some ((i, rvident j)::nil)
    | kident j => Some ((i, rvkident j)::nil)
    | var j => None
    end
  end
end.

```

```

Lemma match_term_sound :
  forall (T1 T2 : term)(s : substitution),
    ground_term T2 = true ->
    match_term T1 T2 = Some s ->
    multisubs_term s T1 = T2.

```

Proof.

```

  intros T1 T2 s H0 H1.
  induction T1; induction T2.
  unfold match_term in H1.
  induction (eq_nat_dec n n0).
  rewrite a.
  clear H1.
  induction s.
  simpl. trivial.
  simpl. induction a0. simpl. assumption.
  discriminate.
  simpl in H1. discriminate.
  simpl in H1. discriminate.
  simpl in H1. discriminate.
  simpl in H1.
  induction (eq_kthing_dec k k0).
  injection H1. intro H2. clear H1.
  rewrite <- H2.
  simpl.
  rewrite a. trivial.
  discriminate.
  simpl in H1. discriminate.
  simpl in H1.
  injection H1; intro H2; clear H1.
  rewrite <- H2. simpl.
  induction (eq_nat_dec n n). trivial.
  assert (n=n). trivial. contradiction.
  simpl in H1.
  injection H1; intro H2; clear H1.

```

```

rewrite <- H2. simpl.
induction (eq_nat_dec n n). trivial.
assert (n=n). trivial. contradiction.
simpl in H0. discriminate.
Qed.

Lemma subs_term_ident :
  forall (s : nat * rval)(i : nat),
    subs_term s (ident i) = ident i.
Proof.
  intros.
  induction s. simpl. trivial.
Qed.

Lemma subs_term_kident :
  forall (s : nat * rval)(i : kthing),
    subs_term s (kident i) = kident i.
Proof.
  intros.
  induction s. simpl. trivial.
Qed.

Lemma multisubs_term_ident :
  forall (s : substitution)(i : nat),
    multisubs_term s (ident i) = ident i.
Proof.
  intros s i.
  induction s. simpl. trivial.
  simpl. induction a. unfold subs_term. assumption.
Qed.

Lemma multisubs_term_kident :
  forall (s : substitution)(i : kthing),
    multisubs_term s (kident i) = kident i.
Proof.
  intros s i.
  induction s. simpl. trivial.
  simpl. induction a. unfold subs_term. assumption.
Qed.

Lemma multisubs_term_list_ident :
  forall (s : substitution)(i : nat)(L : list term),
    multisubs_term_list s ((ident i) :: L) = ident i :: multisubs_term_list s L.
Proof.
  intros s i L.
  induction s. simpl. trivial.
  simpl. induction a. unfold subs_term.
  assert (multisubs_term s (ident i) = ident i).
  apply multisubs_term_ident.
  rewrite H. trivial.
Qed.

```

```

Lemma multisubs_term_list_kident :
  forall (s : substitution)(i : kthing)(L : list term),
    multisubs_term_list s ((kident i) :: L) = kendit i :: multisubs_term_list s
      L.

```

Proof.

```

  intros s i L.
  induction s. simpl. trivial.
  simpl. induction a. unfold subs_term.
  assert (multisubs_term s (kident i) = kendit i).
  apply multisubs_term_kident.
  rewrite H. trivial.

```

Qed.

```

Lemma match_term_complete :
  forall (T1 T2 : term)(s : substitution),
    ground_term T2 = true ->
    multisubs_term s T1 = T2 ->
    (exists sbs, match_term T1 T2 = Some sbs).

```

Proof.

```

  intros T1 T2 s H0 H1.
  induction T1; induction T2; simpl in H0; simpl in H1; simpl.
  induction s.
  simpl in H1.
  injection H1; intro H2.
  exists (nil:substitution).
  induction (eq_nat_dec n n0). trivial. contradiction.
  induction a. simpl in H1. apply IHs. assumption.
  assert (multisubs_term s (ident n) = ident n).
  apply multisubs_term_ident.
  rewrite H in H1. discriminate.
  assert (multisubs_term s (ident n) = ident n).
  apply multisubs_term_ident.
  rewrite H in H1. discriminate.
  assert (multisubs_term s (kident k) = kendit k).
  apply multisubs_term_kident.
  rewrite H in H1. discriminate.
  assert (multisubs_term s (kident k) = kendit k).
  apply multisubs_term_kident.
  rewrite H in H1.
  injection H1; intro H2; clear H1.
  exists (nil:substitution).
  induction (eq_kthing_dec k k0).
  trivial. contradiction.
  discriminate.
  exists ((n, rvident n0)::nil).
  trivial.
  exists ((n, rvkident k)::nil).
  trivial.
  discriminate.

```

Qed.

```

(* Bit of subtlety: once we have partial subs, do the subs in the rest of LT1. *)
)
(* ALSO, this forces structural induction to be over LT2, otherwise can't be
guaranteed to be principal induction. *)
Fixpoint match_term_list (LT1 LT2 : list term) {struct LT2} : option substitutio
n :=
  match LT1 with
  | nil =>
    match LT2 with
    | nil => Some nil
    | _ => None
    end
  | hd1 :: tl1 =>
    match LT2 with
    | nil => None
    | hd2 :: tl2 =>
      match match_term hd1 hd2 with
      | None => None
      | Some s =>
        match match_term_list (multisubs_term_list s tl1) tl2 with
        | None => None
        | Some s2 => Some (s ++ s2)
        end
      end
    end
  end
end.

Lemma app_nil :
  forall (A : Set)(L : list A),
    L ++ nil = L.
Proof.
  intros A L.
  induction L. simpl. trivial.
  simpl. rewrite IHL. trivial.
Qed.

Lemma app_nil_left :
  forall (A : Set)(L : list A),
    nil ++ L = L.
Proof.
  intros. induction L; auto.
Qed.

Lemma elim_option :
  forall (A : Set)(T : option A),
    (exists s, T = Some s) \ / (T = None).
Proof.
  intros A T.
  induction T.
  left; exists a. trivial.

```

```

    right. trivial.
Qed.

```

```

Lemma length_multisubs_term_list :
  forall (LT : list term)(s : substitution),
    length (multisubs_term_list s LT) = length LT.

```

```

Proof.
  intros LT s.
  induction LT.
  simpl. trivial.
  simpl. rewrite IHLT. trivial.
Qed.

```

```

Lemma length_multisubs_atomic_list :
  forall (LA : list atomic)(s : substitution),
    length (multisubs_atomic_list s LA) = length LA.

```

```

Proof.
  intros LA s.
  induction LA.
  simpl. trivial.
  simpl. rewrite IHLA. trivial.
Qed.

```

```

Lemma multisubs_nested :
  forall (sbs : substitution)(n0 : nat)(n1 : rval)(LT1 : list term),
    multisubs_term_list sbs (multisubs_term_list ((n0, n1) :: nil) LT1)
    = multisubs_term_list ((n0, n1) :: sbs) LT1.

```

```

Proof.
  intros sbs n0 n1 LT1.
  induction LT1.
  simpl. trivial.
  induction a.
  assert (multisubs_term_list ((n0, n1) :: sbs) (ident n :: LT1) = ident n :: (m
    ultisubs_term_list ((n0, n1) :: sbs) LT1)).
  apply multisubs_term_list_ident.
  rewrite H.
  assert (multisubs_term_list ((n0, n1) :: nil) (ident n :: LT1) = ident n :: (m
    ultisubs_term_list ((n0, n1) :: nil) LT1)).
  apply multisubs_term_list_ident.
  rewrite H0.
  assert (multisubs_term_list sbs (ident n :: multisubs_term_list ((n0, n1) :: n
    il) LT1) =
    ident n :: multisubs_term_list sbs (multisubs_term_list ((n0, n1) :: nil) LT
    1)).
  apply multisubs_term_list_ident.
  rewrite H1.
  rewrite IHLT1. trivial.
  simpl.
  (* induction (eq_nat_dec n0 n). *)
  rewrite IHLT1. trivial.
  simpl.

```

```

    rewrite IHLT1. trivial.
Qed.

Lemma ground_term_list_ident :
  forall (n : nat)(LT : list term),
    ground_term_list (ident n :: LT) = true ->
      ground_term_list LT = true.
Proof.
  intros n LT H.
  unfold ground_term_list in H.
  unfold forall_bool in H.
  fold forall_bool in H.
  simpl in H.
  unfold ground_term_list. assumption.
Qed.

Lemma ground_term_list_kident :
  forall (n : kthing)(LT : list term),
    ground_term_list (kident n :: LT) = true ->
      ground_term_list LT = true.
Proof.
  intros n LT H.
  unfold ground_term_list in H.
  unfold forall_bool in H.
  fold forall_bool in H.
  simpl in H.
  unfold ground_term_list. assumption.
Qed.

Lemma multisubs_term_list_nil :
  forall (LT : list term),
    multisubs_term_list nil LT = LT.
Proof.
  intro LT.
  induction LT.
  simpl. trivial.
  simpl. rewrite IHLT. trivial.
Qed.

(* Structure match_term_list_sound as induction on length of LT1 *)
Lemma match_term_list_sound_aux :
  forall (n : nat)(LT1 LT2 : list term)(s : substitution),
    length LT1 = n ->
      ground_term_list LT2 = true ->
        match_term_list LT1 LT2 = Some s ->
          multisubs_term_list s LT1 = LT2.
Proof.
  induction n.
  induction LT1.
  induction LT2.

```



```

intros. simpl. trivial.
simpl. intros. discriminate.
simpl. intros. discriminate.
intros LT1 LT2 s H1 Hg H2.
induction LT1.
simpl in H1. discriminate.
clear IHLT1.
simpl in H1. injection H1; intro H3; clear H1.
induction LT2.
simpl. discriminate.
clear IHLT2.
(* OK, stupid crap taken care of. *)

induction a; induction a0; simpl in H2; try discriminate.

(* For ident ident case *)
assert (multisubs_term_list s (ident n0 :: LT1) = ident n0 :: multisubs_term_l
ist s LT1).
apply multisubs_term_list_ident.
rewrite H.
simpl in H2.
induction (eq_nat_dec n0 n1).
rewrite a.
assert (multisubs_term_list nil LT1 = LT1).
apply multisubs_term_list_nil.
rewrite H0 in H2.
simpl in H2.
assert (match_term_list LT1 LT2 = Some s).
rewrite <- H2.
induction (match_term_list LT1 LT2). trivial. trivial.
assert (multisubs_term_list s LT1 = LT2).
apply IHn. assumption. assumption. assumption.
rewrite H4. trivial.
discriminate.
(* kident kident case *)
assert (multisubs_term_list s (kident k :: LT1) = kident k :: multisubs_term_l
ist s LT1).
apply multisubs_term_list_kident.
rewrite H.
induction (eq_kthing_dec k k0).
rewrite a.
assert (multisubs_term_list nil LT1 = LT1).
apply multisubs_term_list_nil.
rewrite H0 in H2.
simpl in H2.
assert (match_term_list LT1 LT2 = Some s).
rewrite <- H2.
induction (match_term_list LT1 LT2). trivial. trivial.
assert (multisubs_term_list s LT1 = LT2).
apply IHn. assumption. assumption. assumption.
rewrite H4. trivial.

```

```

discriminate.
(* var ident case *)

set (mt:=match_term_list (multisubs_term_list ((n0, rvident n1) :: nil) LT1) L
T2).
fold mt in H2.
assert ((exists sbs, mt = Some sbs) \/ mt = None).
apply elim_option.
elim H. intro H4. clear H.
elim H4. intros sbs H5. clear H4.
rewrite H5 in H2.
injection H2. intro H6. clear H2.

assert (multisubs_term_list sbs (multisubs_term_list ((n0, rvident n1) :: nil)
LT1) = LT2).
apply IHn.
assert (length (multisubs_term_list ((n0, rvident n1) :: nil) LT1) = length LT
1).
apply length_multisubs_term_list.
rewrite H. assumption.
apply ground_term_list_ident with (n:=n1).
assumption.
assumption.
simpl.
rewrite <- H6.
assert (multisubs_term ((n0, rvident n1) :: sbs) (var n0) = ident n1).
simpl.
induction (eq_nat_dec n0 n0).
apply multisubs_term_ident.
assert (n0=n0). trivial. contradiction.
rewrite H0.
assert (multisubs_term_list sbs (multisubs_term_list ((n0, rvident n1) :: nil)
LT1)
= multisubs_term_list ((n0, rvident n1) :: sbs) LT1).
apply multisubs_nested.
rewrite <- H1.
rewrite H. trivial.
intro H4.
rewrite H4 in H2. discriminate.

(* var kident case *)
set (mt:=match_term_list (multisubs_term_list ((n0, rvkident k) :: nil) LT1) L
T2).
fold mt in H2.
assert ((exists sbs, mt = Some sbs) \/ mt = None).
apply elim_option.
elim H. intro H4. clear H.
elim H4. intros sbs H5. clear H4.
rewrite H5 in H2.
injection H2. intro H6. clear H2.

```

```

assert (multisubs_term_list sbs (multisubs_term_list ((n0, rvkident k) :: nil)
  LT1) = LT2).
apply IHn.
assert (length (multisubs_term_list ((n0, rvkident k) :: nil) LT1) = length LT
1).
apply length_multisubs_term_list.
rewrite H. assumption.
apply ground_term_list_kident with (n:=k).
assumption.
assumption.
simpl.
rewrite <- H6.
assert (multisubs_term ((n0, rvkident k) :: sbs) (var n0) = kident k).
simpl.
induction (eq_nat_dec n0 n0).
apply multisubs_term_kident.
assert (n0=n0). trivial. contradiction.
rewrite H0.
assert (multisubs_term_list sbs (multisubs_term_list ((n0, rvkident k) :: nil)
  LT1)
  = multisubs_term_list ((n0, rvkident k) :: sbs) LT1).
apply multisubs_nested.
rewrite <- H1.
rewrite H. trivial.
intro H4.
rewrite H4 in H2. discriminate.

```

Qed.

```

Lemma match_term_list_sound :
  forall (LT1 LT2 : list term)(s : substitution),
    ground_term_list LT2 = true ->
    match_term_list LT1 LT2 = Some s ->
    multisubs_term_list s LT1 = LT2.

```

Proof.

```

intros LT1 LT2 s H1 H2.
apply match_term_list_sound_aux with (n:=length LT1).
trivial. assumption. assumption.

```

Qed.

```

Fixpoint filter_out (A : Set)(L : list A)(P : A -> bool) {struct L} : list A :=
  match L with
  | nil => nil
  | hd :: tl =>
    let rest := filter_out A tl P in
    if P hd then rest else hd :: rest
  end.

```

Implicit Arguments filter_out [A].

```

(* Given a subs and a var name, remove all assignments for that var *)
Definition filter_out_n (s : substitution)(n : nat) : substitution :=

```

```

filter_out s (fun pr => match pr with (n0, v0) =>
  if eq_nat_dec n0 n then true else false
end).

(* Booleans are decidable *)
Lemma dec_bool :
  forall (A : bool),
    {A=true}+{A=false}.
Proof.
  intro A.
  induction A. left; trivial. right; trivial.
Qed.

Lemma ground_term_list_true :
  forall (a : term)(LT2 : list term),
    ground_term_list (a :: LT2) = true ->
    ground_term_list LT2 = true.
Proof.
  intros a LT2 H.
  unfold ground_term_list in H.
  simpl in H.
  assert (forall_bool ground_term LT2 = true).
  assert ({forall_bool ground_term LT2 = true}+{forall_bool ground_term LT2 = false}).
  apply dec_bool.
  elim H0; intro H4; clear H0. assumption.
  rewrite H4 in H.
  assert ({ground_term a = true}+{ground_term a = false}).
  apply dec_bool.
  elim H0; intro H1; clear H0.
  rewrite H1 in H. simpl in H. discriminate.
  rewrite H1 in H. simpl in H. discriminate.
  unfold ground_term_list. assumption.
Qed.

Lemma multisubs_term_list_length :
  forall (s : substitution)(LT : list term),
    length LT = length (multisubs_term_list s LT).
Proof.
  intros s LT.
  induction LT.
  simpl. trivial.
  simpl. rewrite IHLT. trivial.
Qed.

Lemma multisubs_term_list_lemma :
  forall (n0 : nat)(n1 : rval)(LT : list term),
    multisubs_term_list ((n0,n1) :: nil) LT =
    subs_term_list (n0,n1) LT.
Proof.

```

```

intros.
induction LT.
simpl. trivial.
simpl.
rewrite IHLT. trivial.
Qed.

Lemma multisubs_term_list_lemma2 :
  forall (n0 : nat)(n1 : rval)(s : substitution)(LT : list term),
    multisubs_term_list ((n0,n1) :: s) LT =
      multisubs_term_list s (subs_term_list (n0,n1) LT).
Proof.
  intros.
  induction LT.
  simpl. trivial.
  simpl.
  rewrite IHLT. trivial.
Qed.

Lemma multisubs_term_list_dup :
  forall (a : nat)(b : rval)(s : substitution)(LT : list term),
    multisubs_term_list ((a,b)::(a,b)::s) LT =
      multisubs_term_list ((a,b)::s) LT.
Proof.
  intros a b s LT.
  induction LT.
  simpl. trivial.
  induction a0.
  assert (multisubs_term_list ((a,b)::(a,b)::s) (ident n :: LT) =
    ident n :: (multisubs_term_list ((a,b)::(a,b)::s) LT)).
  apply multisubs_term_list_ident.
  rewrite H.
  assert (multisubs_term_list ((a,b)::s) (ident n :: LT) =
    ident n :: (multisubs_term_list ((a,b)::s) LT)).
  apply multisubs_term_list_ident.
  rewrite H0.
  rewrite IHLT. trivial.
  simpl.
  (* induction (eq_nat_dec a n). *)
  rewrite IHLT; trivial.
  simpl.
  induction (eq_nat_dec a n).
  rewrite IHLT.
  induction b. trivial. trivial.
  induction (eq_nat_dec a n). contradiction.
  rewrite IHLT. trivial.
Qed.

Lemma subs_lemma_n :
  forall (n : nat)(s : substitution)(LT1 LT2 : list term)(n0 n1 : nat),
    length LT1 = n ->

```

```

    multisubs_term_list s LT1 = LT2 ->
    multisubs_term s (var n0) = ident n1 ->
    multisubs_term_list s (multisubs_term_list ((n0,rvident n1)::nil) LT1) = LT2
.
Proof.
  induction n.
  intros s LT1 LT2 n0 n1 H1 H2 H3.
  induction LT1. simpl. simpl in H2. assumption.
  simpl in H1. discriminate.

  intros s LT1 LT2 n0 n1 H1 H2 H3.
  induction LT1.
  simpl in H1. discriminate.
  simpl in H1. injection H1. intro H4.
  clear IHLT1. (* dont need, have IHn *)
  clear H1.
  simpl in H2.

  induction LT2. discriminate.
  clear IHLT2.
  injection H2. intros H5 H6. clear H2.
  induction a.

  (* ident case *)
  simpl.
  rewrite H6.
  assert (multisubs_term_list s (multisubs_term_list ((n0, rvident n1) :: nil) L
T1) = LT2).
  apply IHn. assumption. assumption. assumption.
  rewrite H. trivial.
  (* kident case *)
  simpl.
  rewrite H6.
  assert (multisubs_term_list s (multisubs_term_list ((n0, rvident n1) :: nil) L
T1) = LT2).
  apply IHn. assumption. assumption. assumption.
  rewrite H. trivial.

  (* var case *)
  simpl.
  induction (eq_nat_dec n0 n2).
  (* = *)
  assert (multisubs_term s (ident n1) = ident n1).
  apply multisubs_term_ident.
  rewrite H.
  rewrite <- H6.
  rewrite <- a.
  rewrite H3.
  assert (multisubs_term_list s (multisubs_term_list ((n0, rvident n1) :: nil) L
T1) = LT2).
  apply IHn. assumption. assumption. assumption.

```

```

rewrite H0. trivial.

(* <> *)
rewrite H6.
assert (multisubs_term_list s (multisubs_term_list ((n0, rvident n1) :: nil) L
T1) = LT2).
apply IHn. assumption. assumption. assumption.
rewrite H. trivial.
Qed.

Lemma ksubs_lemma_n :
  forall (n : nat)(s : substitution)(LT1 LT2 : list term)(n0 : nat)(n1 : kthing)
  ,
    length LT1 = n ->
    multisubs_term_list s LT1 = LT2 ->
    multisubs_term s (var n0) = kident n1 ->
    multisubs_term_list s (multisubs_term_list ((n0,rvkident n1)::nil) LT1) = LT
    2.
Proof.
  induction n.
  intros s LT1 LT2 n0 n1 H1 H2 H3.
  induction LT1. simpl. simpl in H2. assumption.
  simpl in H1. discriminate.

  intros s LT1 LT2 n0 n1 H1 H2 H3.
  induction LT1.
  simpl in H1. discriminate.
  simpl in H1. injection H1. intro H4.
  clear IHLT1. (* dont need, have IHn *)
  clear H1.
  simpl in H2.

  induction LT2. discriminate.
  clear IHLT2.
  injection H2. intros H5 H6. clear H2.
  induction a.

  (* ident case *)
  simpl.
  rewrite H6.
  assert (multisubs_term_list s (multisubs_term_list ((n0, rvkident n1) :: nil)
LT1) = LT2).
  apply IHn. assumption. assumption. assumption.
  rewrite H. trivial.
  (* kident case *)
  simpl.
  rewrite H6.
  assert (multisubs_term_list s (multisubs_term_list ((n0, rvkident n1) :: nil)
LT1) = LT2).
  apply IHn. assumption. assumption. assumption.
  rewrite H. trivial.

```

```

(* var case *)
simpl.
induction (eq_nat_dec n0 n2).
(* = *)
assert (multisubs_term s (kident n1) = kident n1).
apply multisubs_term_kident.
rewrite H.
rewrite <- H6.
rewrite <- a.
rewrite H3.
assert (multisubs_term_list s (multisubs_term_list ((n0, rvkident n1) :: nil)
LT1) = LT2).
apply IHn. assumption. assumption. assumption.
rewrite H0. trivial.

(* <> *)
rewrite H6.
assert (multisubs_term_list s (multisubs_term_list ((n0, rvkident n1) :: nil)
LT1) = LT2).
apply IHn. assumption. assumption. assumption.
rewrite H. trivial.
Qed.

Lemma subs_lemma :
forall (s : substitution)(LT1 LT2 : list term)(n0 n1 : nat),
  multisubs_term_list s LT1 = LT2 ->
  multisubs_term s (var n0) = ident n1 ->
  multisubs_term_list s (multisubs_term_list ((n0, rvident n1)::nil) LT1) = LT
  2.
Proof.
intros s LT1 LT2 n0 n1 H1 H2.
apply subs_lemma_n with (n:=length LT1).
trivial. assumption. assumption.
Qed.

Lemma ksubs_lemma :
forall (s : substitution)(LT1 LT2 : list term)(n0 : nat)(n1 : kthing),
  multisubs_term_list s LT1 = LT2 ->
  multisubs_term s (var n0) = kident n1 ->
  multisubs_term_list s (multisubs_term_list ((n0, rvkident n1)::nil) LT1) = L
  T2.
Proof.
intros s LT1 LT2 n0 n1 H1 H2.
apply ksubs_lemma_n with (n:=length LT1).
trivial. assumption. assumption.
Qed.

Lemma match_term_list_complete_n :
forall (n : nat)(LT1 LT2 : list term)(s : substitution),
  length LT1 = n ->

```



```

length LT2 = n ->
ground_term_list LT2 = true ->
multisubs_term_list s LT1 = LT2 ->
(exists sbs, match_term_list LT1 LT2 = Some sbs).
Proof.
  induction n.
  intros LT1 LT2 s H1 H2 H3 H4.
  exists (nil:substitution).
  induction LT1; induction LT2.
  simpl. trivial. simpl in H2. discriminate. simpl in H1. discriminate.
  simpl in H1. discriminate.

  intros LT1 LT2 s H1 H2 H3 H4.
  induction LT1; induction LT2.
  simpl in H1. discriminate. simpl in H1. discriminate.
  simpl in H2. discriminate.

  clear IHLT1. clear IHLT2.

  (* OK, now we have the correct inductive hypothesis and goal *)
  (* Now just do cases of LT1 and LT2 *)
  induction a; induction a0.
  (* ident ident *)
  simpl.
  induction (eq_nat_dec n0 n1).
  simpl.
  assert (multisubs_term_list nil LT1 = LT1).
  apply multisubs_term_list_nil.
  rewrite H.
  assert (exists sbs : substitution, match_term_list LT1 LT2 = Some sbs).
  apply IHn with (s:=s).
  simpl in H1. injection H1. auto.
  simpl in H2. injection H2. auto.
  apply ground_term_list_ident with (n:=n1). assumption.
  simpl in H4. injection H4. auto.
  elim H0. intros sbs H5.
  exists sbs. rewrite H5. trivial.

  (* ident ident, different *)
  simpl in H4.
  injection H4. intros H5 H6.
  assert (multisubs_term s (ident n0) = ident n0).
  apply multisubs_term_ident.
  rewrite H in H6. injection H6. intro H7. contradiction.

  (* ident kident *)
  simpl in H4.
  injection H4; intros H5 H6; clear H4.
  assert (multisubs_term s (ident n0) = ident n0).
  apply multisubs_term_ident.
  rewrite H in H6. discriminate.

```

```

(* ident var *)
unfold ground_term_list in H3.
unfold forall_bool in H3.
assert (ground_term (var n1) = false).
simpl. trivial.
rewrite H in H3. simpl in H3. discriminate.

(* kident ident *)
simpl in H4.
injection H4; intros H5 H6; clear H4.
assert (multisubs_term s (kident k) = kident k).
apply multisubs_term_kident.
rewrite H in H6. discriminate.

(* kident kident *)
simpl.
induction (eq_kthing_dec k k0).
simpl.
assert (multisubs_term_list nil LT1 = LT1).
apply multisubs_term_list_nil.
rewrite H.
assert (exists sbs : substitution, match_term_list LT1 LT2 = Some sbs).
apply IHn with (s:=s).
simpl in H1. injection H1. auto.
simpl in H2. injection H2. auto.
apply ground_term_list_kident with (n:=k0). assumption.
simpl in H4. injection H4. auto.
elim H0. intros sbs H5.
exists sbs. rewrite H5. trivial.

(* kident kident, different *)
simpl in H4.
injection H4. intros H5 H6.
assert (multisubs_term s (kident k) = kident k).
apply multisubs_term_kident.
rewrite H in H6. injection H6. intro H7. contradiction.

(* kident var *)
unfold ground_term_list in H3.
unfold forall_bool in H3.
assert (ground_term (var n0) = false).
simpl. trivial.
rewrite H in H3. simpl in H3. discriminate.

(* var ident *)
simpl.
assert (exists sbs, match_term_list (multisubs_term_list ((n0, rident n1)::nil) LT1) LT2 = Some sbs).
apply IHn with (s:=s).

```

```

assert (length LT1 = length (multisubs_term_list ((n0, rvident n1)::nil) LT1))
.
apply multisubs_term_list_length with (LT:=LT1)(s:=(n0, rvident n1)::nil).
rewrite <- H. simpl in H1. injection H1. auto.
simpl in H2. injection H2. auto.
apply ground_term_list_ident with (n:=n1). assumption.

clear IHn.
apply subs_lemma.
simpl in H4. injection H4. auto.
simpl in H4. injection H4. auto.
clear IHn.
elim H. intros sbs H5.
exists ((n0, rvident n1)::sbs).
rewrite H5. trivial.

(* var kident *)
simpl.
assert (exists sbs, match_term_list (multisubs_term_list ((n0, rvkident k)::nil) LT1) LT2 = Some sbs).
apply IHn with (s:=s).

assert (length LT1 = length (multisubs_term_list ((n0, rvkident k)::nil) LT1))
.
apply multisubs_term_list_length with (LT:=LT1)(s:=(n0, rvkident k)::nil).
rewrite <- H. simpl in H1. injection H1. auto.
simpl in H2. injection H2. auto.
apply ground_term_list_kident with (n:=k). assumption.

clear IHn.
apply ksubs_lemma.
simpl in H4. injection H4. auto.
simpl in H4. injection H4. auto.
clear IHn.
elim H. intros sbs H5.
exists ((n0, rvkident k)::sbs).
rewrite H5. trivial.

(* var var case *)
unfold ground_term_list in H3.
unfold forall_bool in H3.
assert (ground_term (var n1) = false).
simpl. trivial.
rewrite H in H3. simpl in H3. discriminate.
Qed.

Lemma match_term_list_complete :
  forall (LT1 LT2 : list term)(s : substitution),
    ground_term_list LT2 = true ->
    multisubs_term_list s LT1 = LT2 ->
    (exists sbs, match_term_list LT1 LT2 = Some sbs).

```

Proof.

```
intros.
apply match_term_list_complete_n with (n:=length LT1)(s:=s).
trivial.
assert (length LT1 = length (multisubs_term_list s LT1)).
apply multisubs_term_list_length.
rewrite H1. rewrite H0. trivial.
assumption. assumption.
```

Qed.

Definition match_atomic (A1 A2 : atomic) : option substitution :=

```
match A1 with
| bare p1 LT1 =>
  match A2 with
  | bare p2 LT2 =>
    if eq_nat_dec p1 p2 then
      match_term_list LT1 LT2
    else None
  | says w2 p2 LT2 => None
  | extbare k => None
  | extsays w k => None
  end
| says w1 p1 LT1 =>
  match A2 with
  | bare p2 LT2 => None
  | says w2 p2 LT2 =>
    if eq_nat_dec p1 p2 then
      if eq_nat_dec w1 w2 then
        match_term_list LT1 LT2
      else None
    else None
  | extbare k => None
  | extsays w k => None
  end
| extbare k =>
  match A2 with
  | bare p2 LT2 => None
  | says w2 p2 LT2 => None
  | extbare k2 =>
    match_term k k2
  | extsays w2 k2 => None
  end
| extsays w k =>
  match A2 with
  | bare p2 LT2 => None
  | says w2 p2 LT2 => None
  | extbare k2 => None
  | extsays w2 k2 =>
    if eq_nat_dec w w2 then
      match_term k k2
    else None
```

```

    end
end.

Lemma match_atomic_sound :
  forall (A1 A2 : atomic)(s : substitution),
    ground_atomic A2 = true ->
    match_atomic A1 A2 = Some s ->
    multisubs_atomic s A1 = A2.
Proof.
  intros A1 A2 s H1 H2.
  induction A1; induction A2; simpl in H2; try discriminate.
  (* bare *)
  induction (eq_nat_dec n n0); simpl in H2; try discriminate.
  assert (match_term_list l l0 = Some s).
  induction (match_term_list l l0).
  injection H2; intros H3; clear H2. rewrite <- H3. trivial.
  discriminate.
  clear H2.
  simpl.
  assert (multisubs_term_list s l = l0).
  apply match_term_list_sound.
  simpl in H1. assumption. assumption.
  unfold multisubs_term_list in H0.
  rewrite H0. rewrite a. trivial.

  (* says *)
  induction (eq_nat_dec n0 n2); induction (eq_nat_dec n n1); simpl in H2; try di
  discriminate.
  assert (match_term_list l l0 = Some s).
  induction (match_term_list l l0).
  injection H2; intros H4; clear H2. rewrite <- H4. trivial.
  discriminate.
  clear H2.
  simpl.
  assert (multisubs_term_list s l = l0).
  apply match_term_list_sound.
  simpl in H1. assumption. assumption.
  unfold multisubs_term_list in H0.
  rewrite H0. rewrite a. rewrite a0. trivial.

  (* extbare *)
  simpl.
  cut (multisubs_term s t = t0).
  intro H3.
  rewrite H3. trivial.
  assert (match_term t t0 = Some s).
  induction (match_term t t0).
  injection H2; intros H4; clear H2.
  rewrite H4. trivial.
  discriminate.
  apply match_term_sound.

```

```

    simpl in H1. assumption.
    assumption.

    (* extsays *)
    simpl.
    induction (eq_nat_dec n n0).
    cut (multisubs_term s t = t0).
    intro H3.
    rewrite H3. rewrite a. trivial.
    assert (match_term t t0 = Some s).
    induction (match_term t t0).
    injection H2; intros H4; clear H2.
    rewrite H4. trivial. discriminate.
    apply match_term_sound.
    simpl in H1. assumption. assumption.
    discriminate.
Qed.

Lemma match_atomic_complete :
  forall (A1 A2 : atomic)(s : substitution),
    ground_atomic A2 = true ->
    multisubs_atomic s A1 = A2 ->
    (exists sbs, (match_atomic A1 A2 = Some sbs)).
Proof.
  intros A1 A2 s H1 H2.
  induction A1; induction A2; simpl in H1; simpl in H2; simpl; try discriminate.
  (* bare *)
  induction (eq_nat_dec n n0).
  injection H2; intros H3 H4; clear H2.
  assert (exists s, match_term_list l l0 = Some s).
  apply match_term_list_complete with (s:=s).
  assumption. unfold multisubs_term_list. assumption.
  elim H; intros s2 H5; clear H.
  exists (s2).
  rewrite H5.
  simpl. trivial.
  injection H2; intros. contradiction.

  (* says *)
  injection H2; intros H3 H4 H5; clear H2.
  induction (eq_nat_dec n0 n2); induction (eq_nat_dec n n1); simpl.
  assert (exists s, match_term_list l l0 = Some s).
  apply match_term_list_complete with (s:=s).
  assumption. unfold multisubs_term_list. assumption.
  elim H; intros s2 H6; clear H.
  exists (s2).
  rewrite H6.
  simpl. trivial.
  contradiction. contradiction. contradiction.

  (* extbare *)

```

```

(* induction (eq_nat_dec n n0). *)
  injection H2; intros H3; clear H2.
  assert (exists s, match_term t t0 = Some s).
  apply match_term_complete with (s := s).
  assumption.
  elim H; intros ks2 H4; clear H.
  exists (ks2).
  rewrite H4. simpl.
  trivial.

(* extsays *)
  injection H2; intros; clear H2.
  assert (exists ks, match_term t t0 = Some ks).
  apply match_term_complete with (s := s).
  assumption.
  elim H2; intros ks2 H5; clear H2.
  exists (ks2).
  induction (eq_nat_dec n n0).
  rewrite H5.
  trivial. contradiction.
Qed.

(* Given atomic A and list of things to match against LA, return
list of all possible substitutions that work *)
Fixpoint match_atomic_list (A : atomic)(LA : list atomic) {struct LA} : list substitution :=
  match LA with
  | nil => nil
  | LAhd :: LAtl =>
    match match_atomic A LAhd with
    | Some s => s :: (match_atomic_list A LAtl)
    | None => match_atomic_list A LAtl
    end
  end.

Lemma option_dec :
  forall (A : Set)(B : option A),
    {exists x, B = Some x}+{B = None}.
Proof.
  intros.
  induction B.
  left. exists a. trivial. right. trivial.
Qed.

Lemma match_atomic_list_sound :
  forall (A : atomic)(LA : list atomic)(s : substitution),
    ground_atomic_list LA = true ->
    In s (match_atomic_list A LA) ->
    In (multisubs_atomic s A) LA.
Proof.
  intros A LA s Hg H.

```

```

induction LA.
simpl in H. contradiction.

assert (ground_atomic a = true).

unfold ground_atomic_list in Hg.
unfold forall_bool in Hg.
induction (ground_atomic a). trivial.
simpl in Hg. discriminate.

assert (ground_atomic_list LA = true).

unfold ground_atomic_list in Hg.
unfold forall_bool in Hg.
fold forall_bool in Hg.
unfold ground_atomic_list.
induction (forall_bool ground_atomic LA).
trivial.
induction (ground_atomic a).
simpl in Hg. discriminate.
simpl in Hg. discriminate.

simpl in H.
assert ({exists s, match_atomic A a = Some s}+{match_atomic A a = None}).
apply option_dec.
elim H2; intro H3; clear H2.
elim H3; intros sbs H4; clear H3.
rewrite H4 in H.
simpl in H.
elim H; intro H5; clear H.
simpl. left.
assert (multisubs_atomic s A = a).
apply match_atomic_sound.
assumption.
rewrite <- H5. trivial.

rewrite H. trivial.

simpl. right.
apply IHLA.
assumption.
assumption.

rewrite H3 in H.

assert (In (multisubs_atomic s A) LA).
apply IHLA.
assumption. assumption.

simpl. right. assumption.
Qed.

```



```

Lemma match_atomic_list_complete :
  forall (A : atomic)(LA : list atomic)(s : substitution),
    ground_atomic_list LA = true ->
      In (multisubs_atomic s A) LA ->
        (exists sbs, In sbs (match_atomic_list A LA) /\ (multisubs_atomic sbs A = multisubs_atomic s A)).
Proof.
  intros A LA s H1 H2.
  induction LA.
  simpl in H2. contradiction.

  assert (ground_atomic a = true).

  unfold ground_atomic_list in H1.
  unfold forall_bool in H1.
  induction (ground_atomic a). trivial.
  simpl in H1. discriminate.

  assert (ground_atomic_list LA = true).

  unfold ground_atomic_list in H1.
  unfold forall_bool in H1.
  fold forall_bool in H1.
  unfold ground_atomic_list.
  induction (forall_bool ground_atomic LA).
  trivial.
  induction (ground_atomic a).
  simpl in H1. discriminate.
  simpl in H1. discriminate.

  clear H1.

  simpl in H2.
  elim H2; intro H3; clear H2.
  simpl.
  assert ({exists p, match_atomic A a = Some p}+{match_atomic A a = None}).
  apply option_dec.
  elim H1; intros H4; clear H1.
  elim H4; intros p H5; clear H4.

  rewrite H5.

  exists p.
  split.
  simpl. left. trivial.
  rewrite <- H3.
  apply match_atomic_sound.
  assumption.
  assumption.

```

```

assert (exists sbs, match_atomic A a = Some sbs).
apply match_atomic_complete with (s:=s).
assumption. rewrite H3. trivial.
elim H1. intros sbs H5.
rewrite H4 in H5. discriminate.

assert (exists sbs, In sbs (match_atomic_list A LA) /\
  multisubs_atomic sbs A = multisubs_atomic s A).
apply IHLA. assumption. assumption.
elim H1. intros sbs H4.
exists sbs.
elim H4; intros H5 H6; clear H4.
clear IHLA H1.
split.

simpl.
induction (match_atomic A a).
simpl. right. assumption. assumption.
assumption.
Qed.

(* Same as match_atomic_list but with lists of atomic formulas as the matcher *)
(* That means we give it a body of a clause and the db of facts, returns all possible
substitutions that satisfy the body. Guts of the extension procedure. *)
(* LA2 is the db, LA1 is the body of the clause *)

(* Because of recursive constraints and positivity, we have to pass
around an auxiliary var that represents subs to do before actually
matching. Do subs on LA1 part (thing with variables). *)
Fixpoint match_list_atomic_list_aux (LA1 LA2 : list atomic)(sbs : substitution)
{struct LA1} : list substitution :=
  match LA1 with
  | nil => nil :: nil
  | hd :: tl =>
    (* Match the first goal in the body *)
    let res := match_atomic_list (multisubs_atomic sbs hd) LA2 in
    (* For each result of the first match, do another match with old subs done *)
    (* Combine original substitution with results *)
    let func := fun s => map_append s
      (match_list_atomic_list_aux tl LA2 (sbs ++ s)) in
    let multires := map func res in
    (* At this point we have a list of lists of substitutions, want list of subs
    *)
    flatten multires
  end.
Definition match_list_atomic_list (LA1 LA2 : list atomic) : list substitution :=
  match_list_atomic_list_aux LA1 LA2 nil.

```

```

Lemma in_append :
  forall (A : Set)(s : A)(L1 L2 : list A),
    In s (L1 ++ L2) ->
      (In s L1  $\vee$  In s L2).
Proof.
  intros A s L1 L2 H.
  induction L1. simpl in H. right. assumption.
  simpl in H.
  elim H; intro H1; clear H.
  left. simpl. left. assumption.
  assert (In s L1  $\vee$  In s L2).
  apply IHL1. assumption.
  elim H; intro H2; clear H.
  left. simpl. right. assumption.
  right. assumption.
Qed.

Lemma in_map :
  forall (A : Set)(B : Set)(f : A -> B)(L : list A)(x : B),
    In x (map f L) ->
      (exists y, In y L /\ x = f y).
Proof.
  intros A B f L x H.
  induction L.
  simpl in H. contradiction.
  simpl in H.
  elim H; intro H2; clear H.
  exists a. split. simpl. left. trivial.
  rewrite H2. trivial.
  assert (exists y, In y L /\ x = f y).
  apply IHL. assumption.
  elim H. intros y2 H3.
  exists y2.
  elim H3; intros H4 H5; clear H3.
  split.
  simpl. right. assumption. assumption.
Qed.

Lemma in_map2 :
  forall (A : Set)(B : Set)(f : A -> B)(L : list A)(x : B),
    (exists y, In y L /\ x = f y) ->
      In x (map f L).
Proof.
  intros.
  elim H; intros y H2; clear H.
  elim H2; intros H3 H4; clear H2.
  induction L.
  simpl in H3. contradiction.
  simpl in H3.
  elim H3; intro H5; clear H3.

```

```

    simpl. left. rewrite H5. rewrite H4. trivial.
    simpl. right. apply IHL. assumption.
Qed.

```

```

Lemma in_flatten :
  forall (A : Set)(s : A)(L : list (list A)),
    In s (flatten L) ->
      (exists X, In X L /\ In s X).

```

```

Proof.
  intros A s L H.
  induction L.
  simpl in H. contradiction.
  simpl in H.
  assert (In s a \/ In s (flatten L)).
  apply in_append. assumption.
  elim H0; intro H1; clear H0.
  exists a.
  split. simpl. left. trivial. assumption.
  assert (exists X, In X L /\ In s X).
  apply IHL. assumption.
  elim H0. intros x H2.
  elim H2; intros H3 H4; clear H2.
  exists x.
  split. simpl. right. assumption. assumption.
Qed.

```

```

Lemma flatten_in :
  forall (A : Set)(L : list (list A))(s : A),
    (exists X, In X L /\ In s X) ->
      In s (flatten L).

```

```

Proof.
  induction L.
  intros s H. simpl in H. elim H. intros. elim H0. intros. contradiction.
  intros s H.
  simpl. apply in_or_app.
  elim H. intros x H2. clear H.
  elim H2; intros H3 H4; clear H2.
  simpl in H3.
  elim H3; intro H5; clear H3.
  rewrite <- H5 in H4.
  left. assumption.
  right. apply IHL.
  exists x. split; assumption.
Qed.

```

```

Lemma multisubs_atomic_nil :
  forall (A : atomic),
    multisubs_atomic nil A = A.

```

```

Proof.
  intros A.
  induction A.

```

```

    unfold multisubs_atomic.
    unfold lift_term.
    induction l.
    simpl. trivial.
    simpl. injection IHl. intro H1. rewrite H1. trivial.
    unfold multisubs_atomic.
    unfold lift_term.
    induction l.
    simpl. trivial.
    simpl. injection IHl. intro H1. rewrite H1. trivial.
    simpl. trivial.
    simpl. trivial.
Qed.

```

```

Lemma multisubs_atomic_list_nil :
  forall (LA : list atomic),
    multisubs_atomic_list nil LA = LA.

```

```

Proof.
  induction LA.
  simpl. trivial.
  simpl. rewrite IHLA.
  assert (multisubs_atomic nil a = a).
  apply multisubs_atomic_nil.
  rewrite H. trivial.
Qed.

```

```

Lemma multisubs_cons :
  forall (a : nat*rval)(s : substitution)(A : atomic),
    multisubs_atomic (a :: s) A = multisubs_atomic s (subs_atomic a A).

```

```

Proof.
  intros a s A.
  unfold multisubs_atomic.
  induction A.
  unfold subs_atomic.
  unfold lift_term.
  induction l. simpl. trivial.
  injection IHl. intro H1.

  simpl. rewrite H1. trivial.
  unfold subs_atomic.
  unfold lift_term.
  induction l. simpl. trivial.
  injection IHl. intro H1.

  simpl. rewrite H1. trivial.

  simpl. trivial.
  simpl. trivial.
Qed.

```

```

Lemma multisubs_atomic_list_cons :

```

```

forall (a : nat*rval)(s : substitution)(LA : list atomic),
  multisubs_atomic_list (a :: s) LA = multisubs_atomic_list s (subs_atomic_list a LA).

```

Proof.

```

intros a s LA.
unfold multisubs_atomic_list.
induction LA.
simpl. trivial.
simpl.
rewrite IHLA.
assert (multisubs_atomic (a :: s) a0 =
  multisubs_atomic s (subs_atomic a a0)).
apply multisubs_cons.
rewrite H. trivial.

```

Qed.

Lemma multisubs_append :

```

forall (s1 s2 : substitution)(A : atomic),
  multisubs_atomic (s1 ++ s2) A = multisubs_atomic s2 (multisubs_atomic s1 A).

```

Proof.

```

induction s1.
intros s2 A.
simpl.
assert (multisubs_atomic nil A = A).
apply multisubs_atomic_nil.
rewrite H. trivial.
intros s2 A.
assert ((a :: s1) ++ s2 = a :: (s1 ++ s2)).
simpl. trivial.
rewrite H.
assert (multisubs_atomic (a :: (s1 ++ s2)) A =
  multisubs_atomic (s1 ++ s2) (subs_atomic a A)).
apply multisubs_cons.
rewrite H0.
rewrite IHs1.
assert (multisubs_atomic (a :: s1) A = multisubs_atomic s1 (subs_atomic a A)).
apply multisubs_cons.
rewrite H1. trivial.

```

Qed.

Lemma multisubs_atomic_list_append :

```

forall (LA : list atomic)(s sbs : substitution),
  multisubs_atomic_list (sbs ++ s) LA =
  multisubs_atomic_list s (multisubs_atomic_list sbs LA).

```

Proof.

```

intros LA s sbs.
induction LA.
simpl. trivial.
simpl.
rewrite IHLA.
assert (multisubs_atomic (sbs ++ s) a = multisubs_atomic s (multisubs_atomic sbs a)).

```

```

    bs a)).
    apply multisubs_append.
    rewrite H. trivial.
Qed.

```

```

Lemma ground_term_list_in :
  forall (LA : list term)(x : term),
    ground_term_list LA = true ->
      In x LA ->
        ground_term x = true.

```

```

Proof.
  intros LA x H1 H2.
  induction LA. simpl in H2. contradiction.
  simpl in H2.
  unfold ground_term_list in H1.
  unfold ground_term_list in IHLA.
  simpl in H1.
  assert (ground_term a = true).
  induction (ground_term a). trivial. simpl in H1. discriminate.
  assert (forall_bool ground_term LA = true).
  induction (forall_bool ground_term LA). trivial.
  induction (ground_term a). simpl in H1. discriminate. assumption.

  elim H2; intro H3; clear H2.
  rewrite <- H3. assumption.
  apply IHLA. assumption. assumption.
Qed.

```

```

Lemma ground_atomic_list_in :
  forall (LA : list atomic)(x : atomic),
    ground_atomic_list LA = true ->
      In x LA ->
        ground_atomic x = true.

```

```

Proof.
  intros LA x H1 H2.
  induction LA. simpl in H2. contradiction.
  simpl in H2.
  unfold ground_atomic_list in H1.
  unfold ground_atomic_list in IHLA.
  simpl in H1.
  assert (ground_atomic a = true).
  induction (ground_atomic a). trivial. simpl in H1. discriminate.
  assert (forall_bool ground_atomic LA = true).
  induction (forall_bool ground_atomic LA). trivial.
  induction (ground_atomic a). simpl in H1. discriminate. assumption.

  elim H2; intro H3; clear H2.
  rewrite <- H3. assumption.
  apply IHLA. assumption. assumption.
Qed.

```

```

Lemma subs_term_ground :
  forall (x : nat)(y : rval)(a : term),
    ground_term a = true ->
      subs_term (x, y) a = a.
Proof.
  intros x y a H.
  induction a.
  simpl. trivial. simpl in H. simpl. trivial. discriminate.
Qed.

Lemma multisubs_term_ground :
  forall (s : substitution)(a : term),
    ground_term a = true ->
      multisubs_term s a = a.
Proof.
  intros s a H.
  induction s.
  unfold multisubs_term.
  unfold fold_left. trivial.
  unfold multisubs_term.
  simpl.
  unfold multisubs_term in IHs.
  assert (subs_term a0 a = a).
  induction a0.
  apply subs_term_ground. assumption.
  rewrite H0.
  rewrite IHs. trivial.
Qed.

Lemma subs_term_list_ground :
  forall (x : nat)(y : rval)(a : list term),
    ground_term_list a = true ->
      subs_term_list (x, y) a = a.
Proof.
  intros x y a H.
  induction a. simpl. trivial.
  unfold ground_term_list in H.
  simpl in H.
  simpl.
  assert (ground_term a = true).
  induction (ground_term a). trivial. simpl in H. discriminate.

  assert (subs_term (x, y) a = a).
  apply subs_term_ground.
  assumption. unfold subs_term in H1.
  rewrite H1.

  assert (forall_bool ground_term a0 = true). induction (forall_bool ground_term
    a0).
  trivial. rewrite H0 in H. simpl in H. discriminate.
  assert (subs_term_list (x, y) a0 = a0).

```



```

    apply IHa.
    unfold ground_term_list. assumption.
    rewrite H3. trivial.
Qed.

Lemma multisubs_term_list_ground :
  forall (s : substitution)(a : list term),
    ground_term_list a = true ->
      multisubs_term_list s a = a.
Proof.
  intros s a H.
  unfold multisubs_term_list.
  induction a. simpl. trivial.
  simpl.
  assert (multisubs_term s a = a).
  apply multisubs_term_ground.
  unfold ground_term_list in H.
  apply forall_bool_in with (L:=(a :: a0)).
  assumption. simpl. left. trivial.
  rewrite H0.
  assert (map (multisubs_term s) a0 = a0).
  apply IHa.
  unfold ground_term_list in H.
  simpl in H.
  unfold ground_term_list.
  induction (forall_bool ground_term a0). trivial.
  induction (ground_term a). simpl in H. discriminate. simpl in H. discriminate.
  rewrite H1. trivial.
Qed.

Lemma subs_atomic_ground :
  forall (x : nat)(y : rval)(a : atomic),
    ground_atomic a = true ->
      subs_atomic (x, y) a = a.
Proof.
  intros x y a H.
  induction a.
  unfold subs_atomic.
  unfold lift_term.
  assert (subs_term_list (x, y) l = l).
  apply subs_term_list_ground.
  unfold ground_atomic in H. assumption.
  unfold subs_term_list in H0.
  rewrite H0. trivial.
  unfold subs_atomic.
  unfold lift_term.
  assert (subs_term_list (x, y) l = l).
  apply subs_term_list_ground.
  unfold ground_atomic in H. assumption.
  unfold subs_term_list in H0.
  rewrite H0. trivial.

```

```

induction t; simpl in H; try discriminate.
simpl. trivial.

simpl. trivial.

induction t; simpl in H; try discriminate.
simpl. trivial.
simpl. trivial.
Qed.

Lemma multisubs_atomic_ground :
  forall (s : substitution)(a : atomic),
    ground_atomic a = true ->
    multisubs_atomic s a = a.
Proof.
  intros s a H.
  induction a.
  simpl.
  assert (multisubs_term_list s l = l).
  apply multisubs_term_list_ground.
  unfold ground_atomic in H. assumption.
  unfold multisubs_term_list in H0.
  rewrite H0. trivial.
  simpl.
  assert (multisubs_term_list s l = l).
  apply multisubs_term_list_ground.
  unfold ground_atomic in H. assumption.
  unfold multisubs_term_list in H0.
  rewrite H0. trivial.

  simpl.
  assert (multisubs_term s t = t).
  apply multisubs_term_ground.
  simpl in H. assumption.
  rewrite H0. trivial.

  simpl.
  assert (multisubs_term s t = t).
  apply multisubs_term_ground.
  simpl in H. assumption.
  rewrite H0. trivial.
Qed.

Lemma match_list_atomic_list_aux_sound :
  forall (n : nat)(LA1 LA2 : list atomic)(sbs s : substitution),
    length LA1 = n ->
    ground_atomic_list LA2 = true ->
    In s (match_list_atomic_list_aux LA1 LA2 sbs) ->
    incl (multisubs_atomic_list (sbs ++ s) LA1) LA2.
Proof.

```

```

induction n.
(* n = 0 *)
intros LA1 LA2 sbs s H1 H2 H3.
induction LA1.
(* LA1 nil case *)
simpl. unfold incl. simpl. intros. contradiction.
(* LA1 cons case *)
simpl in H1. discriminate.

(* n > 0 *)
intros LA1 LA2 sbs s H1 H2 H3.
induction LA1.
(* LA1 nil *)
simpl in H1. discriminate.
(* LA1 cons case *)
clear IHLA1. (* useless, we have IHn which is induction on length of LA1 *)

(* Start the real work *)

simpl in H3.
(* If s is in the flatten, it must have been in an element of the flatten *)
set (se1:=(match_atomic_list (multisubs_atomic sbs a) LA2)) in H3.
set (se2:=(map
  (fun s : list (nat * rval) =>
    map_append s (match_list_atomic_list_aux LA1 LA2 (sbs ++ s)))
  se1)) in H3.
assert (exists x, In x se2 /\ In s x).
apply in_flatten.
assumption.
elim H. intros x H4.
elim H4; intros H5 H6. clear H4. clear H. clear H3.
set (f:=(fun s : list (nat * rval) =>
  map_append s (match_list_atomic_list_aux LA1 LA2 (sbs ++ s)))) in se
2.
assert (exists y, In y se1 /\ x = f y).
apply in_map. assumption.
elim H; intros y H7.
elim H7; intros H8 H9; clear H7. clear H.

(* Do stuff with a multisubs and a *)
assert (In (multisubs_atomic y (multisubs_atomic sbs a)) LA2).
apply match_atomic_list_sound. assumption. assumption.
assert (multisubs_atomic (sbs ++ y) a = multisubs_atomic y (multisubs_atomic s
bs a)).
apply multisubs_append.
rewrite <- H0 in H.
clear H0.
rewrite H9 in H6.

(* Since s is in f y, use def of f to show starts with y *)
unfold f in H6.

```

```

unfold map_append in H6.
assert (exists z, In z (match_list_atomic_list_aux LA1 LA2 (sbs ++ y)) /\
  s = (fun x : list (nat * rval) => y ++ x) z).
apply in_map. assumption.
elim H0. intros z H10. clear H0.
elim H10; intros H11 H12; clear H10.
rewrite H12.

(* Almost there, need to show that doing sbs ++ y ++ z to a
   gives something in LA2. Know that doing sbs ++y works, doing
   more works because sbs ++ y puts it to ground and then subs have
   no effect. *)

assert (ground_atomic (multisubs_atomic (sbs ++ y) a) = true).
apply ground_atomic_list_in with (x:=(multisubs_atomic (sbs ++ y) a))
  (LA:=LA2).
assumption. assumption.

(* Remaining: show that subsing a with sbs ++ y ++ z gets something in LA2 *)
unfold incl in IHn.
unfold incl.
intros a0 H13.
simpl in H13.
elim H13; intro H14; clear H13.
rewrite <- H14. clear H14 a0 H6 IHn.

assert (multisubs_atomic ((sbs ++ y) ++ z) a =
  multisubs_atomic z (multisubs_atomic (sbs ++ y) a)).
apply multisubs_append.
assert ((sbs ++ y) ++ z = sbs ++ y ++ z).
apply app_ass.
rewrite <- H4.
rewrite H3.
assert (multisubs_atomic z (multisubs_atomic (sbs ++ y) a) = multisubs_atomic
  (sbs ++ y) a).
apply multisubs_atomic_ground.
assumption.
rewrite H6. assumption.

(* Inductive case *)
apply IHn with (a:=a0)(LA1:=LA1)(LA2:=LA2)(sbs:=sbs++y)(s:=z).
clear IHn H0 H6 H H5 H8 H9 H12 x f se1 se2.
simpl in H1.
injection H1. trivial.
assumption. assumption.
assert ((sbs ++ y) ++ z = sbs ++ y ++ z).
apply app_ass.
rewrite H3. assumption.
Qed.

Lemma match_list_atomic_list_sound :

```

```

forall (LA1 LA2 : list atomic)(s : substitution),
  ground_atomic_list LA2 = true ->
  In s (match_list_atomic_list LA1 LA2) ->
  incl (multisubs_atomic_list s LA1) LA2.
Proof.
  intros LA1 LA2 s H1 H2.
  assert (match_list_atomic_list LA1 LA2 = match_list_atomic_list_aux LA1 LA2 nil).
  unfold match_list_atomic_list. trivial.
  rewrite H in H2.
  assert (s = nil ++ s).
  simpl. trivial.
  rewrite H0.
  apply match_list_atomic_list_aux_sound with (n:=length LA1).
  trivial. assumption. assumption.
Qed.

Lemma map_func_eq :
  forall (A B : Set)(f g : A -> B)(L : list A),
    (forall (x : A), f x = g x) ->
    map f L = map g L.
Proof.
  intros A B f g L H.
  induction L. simpl. trivial.
  simpl. rewrite H. rewrite IHL. trivial.
Qed.

Lemma match_list_atomic_list_aux_presubs :
  forall (LA1 LA2 : list atomic)(sbs s : substitution),
    match_list_atomic_list_aux LA1 LA2 (sbs ++ s) =
    match_list_atomic_list_aux (multisubs_atomic_list sbs LA1) LA2 s.
Proof.
  induction LA1.
  intros. simpl. trivial.

  intros LA2 sbs s.
  simpl.

  (* First get rid of flattens *)
  cut (
    (map
      (fun s0 : list (nat * rval) =>
        map_append s0
        (match_list_atomic_list_aux LA1 LA2 ((sbs ++ s) ++ s0)))
      (match_atomic_list (multisubs_atomic (sbs ++ s) a) LA2)) =
    (map
      (fun s0 : list (nat * rval) =>
        map_append s0
        (match_list_atomic_list_aux (multisubs_atomic_list sbs LA1) LA2
          (s ++ s0)))
      (match_atomic_list (multisubs_atomic s (multisubs_atomic sbs a)) LA2))).

```

```

intro H2. rewrite H2. trivial.

(* Rewrite list that map is applied to to be the same *)

assert (multisubs_atomic (sbs ++ s) a = multisubs_atomic s (multisubs_atomic s
bs a)).
apply multisubs_append.
rewrite H. clear H.

(* Get rid of map and list to which it applies *)
(* Maps are equal if functions give same answer for every input *)
apply map_func_eq.
intro s0.

(* Associate appends *)
assert ((sbs ++ s) ++ s0 = sbs ++ (s ++ s0)).
apply app_ass.
rewrite H. clear H.

(* Use IH *)
assert (match_list_atomic_list_aux LA1 LA2 (sbs ++ (s ++ s0)) =
  match_list_atomic_list_aux (multisubs_atomic_list sbs LA1) LA2 (s ++ s0)).
apply IHLA1 with (LA2:=LA2)(sbs:=sbs).
rewrite H.
trivial.
Qed.

Lemma match_list_atomic_list_aux_presubs2 :
  forall (LA1 LA2 : list atomic)(sbs : substitution),
    match_list_atomic_list_aux LA1 LA2 sbs =
    match_list_atomic_list (multisubs_atomic_list sbs LA1) LA2.
Proof.
  intros LA1 LA2 sbs.
  unfold match_list_atomic_list.
  assert (sbs ++ nil = sbs).
  apply app_nil.
  rewrite <- H.
  assert (
    match_list_atomic_list_aux LA1 LA2 (sbs ++ nil) =
    match_list_atomic_list_aux (multisubs_atomic_list sbs LA1) LA2 nil).
  apply match_list_atomic_list_aux_presubs.
  rewrite H0.
  rewrite H. trivial.
Qed.

Lemma in_append_flatten :
  forall (s3 s4 : substitution)(L1 L2 : list substitution),
    In s4 L1 ->
    In s3 L2 ->
    In (s3 ++ s4) (flatten (map (fun s0 => map_append s0 L1) L2)).
Proof.

```

```

intros s3 s4 L1 L2 H1 H2.
induction L2.
simpl in H2. contradiction.

simpl in H2.
elim H2; intro H3; clear H2.

simpl.
rewrite H3.
clear IHL2.
apply in_or_app. left.
induction L1. simpl in H1. contradiction.
simpl in H1. elim H1; intro H2; clear H1.
rewrite H2. simpl.
left. trivial.
simpl. right. apply IHL1. assumption.

simpl.
apply in_or_app.
right. apply IHL2. assumption.
Qed.

(* This is tricky part of match_list_atomic_list_complete *)
(* Basically it says we allowed to do presubstitution in body
after matching the head and we still match and get all substitutions *)

(*
Lemma presubs2 : DONT USE
  forall (LT2 : list term)(s s3 : substitution)(a LT1 : list term),
    ground_term_list LT2 = true ->
    match_term_list a LT2 = Some s3 ->
    multisubs_term_list s (multisubs_term_list s3 LA1)
    = multisubs_term_list s LA1.
*)

(* An unambiguous substitution is one that doesn't repeat vars *)
(* subs tail, head element, remaining to check *)
Inductive unambiguous_subs_aux : (nat * rval) -> substitution -> Prop :=
| unambiguous_subs_aux_nil :
  forall (x : nat)(v : rval),
    unambiguous_subs_aux (x, v) nil
| unambiguous_subs_aux_cons :
  forall (x x2 : nat)(v v2 : rval)(L : substitution),
    unambiguous_subs_aux (x, v) L ->
    (x <> x2) ->
    unambiguous_subs_aux (x, v) ((x2, v2) :: L).
Inductive unambiguous_subs : substitution -> Prop :=
| unambiguous_subs_nil :
  unambiguous_subs nil
| unambiguous_subs_cons :
  forall (H : (nat * rval))(T : substitution),

```

```

    unambiguous_subs T ->
    unambiguous_subs_aux H T ->
    unambiguous_subs (H :: T).

Lemma multisubs_term_list_subs :
  forall (a : (nat * rval))(s : substitution)(l : list term),
    multisubs_term_list (a :: s) l = multisubs_term_list s (subs_term_list a l).
Proof.
  unfold multisubs_term_list.
  induction l. simpl. trivial.
  simpl. rewrite IHl. trivial.
Qed.

Lemma var_occurs_term_list_multisubs :
  forall (s : substitution)(x : nat)(l : list term),
    var_occurs_term_list x (multisubs_term_list s l) = true ->
    var_occurs_term_list x l = true.
Proof.
  induction s; intros x l H.
  assert (multisubs_term_list nil l = l).
  apply multisubs_term_list_nil.
  rewrite H0 in H. assumption.

  assert (multisubs_term_list (a :: s) l = multisubs_term_list s (subs_term_list
    a l)).
  apply multisubs_term_list_subs.
  rewrite H0 in H. clear H0.
  assert (var_occurs_term_list x (subs_term_list a l) = true).
  apply IHs. assumption.
  clear IHs H.

  induction l.
  simpl in H0. assumption.
  simpl in H0.
  unfold var_occurs_term_list in H0.
  unfold exists_bool in H0.
  fold exists_bool in H0.
  assert (var_occurs_term x (subs_term a a0) = true \/ var_occurs_term x (subs_t
    erm a a0) = false).
  induction (var_occurs_term x (subs_term a a0)); auto.
  elim H; intro H1; clear H.
  induction a0.
  assert (subs_term a (ident n) = ident n).
  apply subs_term_ident.
  rewrite H in H1.
  clear H. simpl in H1. discriminate.

  induction a.
  simpl in H1. discriminate.
  induction a.
  simpl in H1.

```



```

induction (eq_nat_dec a n).
induction b; simpl in H1; discriminate.
simpl in H1.
induction (eq_nat_dec n x).
rewrite a0.
unfold var_occurs_term_list.
unfold exists_bool.
simpl.
induction (eq_nat_dec x x). simpl. trivial.
contradiction b1. trivial.
discriminate.

rewrite H1 in H0.
simpl in H0. clear H1.
unfold var_occurs_term_list in IH1.
unfold var_occurs_term_list.
assert (exists_bool (var_occurs_term x) 1 = true).
apply IH1. assumption. clear H0 IH1.
simpl. rewrite H.
induction (var_occurs_term x a0); auto.
Qed.

Lemma var_occurs_term_multisubs:
  forall (x : nat)(s : substitution)(t : term),
    var_occurs_term x (multisubs_term s t) = true ->
      var_occurs_term x t = true.
Proof.
  intros.
  induction t.
  assert (multisubs_term s (ident n) = ident n).
  apply multisubs_term_ident.
  rewrite H0 in H. assumption.
  assert (multisubs_term s (kident k) = kident k).
  apply multisubs_term_kident.
  rewrite H0 in H. assumption.
  simpl.
  induction s.
  simpl in H. assumption.
  induction (eq_nat_dec n x). trivial.
  simpl in H.
  apply IHs.
  induction a.
  simpl in H.
  induction (eq_nat_dec a n).
  induction b0.
  assert (multisubs_term s (ident n0) = ident n0).
  apply multisubs_term_ident.
  rewrite H0 in H. simpl in H. discriminate.
  assert (multisubs_term s (kident k) = kident k).
  apply multisubs_term_kident.
  rewrite H0 in H. simpl in H. discriminate.

```

```

    assumption.

Qed.

Lemma var_occurs_atomic_multisubs :
  forall (s : substitution)(x : nat)(a : atomic),
    var_occurs_atomic x (multisubs_atomic s a) = true ->
      var_occurs_atomic x a = true.
Proof.
  intros.
  induction a.
  simpl. simpl in H.
  apply var_occurs_term_list_multisubs with (s:=s).
  unfold multisubs_term_list. assumption.
  simpl. simpl in H.
  apply var_occurs_term_list_multisubs with (s:=s).
  unfold multisubs_term_list. assumption.

  simpl in H. simpl. apply var_occurs_term_multisubs with (s:=s). assumption.
  simpl in H. simpl. apply var_occurs_term_multisubs with (s:=s). assumption.
Qed.

Lemma var_occurs_atomic_list_multisubs :
  forall (s : substitution)(x : nat)(la : list atomic),
    var_occurs_atomic_list x (multisubs_atomic_list s la) = true ->
      var_occurs_atomic_list x la = true.
Proof.
  induction la.
  simpl. trivial.
  intros.
  assert (var_occurs_atomic x (multisubs_atomic s a) = true /\ var_occurs_atomic
    _list x (multisubs_atomic_list s la) = true).
  simpl in H.
  unfold var_occurs_atomic_list in H.
  simpl in H.
  induction (var_occurs_atomic x (multisubs_atomic s a)). left; trivial.
  simpl in H. right. unfold var_occurs_atomic_list. assumption.
  unfold var_occurs_atomic_list.
  simpl.
  elim H0; intro H1; clear H0.
  assert (var_occurs_atomic x a = true).
  apply var_occurs_atomic_multisubs with (s:=s).
  assumption.
  induction (var_occurs_atomic x a).
  simpl. trivial. discriminate.
  induction (var_occurs_atomic x a). simpl. trivial.
  simpl.
  unfold var_occurs_atomic_list in IH1a.
  apply IH1a.
  unfold var_occurs_atomic_list in H1. assumption.
Qed.

```

```

Lemma var_occurs_match_term_list_aux :
  forall (n : nat)(l l0 : list term)(s : substitution),
    length l = n ->
      match_term_list l l0 = Some s ->
        forall (x : nat)(v : rval),
          In (x, v) s ->
            var_occurs_term_list x l = true.
Proof.
  induction n.
  intros.
  induction l.
  induction l0.
  simpl in H0. injection H0; intro H2. rewrite <- H2 in H1. simpl in H1. contradict
  iction.
  simpl in H0. discriminate.
  simpl in H. discriminate.
  intros.
  induction l; induction l0.
  simpl in H. discriminate.
  simpl in H. discriminate.
  simpl in H0. discriminate.
  clear IHl IHl0.
  simpl in H. injection H; intro Hlen; clear H.

  simpl in H0.
  assert ({exists s2, match_term a a0 = Some s2}+{match_term a a0 = None}).
  apply option_dec.
  elim H; intro H2; clear H.
  elim H2; intros s2 H3; clear H2.
  rewrite H3 in H0.

  assert ({exists s3, match_term_list (multisubs_term_list s2 l) l0 = Some s3}+
    {match_term_list (multisubs_term_list s2 l) l0 = None}).
  apply option_dec.
  elim H; intro H4; clear H.
  elim H4; intros s3 H5; clear H4.
  rewrite H5 in H0.
  injection H0; intro H6; clear H0.
  rewrite <- H6 in H1.
  assert (In (x,v) s2 \/ In (x,v) s3).
  apply in_app_or. assumption.
  clear H1 H6.
  elim H; intro H6; clear H.

  (* Don't need IH for this side, x in s2 *)
  clear IHn.
  induction a; induction a0.
  simpl in H3.
  induction (eq_nat_dec n0 n1).
  injection H3; intro H7; clear H3. rewrite <- H7 in H6. simpl in H6. contradict

```

```

ion.
discriminate.
simpl in H3. discriminate.
simpl in H3; discriminate.
simpl in H3; discriminate.
simpl in H3.
induction (eq_kthing_dec k k0).
injection H3; intro H7; clear H3. rewrite <- H7 in H6. simpl in H6. contradict
ion.
discriminate.
simpl in H3. discriminate.
simpl in H3. injection H3; intro H7; clear H3.
rewrite <- H7 in H6. simpl in H6.
elim H6; intro H8; clear H6.
injection H8; intros H9 H10.
rewrite <- H10.
unfold var_occurs_term_list.
unfold exists_bool.
simpl. induction (eq_nat_dec n0 n0). auto.
unfold not in b. contradiction b. trivial.
contradiction.

simpl in H3. injection H3; intro H7; clear H3.
rewrite <- H7 in H6. simpl in H6.
elim H6; intro H8; clear H6.
injection H8; intros H9 H10.
rewrite <- H10.
unfold var_occurs_term_list.
unfold exists_bool.
simpl. induction (eq_nat_dec n0 n0). auto.
unfold not in b. contradiction b. trivial.
contradiction.

simpl in H3. discriminate.

(* Need IH for this side, x in s3 *)
assert (var_occurs_term_list x (multisubs_term_list s2 l) = true).
apply IHn with (l:=multisubs_term_list s2 l)(l0:=l0)(x:=x)(v:=v)(s:=s3).
assert (length (multisubs_term_list s2 l) = length l).
apply length_multisubs_term_list.
rewrite H. assumption. assumption. assumption.
assert (var_occurs_term_list x l = true).
apply var_occurs_term_list_multisubs with (s:=s2). assumption.
unfold var_occurs_term_list.
unfold var_occurs_term_list in H0.
simpl.
rewrite H0.
induction (var_occurs_term x a); auto.

rewrite H4 in H0. discriminate.

```

```

    rewrite H2 in H0. discriminate.
Qed.

```

```

Lemma var_occurs_match_term_list :
  forall (l l0 : list term)(s : substitution),
    match_term_list l l0 = Some s ->
      forall (x : nat)(v : rval),
        In (x, v) s ->
          var_occurs_term_list x l = true.

```

```

Proof.
  intros.
  apply var_occurs_match_term_list_aux with (x:=x)(l:=l)(n:=length l)(l0:=l0)(s:=s)(v:=v); auto.
Qed.

```

```

Lemma subs_agree :
  forall (s s3 : substitution),
    (forall (x : nat*rval)(LA1 : list atomic),
      In x s3 ->
        multisubs_atomic_list s (subs_atomic_list x LA1)
        = multisubs_atomic_list s LA1) ->
    (forall (LA1 : list atomic),
      multisubs_atomic_list s (multisubs_atomic_list s3 LA1)
      = multisubs_atomic_list s LA1).

```

```

Proof.
  induction s3.
  intros H1 LA1.
  assert (multisubs_atomic_list nil LA1 = LA1).
  apply multisubs_atomic_list_nil.
  rewrite H. trivial.
  intros H LA1.
  assert (multisubs_atomic_list (a :: s3) LA1 =
    multisubs_atomic_list s3 (subs_atomic_list a LA1)).
  apply multisubs_atomic_list_cons.
  rewrite H0.
  clear H0.
  assert (multisubs_atomic_list s (subs_atomic_list a LA1) =
    multisubs_atomic_list s LA1).
  apply H. simpl. left. trivial.
  rewrite <- H0.
  clear H0.
  apply IHs3 with (LA1:=subs_atomic_list a LA1).
  intros x LA2 H2.
  apply H.
  simpl. right. assumption.
Qed.

```

```

Lemma multisubs_term_nil :
  forall (t : term),
    multisubs_term nil t = t.

```

Proof.

```
  intros.
  unfold multisubs_term.
  simpl. trivial.
```

Qed.

Lemma agreemulti_term :

```
  forall (s3 s : substitution)(a : term)(x : nat),
    multisubs_term s3 a = multisubs_term s a ->
    var_occurs_term x a = true ->
    multisubs_term s3 (var x) = multisubs_term s (var x).
```

Proof.

```
  intros.
  induction a.
  simpl in H0. discriminate.
  simpl in H0; discriminate.
  simpl in H0. induction (eq_nat_dec n x).
  rewrite <- a. assumption. discriminate.
```

Qed.

Lemma agreemulti :

```
  forall (s3 s : substitution)(a : atomic)(x : nat),
    multisubs_atomic s3 a = multisubs_atomic s a ->
    var_occurs_atomic x a = true ->
    multisubs_term s3 (var x) = multisubs_term s (var x).
```

Proof.

```
  intros.
  induction a.
  induction l.
  simpl in H0.
  unfold var_occurs_term_list in H0.
  simpl in H0. discriminate.
  simpl in H0.
  unfold var_occurs_term_list in H0.
  simpl in H0.
  assert (var_occurs_term x a = true /\ exists_bool (var_occurs_term x) l = true)
  ).
  induction (var_occurs_term x a). left. trivial. right. simpl in H0. assumption
  .
  clear H0.
  elim H1; intro H2; clear H1.
  simpl in H.
  injection H. intros H3 H4. clear H.
  (* apply agreemulti_term with (s:=s)(s3:=s3)(a:=var x). *)
```

```
  (* The a here is a term, not atomic, should change the name *)
  induction a. simpl in H2. discriminate.
  simpl in H2. discriminate.
  simpl in H2.
  assert (n0 = x). induction (eq_nat_dec n0 x). assumption. discriminate.
  clear H2.
```

```

rewrite <- H. assumption.

apply IH1.
simpl in H.
simpl.
injection H; intros H3 H4; clear H.
rewrite H3. trivial.
unfold var_occurs_atomic.
unfold var_occurs_term_list. assumption.

induction l.
simpl in H0.
unfold var_occurs_term_list in H0.
simpl in H0. discriminate.
simpl in H0.
unfold var_occurs_term_list in H0.
simpl in H0.
assert (var_occurs_term x a = true  $\wedge$  exists_bool (var_occurs_term x) l = true
).
induction (var_occurs_term x a). left. trivial. right. simpl in H0. assumption
.
clear H0.
elim H1; intro H2; clear H1.
simpl in H.
injection H. intros H3 H4. clear H.
(* The a here is a term, not atomic, should change the name *)
apply agreemulti_term with (a:=a). assumption. assumption.

apply IH1.
simpl in H.
simpl.
injection H; intros H3 H4; clear H.
rewrite H3. trivial.
unfold var_occurs_atomic.
unfold var_occurs_term_list. assumption.

simpl in H0.
simpl in H.
injection H. intro H3. clear H.
apply agreemulti_term with (a:=t).
assumption. assumption.

simpl in H.
injection H; intro H3; clear H.
simpl in H0.
apply agreemulti_term with (a:=t).
assumption. assumption.
Qed.

Lemma unambiguous_notin :
  forall (s : substitution)(x : nat)(v : rval),

```

```

unambiguous_subs ((x, v) :: s) ->
  (forall (x2 : nat)(v2 : rval), In (x2, v2) s -> x <> x2).

```

Proof.

```

intros s x v H.
inversion_clear H.
clear H1.
induction s.
auto.
intros.
simpl in H.
elim H; intro H3; clear H.
rewrite H3 in H2. clear H3.
inversion H2. assumption.
apply IHs with (x2:=x2)(v2:=v2).
inversion H2. assumption. assumption.

```

Qed.

```

Definition val_of_rval (v : rval) :=
  match v with
  | rvident i => ident i
  | rvkident k => kident k
  end.

```

```

Lemma unambiguous_lemma :
  forall (s3 : substitution)(x : nat)(v : rval),
    In (x, v) s3 ->
      unambiguous_subs s3 ->
        multisubs_term s3 (var x) = val_of_rval v.

```

Proof.

```

intros.
induction s3.
simpl in H; contradiction.
simpl in H.
elim H; intro H1; clear H.
simpl.
rewrite H1.
assert (subs_term (x, v) (var x) = val_of_rval v).
simpl. induction (eq_nat_dec x x). trivial. contradiction b. trivial.
rewrite H.
induction v.
simpl.
apply multisubs_term_ident.
simpl.
apply multisubs_term_kident.
simpl.
induction a.
assert (a<>x).
apply unambiguous_notin with (x:=a)(v:=b)(x2:=x)(v2:=v)(s:=s3).
assumption. assumption.
simpl.
induction (eq_nat_dec a x). contradiction.

```



```

clear b0.
apply IHs3. assumption.
inversion_clear H0. assumption.
Qed.

```

```

Lemma presubs_pre :
  forall (s3 s : substitution)(a : atomic)(x : nat)(v : rval),
    multisubs_atomic s3 a = multisubs_atomic s a ->
      In (x, v) s3 ->
        var_occurs_atomic x a = true ->
          unambiguous_subs s3 ->
            multisubs_term s (var x) = val_of_rval v.

```

```

Proof.
  intros.
  assert (multisubs_term s3 (var x) = multisubs_term s (var x)).
  apply agreemulti with (a:=a).
  assumption. assumption.
  rewrite <- H3. clear H3.
  apply unambiguous_lemma. assumption. assumption.
Qed.

```

```

Lemma match_term_unambiguous :
  forall (t t0 : term)(s : substitution),
    match_term t t0 = Some s ->
      unambiguous_subs s.

```

```

Proof.
  intros.
  induction t; induction t0; simpl in H.
  induction (eq_nat_dec n n0). injection H; intro H2. rewrite <- H2. apply unamb
  iguous_subs_nil.
  discriminate. discriminate. discriminate. discriminate.
  induction (eq_kthing_dec k k0).
  injection H; intro H2; rewrite <- H2. apply unambiguous_subs_nil.
  discriminate. discriminate.
  injection H; intro H2. rewrite <- H2. apply unambiguous_subs_cons.
  apply unambiguous_subs_nil.
  apply unambiguous_subs_aux_nil.
  injection H; intro H2. rewrite <- H2. apply unambiguous_subs_cons.
  apply unambiguous_subs_nil.
  apply unambiguous_subs_aux_nil.
  discriminate.
Qed.

```

```

Lemma unambiguous_notin_reverse :
  forall (s : substitution)(x : nat)(v : rval),
    (forall (x2 : nat)(v2 : rval), In (x2, v2) s -> x <> x2) ->
      unambiguous_subs_aux (x, v) s.

```

```

Proof.
  intros.
  induction s.
  apply unambiguous_subs_aux_nil.

```

```

induction a.
apply unambiguous_subs_aux_cons.
apply IHs.
intros.
apply H with (x2:=x2)(v2:=v2).
simpl; right; assumption.
apply H with (x2:=a)(v2:=b).
simpl. left; trivial.
Qed.

Lemma unambiguous_plus_unambiguous :
forall (s s2 : substitution),
  unambiguous_subs s ->
  unambiguous_subs s2 ->
  (forall (x x2 : nat)(v v2 : rval), In (x, v) s -> In (x2, v2) s2 -> x <> x2)
  ->
  unambiguous_subs (s ++ s2).
Proof.
  intros.
  induction s.
  simpl. assumption.
  simpl.
  assert (unambiguous_subs (s ++ s2)).
  apply IHs.
  inversion H. assumption.
  intros.
  apply H1 with (v:=v)(v2:=v2).
  simpl; right; assumption. assumption.
  apply unambiguous_subs_cons. assumption.
  induction a.
  cut (forall (x3 : nat)(v3 : rval), In (x3, v3) (s ++ s2) -> a <> x3).
  intro H3.
  apply unambiguous_notin_reverse. assumption.
  intros.
  assert (In (x3, v3) s \ / In (x3, v3) s2).
  apply in_app_or.
  assumption. clear H3.
  elim H4; intro H5; clear H4.
  apply unambiguous_notin with (s:=s)(x:=a)(v:=b)(x2:=x3)(v2:=v3).
  assumption. assumption.
  clear IHs.
  apply H1 with (x:=a)(v:=b)(x2:=x3)(v2:=v3).
  simpl; left; trivial. assumption.
Qed.

Lemma match_term_var_appears :
forall (t0 t : term)(s : substitution)(x : nat)(v : rval),
  match_term t t0 = Some s ->
  In (x, v) s ->
  var_occurs_term x t = true.

```

Proof.

```

intros.
induction t; induction t0; simpl in H.
induction (eq_nat_dec n n0).
injection H; intro H1. rewrite <- H1 in H0. simpl in H0; contradiction.
discriminate. discriminate. discriminate. discriminate.
induction (eq_kthing_dec k k0).
injection H; intro H1; rewrite <- H1 in H0. simpl in H0. contradiction.
discriminate. discriminate.
injection H; intro H1. rewrite <- H1 in H0.
simpl in H0. elim H0; intro H2; clear H0.
clear H H1.
injection H2; intros H3 H4; clear H2.
rewrite H4.
simpl.
induction (eq_nat_dec x x). trivial. contradiction b. trivial. contradiction.
injection H; intro H1. rewrite <- H1 in H0.
simpl in H0. elim H0; intro H2; clear H0.
clear H H1.
injection H2; intros H3 H4; clear H2.
rewrite H4.
simpl.
induction (eq_nat_dec x x). trivial. contradiction b. trivial. contradiction.

discriminate.

```

Qed.

Lemma var_occurs_multisubs:

```

forall (s : substitution)(x : nat)(l : list term),
  var_occurs_term_list x (multisubs_term_list s l) = true ->
  var_occurs_term_list x l = true.

```

Proof.

```

induction s. intros.
assert (multisubs_term_list nil l = l).
apply multisubs_term_list_nil.
rewrite H0 in H. assumption.
intros.
induction a.
assert (multisubs_term_list ((a,b)::s) l = multisubs_term_list s (subs_term_list (a,b) l)).
apply multisubs_term_list_lemma2.
rewrite H0 in H. clear H0.
assert (var_occurs_term_list x (subs_term_list (a, b) l) = true).
apply IHs with (l:=subs_term_list (a,b) l).
assumption.
clear H. clear IHs.
induction l.
simpl in H0; assumption.
unfold var_occurs_term_list in H0.
unfold var_occurs_term_list in IH1.
unfold var_occurs_term_list.

```

```

induction a0.
simpl in H0. apply IH1; assumption.
simpl in H0. apply IH1; assumption.

simpl in H0. induction (eq_nat_dec a n).
induction b.
simpl in H0.
simpl. assert (exists_bool (var_occurs_term x) 1 = true).
apply IH1; assumption.
induction (eq_nat_dec n x). simpl. trivial.
simpl. assumption.
simpl in H0.
simpl. assert (exists_bool (var_occurs_term x) 1 = true).
apply IH1; assumption.
induction (eq_nat_dec n x). simpl. trivial.
simpl. assumption.

simpl in H0.
induction (eq_nat_dec n x).
rewrite <- a0.
simpl. induction (eq_nat_dec n n). simpl. trivial. contradiction b1. trivial.
simpl in H0.
assert (exists_bool (var_occurs_term x) 1 = true).
apply IH1; assumption.
simpl. induction (eq_nat_dec n x). simpl; trivial. simpl. assumption.
Qed.

Lemma match_term_list_var_appears :
  forall (l0 l : list term)(s : substitution)(x : nat)(v : rval),
    match_term_list l l0 = Some s ->
      In (x, v) s ->
        var_occurs_term_list x l = true.
Proof.
  induction l0; induction l; intros; simpl in H.
  injection H; intro H1. rewrite <- H1 in H0; simpl in H0; contradiction.
  discriminate. discriminate.
  clear IH1.
  assert ({exists s2, match_term a0 a = Some s2}+{match_term a0 a = None}).
  apply option_dec.
  elim H1; intro H2; clear H1.
  elim H2; intros s2 H3; clear H2.
  rewrite H3 in H.
  assert ({exists s3, match_term_list (multisubs_term_list s2 l) l0 = Some s3}+{
    match_term_list (multisubs_term_list s2 l) l0 = None}).
  apply option_dec.
  elim H1; intro H2; clear H1.
  elim H2; intros s3 H4; clear H2.
  rewrite H4 in H.
  injection H; intro H5; clear H.
  rewrite <- H5 in H0.
  assert (In (x, v) s2 \/ In (x, v) s3).

```

```

apply in_app_or. assumption. clear H0.
elim H; intro H6; clear H.
assert (var_occurs_term x a0 = true).
apply match_term_var_appears with (t0:=a)(t:=a0)(s:=s2)(x:=x)(v:=v).
assumption.
unfold var_occurs_term_list.
simpl.
induction (var_occurs_term x a0). simpl. trivial.
discriminate.
assert (var_occurs_term_list x (multisubs_term_list s2 l) = true).
apply IH10 with (l:=multisubs_term_list s2 l)(s:=s3)(x:=x)(v:=v).
assumption. assumption.

assert (var_occurs_term_list x l = true).
apply var_occurs_multisubs with (s:=s2)(x:=x)(l:=l). assumption.
unfold var_occurs_term_list. simpl.
induction (var_occurs_term x a0).
simpl. trivial. simpl. unfold var_occurs_term_list in H0. assumption.

rewrite H2 in H. discriminate.
rewrite H2 in H. discriminate.
Qed.

Lemma match_atomic_var_appears :
  forall (a0 a : atomic)(s : substitution)(x : nat)(v : rval),
    match_atomic a a0 = Some s ->
      In (x, v) s ->
        var_occurs_atomic x a = true.
Proof.
  intros.
  induction a; induction a0.
  simpl in H.
  induction (eq_nat_dec n n0).
  simpl.
  apply match_term_list_var_appears with (l:=l)(l0:=l0)(s:=s)(x:=x)(v:=v).
  assumption. assumption.
  discriminate.
  simpl in H. discriminate.
  simpl in H. discriminate.
  simpl in H. discriminate.
  simpl in H. discriminate.
  simpl in H.
  induction (eq_nat_dec n0 n2).
  induction (eq_nat_dec n n1).
  simpl.
  apply match_term_list_var_appears with (l:=l)(l0:=l0)(s:=s)(x:=x)(v:=v).
  assumption. assumption.
  discriminate. discriminate.

  simpl in H. discriminate.
  simpl in H. discriminate.

```

```

simpl in H. discriminate.
simpl in H. discriminate.

simpl in H. simpl.
induction t; induction t0; simpl in H; try (induction (eq_nat_dec n n0); discriminate).
induction (eq_nat_dec n n0); try discriminate.
injection H; intro H2; rewrite <- H2 in H0; simpl in H0; contradiction.
discriminate. discriminate.
induction (eq_kthing_dec k k0); try discriminate.
injection H; intro H2; rewrite <- H2 in H0; simpl in H0; contradiction.
discriminate.
injection H; intro H2. rewrite <- H2 in H0.
simpl in H0. elim H0; intro H1; clear H0. injection H1; intros H3 H4; clear H1
.
rewrite H4. simpl. induction (eq_nat_dec x x). trivial. absurd (x=x). assumption.
trivial. contradiction.
injection H; intro H2; clear H. rewrite <- H2 in H0. simpl in H0.
elim H0; intro H1; clear H0.
injection H1; intros H3 H4; clear H1.
rewrite H4. simpl. induction (eq_nat_dec x x). trivial. absurd (x=x). assumption.
trivial. contradiction.
simpl in H. discriminate.
simpl in H. discriminate.
simpl in H. discriminate.
simpl in H. discriminate.

simpl in H. simpl.
induction t; induction t0; simpl in H; try (induction (eq_nat_dec n n0); discriminate).
induction (eq_nat_dec n n0); induction (eq_nat_dec n1 n2); try discriminate.
injection H; intro H2; rewrite <- H2 in H0; simpl in H0; contradiction.
induction (eq_nat_dec n n0); try discriminate.
induction (eq_kthing_dec k k0); try discriminate.
injection H; intro H2; rewrite <- H2 in H0; simpl in H0; contradiction.
induction (eq_nat_dec n n0); try discriminate.
injection H; intro H2. rewrite <- H2 in H0.
simpl in H0. elim H0; intro H1; clear H0. injection H1; intros H3 H4; clear H1
.
rewrite H4. simpl. induction (eq_nat_dec x x). trivial. absurd (x=x). assumption.
trivial. contradiction.
induction (eq_nat_dec n n0); try discriminate.
injection H; intro H2; clear H. rewrite <- H2 in H0. simpl in H0.
elim H0; intro H1; clear H0.
injection H1; intros H3 H4; clear H1.
rewrite H4. simpl. induction (eq_nat_dec x x). trivial. absurd (x=x). assumption.
trivial. contradiction.

```

Qed.

Lemma match_atomic_list_var_appears :

```

forall (a : atomic)(LA : list atomic)(s : substitution)(x : nat)(v : rval),

```

```

      In s (match_atomic_list a LA) ->
      In (x, v) s ->
      var_occurs_atomic x a = true.
Proof.
  intros.
  induction LA.
  simpl in H. contradiction.
  simpl in H.
  assert ({exists s2, match_atomic a a0 = Some s2}+{match_atomic a a0 = None}).
  apply option_dec.
  elim H1; intro H2; clear H1.
  elim H2; intros s2 H3; clear H2.
  rewrite H3 in H.
  simpl in H.
  elim H; intro H4; clear H.
  apply match_atomic_var_appears with (a0:=a0)(a:=a)(x:=x)(v:=v)(s:=s2).
  assumption.
  rewrite H4. assumption.
  apply IHLA. assumption.
  rewrite H2 in H.
  apply IHLA. assumption.
Qed.

```

```

Lemma var_occurs_term_list_subs_not :
  forall (x : nat)(v : rval)(l : list term),
    var_occurs_term_list x (subs_term_list (x,v) l) = false.

```

```

Proof.
  intros.
  induction l.
  simpl.
  unfold var_occurs_term_list.
  simpl. trivial.
  unfold var_occurs_term_list.
  unfold var_occurs_term_list in IHl.
  induction a.
  simpl. assumption.
  simpl. assumption.
  simpl.
  induction (eq_nat_dec x n).
  induction v.
  simpl. assumption.
  simpl. assumption.
  simpl.
  induction (eq_nat_dec n x).
  contradiction b. rewrite a; trivial.
  simpl. assumption.
Qed.

```

```

Lemma one_more_sub_notin :
  forall (x x2 : nat)(v : rval)(l : list term),
    var_occurs_term_list x2 l = false ->
      var_occurs_term_list x2 (subs_term_list (x, v) l) = false.
Proof.
  intros.
  induction l.
  simpl. assumption.
  unfold var_occurs_term_list in H.
  induction a.
  simpl in H.
  assert (var_occurs_term_list x2 (subs_term_list (x, v) l) = false).
  apply IHl.
  unfold var_occurs_term_list in IHl.
  unfold var_occurs_term_list. assumption.
  clear IHl H.
  unfold var_occurs_term_list.
  simpl. unfold var_occurs_term_list in H0.
  assumption.

  simpl in H.
  assert (var_occurs_term_list x2 (subs_term_list (x, v) l) = false).
  apply IHl.
  unfold var_occurs_term_list in IHl.
  unfold var_occurs_term_list. assumption.
  clear IHl H.
  unfold var_occurs_term_list.
  simpl. unfold var_occurs_term_list in H0.
  assumption.

  unfold var_occurs_term_list in IHl.
  unfold var_occurs_term_list.
  simpl in H.
  induction (eq_nat_dec n x2).
  simpl in H. discriminate.
  simpl in H. simpl.
  induction (eq_nat_dec x n).
  induction v.
  simpl. apply IHl. assumption.
  simpl. apply IHl. assumption.
  simpl. induction (eq_nat_dec n x2).
  contradiction.
  simpl.
  apply IHl. assumption.
Qed.

Lemma var_occurs_term_list_multisubs_not :
  forall (s : substitution)(x : nat)(l : list term),
    var_occurs_term_list x l = false ->
      var_occurs_term_list x (multisubs_term_list s l) = false.

```



```

Proof.
  induction s.
  intros.
  assert (multisubs_term_list nil l = l).
  apply multisubs_term_list_nil.
  rewrite H0. clear H0. assumption.
  intros.
  induction a.
  assert (multisubs_term_list ((a,b)::s) l = multisubs_term_list s (subs_term_list (a,b) l)).
  apply multisubs_term_list_lemma2.
  rewrite H0. clear H0.
  apply IHs.
  apply one_more_sub_notin. assumption.
Qed.

```

```

Lemma var_occurs_term_list_multisubs_not_in :
  forall (s : substitution)(l : list term)(x : nat)(v : rval),
    var_occurs_term_list x (multisubs_term_list s l) = true ->
      In (x, v) s ->
        False.

```

```

Proof.
  induction s.
  intros. simpl in H0. contradiction.
  intros.
  induction a.
  assert (multisubs_term_list ((a,b) :: s) l = multisubs_term_list s (subs_term_list (a,b) l)).
  apply multisubs_term_list_lemma2.
  rewrite H1 in H. clear H1.
  simpl in H0.
  elim H0; intro H1; clear H0.
  injection H1; intros H2 H3; clear H1.
  rewrite H3 in H.
  clear a v H2 H3 IHs.
  assert (var_occurs_term_list x (multisubs_term_list s (subs_term_list (x,b) l)) = false).
  apply var_occurs_term_list_multisubs_not.
  apply var_occurs_term_list_subs_not.
  rewrite H0 in H. discriminate.

  apply IHs with (l:=subs_term_list (a,b) l)(x:=x)(v:=v).
  assumption. assumption.
Qed.

```

```

Lemma match_term_list_unambiguous :
  forall (l0 l : list term)(s : substitution),
    match_term_list l l0 = Some s ->
      unambiguous_subs s.

```

Proof.

```

induction l0; induction l; intros.
simpl in H. injection H; intro H1. rewrite <- H1. apply unambiguous_subs_nil.
simpl in H. discriminate.
simpl in H. discriminate.
clear IH1.
simpl in H.
assert ({exists s2, match_term a0 a = Some s2}+{match_term a0 a = None}).
apply option_dec.
elim H0; intro H1; clear H0.
elim H1; intros s3 H2; clear H1.
rewrite H2 in H.
assert ({exists s4, match_term_list (multisubs_term_list s3 l) l0 = Some s4}+{
match_term_list (multisubs_term_list s3 l) l0 = None}).
apply option_dec.
elim H0; intro H3; clear H0.
elim H3; intros s4 H4; clear H3.
rewrite H4 in H.
injection H; intro H5; clear H.
rewrite <- H5; clear H5.
apply unambiguous_plus_unambiguous.
apply match_term_unambiguous with (t:=a0)(t0:=a).
assumption.
apply IHl0 with (l:=multisubs_term_list s3 l).
assumption.

```

```

intros.
assert (var_occurs_term_list x2 (multisubs_term_list s3 l) = true).
apply match_term_list_var_appears with (l0:=l0)(l:=multisubs_term_list s3 l)(s
:=s4)(x:=x2)(v:=v2). assumption. assumption.

```

(* Where I'm at: showing that if a var appears in result of match_term list, must have been in first arg *)

(* Then show after substitution, it cannot appear there *)

(* This will show that match_term_list is unambiguous *)

(* Bigger picture: doing lemmas for presubs, need to show result from match_atom ic_list is unambiguous *)

```

intros.
assert (In (x2, v) s3 -> False).
intro H5.
apply var_occurs_term_list_multisubs_not_in with (s:=s3)(l:=l)(x:=x2)(v:=v).
assumption. assumption.
unfold not.
intro H5. rewrite H5 in H.
contradiction.

```

```

rewrite H3 in H. discriminate.
rewrite H1 in H. discriminate.

```

Qed.

```

Lemma match_atomic_unambiguous :
  forall (s : substitution)(a a0 : atomic),
    match_atomic a a0 = Some s ->
      unambiguous_subs s.
Proof.
  intros.
  induction a.
  induction a0.
  simpl in H.
  induction (eq_nat_dec n n0).
  apply match_term_list_unambiguous with (l:=1)(l0:=l0).
  assumption.
  discriminate.
  simpl in H. discriminate.
  simpl in H. discriminate.
  simpl in H. discriminate.

  induction a0.
  simpl in H. discriminate.
  simpl in H.
  induction (eq_nat_dec n0 n2).
  induction (eq_nat_dec n n1).
  apply match_term_list_unambiguous with (l:=1)(l0:=l0).
  assumption.
  discriminate.
  discriminate.

  simpl in H. discriminate.
  simpl in H. discriminate.

  induction a0; try (simpl in H; discriminate).
  simpl in H.
  induction t; induction t0; simpl in H; try induction (eq_nat_dec n n0); try di
  discriminate.
  injection H; intro H1; clear H.
  rewrite <- H1.
  apply unambiguous_subs_nil.
  induction (eq_kthing_dec k k0).
  injection H; intro H1; clear H.
  rewrite <- H1.
  apply unambiguous_subs_nil. discriminate.
  injection H; intro H1; clear H.
  rewrite <- H1.
  apply unambiguous_subs_cons.
  apply unambiguous_subs_nil.
  apply unambiguous_subs_aux_nil.
  injection H; intro H1; clear H.
  rewrite <- H1.
  apply unambiguous_subs_cons.
  apply unambiguous_subs_nil.
  apply unambiguous_subs_aux_nil.

```

```

injection H; intro H1; clear H.
rewrite <- H1.
apply unambiguous_subs_cons.
apply unambiguous_subs_nil.
apply unambiguous_subs_aux_nil.

induction a0; try (simpl in H; discriminate).
simpl in H.
induction (eq_nat_dec n n0).
induction t; induction t0; simpl in H; try induction (eq_nat_dec n1 n2); try d
iscriminate; try induction (eq_nat_dec n3 n4); try discriminate.
injection H; intro H1; clear H.

rewrite <- H1.
apply unambiguous_subs_nil.
induction (eq_kthing_dec k k0).
injection H; intro H1; clear H.
rewrite <- H1.
apply unambiguous_subs_nil. discriminate.
injection H; intro H1; clear H.
rewrite <- H1.
apply unambiguous_subs_cons.
apply unambiguous_subs_nil.
apply unambiguous_subs_aux_nil.
injection H; intro H1; clear H.
rewrite <- H1.
apply unambiguous_subs_cons.
apply unambiguous_subs_nil.
apply unambiguous_subs_aux_nil.
injection H; intro H1; clear H.
rewrite <- H1.
apply unambiguous_subs_cons.
apply unambiguous_subs_nil.
apply unambiguous_subs_aux_nil.

discriminate.
Qed.

Lemma match_atomic_list_unambiguous :
  forall (s : substitution)(a : atomic)(LA : list atomic),
    In s (match_atomic_list a LA) ->
      unambiguous_subs s.
Proof.
  intros.
  induction LA.
  simpl in H. contradiction.
  simpl in H.
  assert ({exists o, match_atomic a a0 = Some o}+{match_atomic a a0 = None}).
  apply option_dec.
  elim H0; intro H1; clear H0.
  elim H1; intros s2 H2; clear H1.

```

```

rewrite H2 in H.
simpl in H.
elim H; intro H3; clear H.
rewrite H3 in H2.
apply match_atomic_unambiguous with (a:=a)(a0:=a0)(s:=s).
assumption.
apply IHLA. assumption.
rewrite H1 in H.
apply IHLA. assumption.
Qed.

(* It's ok to presubstitute values from a match_atomic_list *)

Lemma presubs :
  forall (LA2 : list atomic)(s s3 : substitution)(a : atomic)(LA1 : list atomic)
  ,
    ground_atomic_list LA2 = true ->
    In s3 (match_atomic_list a LA2) ->
    multisubs_atomic s3 a = multisubs_atomic s a ->
    multisubs_atomic_list s (multisubs_atomic_list s3 LA1)
    = multisubs_atomic_list s LA1.
Proof.
  intros LA2 s s3 a LA1 H1 H2 H3.
  apply subs_agree.
  intros xx LA3 H4.
  induction xx as [x v].
  assert (var_occurs_atomic x a = true).
  apply match_atomic_list_var_appears with (v:=v)(s:=s3)(LA:=LA2).
  (* apply appears_in_match_atomic_list with (v:=v)(s3:=s3)(LA2:=LA2). *)
  assumption. assumption.
  cut (multisubs_term s (var x) = val_of_rval v). intro H5.
  clear H1 H2 H3 H4 H.
  induction LA3. simpl. trivial.
  simpl. rewrite IHLA3.
  cut (multisubs_atomic s (subs_atomic (x, v) a0) = multisubs_atomic s a0).
  intro H6. rewrite H6. trivial.
  clear IHLA3.
  induction a0.
  induction l.
  simpl. trivial.
  simpl.
  simpl in IHl. injection IHl. intro H6. clear IHl.
  rewrite <- H6.
  cut (
    (multisubs_term s
      match a0 with
      | ident _ => a0
      | kident _ => a0
      | var y => if eq_nat_dec x y then match v with | rvident i => ident i |
      rvkident k => kident k end else a0
    end :: map (multisubs_term s) (map (subs_term (x, v)) l)) =

```

```

(multisubs_term s a0
  :: map (multisubs_term s) (map (subs_term (x, v)) l)) ).
intro H7. rewrite H7. trivial.
cut (
  multisubs_term s
  match a0 with
  | ident _ => a0
  | kident _ => a0
  | var y => if eq_nat_dec x y then match v with | rvident i => ident i | rv
    kident k => kident k end else a0
  end = multisubs_term s a0).
intro H7. rewrite H7. trivial.
clear H6.
induction a0. trivial. trivial.
induction (eq_nat_dec x n0).
rewrite <- a0.
rewrite H5.
induction v; simpl.
apply multisubs_term_ident.
apply multisubs_term_kident.
trivial.

induction l.
simpl. trivial.
simpl.
simpl in IH1. injection IH1. intro H6. clear IH1.
rewrite <- H6.
cut (
  (multisubs_term s
    match a0 with
    | ident _ => a0
    | kident _ => a0
    | var y => if eq_nat_dec x y then match v with | rvident i => ident i | rv
      rvkident k => kident k end else a0
    end :: map (multisubs_term s) (map (subs_term (x, v)) l)) =
  (multisubs_term s a0
    :: map (multisubs_term s) (map (subs_term (x, v)) l)) ).
intro H7. rewrite H7. trivial.
cut (
  multisubs_term s
  match a0 with
  | ident _ => a0
  | kident _ => a0
  | var y => if eq_nat_dec x y then match v with | rvident i => ident i | rv
    kident k => kident k end else a0
  end = multisubs_term s a0).
intro H7. rewrite H7. trivial.
clear H6.
induction a0. trivial. trivial.
induction (eq_nat_dec x n1).
rewrite <- a0.

```

```

rewrite H5.
induction v; simpl.
apply multisubs_term_ident.
apply multisubs_term_kident.
trivial.

induction t.
simpl. trivial.
simpl. trivial.
simpl.
induction (eq_nat_dec x n).
induction v; simpl; simpl in H5.
rewrite <- a0.
rewrite H5.
assert (multisubs_term s (ident n0) = ident n0).
apply multisubs_term_ident.
rewrite H. trivial.
rewrite <- a0.
rewrite H5.
assert (multisubs_term s (kident k) = kident k).
apply multisubs_term_kident.
rewrite H. trivial.
trivial.

induction t.
simpl. trivial.
simpl. trivial.
simpl.
induction (eq_nat_dec x n0).
induction v; simpl; simpl in H5.
rewrite <- a0.
rewrite H5.
assert (multisubs_term s (ident n1) = ident n1).
apply multisubs_term_ident.
rewrite H. trivial.
rewrite <- a0.
rewrite H5.
assert (multisubs_term s (kident k) = kident k).
apply multisubs_term_kident.
rewrite H. trivial.
trivial.

cut (unambiguous_subs s3). intro H5.
apply presubs_pre with (s:=s)(s3:=s3)(x:=x)(v:=v)(a:=a); assumption.
apply match_atomic_list_unambiguous with (a:=a)(LA:=LA2)(s:=s3).
assumption.
Qed.

Lemma match_list_atomic_list_complete_n :
  forall (n : nat)(LA1 LA2 : list atomic)(s : substitution),
    length LA1 = n ->

```

```

ground_atomic_list LA2 = true ->
incl (multisubs_atomic_list s LA1) LA2 ->
(exists s2, In s2 (match_list_atomic_list LA1 LA2) /\ (multisubs_atomic_list
  s2 LA1 = multisubs_atomic_list s LA1)).
Proof.
induction n.

(* n = 0 *)
intros LA1 LA2 s H1 H2 H3.
induction LA1.
simpl.
exists (nil:substitution).
split. left. trivial. trivial.
simpl in H1. discriminate.

(* n > 0 *)

intros LA1 LA2 s H1 H2 H3.
induction LA1.
simpl in H1. discriminate.
clear IH1A1.

(* Split the incl hypothesis into parts *)

assert (In (multisubs_atomic s a) LA2).
simpl in H3.
unfold incl in H3. simpl in H3.
apply H3. left. trivial.

assert (incl (multisubs_atomic_list s LA1) LA2).
simpl in H3. unfold incl in H3. unfold incl.
intros a0 H4. apply H3. simpl. right. assumption.
clear H3.

(* What do we need to prove? *)
unfold match_list_atomic_list.
simpl.

(* Do some reasoning about match_atomic_list *)
(* Use atomic completeness *)
assert (exists s3, In s3 (match_atomic_list a LA2) /\ (multisubs_atomic s3 a =
  multisubs_atomic s a)).
apply match_atomic_list_complete.
assumption. assumption.
elim H3. intros s3 H4. clear H3. elim H4; intros H5 H6; clear H4.

(* Use IH with LA1 substituted *)
(* Check IHn(multisubs_atomic_list s3 LA1)(LA2). *)
assert (
  exists s4 : substitution,
    In s4 (match_list_atomic_list (multisubs_atomic_list s3 LA1) LA2) /\

```



```

    multisubs_atomic_list s4 (multisubs_atomic_list s3 LA1) =
      multisubs_atomic_list s (multisubs_atomic_list s3 LA1)).
  apply IHn with (LA1:=multisubs_atomic_list s3 LA1)(LA2:=LA2).
  assert (length (multisubs_atomic_list s3 LA1) = length LA1).
  apply length_multisubs_atomic_list.
  rewrite H3. simpl in H1. injection H1. trivial.
  assumption.

  assert (multisubs_atomic_list s (multisubs_atomic_list s3 LA1) =
    multisubs_atomic_list s LA1).
  apply presubs with (a:=a)(LA1:=LA1)(LA2:=LA2)(s3:=s3).
  assumption. assumption. assumption.
  rewrite H3. assumption.

  (* Get s4 into assumptions *)
  elim H3; intros s4 H7; clear H3.
  elim H7; intros H8 H9; clear H7.

  (* The final s2 that we need is s3 ++ s4 *)
  exists (s3 ++ s4).
  split.
  (* We now have two ugly things to prove. First one is technical relating
  to knowing that s3++s4 actually comes out of the match. The second is an
  equality between substitutions. *)
  assert (multisubs_atomic nil a = a).
  apply multisubs_atomic_nil.
  rewrite H3.
  assert (
    match_list_atomic_list_aux LA1 LA2 s3 =
      match_list_atomic_list (multisubs_atomic_list s3 LA1) LA2).
  apply match_list_atomic_list_aux_presubs2.
  (* Don't need lots of equations for this part *)
  clear H3 H9 H6 H0 H H2 H1 IHn.

  apply flatten_in.
  exists (map_append s3 (match_list_atomic_list_aux LA1 LA2 s3)).
  split.

  induction (match_atomic_list a LA2).
  simpl in H5. contradiction.
  simpl.
  simpl in H5.
  elim H5; intro H9; clear H5.
  left. rewrite H9. trivial.
  right. apply IH1. assumption.

  rewrite H4. clear H4.
  induction (match_list_atomic_list (multisubs_atomic_list s3 LA1) LA2).
  simpl in H8. contradiction.
  simpl in H8. elim H8; intro H9; clear H8.
  simpl. left. rewrite H9. trivial.

```

```

simpl. right. apply IH1. assumption.

(* Here is equality part *)

(* Heads must equal, and they do because after doing s3 we go to ground
   and s4 is ignored *)
assert (multisubs_atomic (s3 ++ s4) a = multisubs_atomic s a).
assert (multisubs_atomic (s3 ++ s4) a = multisubs_atomic s4 (multisubs_atomic
s3 a)).
apply multisubs_append.
rewrite H3. clear H3.
rewrite H6.
apply multisubs_atomic_ground.
apply ground_atomic_list_in with (LA:=LA2). assumption. assumption.
rewrite H3. clear H3.

(* Now show bodies are equal; they are by previous annoying presubs lemma
   that said doing s3 then s is the same as doing s *)
assert (multisubs_atomic_list (s3 ++ s4) LA1 =
  multisubs_atomic_list s4 (multisubs_atomic_list s3 LA1)).
apply multisubs_atomic_list_append.
rewrite H3. clear H3.
rewrite H9.
assert (multisubs_atomic_list s (multisubs_atomic_list s3 LA1) =
  multisubs_atomic_list s LA1).
apply presubs with (a:=a)(LA1:=LA1)(LA2:=LA2)(s3:=s3).
assumption. assumption. assumption.
rewrite H3. trivial.
Qed.

Lemma match_list_atomic_list_complete :
  forall (LA1 LA2 : list atomic)(s : substitution),
    ground_atomic_list LA2 = true ->
    incl (multisubs_atomic_list s LA1) LA2 ->
    (exists sbs, In sbs (match_list_atomic_list LA1 LA2) /\ (multisubs_atomic_li
st sbs LA1 = multisubs_atomic_list s LA1)).
Proof.
  induction LA1.
  intros LA2 s H1 H2.
  apply match_list_atomic_list_complete_n with (n:=0).
  simpl. trivial. assumption. assumption.
  intros LA2 s H1 H2.
  apply match_list_atomic_list_complete_n with (n:=S (length LA1)).
  simpl. trivial. assumption. assumption.
Qed.

(* Given a formula and db of facts, give set of all possible heads derivable
   using the formula *)
Definition match_form_atomic_list (F : form)(LA : list atomic) : list atomic :=
  (* Match the body with the db, get list of substitutions that satisfy body *)

```

```

let res := match_list_atomic_list (body F) LA in
(* Now apply all the substitutions to the head *)
map (fun s => multisubs_atomic s (head F)) res.

(* Given a set of formulas and db of facts, get set of all possible derivable
facts (in one step) *)
Definition match_form_list_atomic_list (LF : list form)(LA : list atomic) : list
atomic :=
  flatten (map (fun f => match_form_atomic_list f LA) LF).

Definition kthing_start_set_aux (LK : list kthing) : list atomic :=
  let pr := list_prod LK LK in
  let pr2 := filter
    (fun pr => match pr with (x, y) => if exttype_dec x y then true else false e
      nd)
    pr in
  map (fun pr => match pr with (x, y) => extbare (kident y) end) pr2.

Definition kthing_set_ext (LK : list kthing)(DB : list atomic) : list atomic :=
  let pr := list_prod LK LK in
  let pr2 := list_prod pr LK in
  let pr3 := filter
    (fun p => match p with (p2, k3) => match p2 with (k1, k2) =>
      if In_dec eq_atomic_dec (extbare (k
        ident k1)) DB then (
          if extdot_dec k1 k2 k3 then true
          else false
        ) else false
      end end)
    pr2 in
  map (fun p => match p with (p2, k3) => extbare (kident k3) end) pr3.

Definition kthing_start_set (LF : list form) : list atomic :=
  let M := kthing_start_set_aux (extset LF) in
  (set_union eq_atomic_dec nil M).

Definition kthing_start_set2 (LF : list form) : list atomic :=
  map (fun k => extbare (kident k)) (extset LF).

Lemma in_prod_l:
  forall (A B : Type)(LA : list A)(LB : list B)(x : A)(y : B),
    In (x, y) (list_prod LA LB) -> In x LA.
Proof.
  intros.
  assert (In (x,y) (list_prod LA LB) <-> In x LA /\ In y LB).
  apply in_prod_iff.
  elim H0; intros.
  assert (In x LA /\ In y LB).
  apply H1. assumption. elim H3; auto.
Qed.

```

```

Lemma in_prod_r:
  forall (A B : Type)(LA : list A)(LB : list B)(x : A)(y : B),
    In (x, y) (list_prod LA LB) -> In y LB.
Proof.
  intros.
  assert (In (x,y) (list_prod LA LB) <-> In x LA /\ In y LB).
  apply in_prod_iff.
  elim H0; intros.
  assert (In x LA /\ In y LB).
  apply H1. assumption. elim H3; auto.
Qed.

Lemma in_filter_in:
  forall (A : Type)(f : A -> bool)(L : list A)(x : A),
    In x (filter f L) -> In x L.
Proof.
  intros.
  induction L. simpl in H. contradiction.
  simpl in H. destruct (f a). simpl in H. elim H; intros; clear H.
  subst a. simpl. auto. simpl. right. apply IHL. assumption.
  simpl. right. apply IHL. assumption.
Qed.

Lemma in_kthing_start_set_in_kthing_start_set2:
  forall k LF,
    In k (kthing_start_set LF) ->
    In k (kthing_start_set2 LF).
Proof.
  intros.
  unfold kthing_start_set in H.
  assert (set_In k nil \/ set_In k (kthing_start_set_aux (extset LF))).
  apply set_union_elim with (Aeq_dec := eq_atomic_dec).
  assumption. elim H0; intros; clear H0. simpl in H1. contradiction.
  unfold kthing_start_set_aux in H1.
  assert (exists y, In y (filter
    (fun pr : kthing * kthing =>
      let (x, y) := pr in if exttype_dec x y then true else false)
    (list_prod (extset LF) (extset LF))) /\ k = ((fun pr : kthing * k
    thing =>
      let (_, y) := pr in extbare (kident y))) y).
  apply in_map. assumption. clear H1.
  elim H0; intros; clear H0.
  elim H1; intros; clear H1. destruct x.
  unfold kthing_start_set2.
  subst k.
  clear H.
  assert (In (k0, k1) (list_prod (extset LF) (extset LF))).
  apply in_filter_in with (f := (fun pr : kthing * kthing =>
    let (x, y) := pr in if exttype_dec x y then true else false)).
  assumption.
  assert (In k1 (extset LF)).

```

```

    apply in_prod_r with (x := k0)(LA := (extset LF)).
    assumption.
    clear H0 H.
    apply in_map2.
    exists k1. split. assumption. trivial.
Qed.

(* Given program and db, give new db *)
(* Must remove duplicates *)
Definition extend_one (LF : list form)(LA : list atomic) : list atomic :=
  set_union eq_atomic_dec LA (match_form_list_atomic_list LF LA).

Definition extend_one_k (LF : list form)(LA : list atomic) : list atomic :=
  set_union eq_atomic_dec LA (kthing_set_ext (extset LF) LA).

Definition extend_one_one_k (LF : list form)(LA : list atomic) : list atomic :=
  extend_one_k LF (extend_one LF LA).

(* Algorithm is using iter on this *)
Fixpoint datalog_alg_aux (LF : list form)(LA : list atomic)(A : atomic)(n : nat)
  {struct n} : bool :=
  match n with
  | 0 => false
  | S p => let res := extend_one_one_k LF LA in
    if eq_atomic_list_dec LA res then
      if (In_dec eq_atomic_dec A LA) then true
      else false
    else
      datalog_alg_aux LF res A p
  end.

Definition datalog_alg (LF : list form)(A : atomic)(n : nat) :=
  datalog_alg_aux LF (kthing_start_set LF) A n.

Lemma match_form_atomic_list_sound :
  forall (F : form)(LF : list form)(A : atomic)(LA : list atomic),
    In F LF ->
    ground_atomic_list LA = true ->
    In A (match_form_atomic_list F LA) ->
    forall (der LF) LA ->
    der LF A.
Proof.
  intros.
  unfold match_form_atomic_list in H1.
  assert (exists s2, In s2 (match_list_atomic_list (body F) LA) /\ A = multisubs
    _atomic s2 (head F)).
  apply in_map with (f:=(fun s => multisubs_atomic s (head F))). assumption.
  elim H3; intros s2 H4; clear H3.

```

```

elim H4; intros H5 H6; clear H4.
rewrite H6.
apply modus_ponens.
assumption.
unfold forallelts.
intros A2 H7.
assert (incl (multisubs_atomic_list s2 (body F)) LA).
apply match_list_atomic_list_sound.
assumption. assumption.
unfold incl in H3.
assert (In (multisubs_atomic s2 A2) (multisubs_atomic_list s2 (body F))).
Lemma multisubs_atomic_list_in :
  forall (A : atomic)(LA : list atomic)(s : substitution),
    In A LA ->
      In (multisubs_atomic s A) (multisubs_atomic_list s LA).
Proof.
  induction LA.
  intros. simpl in H. contradiction.
  intros.
  simpl in H. elim H; intro H2; clear H.
  simpl. left. rewrite H2. trivial.
  simpl. right. apply IHLA. assumption.
Qed.
apply multisubs_atomic_list_in; assumption.
assert (In (multisubs_atomic s2 A2) LA).
apply H3. assumption.
unfold forallelts in H2.
apply H2. assumption.
Qed.

Definition no_head_vars_form (F : form) : Prop :=
  match F with
  | clause H T =>
    forall (x : nat),
      var_occurs_atomic x H = true ->
      var_occurs_atomic_list x T = true
  end.

Definition no_head_vars_form_list (LF : list form) : Prop :=
  forallelts form no_head_vars_form LF.

Lemma multisubs_atomic_equal:
  forall (s sbs : substitution)(l : list atomic)(a : atomic),
    multisubs_atomic_list s l = multisubs_atomic_list sbs l ->
    (forall x : nat,
      var_occurs_atomic x a = true -> var_occurs_atomic_list x l = true) ->
    multisubs_atomic s a = multisubs_atomic sbs a.
Proof.
  intros.
  induction a.
  (* BARE *)

```

```

induction l0.
simpl. trivial.
simpl.
simpl in IHl0.
assert (bare n (map (multisubs_term s) l0) =
  bare n (map (multisubs_term sbs) l0)).
apply IHl0.
intros.
apply H0.
simpl.
unfold var_occurs_term_list.
simpl.
induction (var_occurs_term x a). simpl. trivial.
simpl. unfold var_occurs_term_list in H1. assumption.
assert (map (multisubs_term s) l0 = map (multisubs_term sbs) l0).
injection H1. trivial.
clear H1.
rewrite H2.
cut (multisubs_term s a = multisubs_term sbs a).
intro H7.
rewrite H7. trivial.
clear H2 IHl0.

(* We're getting down to the term level *)
(* Simplify the var_occurs_atomic to a var_occurs_term *)
assert (forall x, var_occurs_term x a = true -> var_occurs_atomic_list x l = true).
intros.
apply H0.
simpl.
unfold var_occurs_term_list.
simpl.
rewrite H1.
simpl. trivial.
clear H0.

(* ident case is fast *)
induction a.
assert (multisubs_term s (ident n0) = ident n0).
apply multisubs_term_ident.
assert (multisubs_term sbs (ident n0) = ident n0).
apply multisubs_term_ident.
rewrite H0. rewrite H2. trivial.

(* kident case *)
assert (multisubs_term s (kident k) = kident k).
apply multisubs_term_kident.
assert (multisubs_term sbs (kident k) = kident k).
apply multisubs_term_kident.
rewrite H0. rewrite H2. trivial.

```

```

(* var case *)
assert (var_occurs_atomic_list n0 l = true).
apply H1. simpl.
induction (eq_nat_dec n0 n0). trivial.
contradiction b. trivial.
clear H1.
unfold var_occurs_atomic_list in H0.
induction l. simpl in H0. discriminate.
simpl in H0.
assert ({var_occurs_atomic n0 a = true} + {var_occurs_atomic n0 a = false}).
induction (var_occurs_atomic n0 a). left; trivial. right; trivial.
elim H1; intro H7; clear H1.
simpl in H.
injection H; intros H8 H9; clear H.
clear H0 IH1 H8.

induction a.
simpl in H7.
simpl in H9.
injection H9; intro H10; clear H9.
induction l1.
unfold var_occurs_term_list in H7.
simpl in H7. discriminate.
simpl in H7.
unfold var_occurs_term_list in H7.
simpl in H7.
assert ({var_occurs_term n0 a = true} + {var_occurs_term n0 a = false}).
induction (var_occurs_term n0 a).
left; trivial. right; trivial.
elim H; intro H11; clear H.
simpl in H10.
injection H10; intros H12 H13; clear H10.
induction a.
simpl in H11. discriminate.
simpl in H11. discriminate.
simpl in H11.

induction (eq_nat_dec n2 n0).
rewrite <- a. assumption.
discriminate.

rewrite H11 in H7.
simpl in H7.
unfold var_occurs_term_list in IH11.
simpl in H10.
injection H10; intros H12 H13; clear H10.
apply IH11.
assumption. assumption.

simpl in H7.
simpl in H9.

```



```

injection H9; intro H10; clear H9.
induction l1.
unfold var_occurs_term_list in H7.
simpl in H7. discriminate.
simpl in H7.
unfold var_occurs_term_list in H7.
simpl in H7.
assert ({var_occurs_term n0 a = true} + {var_occurs_term n0 a = false}).
induction (var_occurs_term n0 a).
left; trivial. right; trivial.
elim H; intro H11; clear H.
simpl in H10.
injection H10; intros H12 H13; clear H10.
induction a.
simpl in H11. discriminate.
simpl in H11. discriminate.
simpl in H11.
induction (eq_nat_dec n3 n0).
rewrite <- a. assumption.
discriminate.

rewrite H11 in H7.
simpl in H7.
unfold var_occurs_term_list in IH11.
simpl in H10.
injection H10; intros H12 H13; clear H10.
apply IH11.
assumption. assumption.

simpl in H7.
simpl in H9.
injection H9; intro H10; clear H9.
induction t.
simpl in H7. discriminate.
simpl in H7. discriminate.
simpl in H7.
induction (eq_nat_dec n1 n0); try discriminate.
rewrite <- a. clear a. clear H7.

assumption.
simpl in H7.
simpl in H9.
injection H9; intro H10; clear H9.
induction t.
simpl in H7. discriminate.
simpl in H7. discriminate.
simpl in H7.
induction (eq_nat_dec n2 n0); try discriminate.
rewrite <- a. clear a. clear H7.
assumption.

```

```

rewrite H7 in H0.
simpl in H0.
simpl in H.
injection H; intros H12 H13; clear H.
apply IH1.
assumption.
assumption.

(* SAYS *)

induction l0.
simpl. trivial.
simpl.
simpl in IH10.
assert (says n n0 (map (multisubs_term s) l0) =
  says n n0 (map (multisubs_term sbs) l0)).
apply IH10.
intros.
apply H0.
simpl.
unfold var_occurs_term_list.
simpl.
induction (var_occurs_term x a). simpl. trivial.
simpl. unfold var_occurs_term_list in H1. assumption.
assert (map (multisubs_term s) l0 = map (multisubs_term sbs) l0).
injection H1. trivial.
clear H1.
rewrite H2.
cut (multisubs_term s a = multisubs_term sbs a).
intro H7.
rewrite H7. trivial.
clear H2 IH10.

(* We're getting down to the term level *)
(* Simplify the var_occurs_atomic to a var_occurs_term *)
assert (forall x, var_occurs_term x a = true -> var_occurs_atomic_list x l = t
rue).
intros.
apply H0.
simpl.
unfold var_occurs_term_list.
simpl.
rewrite H1.
simpl. trivial.
clear H0.

(* ident case is fast *)
induction a.
assert (multisubs_term s (ident n1) = ident n1).
apply multisubs_term_ident.

```

```

assert (multisubs_term sbs (ident n1) = ident n1).
apply multisubs_term_ident.
rewrite H0. rewrite H2. trivial.

(* kident case *)
assert (multisubs_term s (kident k) = kident k).
apply multisubs_term_kident.
assert (multisubs_term sbs (kident k) = kident k).
apply multisubs_term_kident.
rewrite H0. rewrite H2. trivial.

(* var case *)
assert (var_occurs_atomic_list n1 l = true).
apply H1. simpl.
induction (eq_nat_dec n1 n1). trivial.
contradiction b. trivial.
clear H1.
unfold var_occurs_atomic_list in H0.
induction l. simpl in H0. discriminate.
simpl in H0.
assert ({var_occurs_atomic n1 a = true} + {var_occurs_atomic n1 a = false}).
induction (var_occurs_atomic n1 a). left; trivial. right; trivial.
elim H1; intro H7; clear H1.
simpl in H.
injection H; intros H8 H9; clear H.
clear H0 IH1 H8.

induction a.
simpl in H7.
simpl in H9.
injection H9; intro H10; clear H9.
induction l1.
unfold var_occurs_term_list in H7.
simpl in H7. discriminate.
simpl in H7.
unfold var_occurs_term_list in H7.
simpl in H7.
assert ({var_occurs_term n1 a = true} + {var_occurs_term n1 a = false}).
induction (var_occurs_term n1 a).
left; trivial. right; trivial.
elim H; intro H11; clear H.
simpl in H10.
injection H10; intros H12 H13; clear H10.
induction a.
simpl in H11. discriminate.
simpl in H11. discriminate.
simpl in H11.
induction (eq_nat_dec n3 n1).
rewrite <- a. assumption.
discriminate.

```

```

rewrite H11 in H7.
simpl in H7.
unfold var_occurs_term_list in IH11.
simpl in H10.
injection H10; intros H12 H13; clear H10.
apply IH11.
assumption. assumption.

simpl in H7.
simpl in H9.
injection H9; intro H10; clear H9.
induction l1.
unfold var_occurs_term_list in H7.
simpl in H7. discriminate.
simpl in H7.
unfold var_occurs_term_list in H7.
simpl in H7.
assert ({var_occurs_term n1 a = true} + {var_occurs_term n1 a = false}).
induction (var_occurs_term n1 a).
left; trivial. right; trivial.
elim H; intro H11; clear H.
simpl in H10.
injection H10; intros H12 H13; clear H10.
induction a.
simpl in H11. discriminate.
simpl in H11. discriminate.
simpl in H11.
induction (eq_nat_dec n4 n1).
rewrite <- a. assumption.
discriminate.

rewrite H11 in H7.
simpl in H7.
unfold var_occurs_term_list in IH11.
simpl in H10.
injection H10; intros H12 H13; clear H10.
apply IH11.
assumption. assumption.

```

```

(* *** *)
simpl in H7.
simpl in H9.
injection H9; intro H10; clear H9.
induction t.
simpl in H7. discriminate.
simpl in H7. discriminate.
simpl in H7.
induction (eq_nat_dec n2 n1); try discriminate.
rewrite <- a. clear a. clear H7. assumption.

simpl in H7.

```

```

simpl in H9.
injection H9; intro H10; clear H9.
induction t.
simpl in H7. discriminate.
simpl in H7. discriminate.
simpl in H7.
induction (eq_nat_dec n3 n1); try discriminate.
rewrite <- a. clear a. clear H7.

assumption.

(* *** *)

rewrite H7 in H0.
simpl in H0.
simpl in H.
injection H; intros H12 H13; clear H.
apply IH1.
assumption.
assumption.

(* EXTBAE *)
simpl.
simpl in H0.
cut (multisubs_term s t = multisubs_term sbs t).
intro H1.
rewrite H1. trivial.
induction t.
assert (multisubs_term s (ident n) = ident n).
apply multisubs_term_ident.
assert (multisubs_term sbs (ident n) = ident n).
apply multisubs_term_ident.
rewrite H1. rewrite H2.
trivial.
assert (multisubs_term s (kident k) = kident k).
apply multisubs_term_kident.
assert (multisubs_term sbs (kident k) = kident k).
apply multisubs_term_kident.
rewrite H1. rewrite H2.
trivial.
assert (var_occurs_atomic_list n l = true).
apply H0. simpl.
induction (eq_nat_dec n n). trivial. absurd (n = n). assumption. trivial.
clear H0.
unfold var_occurs_atomic_list in H1.
induction l.
unfold exists_bool in H1. discriminate.
simpl in H.
injection H; intros H2 H3; clear H.
simpl in H1.
assert (var_occurs_atomic n a = true /\ exists_bool (var_occurs_atomic n) l =

```

```

true).
induction (var_occurs_atomic n a). left; trivial. right.
simpl in H1. assumption.
clear H1.
elim H; intro H4; clear H.
induction a.
simpl in H3. simpl in H4.
injection H3; intro H5; clear H3.
unfold var_occurs_term_list in H4.
induction l0.
unfold exists_bool in H4.
discriminate.
simpl in H5.
injection H5; intros H6 H7; clear H5.
simpl in H4.
assert (var_occurs_term n a = true /\ exists_bool (var_occurs_term n) l0 = true).
induction (var_occurs_term n a). left; trivial. right.
simpl in H4. assumption. clear H4.
elim H; intro H8; clear H.
induction a. simpl in H8. discriminate.
simpl in H8. discriminate.
simpl in H8.
induction (eq_nat_dec n1 n).
rewrite a in H7. assumption. discriminate.

apply IHl0. assumption.
assumption.

simpl in H3. simpl in H4.
injection H3; intro H5; clear H3.
unfold var_occurs_term_list in H4.
induction l0.
unfold exists_bool in H4.
discriminate.
simpl in H5.
injection H5; intros H6 H7; clear H5.
simpl in H4.
assert (var_occurs_term n a = true /\ exists_bool (var_occurs_term n) l0 = true).
induction (var_occurs_term n a). left; trivial. right.
simpl in H4. assumption. clear H4.
elim H; intro H8; clear H.
induction a. simpl in H8. discriminate.
simpl in H8. discriminate.
simpl in H8.
induction (eq_nat_dec n2 n).
rewrite a in H7. assumption.

discriminate.
apply IHl0. assumption.

```

```

assumption.

simpl in H3. simpl in H4.
injection H3; intro H5; clear H3.
induction t.
simpl in H4. discriminate.
simpl in H4. discriminate.
simpl in H4.
induction (eq_nat_dec n0 n).
rewrite <- a. assumption. discriminate.

simpl in H3. simpl in H4.
injection H3; intro H5; clear H3.
induction t.
simpl in H4. discriminate.
simpl in H4. discriminate.
simpl in H4.
induction (eq_nat_dec n1 n).
rewrite <- a. assumption. discriminate.

apply IH1.
assumption. assumption.

(* EXTSAAYS *)

simpl.
simpl in H0.
cut (multisubs_term s t = multisubs_term sbs t).
intro H1.
rewrite H1. trivial.
induction t.
assert (multisubs_term s (ident n0) = ident n0).
apply multisubs_term_ident.
assert (multisubs_term sbs (ident n0) = ident n0).
apply multisubs_term_ident.
rewrite H1. rewrite H2.
trivial.
assert (multisubs_term s (kident k) = kident k).
apply multisubs_term_kident.
assert (multisubs_term sbs (kident k) = kident k).
apply multisubs_term_kident.
rewrite H1. rewrite H2.
trivial.
assert (var_occurs_atomic_list n0 l = true).
apply H0. simpl.
induction (eq_nat_dec n0 n0). trivial. absurd (n0 = n0). assumption. trivial.
clear H0.
unfold var_occurs_atomic_list in H1.
induction l.
unfold exists_bool in H1. discriminate.
simpl in H.

```

```

injection H; intros H2 H3; clear H.
simpl in H1.
assert (var_occurs_atomic n0 a = true /\ exists_bool (var_occurs_atomic n0) 1
= true).
induction (var_occurs_atomic n0 a). left; trivial. right.
simpl in H1. assumption.
clear H1.
elim H; intro H4; clear H.
induction a.
simpl in H3. simpl in H4.
injection H3; intro H5; clear H3.
unfold var_occurs_term_list in H4.
induction l0.
unfold exists_bool in H4.
discriminate.
simpl in H5.
injection H5; intros H6 H7; clear H5.
simpl in H4.
assert (var_occurs_term n0 a = true /\ exists_bool (var_occurs_term n0) l0 = t
rue).
induction (var_occurs_term n0 a). left; trivial. right.
simpl in H4. assumption. clear H4.
elim H; intro H8; clear H.
induction a. simpl in H8. discriminate.
simpl in H8. discriminate.
simpl in H8.
induction (eq_nat_dec n2 n0).
rewrite a in H7. assumption.

discriminate.
apply IHl0. assumption.
assumption.

simpl in H3. simpl in H4.
injection H3; intro H5; clear H3.
unfold var_occurs_term_list in H4.
induction l0.
unfold exists_bool in H4.
discriminate.
simpl in H5.
injection H5; intros H6 H7; clear H5.
simpl in H4.
assert (var_occurs_term n0 a = true /\ exists_bool (var_occurs_term n0) l0 = t
rue).
induction (var_occurs_term n0 a). left; trivial. right.
simpl in H4. assumption. clear H4.
elim H; intro H8; clear H.
induction a. simpl in H8. discriminate.
simpl in H8. discriminate.
simpl in H8.
induction (eq_nat_dec n3 n0).

```



```

rewrite a in H7. assumption.

discriminate.
apply IH10. assumption.
assumption.

simpl in H3. simpl in H4.
injection H3; intro H5; clear H3.
induction t.
simpl in H4. discriminate.
simpl in H4. discriminate.
simpl in H4.
induction (eq_nat_dec n1 n0).
rewrite <- a. assumption. discriminate.

simpl in H3. simpl in H4.
injection H3; intro H5; clear H3.
induction t.
simpl in H4. discriminate.
simpl in H4. discriminate.
simpl in H4.
induction (eq_nat_dec n2 n0).
rewrite <- a. assumption. discriminate.

apply IH1.
assumption. assumption.
Qed.

Lemma match_form_atomic_list_complete :
  forall (F : form)(LF : list form)(LA : list atomic)(s : substitution),
    no_head_vars_form_list LF ->
    ground_atomic_list LA = true ->
    In F LF ->
    forallelts atomic (fun x => In (multisubs_atomic s x) LA) (body F) ->
    In (multisubs_atomic s (head F)) (match_form_atomic_list F LA).
Proof.
  intros.
  unfold match_form_atomic_list.
  cut (exists z, In z (match_list_atomic_list (body F) LA) /\ multisubs_atomic s
    (head F) = (fun s0 : substitution => multisubs_atomic s0 (head F)) z).
  intro H3.
  apply in_map2. assumption.

  assert ((exists sbs, In sbs (match_list_atomic_list (body F) LA) /\ (multisubs
    _atomic_list sbs (body F) = multisubs_atomic_list s (body F)))).
  apply match_list_atomic_list_complete.
  assumption.
  unfold incl.
  intros.
  unfold forallelts in H2.
  unfold multisubs_atomic_list in H3.

```

```

assert (exists zz, In zz (body F) /\ a = (multisubs_atomic s) zz).
apply in_map.
assumption.
elim H4; intros x H5; clear H4.
elim H5; intros H6 H7; clear H5.
rewrite H7.
apply H2. assumption.

elim H3; intros sbs H4; clear H3.
elim H4; intros H5 H6; clear H4.
exists sbs.
split. assumption.
(* Now we just have to show that since the substitution makes the bodies equal
, it makes the heads equal *)
(* Need no_head_vars_form_list as assumption to show that there are no extra v
ariables that might screw things up *)
unfold no_head_vars_form_list in H.
unfold forallelts in H.
assert (no_head_vars_form F).
apply H. assumption. clear H.
induction F.
unfold no_head_vars_form in H3.
simpl in H6.
simpl in H5.
clear H1 H2.
simpl.
symmetry in H6.
apply multisubs_atomic_equal with (s := s)(sbs := sbs)(l := l)(a := a).
assumption.
assumption.
Qed.

Lemma match_form_list_atomic_list_sound :
  forall (LF : list form)(A : atomic)(LA : list atomic),
    ground_atomic_list LA = true ->
      In A (match_form_list_atomic_list LF LA) ->
        forallelts atomic (der LF) LA ->
          der LF A.
Proof.
  intros.
  unfold match_form_list_atomic_list in H0.
  assert (exists z, In z (map (fun f => match_form_atomic_list f LA) LF) /\ In A
    z).
  apply in_flatten; assumption.
  elim H2; intros z H3; clear H2.
  elim H3; intros H4 H5; clear H3.
  assert (exists y, In y LF /\ z = (fun f => match_form_atomic_list f LA) y).
  apply in_map with (f:=(fun f => match_form_atomic_list f LA)); assumption.
  elim H2; intros y H6; clear H2.
  elim H6; intros H7 H8; clear H6.
  rewrite H8 in H5.

```

```

    apply match_form_atomic_list_sound with (A:=A)(F:=y)(LA:=LA); assumption.
Qed.

```

```

Lemma match_form_list_atomic_list_complete :
  forall (F : form)(LF : list form)(LA : list atomic)(s : substitution),
    no_head_vars_form_list LF ->
    ground_atomic_list LA = true ->
    In F LF ->
    forallelts atomic (fun x => In (multisubs_atomic s x) LA) (body F) ->
    In (multisubs_atomic s (head F)) (match_form_list_atomic_list LF LA).

```

```

Proof.
  intros.
  unfold match_form_list_atomic_list.
  apply flatten_in.
  exists (match_form_atomic_list F LA).
  split.
  apply in_map2.
  exists F. split.
  assumption. trivial.
  apply match_form_atomic_list_complete with (LF := LF).
  assumption. assumption.
  assumption.
  assumption.
Qed.

```

```

Lemma var_occurs_match_atomic :
  forall (a a0 : atomic)(s : substitution),
    match_atomic a a0 = Some s ->
    forall (x : nat)(v : rval),
      In (x, v) s ->
      var_occurs_atomic x a = true.

```

```

Proof.
  intros.
  induction a; induction a0; simpl; simpl in H; try discriminate.
  induction (eq_nat_dec n n0).
  apply var_occurs_match_term_list with (s:=s)(x:=x)(v:=v)(l:=l)(l0:=l0); assumption.
  discriminate.
  induction (eq_nat_dec n0 n2); induction (eq_nat_dec n n1); simpl in H; try discriminate.
  apply var_occurs_match_term_list with (s:=s)(x:=x)(v:=v)(l:=l)(l0:=l0); assumption.
  discriminate.

  induction t; induction t0; simpl; simpl in H; try discriminate; try (induction
    (eq_nat_dec n n0); simpl in H; try discriminate).
  injection H; intro H1; clear H; rewrite <- H1 in H0; simpl in H0; contradiction.
  induction (eq_kthing_dec k k0).
  injection H; intro H1; clear H; rewrite <- H1 in H0; simpl in H0; contradiction.

```

```

discriminate.
induction (eq_nat_dec n x). trivial.
injection H; intro H1; clear H; rewrite <- H1 in H0; simpl in H0.
elim H0; intro H2; clear H0; try contradiction.
injection H2; intros H3 H4; clear H2.
contradiction.
induction (eq_nat_dec n x). trivial.
injection H; intro H1; clear H; rewrite <- H1 in H0; simpl in H0.
elim H0; intro H2; clear H0; try contradiction.
injection H2; intros H3 H4; clear H2.
contradiction.

induction (eq_nat_dec n x); simpl in H; try discriminate.
trivial.

injection H; intro H1; clear H.
rewrite <- H1 in H0. simpl in H0.
elim H0; intros. injection H; intros. contradiction. contradiction.
induction (eq_nat_dec n n0).

induction t; induction t0; simpl; simpl in H; try discriminate; try (induction
  (eq_nat_dec n1 n2); simpl in H; try discriminate).
injection H; intro H1; clear H; rewrite <- H1 in H0; simpl in H0; contradictio
n.
induction (eq_kthing_dec k k0).
injection H; intro H1; clear H; rewrite <- H1 in H0; simpl in H0; contradictio
n.
discriminate.
induction (eq_nat_dec n1 x). trivial.
injection H; intro H1; clear H; rewrite <- H1 in H0; simpl in H0.
elim H0; intro H2; clear H0; try contradiction.
injection H2; intros H3 H4; clear H2.
contradiction.
induction (eq_nat_dec n1 x). trivial.
injection H; intro H1; clear H; rewrite <- H1 in H0; simpl in H0.
elim H0; intro H2; clear H0; try contradiction.
injection H2; intros H3 H4; clear H2.
contradiction.

induction (eq_nat_dec n1 x); simpl in H; try discriminate.
trivial.

injection H; intro H1; clear H.
rewrite <- H1 in H0. simpl in H0.
elim H0; intros. injection H; intros. contradiction. contradiction.
discriminate.
Qed.

```

```

Lemma var_occurs_match_atomic_list :
  forall (la : list atomic)(a : atomic)(s : substitution),
    In s (match_atomic_list a la) ->

```

```

    forall (x : nat)(v : rval),
      In (x, v) s ->
        var_occurs_atomic x a = true.
Proof.
  induction la.
  simpl. contradiction.
  intros a0 s1 H x v H1.
  simpl in H.
  assert ({exists s2, match_atomic a0 a = Some s2}+{match_atomic a0 a = None}).
  apply option_dec.
  elim H0; intro H2; clear H0.
  elim H2; intros s2 H3; clear H2.
  rewrite H3 in H.
  simpl in H.
  elim H; intro H4; clear H.
  apply var_occurs_match_atomic with (a:=a0)(a0:=a)(x:=x)(v:=v)(s:=s2).
  assumption. rewrite H4. assumption.
  apply IHla with (s:=s1)(v:=v). assumption. assumption.
  rewrite H2 in H.
  apply IHla with (s:=s1)(v:=v). assumption. assumption.
Qed.

Lemma var_occurs_match_list_atomic_list_aux :
  forall (l l0 : list atomic)(s s2 : substitution),
    In s (match_list_atomic_list_aux l l0 s2) ->
      forall (x : nat)(v : rval),
        In (x, v) s ->
          var_occurs_atomic_list x (multisubs_atomic_list s2 l) = true.
Proof.
  induction l.
  simpl.
  intros.
  elim H; intro H1; clear H.
  rewrite <- H1 in H0.
  simpl in H0. contradiction.
  contradiction.
  intros l0 s3 s2 H x v H1.
  simpl in H.
  simpl.
  unfold var_occurs_atomic_list.
  simpl.
  assert (exists z, In z (map
    (fun s : list (nat * rval) =>
      map_append s (match_list_atomic_list_aux l l0 (s2 ++ s)))
    (match_atomic_list (multisubs_atomic s2 a) l0)) /\ In s3 z).
  apply in_flatten. assumption.
  clear H.
  elim H0; intros z H2; clear H0.
  elim H2; intros H3 H4; clear H2.
  assert (exists y, In y (match_atomic_list (multisubs_atomic s2 a) l0) /\ z = (
  fun s : list (nat * rval) =>

```

```

      map_append s (match_list_atomic_list_aux 1 l0 (s2 ++ s))) y).
apply in_map with (f:=(fun s : list (nat * rval) =>
      map_append s (match_list_atomic_list_aux 1 l0 (s2 ++ s)))).
assumption.
clear H3.
elim H; intros y H5; clear H.
elim H5; intros H6 H7; clear H5.
unfold map_append in H7.
rewrite H7 in H4.
assert (exists w, In w (match_list_atomic_list_aux 1 l0 (s2 ++ y)) /\ s3 = (fun
n x => y ++ x) w).
apply in_map.
assumption.
elim H; intros w H5; clear H.
elim H5; intros H8 H9; clear H5.
clear H4.
clear H7.
rewrite H9 in H1.
assert (In (x,v) y \/ In (x,v) w).
apply in_append; assumption.
clear H1.
elim H; intro H10; clear H.
assert (var_occurs_atomic x (multisubs_atomic s2 a) = true).
apply var_occurs_match_atomic_list with (s:=y)(la:=l0)(v:=v).
assumption. assumption.
induction (var_occurs_atomic x (multisubs_atomic s2 a)).
simpl. trivial. discriminate.

assert (var_occurs_atomic_list x (multisubs_atomic_list (s2 ++ y) l) = true).
apply IH1 with (x:=x)(v:=v)(s:=w)(l0:=l0).
assumption. assumption.
assert (multisubs_atomic_list (s2 ++ y) l = multisubs_atomic_list y (multisubs
_atomic_list s2 l)).
apply multisubs_atomic_list_append.
rewrite H0 in H. clear H0.
assert (var_occurs_atomic_list x (multisubs_atomic_list s2 l) = true).
apply var_occurs_atomic_list_multisubs with (x:=x)(la:=multisubs_atomic_list s
2 l)(s:=y).
assumption.
clear H.
unfold var_occurs_atomic_list in H0.
induction (var_occurs_atomic x (multisubs_atomic s2 a)).
simpl. trivial. simpl. assumption.
Qed.

```

```

Lemma var_occurs_match_list_atomic_list :
  forall (l l0 : list atomic)(s : substitution),
    In s (match_list_atomic_list l l0) ->
    forall (x : nat)(v : rval),
      In (x, v) s ->

```

```

    var_occurs_atomic_list x l = true.
Proof.
  intros.
  unfold match_list_atomic_list in H.
  assert (multisubs_atomic_list nil l = l).
  apply multisubs_atomic_list_nil.
  rewrite <- H1.
  clear H1.
  apply var_occurs_match_list_atomic_list_aux with (l:=l)(l0:=l0)(s:=s)(s2:=nil:
    (list (nat*rval)))(x:=x)(v:=v).
  assumption. assumption.
Qed.

Lemma var_occurs_match_term2 :
  forall (l l0 : term)(s : substitution),
    match_term l l0 = Some s ->
      forall (x : nat),
        var_occurs_term x l = true ->
          exists v, In (x, v) s.
Proof.
  intros.
  induction l; induction l0; simpl in H; try discriminate.
  simpl in H0.
  induction (eq_nat_dec n x).
  rewrite <- a.
  injection H. intros. rewrite <- H1. simpl. exists (rvident n0). left. trivial.
  discriminate.

  simpl in H0.
  induction (eq_nat_dec n x).
  rewrite <- a.
  injection H. intros. rewrite <- H1. simpl. exists (rvkident k). left. trivial.
  discriminate.
Qed.

Lemma xvin_dec :
  forall (x : nat)(s : substitution),
    {exists v, In (x,v) s}+{~(exists v, In (x,v) s)}.
Proof.
  intros.
  induction s.
  right. unfold not. intro H.
  elim H; intros v H2. simpl in H2. contradiction.
  simpl.
  elim IHs.
  intro H.
  left.
  elim H.
  intros v H1.
  exists v. right. assumption.
  intro H.

```

```

induction a.
assert ({a=x}+{a<>x}).
apply eq_nat_dec.
elim H0; intro H1; clear H0.
left.
exists b. left. rewrite H1. trivial.
right.
unfold not.
intro H2.
elim H2; intros v H3; clear H2.
elim H3; intro H4; clear H3.
injection H4; intros H5 H6; clear H4.
contradiction.
unfold not in H.
apply H.
exists v. assumption.
Qed.

Lemma var_occurs_match_term_list2_n :
  forall (n : nat)(l0 l : list term)(s : substitution),
    length l = n ->
    match_term_list l l0 = Some s ->
    forall (x : nat),
      var_occurs_term_list x l = true ->
      exists v, In (x, v) s.
Proof.
  induction n.
  intros.
  induction l.
  induction l0.
  unfold var_occurs_term_list in H1.
  simpl in H1. discriminate.
  simpl in H0. discriminate.
  simpl in H. discriminate.
  intros.
  induction l.
  simpl in H. discriminate.
  induction l0.
  simpl in H0. discriminate.
  clear IH1 IH10.
  simpl in H.
  injection H; intro H2. clear H.
  (* OK, base cases are done, real work now *)

  simpl in H0.
  assert ({exists s2, match_term a a0 = Some s2}+{match_term a a0 = None}).
  apply option_dec.
  elim H; intro H3; clear H.
  elim H3; intros s2 H4; clear H3.
  rewrite H4 in H0.
  assert ({exists s3, match_term_list (multisubs_term_list s2 l) l0 = Some s3}+{

```



```

match_term_list (multisubs_term_list s2 l) l0 = None}).
apply option_dec.
elim H; intro H5; clear H.
elim H5; intros s3 H6; clear H5.
rewrite H6 in H0.
injection H0; intro H7; clear H0.
rewrite <- H7.
clear H7.
unfold var_occurs_term_list in H1.
simpl in H1.
assert (var_occurs_term x a = true /\ exists_bool (var_occurs_term x) l = true
).
induction (var_occurs_term x a). left; trivial.
right. simpl in H1. assumption.
clear H1.
elim H; intro H7; clear H.
assert (exists v, In (x,v) s2).
apply var_occurs_match_term2 with (l:=a)(l0:=a0)(s:=s2).
assumption. assumption.
elim H; intros v H8; clear H.
exists v.
apply in_or_app.
left. assumption.
(* tricky bit here: variable x might already have been in a *)
assert ({exists v, In (x,v) s2}+{~(exists v, In (x,v) s2)}).
apply xvin_dec.
elim H; intro H8; clear H.
elim H8; intros v H9; clear H8.
exists v. apply in_or_app. left; trivial.

assert (exists v, In (x,v) s3).
apply IHn with (l:=multisubs_term_list s2 l)(l0:=l0)(s:=s3).
assert (length (multisubs_term_list s2 l) = length l).
apply length_multisubs_term_list.
rewrite H. assumption.
assumption.
unfold var_occurs_term_list.
clear n a IHn a0 l0 s H2 H4 s3 H6.
induction l.
simpl in H7. discriminate.
simpl in H7.
assert ({var_occurs_term x a = true}+{exists_bool (var_occurs_term x) l = true
}).
induction (var_occurs_term x a). left; trivial.
simpl in H7. right; assumption.
elim H; intro H1; clear H.
simpl.
clear H7 IH1.
assert (var_occurs_term x (multisubs_term s2 a) = true).

induction a.

```

```

(* ident *)
simpl in H1. discriminate.
simpl in H1. discriminate.
induction s2.
simpl.
simpl in H1. assumption.

simpl.
simpl in H1. induction (eq_nat_dec n x).
induction a.
rewrite a0.
rewrite a0 in IHs2.
assert (a<>x).
unfold not in H8.
unfold not.
intro H9.
apply H8. simpl.
exists b. left. rewrite H9. trivial.
assert (subs_term (a,b)(var x) = var x).
simpl.
induction (eq_nat_dec a x).
induction b; contradiction. trivial.
rewrite H0.
apply IHs2.
unfold not.
intro H2.
elim H2; intros v H3.
unfold not in H8.
apply H8.
exists v.
simpl. right. assumption.
discriminate.
rewrite H. simpl. trivial.
simpl.
induction (var_occurs_term x (multisubs_term s2 a)).
simpl. trivial. simpl. apply IH1. assumption.

elim H; intros v H9; clear H.
exists v. apply in_or_app.
right; assumption.
rewrite H5 in H0. discriminate.
rewrite H3 in H0. discriminate.
Qed.

Lemma var_occurs_match_term_list2 :
  forall (l0 l : list term)(s : substitution),
    match_term_list l l0 = Some s ->
      forall (x : nat),
        var_occurs_term_list x l = true ->
          exists v, In (x, v) s.
Proof.

```

```

intros.
apply var_occurs_match_term_list2_n with (n:=length l)(l:=l)(l0:=l0)(s:=s)(x:=
x); try assumption.
trivial.
Qed.

```

```

Lemma var_occurs_match_atomic2 :
  forall (a a0 : atomic)(s : substitution),
    match_atomic a a0 = Some s ->
      forall (x : nat),
        var_occurs_atomic x a = true ->
          exists v, In (x, v) s.

```

Proof.

```

intros.
induction a; induction a0; simpl in H; simpl; try discriminate.
induction (eq_nat_dec n n0).
apply var_occurs_match_term_list2 with (l:=l)(l0:=l0)(s:=s)(x:=x).
assumption.
simpl in H0. assumption.
discriminate.
induction (eq_nat_dec n0 n2).
induction (eq_nat_dec n n1).
apply var_occurs_match_term_list2 with (l:=l)(l0:=l0)(s:=s)(x:=x).
assumption.
simpl in H0. assumption.
discriminate. discriminate.

```

(* induction (eq_nat_dec n n0); simpl in H; try discriminate. *)

```

induction t; induction t0; simpl in H; simpl in H0; try discriminate.
induction (eq_nat_dec n x); try discriminate.
injection H; intro H1; clear H.
rewrite <- H1. simpl.
exists (rvident n0). left. rewrite a; trivial.
induction (eq_nat_dec n x); try discriminate.
injection H; intro H1; clear H.
rewrite <- H1. exists (rvkident k).
rewrite a. simpl. left. trivial.

```

```

induction (eq_nat_dec n n0); simpl in H; try discriminate.
induction t; induction t0; simpl in H; simpl in H0; try discriminate.
induction (eq_nat_dec n1 x); try discriminate.
injection H; intro H1; clear H.
rewrite <- H1. simpl.
exists (rvident n2). left. rewrite a0; trivial.
induction (eq_nat_dec n1 x); try discriminate.
injection H; intro H1; clear H.
rewrite <- H1. simpl.
exists (rvkident k). left. rewrite a0; trivial.

```

Qed.

```

Lemma var_occurs_match_atomic_list2 :
  forall (la : list atomic)(a : atomic)(s : substitution),
    In s (match_atomic_list a la) ->
      forall (x : nat),
        var_occurs_atomic x a = true ->
          exists v, In (x, v) s.

```

Proof.

```

  intros.
  induction la.
  simpl in H. contradiction.
  simpl in H.
  assert ({exists s2, match_atomic a a0 = Some s2}+{match_atomic a a0 = None}).
  apply option_dec.
  elim H1; intro H2; clear H1.
  elim H2; intros s2 H3; clear H2.
  rewrite H3 in H.
  simpl in H.
  elim H; intro H4; clear H.
  rewrite <- H4.
  apply var_occurs_match_atomic2 with (a:=a)(a0:=a0)(s:=s2)(x:=x).
  assumption. assumption.
  apply IHla. assumption.
  rewrite H2 in H.
  apply IHla. assumption.

```

Qed.

```

Lemma var_occurs_match_list_atomic_list_aux2 :
  forall (l l0 : list atomic)(s s2 : substitution),
    In s (match_list_atomic_list_aux l l0 s2) ->
      forall (x : nat),
        var_occurs_atomic_list x (multisubs_atomic_list s2 l) = true ->
          (exists v, In (x, v) s).

```

Proof.

```

  induction l.
  intros.
  unfold var_occurs_atomic_list in H0.
  simpl in H0. discriminate.
  intros.
  simpl in H.
  assert (exists w, In w (map
    (fun s : list (nat * rval) =>
      map_append s (match_list_atomic_list_aux l l0 (s2 ++ s)))
    (match_atomic_list (multisubs_atomic s2 a) l0)) /\ In s w).
  apply in_flatten.
  assumption.
  elim H1; intros w H2; clear H1.
  elim H2; intros H3 H4; clear H2.
  assert (exists z, In z (match_atomic_list (multisubs_atomic s2 a) l0) /\ w = (
    fun s : list (nat * rval) =>
      map_append s (match_list_atomic_list_aux l l0 (s2 ++ s))) z).

```

```

apply in_map with (f:=(fun s : list (nat * rval) =>
      map_append s (match_list_atomic_list_aux 1 10 (s2 ++ s)))).
assumption.
elim H1; intros z H5; clear H1.
elim H5; intros H6 H7; clear H5.
clear H H3.
rewrite H7 in H4.
clear w H7.
unfold map_append in H4.
assert (exists y, In y (match_list_atomic_list_aux 1 10 (s2 ++ z)) /\ s = (fun
  x : list (nat * rval) => z ++ x) y).
apply in_map with (f:=(fun x : list (nat * rval) => z ++ x)).
assumption.
elim H; intros y H5; clear H.
elim H5; intros H7 H8; clear H5.
clear H4.
rewrite H8. clear H8.
unfold var_occurs_atomic_list in H0.
simpl in H0.
assert (var_occurs_atomic x (multisubs_atomic s2 a)=true \/ var_occurs_atomic_
list x (multisubs_atomic_list s2 l) = true).
induction (var_occurs_atomic x (multisubs_atomic s2 a)).
left; trivial.
simpl in H0. right. assumption.
clear H0.
elim H; intro H9; clear H.
assert (exists v, In (x,v) z).
apply var_occurs_match_atomic_list2 with (a:=multisubs_atomic s2 a)(la:=l0)(s:
=z)(x:=x).
assumption. assumption.
elim H; intros v H10; clear H.
exists v. apply in_or_app. left; assumption.

assert ({exists v, In (x,v) z}+{~exists v, In (x,v) z}).
apply xvin_dec.
elim H; intro H10; clear H.
elim H10; intros v H11; clear H10.
exists v. apply in_or_app. left; assumption.
unfold not in H10.
assert (exists v, In (x,v) y).
apply IH1 with (s:=y)(l0:=l0)(s2:=s2++z).
assumption.
clear H7 H6 IH1.
clear a l0 s y.
assert (multisubs_atomic_list (s2 ++ z) l = multisubs_atomic_list z (multisubs
_atomic_list s2 l)).
apply multisubs_atomic_list_append.
rewrite H.
clear H.

```

Lemma blah1 :

```

forall (l : term)(x : nat)(z : substitution),
  var_occurs_term x l = true ->
    (~ exists v, In (x,v) z) ->
      var_occurs_term x (multisubs_term z l) = true.

```

Proof.

```

intros.
induction l.
simpl in H. discriminate.
simpl in H. discriminate.
simpl in H.
induction (eq_nat_dec n x).
clear H.
unfold not in H0.
induction z.
simpl.
induction (eq_nat_dec n x). trivial. contradiction.
simpl.
induction a0.
rewrite a.
simpl.
induction (eq_nat_dec a0 x).
rewrite a1 in H0.
assert False.
apply H0.
simpl.
exists b.
left; trivial. contradiction.
rewrite a in IHZ.
apply IHZ.
intro H1.
elim H1; intros v H2; clear H1.
apply H0.
exists v.
simpl. right; assumption.
discriminate.

```

Qed.

Lemma blah2 :

```

forall (l : list term)(x : nat)(z : substitution),
  var_occurs_term_list x l = true ->
    (~ exists v, In (x,v) z) ->
      var_occurs_term_list x (multisubs_term_list z l) = true.

```

Proof.

```

intros.
induction l.
unfold var_occurs_term_list in H.
simpl in H. discriminate.
unfold var_occurs_term_list in H.
simpl in H.
assert (var_occurs_term x a = true \/ var_occurs_term_list x l = true).
induction (var_occurs_term x a).

```

```

left; trivial. simpl in H. right. unfold var_occurs_term_list. assumption.
clear H.
elim H1; intro H2; clear H1.
assert (var_occurs_term x (multisubs_term z a) = true).
apply blah1. assumption. assumption.
simpl.
unfold var_occurs_term_list. simpl.
rewrite H. simpl. trivial.
assert (var_occurs_term_list x (multisubs_term_list z l) = true).
apply IH1. assumption.
unfold var_occurs_term_list.
simpl.
unfold var_occurs_term_list in H.
rewrite H.
induction (var_occurs_term x (multisubs_term z a)).
simpl; trivial. simpl; trivial.
Qed.

```

```

Lemma blah3 :
  forall (l : atomic)(x : nat)(z : substitution),
    var_occurs_atomic x l = true ->
    (~ exists v, In (x,v) z) ->
    var_occurs_atomic x (multisubs_atomic z l) = true.

```

```

Proof.
  intros.
  induction l.
  simpl in H.
  simpl.
  assert (map (multisubs_term z) l = multisubs_term_list z l).
  unfold multisubs_term_list. trivial.
  rewrite H1. clear H1.
  apply blah2. assumption. assumption.
  simpl in H.
  simpl.
  assert (map (multisubs_term z) l = multisubs_term_list z l).
  unfold multisubs_term_list. trivial.
  rewrite H1. clear H1.
  apply blah2. assumption. assumption.

  simpl in H.
  simpl.
  induction t; simpl in H; try discriminate.
  induction (eq_nat_dec n x); try discriminate.
  rewrite a.
  assert (multisubs_term z (var x) = var x).
  induction z.
  simpl. trivial.
  induction a0.
  simpl in H0. simpl.
  induction (eq_nat_dec a0 x).
  rewrite a1 in H0.

```

```

induction b.
absurd (exists v, (x, rvident n0) = (x, v) \ / In (x, v) z).
assumption.
exists (rvident n0). left; trivial.
absurd (exists v, (x, rvkident k) = (x, v) \ / In (x, v) z).
assumption.
exists (rvkident k). left; trivial.

apply IHz.
unfold not.
intro H2.
apply H0.
elim H2; intros v H3; clear H2.
exists v. right. assumption.
rewrite H1. simpl.
induction (eq_nat_dec x x). trivial. absurd (x = x). assumption. trivial.

simpl in H.
simpl.
induction t; simpl in H; try discriminate.
induction (eq_nat_dec n0 x); try discriminate.
rewrite a.
assert (multisubs_term z (var x) = var x).
induction z.
simpl. trivial.
induction a0.
simpl in H0. simpl.
induction (eq_nat_dec a0 x).
rewrite a1 in H0.
induction b.
absurd (exists v, (x, rvident n1) = (x, v) \ / In (x, v) z).
assumption.
exists (rvident n1). left; trivial.
absurd (exists v, (x, rvkident k) = (x, v) \ / In (x, v) z).
assumption.
exists (rvkident k). left; trivial.

apply IHz.
unfold not.
intro H2.
apply H0.
elim H2; intros v H3; clear H2.
exists v. right. assumption.
rewrite H1. simpl.
induction (eq_nat_dec x x). trivial. absurd (x = x). assumption. trivial.
Qed.

Lemma blah4 :
  forall (l : list atomic)(x : nat)(z : substitution),
    var_occurs_atomic_list x l = true ->
      (~ exists v, In (x,v) z) ->

```



```

    var_occurs_atomic_list x (multisubs_atomic_list z l) = true.
Proof.
  intros.
  induction l.
  unfold var_occurs_atomic_list in H.
  simpl in H. discriminate.
  unfold var_occurs_atomic_list in H.
  simpl in H.
  assert (var_occurs_atomic x a = true /\ exists_bool (var_occurs_atomic x) l
    = true).
  induction (var_occurs_atomic x a).
  left; trivial. right. simpl in H. assumption. clear H.
  elim H1; intro H2; clear H1.
  assert (var_occurs_atomic x (multisubs_atomic z a) = true).
  apply blah3. assumption. assumption.
  unfold var_occurs_atomic_list.
  simpl.
  rewrite H. simpl. trivial.

  assert (var_occurs_atomic_list x (multisubs_atomic_list z l) = true).
  apply IHl.
  unfold var_occurs_atomic_list. assumption.
  unfold var_occurs_atomic_list.
  simpl.
  unfold var_occurs_atomic_list in H.
  rewrite H.
  induction (var_occurs_atomic x (multisubs_atomic z a)).
  simpl. trivial. simpl. trivial.
Qed.

  apply blah4 with (l:=multisubs_atomic_list s2 l)(x:=x)(z:=z).
  assumption. unfold not. assumption.
  elim H; intros v H11; clear H.
  exists v. apply in_or_app.
  right. assumption.
Qed.

```

```

Lemma var_occurs_match_list_atomic_list2 :
  forall (l l0 : list atomic)(s : substitution),
    In s (match_list_atomic_list l l0) ->
      forall (x : nat),
        var_occurs_atomic_list x l = true ->
          (exists v, In (x, v) s).

```

```

Proof.
  intros.
  unfold match_list_atomic_list in H.
  apply var_occurs_match_list_atomic_list_aux2 with (l:=l)(l0:=l0)(s:=s)(s2:=nil
    :(list (nat*rval))).
  assumption.

```

```

assert (multisubs_atomic_list nil 1 = 1).
apply multisubs_atomic_list_nil.
rewrite H1. assumption.
Qed.

```

```

Lemma ground_extend_match_form_atomic_list :
  forall (F : form)(Base : list atomic),
    no_head_vars_form F ->
      ground_atomic_list (match_form_atomic_list F Base) = true.

```

Proof.

```

intros.
induction F.
unfold match_form_atomic_list.
unfold no_head_vars_form in H.
unfold ground_atomic_list.
apply in_forall_bool.
intros a2 H1.
assert (exists y, In y (match_list_atomic_list (body (clause a 1)) Base) /\ a2
  = (fun s => multisubs_atomic s (head (clause a 1))) y).
apply in_map with (f:= (fun s => multisubs_atomic s (head (clause a 1)))).
assumption. clear H1.
simpl in H0.
elim H0; intros y H3; clear H0.
elim H3; intros H4 H5; clear H3.
assert (forall (x : nat),
  var_occurs_atomic_list x 1 = true ->
    (exists v, In (x, v) y)).
intros x H1.
apply var_occurs_match_list_atomic_list2 with (s:=y)(l:=1)(l0:=Base).
assumption. assumption.
assert (forall x, var_occurs_atomic x a = true -> var_occurs_atomic x a2 = false).
intros x H6.
assert (exists v, In (x,v) y).
apply H0.
apply H. assumption.
rewrite H5.
clear a2 H H4 H5 H6 H0.
elim H1; intros v H2; clear H1.
induction y.
simpl in H2. contradiction.
simpl in H2.
elim H2; intro H3; clear H2.
rewrite H3.

```

```

Lemma no_var_occurs_term_multisubs :
  forall (x : nat)(v : rval)(s : substitution)(a : term),
    var_occurs_term x (multisubs_term ((x, v) :: s) a) = false.

```

Proof.

```

intros.

```

```

induction a.
simpl.
assert (multisubs_term s (ident n) = ident n).
apply multisubs_term_ident.
rewrite H. simpl. trivial.
simpl.
assert (multisubs_term s (kident k) = kident k).
apply multisubs_term_kident.
rewrite H. simpl. trivial.
simpl.
induction (eq_nat_dec x n).
induction v.
assert (multisubs_term s (ident n0) = ident n0).
apply multisubs_term_ident.
rewrite H.
simpl. trivial.
assert (multisubs_term s (kident k) = kident k).
apply multisubs_term_kident.
rewrite H.
simpl. trivial.
induction s.
simpl.
induction (eq_nat_dec n x). rewrite a in b. assert (x=x); trivial. contradic
tion. trivial.
simpl.
induction a.
simpl.
induction (eq_nat_dec a n).
induction b0.
assert (multisubs_term s (ident n0) = ident n0).
apply multisubs_term_ident.
rewrite H. simpl. trivial.
assert (multisubs_term s (kident k) = kident k).
apply multisubs_term_kident.
rewrite H. simpl. trivial.
assumption.
Qed.

```

```

Lemma no_var_occurs_term_list_multisubs :
  forall (x : nat)(v : rval)(s : substitution)(a : list term),
    var_occurs_term_list x (multisubs_term_list ((x, v) :: s) a ) = false.

```

Proof.

```

intros.
induction a.
unfold var_occurs_term_list.
simpl. trivial.
assert (var_occurs_term_list x (multisubs_term_list ((x,v)::s)(a::a0)) =
  var_occurs_term x (multisubs_term ((x,v)::s) a) ||
  var_occurs_term_list x (multisubs_term_list ((x,v)::s) a0)).
simpl. trivial.
rewrite H.

```

```

rewrite IHa.
assert (var_occurs_term x (multisubs_term ((x,v)::s) a) = false).
apply no_var_occurs_term_multisubs.
rewrite H0. trivial.
Qed.

Lemma no_var_occurs_atomic_multisubs :
  forall (x : nat)(v : rval)(s : substitution)(a : atomic),
    var_occurs_atomic x (multisubs_atomic ((x, v) :: s) a ) = false.
Proof.
  intros.
  induction a.
  simpl.
  assert (var_occurs_term_list x (multisubs_term_list ((x, v) :: s) l) = false
  ).
  apply no_var_occurs_term_list_multisubs.
  unfold multisubs_term_list in H. assumption.

  simpl.
  assert (var_occurs_term_list x (multisubs_term_list ((x, v) :: s) l) = false
  ).
  apply no_var_occurs_term_list_multisubs.
  unfold multisubs_term_list in H. assumption.

  simpl.
  assert (var_occurs_term x (multisubs_term ((x, v) :: s) t) = false).
  apply no_var_occurs_term_multisubs.
  simpl in H.
  assumption.

  simpl.
  assert (var_occurs_term x (multisubs_term ((x, v) :: s) t) = false).
  apply no_var_occurs_term_multisubs.
  simpl in H.
  assumption.
Qed.

apply no_var_occurs_atomic_multisubs.
assert (var_occurs_atomic x (multisubs_atomic y a) = false).
apply IHy. assumption. clear IHy H3.
assert (multisubs_atomic (a0 :: y) a = multisubs_atomic y (subs_atomic a0 a)).
apply multisubs_cons.
rewrite H0. clear H0.

induction a0.
induction a.

(* bare *)
simpl in H. simpl.
induction l0. simpl. unfold var_occurs_term_list. simpl. trivial.
simpl in H.

```

```

unfold var_occurs_term_list in H.
simpl in H.
assert (var_occurs_term x (multisubs_term y a) = false).
induction (var_occurs_term x (multisubs_term y a)). simpl in H. discriminate.
trivial.
assert (var_occurs_term_list x (map (multisubs_term y) l0) = false).
induction (var_occurs_term x (multisubs_term y a)). discriminate. simpl in H.
unfold var_occurs_term_list.
assumption.
clear H.
assert (var_occurs_term_list x
      (map (multisubs_term y) (map (subs_term (a0, b)) l0)) = false).
apply IHl0.
assumption.
clear IHl0.
induction a.
  (* ident *)
simpl.
assert (multisubs_term y (ident n0) = ident n0).
apply multisubs_term_ident.
rewrite H2.
unfold var_occurs_term_list.
simpl.
assumption.
  (* kident *)
simpl.
assert (multisubs_term y (kident k) = kident k).
apply multisubs_term_kident.
rewrite H2.
unfold var_occurs_term_list.
simpl.
assumption.
  (* var *)
unfold var_occurs_term_list.
simpl.
induction (eq_nat_dec a0 n0).
induction b.
assert (multisubs_term y (ident n1) = ident n1).
apply multisubs_term_ident.
rewrite H2.
clear H2.
simpl. assumption.
assert (multisubs_term y (kident k) = kident k).
apply multisubs_term_kident.
rewrite H2.
clear H2.
simpl. assumption.

unfold var_occurs_term_list in H.
rewrite H.
rewrite H0. simpl. trivial.

```

```

(* says *)
simpl in H. simpl.
induction l0. simpl. unfold var_occurs_term_list. simpl. trivial.
simpl in H.
unfold var_occurs_term_list in H.
simpl in H.
assert (var_occurs_term x (multisubs_term y a) = false).
induction (var_occurs_term x (multisubs_term y a)). simpl in H. discriminate.
trivial.
assert (var_occurs_term_list x (map (multisubs_term y) l0) = false).
induction (var_occurs_term x (multisubs_term y a)). discriminate. simpl in H.
unfold var_occurs_term_list.
assumption.
clear H.
assert (var_occurs_term_list x
      (map (multisubs_term y) (map (subs_term (a0, b)) l0)) = false).
apply IHl0.
assumption.
clear IHl0.
induction a.
  (* ident *)
  simpl.
  assert (multisubs_term y (ident n1) = ident n1).
  apply multisubs_term_ident.
  rewrite H2.
  unfold var_occurs_term_list.
  simpl.
  assumption.
  (* kident *)
  simpl.
  assert (multisubs_term y (kident k) = kident k).
  apply multisubs_term_kident.
  rewrite H2.
  unfold var_occurs_term_list.
  simpl.
  assumption.
  (* var *)
  unfold var_occurs_term_list.
  simpl.
  induction (eq_nat_dec a0 n1).
  induction b.
  assert (multisubs_term y (ident n2) = ident n2).
  apply multisubs_term_ident.
  rewrite H2.
  clear H2.
  simpl. assumption.
  assert (multisubs_term y (kident k) = kident k).
  apply multisubs_term_kident.
  rewrite H2.
  clear H2.

```

```

simpl. assumption.

unfold var_occurs_term_list in H.
rewrite H.
rewrite H0. simpl. trivial.

(* extbare *)

simpl in H. simpl.
induction t; simpl. assumption. assumption.
induction (eq_nat_dec a0 n).
induction b.
assert (multisubs_term y (ident n0) = ident n0).
apply multisubs_term_ident.
rewrite H0. simpl. trivial.
assert (multisubs_term y (kident k) = kendit k).
apply multisubs_term_kident.
rewrite H0. simpl. trivial.
assumption.

(* extsays *)
simpl in H. simpl.
induction t; simpl. assumption. assumption.
induction (eq_nat_dec a0 n0).
induction b.
(* induction (eq_nat_dec a0 n0). *)
assert (multisubs_term y (ident n1) = ident n1).
apply multisubs_term_ident.
rewrite H0. simpl. trivial.
assert (multisubs_term y (kident k) = kendit k).
apply multisubs_term_kident.
rewrite H0. simpl. trivial.
assumption.
clear l Base H H4 H0.

(* If a var occurs in a, then it does not occur in a2.
But a2 is the result of removing free vars from a, so it has strictly less var
s.*)

Lemma no_carryover_vars_ground_term:
  forall (y : list (nat * rval))(a : term),
    (forall (x : nat),
      var_occurs_term x a = true -> var_occurs_term x (multisubs_term y a) = false) -
    >
    ground_term (multisubs_term y a) = true.
Proof.
  intros.
  induction a.
  assert (multisubs_term y (ident n) = ident n).
  apply multisubs_term_ident.
  rewrite H0.

```

```

simpl. trivial.
assert (multisubs_term y (kident k) = kident k).
apply multisubs_term_kident.
rewrite H0.
simpl. trivial.

Lemma subs_term_var :
  forall (y : (nat * rval))(n : nat),
    (subs_term y (var n) = var n  $\wedge$  (exists i, subs_term y (var n) = ident i)  $\wedge$  (exists k, subs_term y (var n) = kident k)).
Proof.
  induction y.
  intros.
  simpl.
  induction (eq_nat_dec a n).
  induction b.
  right; left. exists n0. trivial.
  right. right. exists k; trivial.
  left. trivial.
Qed.

Lemma multisubs_term_var :
  forall (y : list (nat * rval))(n : nat),
    (multisubs_term y (var n) = var n  $\wedge$  (exists i, multisubs_term y (var n) = ident i)  $\wedge$  (exists k, multisubs_term y (var n) = kident k)).
Proof.
  intros.
  induction y.
  simpl. left; trivial.
  simpl.
  elim IHy; intro H1; clear IHy.

  assert (subs_term a (var n) = var n  $\wedge$  (exists i, subs_term a (var n) = ident i)  $\wedge$  (exists k, subs_term a (var n) = kident k)).
  apply subs_term_var.
  elim H; intro H0; clear H.
  rewrite H0. left; assumption.
  elim H0; intro H0v1; clear H0.
  elim H0v1; intros i H2; clear H0v1.
  right. left. exists i. rewrite H2.
  apply multisubs_term_ident.
  elim H0v1; intros k H2; clear H0v1.
  rewrite H2.
  right. right.
  exists k. apply multisubs_term_kident.

  assert (subs_term a (var n) = var n  $\wedge$  (exists i, subs_term a (var n) = ident i)  $\wedge$  (exists k, subs_term a (var n) = kident k)).
  apply subs_term_var.
  elim H; intro H0; clear H.
  rewrite H0.

```



```

    elim H1; intro H2; clear H1.
    elim H2; intros i H3; clear H2.
    rewrite H3. right. left. exists i; trivial.
    elim H2; intros k H3; clear H2.
    rewrite H3. right. right. exists k; trivial.
    elim H0; intro H2; clear H0.
    elim H2; intros i H3; clear H2.
    rewrite H3. right. left. exists i. apply multisubs_term_ident.
    elim H2; intros i H3; clear H2.
    rewrite H3. right. right. exists i. apply multisubs_term_kident.
  Qed.

  assert (multisubs_term y (var n) = var n  $\wedge$  (exists i, multisubs_term y (var
    n) = ident i)  $\wedge$  (exists k, multisubs_term y (var n) = kident k)).
  apply multisubs_term_var.
  elim H0; intro H1; clear H0.
  rewrite H1 in H.
  assert (var_occurs_term n (var n) = false).
  apply H. simpl. induction (eq_nat_dec n n). trivial. contradiction b. trivial.
  simpl in H0. induction (eq_nat_dec n n). discriminate. contradiction b. trivial.
  elim H1; intro H1v1; clear H1.
  elim H1v1; intros i H2; clear H1v1.
  rewrite H2.
  simpl. trivial.
  elim H1v1; intros i H2; clear H1v1.
  rewrite H2.
  simpl. trivial.
  Qed.

  Lemma no_carryover_vars_ground_atomic:
    forall (y : list (nat * rval))(a : atomic),
      (forall (x : nat),
        var_occurs_atomic x a = true -> var_occurs_atomic x (multisubs_atomic y a) = fa
        lse) ->
        ground_atomic (multisubs_atomic y a) = true.
  Proof.
    intros.
    induction a.
    induction l.
    simpl. unfold ground_term_list. simpl. trivial.
    simpl.
    simpl in H.
    simpl in IHl.

    assert (ground_term (multisubs_term y a) = true).
    apply no_carryover_vars_ground_term.
    intros.
    unfold var_occurs_term_list in H.
    simpl in H.

```

```

cut (var_occurs_term x a || exists_bool (var_occurs_term x) l = true).
intros.
assert (var_occurs_term x (multisubs_term y a)
  || exists_bool (var_occurs_term x) (map (multisubs_term y) l) = false).
apply H. assumption.
induction (var_occurs_term x (multisubs_term y a)).
simpl in H2. discriminate. trivial.
rewrite H0. simpl. trivial.

assert (ground_term_list (map (multisubs_term y) l) = true).
apply IH1.
intros.
assert (var_occurs_term_list x (multisubs_term y a :: map (multisubs_term y)
  l) =
  false).
apply H.
unfold var_occurs_term_list. simpl.
unfold var_occurs_term_list in H1.
rewrite H1.
induction (var_occurs_term x a). trivial. simpl. trivial.
unfold var_occurs_term_list in H2.
simpl in H2.
unfold var_occurs_term_list.
induction (var_occurs_term x (multisubs_term y a)).
simpl in H2. discriminate.
simpl in H2. assumption.
unfold ground_term_list.
simpl.
rewrite H0.
unfold ground_term_list in H1.
rewrite H1.
trivial.

induction l.
simpl. unfold ground_term_list. simpl. trivial.
simpl.
simpl in H.
simpl in IH1.

assert (ground_term (multisubs_term y a) = true).
apply no_carryover_vars_ground_term.
intros.
unfold var_occurs_term_list in H.
simpl in H.
cut (var_occurs_term x a || exists_bool (var_occurs_term x) l = true).
intros.
assert (var_occurs_term x (multisubs_term y a)
  || exists_bool (var_occurs_term x) (map (multisubs_term y) l) = false).
apply H. assumption.
induction (var_occurs_term x (multisubs_term y a)).
simpl in H2. discriminate. trivial.

```

```

rewrite H0. simpl. trivial.

assert (ground_term_list (map (multisubs_term y) l) = true).
apply IH1.
intros.
assert (var_occurs_term_list x (multisubs_term y a :: map (multisubs_term y)
  l) =
  false).
apply H.
unfold var_occurs_term_list. simpl.
unfold var_occurs_term_list in H1.
rewrite H1.
induction (var_occurs_term x a). trivial. simpl. trivial.
unfold var_occurs_term_list in H2.
simpl in H2.
unfold var_occurs_term_list.
induction (var_occurs_term x (multisubs_term y a)).
simpl in H2. discriminate.
simpl in H2. assumption.
unfold ground_term_list.
simpl.
rewrite H0.
unfold ground_term_list in H1.
rewrite H1.
trivial.

simpl in H.
simpl.
apply no_carryover_vars_ground_term.
intros.
apply H. assumption.

simpl in H.
simpl.
apply no_carryover_vars_ground_term.
intros.
apply H. assumption.
Qed.

rewrite H5.
apply no_carryover_vars_ground_atomic.
intros.
rewrite H5 in H1.
apply H1. assumption.
Qed.

Lemma ground_extend_match_form_list_atomic_list :
  forall (LF : list form)(Base : list atomic),
    no_head_vars_form_list LF ->
    ground_atomic_list Base = true ->
    ground_atomic_list (match_form_list_atomic_list LF Base) = true.

```

Proof.

```
intros.
unfold match_form_list_atomic_list.
induction LF.
simpl. unfold ground_atomic_list. simpl. trivial.
unfold ground_atomic_list.
simpl.
apply in_forall_bool.
intros atm H1.
assert (In atm (match_form_atomic_list a Base) \/\ In atm (flatten (map (fun f
: form => match_form_atomic_list f Base) LF))))).
apply in_app_or. assumption.
clear H1.
elim H2; intro H3; clear H2.
assert (ground_atomic_list (match_form_atomic_list a Base) = true).
apply ground_extend_match_form_atomic_list.
unfold no_head_vars_form_list in H.
unfold forallelts in H.
simpl in H.
apply H. left; trivial.
unfold ground_atomic_list in H1.
apply forall_bool_in with (A:=atomic)(L:=match_form_atomic_list a Base)(F:=ground_atomic)(x:=atm).
assumption. assumption.
assert (ground_atomic_list
      (flatten (map (fun f : form => match_form_atomic_list f Base) LF)) =
      true).
apply IHLF.
unfold no_head_vars_form_list.
unfold forallelts.
intros x H4.
unfold no_head_vars_form_list in H.
unfold forallelts in H.
apply H.
simpl. right; assumption.
unfold ground_atomic_list in H1.
apply forall_bool_in with (A:=atomic)(L:=(flatten (map (fun f : form => match_form_atomic_list f Base) LF)))(F:=ground_atomic)(x:=atm).
assumption. assumption.
```

Qed.

(* To show finiteness of the extension step, we show that nothing new comes out of extensions.
In particular, no new terms are created, no new arities are generated, and no new predicate names are generated. Any new atoms going into the db are thus from a finite pool, so there is a bound on the number of extensions. *)

Definition term_in_atomic (T : term)(A : atomic) :=

```

match A with
| bare P B =>
  In T B
| says W P B =>
  In T B
| extbare K =>
  T = K
| extsays W K =>
  T = K
end.

Definition term_in_atomic_list (T : term)(LA : list atomic) :=
  exists x, In x LA /\ term_in_atomic T x.

Definition term_in_form (T : term)(F : form) :=
  match F with
  | clause H B =>
    term_in_atomic T H /\
    term_in_atomic_list T B
  end.

Definition term_in_form_list (T : term)(LF : list form) :=
  exists x, In x LF /\ term_in_form T x.

Definition pred_in_atomic (p : nat)(A : atomic) :=
  match A with
  | bare P B => P = p
  | says W P B => P = p
  | extbare K => False
  | extsays W K => False
  end.

Definition pred_in_atomic_list (p : nat)(LA : list atomic) :=
  exists x, In x LA /\ pred_in_atomic p x.

Definition pred_in_form (p : nat)(F : form) :=
  match F with
  | clause H B =>
    pred_in_atomic p H /\
    pred_in_atomic_list p B
  end.

Definition pred_in_form_list (p : nat)(LF : list form) :=
  exists x, In x LF /\ pred_in_form p x.

Definition prin_in_atomic (p : nat)(A : atomic) :=
  match A with
  | bare P B => False
  | says W P B => W = p
  | extbare K => False
  | extsays W K => W = p

```

```

end.

Definition prin_in_atomic_list (p : nat)(LA : list atomic) :=
  exists x, In x LA /\ prin_in_atomic p x.

Definition prin_in_form (p : nat)(F : form) :=
  match F with
  | clause H B =>
    prin_in_atomic p H /\
    prin_in_atomic_list p B
  end.

Definition prin_in_form_list (p : nat)(LF : list form) :=
  exists x, In x LF /\ prin_in_form p x.

Definition arity_in_atomic (a : nat)(A : atomic) :=
  match A with
  | bare P B => length B = a
  | says W P B => length B = a
  | extbare K => False
  | extsays W K => False
  end.

Definition arity_in_atomic_list (a : nat)(LA : list atomic) :=
  exists x, In x LA /\ arity_in_atomic a x.

Definition arity_in_form (a : nat)(F : form) :=
  match F with
  | clause H B =>
    arity_in_atomic a H /\
    arity_in_atomic_list a B
  end.

Definition arity_in_form_list (a : nat)(LF : list form) :=
  exists x, In x LF /\ arity_in_form a x.

(* Show that extension never adds new identifier *)

Lemma term_in_match_term :
  forall (x : nat)(t : rval)(S : substitution)(A B : term),
    match_term A B = Some S ->
    In (x, t) S ->
    B = val_of_rval t.
Proof.
  intros.
  induction A.
  induction B.
  simpl in H.
  induction (eq_nat_dec n n0).
  injection H. intro H1. rewrite <- H1 in H0. simpl in H0. contradiction.

```

```

discriminate.
simpl in H. discriminate.
simpl in H. discriminate.
simpl in H.
induction B. discriminate.
induction (eq_kthing_dec k k0).
injection H. intro H1.
rewrite <- H1 in H0.
simpl in H0. contradiction.
discriminate. discriminate.
induction B.
simpl in H.
injection H; intro H1; clear H.
rewrite <- H1 in H0.
simpl in H0.
elim H0; intro H2; try contradiction.
injection H2; intros H3 H4; clear H2.
rewrite <- H3. simpl. trivial.
simpl in H.
injection H; intro H1; clear H.
rewrite <- H1 in H0.
simpl in H0.
elim H0; intro H2; try contradiction.
injection H2; intros H3 H4; clear H2.
rewrite <- H3. simpl. trivial.
simpl in H. discriminate.
Qed.

Lemma term_in_match_term_list_n :
  forall (n : nat)(x : nat)(t : rval)(S : substitution)(A B : list term),
    length A = n ->
      match_term_list A B = Some S ->
        In (x, t) S ->
          In (val_of_rval t) B.
Proof.
  induction n.
  intros.
  induction A.
  induction B.
  simpl in H0.
  injection H0. intro H2. rewrite <- H2 in H1. simpl in H1. contradiction.
  simpl in H0. discriminate.
  simpl in H. discriminate.
  intros.
  induction B.
  simpl in H0.
  induction A.
  injection H0; intro H2; rewrite <- H2 in H1; simpl in H1; contradiction.
  discriminate.
  clear IHB.
  induction A.

```

```

simpl in H0. discriminate.
clear IHA.

simpl in H0.
assert (exists s2, match_term a0 a = Some s2).
induction (match_term a0 a).
exists a1. trivial.
discriminate.
elim H2; intros s2 H3; clear H2.
rewrite H3 in H0.
assert (exists s3, match_term_list (multisubs_term_list s2 A) B = Some s3).
induction (match_term_list (multisubs_term_list s2 A) B).
exists a1. trivial. discriminate.
elim H2; intros s3 H4; clear H2.
rewrite H4 in H0.
injection H0; intro H5; clear H0.
rewrite <- H5 in H1. clear H5.
assert (In (x, t) s2 /\ In (x, t) s3).
apply in_app_or. assumption.
clear H1.
elim H0; intro H5; clear H0.
assert (a = val_of_rval t).
apply term_in_match_term with (A := a0)(S := s2)(x := x).
assumption. assumption.
rewrite H0. simpl. left; trivial.
simpl. right.
apply IHn with (A := multisubs_term_list s2 A)(B := B)(S := s3)(x := x).
assert (length (multisubs_term_list s2 A) = length A).
apply length_multisubs_term_list. rewrite H0.
simpl in H. injection H; intro H6; clear H. assumption.
assumption. assumption.
Qed.

Lemma term_in_match_term_list :
  forall (x : nat)(t : rval)(S : substitution)(A B : list term),
    match_term_list A B = Some S ->
      In (x, t) S ->
        In (val_of_rval t) B.
Proof.
  intros.
  apply term_in_match_term_list_n with (x := x)(t := t)(S := S)(A := A)(B := B)(
    n := length A); try trivial; try assumption.
Qed.

Lemma term_in_match_atomic:
  forall (x : nat)(t : rval)(S : substitution)(A B : atomic),
    match_atomic A B = Some S ->
      In (x, t) S ->
        term_in_atomic (val_of_rval t) B.
Proof.
  induction A; induction B; simpl; intros; try discriminate.

```



```

induction (eq_nat_dec n n0).
apply term_in_match_term_list with (x := x)(t := t)(S := S)(A := 1)(B := 10);
try assumption.
discriminate.
induction (eq_nat_dec n0 n2).
induction (eq_nat_dec n n1).
apply term_in_match_term_list with (x := x)(t := t)(S := S)(A := 1)(B := 10);
try assumption.
discriminate.
discriminate.
symmetry.
apply term_in_match_term with (x := x)(t := t)(S := S)(A := t0)(B := t1); try
assumption.
induction (eq_nat_dec n n0).
symmetry.
apply term_in_match_term with (x := x)(t := t)(S := S)(A := t0)(B := t1); try
assumption.
discriminate.
Qed.

```

```

Lemma pred_in_match_atomic:
  forall (x : nat)(S : substitution)(A B : atomic),
    match_atomic A B = Some S ->
      pred_in_atomic x A ->
        pred_in_atomic x B.

```

```

Proof.
  induction A; induction B; simpl; intros; try discriminate.
  induction (eq_nat_dec n n0).
  rewrite <- H0.
  rewrite <- a.
  trivial.
  discriminate.
  induction (eq_nat_dec n0 n2).
  induction (eq_nat_dec n n1).
  rewrite <- H0.
  rewrite <- a.
  trivial.
  discriminate. discriminate.
  contradiction.
  contradiction.
Qed.

```

```

Lemma pred_in_match_atomic_list:
  forall (x : nat)(LS : substitution)(A : atomic)(LA : list atomic),
    In LS (match_atomic_list A LA) ->
      pred_in_atomic x A ->
        pred_in_atomic_list x LA.

```

```

Proof.
  intros.
  induction LA.
  simpl in H. contradiction.

```

```

simpl in H. assert (1=1); trivial.
assert ({exists s, match_atomic A a = Some s}+{match_atomic A a = None}).
apply option_dec.
elim H2; intro H3; clear H2.
elim H3; intros s H4; clear H3.
rewrite H4 in H.
assert (pred_in_atomic x a).
apply pred_in_match_atomic with (A := A)(B := a)(S := s).
assumption. assumption.
unfold pred_in_atomic_list.
exists a. split. simpl. left; trivial. assumption.
rewrite H3 in H.
assert (pred_in_atomic_list x LA).
apply IHLA.
assumption.
unfold pred_in_atomic_list.
unfold pred_in_atomic_list in H2.
elim H2; intros x0 H4; clear H2.
elim H4; intros H5 H6; clear H4.
exists x0. split. simpl. right. assumption. assumption.
Qed.

```

```

Lemma prin_in_match_atomic:
  forall (x : nat)(S : substitution)(A B : atomic),
    match_atomic A B = Some S ->
      prin_in_atomic x A ->
      prin_in_atomic x B.

```

```

Proof.
  induction A; induction B; simpl; intros; try discriminate.
  induction (eq_nat_dec n n0).
  contradiction. contradiction.
  induction (eq_nat_dec n0 n2).
  induction (eq_nat_dec n n1).
  rewrite <- H0.
  rewrite <- a0.
  trivial.
  discriminate. discriminate.
  contradiction.
  induction (eq_nat_dec n n0); simpl in H; try discriminate.
  rewrite <- H0.
  rewrite <- a. trivial.
Qed.

```

```

Lemma prin_in_match_atomic_list:
  forall (x : nat)(LS : substitution)(A : atomic)(LA : list atomic),
    In LS (match_atomic_list A LA) ->
      prin_in_atomic x A ->
      prin_in_atomic_list x LA.

```

```

Proof.
  intros.
  induction LA.

```

```

    simpl in H. contradiction.
    simpl in H. assert (1=1); trivial.
    assert ({exists s, match_atomic A a = Some s}+{match_atomic A a = None}).
    apply option_dec.
    elim H2; intro H3; clear H2.
    elim H3; intros s H4; clear H3.
    rewrite H4 in H.
    assert (prin_in_atomic x a).
    apply prin_in_match_atomic with (A := A)(B := a)(S := s).
    assumption. assumption.
    unfold prin_in_atomic_list.
    exists a. split. simpl. left; trivial. assumption.
    rewrite H3 in H.
    assert (prin_in_atomic_list x LA).
    apply IHLA.
    assumption.
    unfold prin_in_atomic_list.
    unfold prin_in_atomic_list in H2.
    elim H2; intros x0 H4; clear H2.
    elim H4; intros H5 H6; clear H4.
    exists x0. split. simpl. right. assumption. assumption.
Qed.

Lemma term_in_match_atomic_list:
  forall (x : nat)(t : rval)(S : substitution)(A : atomic)(LA : list atomic),
    In S (match_atomic_list A LA) ->
    In (x, t) S ->
    term_in_atomic_list (val_of_rval t) LA.
Proof.
  intros.
  induction LA.
  simpl in H. contradiction.
  simpl in H.
  assert ({exists s, match_atomic A a = Some s}+{match_atomic A a = None}).
  apply option_dec.
  elim H1; intro H2; clear H1.
  elim H2; intros s H3; clear H2.
  rewrite H3 in H.
  simpl in H.
  elim H; intro H4; clear H.
  assert (term_in_atomic (val_of_rval t) a).
  apply term_in_match_atomic with (A := A)(S := s)(x := x). assumption.
  rewrite H4. assumption.
  unfold term_in_atomic_list.
  exists a.
  split. simpl. left; trivial. assumption.
  assert (term_in_atomic_list (val_of_rval t) LA).
  apply IHLA. assumption.
  unfold term_in_atomic_list.
  unfold term_in_atomic_list in H.
  elim H; intros y H5; clear H.

```

```

elim H5; intros H6 H7; clear H5.
exists y. split. simpl. right; assumption. assumption.
rewrite H2 in H.
assert (term_in_atomic_list (val_of_rval t) LA).
apply IHLA. assumption.
unfold term_in_atomic_list.
unfold term_in_atomic_list in H1.
elim H1; intros y H5; clear H1.
elim H5; intros H6 H7; clear H5.
exists y. split. simpl. right; assumption. assumption.
Qed.

Lemma term_in_match_list_atomic_list_aux_n:
  forall (n : nat)(x : nat)(t : rval)(S Spre : substitution)(LA1 LA2 : list atom
ic),
    length LA1 = n ->
      In S (match_list_atomic_list_aux LA1 LA2 Spre) ->
        In (x, t) S ->
          term_in_atomic_list (val_of_rval t) LA2.
Proof.
  induction n.
  intros.
  induction LA1.
  simpl in H0.
  elim H0; intro H2; clear H0.
  rewrite <- H2 in H1; simpl in H1; contradiction. contradiction.
  simpl in H. discriminate.
  intros.
  induction LA1.
  simpl in H. discriminate. clear IHLA1.
  induction LA2.
  simpl in H0. contradiction. clear IHLA2.
  simpl in H.
  injection H; intro Hlen. clear H.
  simpl in H0.
  assert (exists x, In x (map
    (fun s : list (nat * rval) =>
      map_append s
        (match_list_atomic_list_aux LA1 (a0 :: LA2) (Spre ++ s)))
    match match_atomic (multisubs_atomic Spre a) a0 with
    | Some s =>
      s :: match_atomic_list (multisubs_atomic Spre a) LA2
    | None => match_atomic_list (multisubs_atomic Spre a) LA2
    end) /\ In S x).
  apply in_flatten. assumption.
  elim H; intros y H2; clear H.
  elim H2; intros H3 H4; clear H2.
  clear H0.
  assert (exists z, In z (match match_atomic (multisubs_atomic Spre a) a0 with
    | Some s => s :: match_atomic_list (multisubs_atomic Spre a) LA2
    | None => match_atomic_list (multisubs_atomic Spre a) LA2

```

```

end) /\ y = (fun s : list (nat * rval) =>
  map_append s
    (match_list_atomic_list_aux LA1 (a0 :: LA2) (Spre ++ s))) z).
apply in_map with (f := (fun s : list (nat * rval) =>
  map_append s
    (match_list_atomic_list_aux LA1 (a0 :: LA2) (Spre ++ s)))).
assumption.
clear H3.
elim H; intros z H5; clear H.
elim H5; intros H6 H7; clear H5.
assert ({exists s2, match_atomic (multisubs_atomic Spre a) a0 = Some s2}+{match_atomic (multisubs_atomic Spre a) a0 = None}).
apply option_dec.
elim H; intro H8; clear H.
elim H8; intros s2 H9; clear H8.
rewrite H9 in H6.
simpl in H6.
elim H6; intro H10; clear H6.
unfold map_append in H7.
rewrite H7 in H4.
assert (exists xx, In xx (match_list_atomic_list_aux LA1 (a0 :: LA2) (Spre ++ z)) /\ S = (fun x : list (nat * rval) => z ++ x) xx).
apply in_map.
assumption.
elim H; intros xx H11; clear H.
elim H11; intros H12 H13; clear H11.
clear H4.
assert (In (x, t) z \/ In (x, t) xx).
apply in_app_or. rewrite H13 in H1. assumption.
elim H; intro H14; clear H.
rewrite <- H10 in H14.
assert (term_in_atomic (val_of_rval t) a0).
apply term_in_match_atomic with (A := multisubs_atomic Spre a)(B := a0)(S := s2)(x := x).
assumption. assumption.
unfold term_in_atomic_list.
exists a0. split. simpl. left; trivial. assumption.
apply IHn with (x := x)(t := t)(S := xx)(Spre := Spre ++ z)(LA1 := LA1)(LA2 := a0 :: LA2).
assumption. assumption. assumption.

unfold map_append in H7.
rewrite H7 in H4.
assert (exists xx, In xx (match_list_atomic_list_aux LA1 (a0 :: LA2) (Spre ++ z)) /\ S = (fun x : list (nat * rval) => z ++ x) xx).
apply in_map.
assumption.
elim H; intros xx H11; clear H.
elim H11; intros H12 H13; clear H11.
clear H4.
assert (In (x, t) z \/ In (x, t) xx).

```

```

apply in_app_or. rewrite H13 in H1. assumption.
elim H; intro H14; clear H.
assert (term_in_atomic_list (val_of_rval t) LA2).
apply term_in_match_atomic_list with (A := multisubs_atomic Spre a)(LA := LA2)
(x := x)(t := t)(S := z).
assumption. assumption.
unfold term_in_atomic_list.
unfold term_in_atomic_list in H.
elim H; intros xxx H15; clear H.
exists xxx. split. simpl. right. elim H15. trivial.
elim H15; trivial.
apply IHn with (x := x)(t := t)(S := xx)(Spre := Spre ++ z)(LA1 := LA1)(LA2 :=
a0 :: LA2).
assumption. assumption. assumption.

rewrite H8 in H6.
unfold map_append in H7.
rewrite H7 in H4.
assert (exists xx, In xx (match_list_atomic_list_aux LA1 (a0 :: LA2) (Spre ++
z)) /\ S = (fun x : list (nat * rval) => z ++ x) xx).
apply in_map.
assumption.
elim H; intros xx H11; clear H.
elim H11; intros H12 H13; clear H11.
clear H4.
assert (In (x, t) z \/ In (x, t) xx).
apply in_app_or. rewrite H13 in H1. assumption.
elim H; intro H14; clear H.
assert (term_in_atomic_list (val_of_rval t) LA2).
apply term_in_match_atomic_list with (A := multisubs_atomic Spre a)(LA := LA2)
(x := x)(t := t)(S := z).
assumption. assumption.
unfold term_in_atomic_list.
unfold term_in_atomic_list in H.
elim H; intros xxx H15; clear H.
exists xxx. split. simpl. right. elim H15. trivial.
elim H15; trivial.
apply IHn with (x := x)(t := t)(S := xx)(Spre := Spre ++ z)(LA1 := LA1)(LA2 :=
a0 :: LA2).
assumption. assumption. assumption.
Qed.

```

Lemma term_in_match_list_atomic_list_aux:

```

forall (x : nat)(t : rval)(S Spre : substitution)(LA1 LA2 : list atomic),
  In S (match_list_atomic_list_aux LA1 LA2 Spre) ->
  In (x, t) S ->
  term_in_atomic_list (val_of_rval t) LA2.

```

Proof.

```

intros.
apply term_in_match_list_atomic_list_aux_n with (n := length LA1)(LA1 := LA1)(
LA2 := LA2)(S := S)(Spre := Spre)(x := x)(t := t).

```

```

    trivial. assumption. assumption.
Qed.

```

```

Lemma term_in_match_list_atomic_list:
  forall (x : nat)(t : rval)(S : substitution)(LA1 LA2 : list atomic),
    In S (match_list_atomic_list LA1 LA2) ->
      In (x, t) S ->
        term_in_atomic_list (val_of_rval t) LA2.

```

```

Proof.
  intros.
  unfold match_list_atomic_list in H.
  apply term_in_match_list_atomic_list_aux with (LA1 := LA1)(LA2 := LA2)(S := S)
    (Spre := nil : (list (nat * rval)))(x := x)(t := t).
  assumption. assumption.
Qed.

```

```

Lemma pred_in_atomic_subs:
  forall (x : nat)(A : atomic)(S : nat * rval),
    pred_in_atomic x A ->
      pred_in_atomic x (subs_atomic S A).

```

```

Proof.
  intros.
  induction S.
  induction A.
  simpl.
  simpl in H. assumption.
  simpl.
  simpl in H. assumption.
  simpl in H. contradiction.
  simpl in H. contradiction.
Qed.

```

```

Lemma pred_in_atomic_subs2:
  forall (x : nat)(A : atomic)(S : nat * rval),
    pred_in_atomic x (subs_atomic S A) ->
      pred_in_atomic x A.

```

```

Proof.
  intros.
  induction S.
  induction A.
  simpl.
  simpl in H. assumption.
  simpl.
  simpl in H. assumption.
  simpl in H. contradiction.
  simpl in H. contradiction.
Qed.

```

```

Lemma prin_in_atomic_subs:
  forall (x : nat)(A : atomic)(S : nat * rval),
    prin_in_atomic x A ->

```

```
    prin_in_atomic x (subs_atomic S A).
```

```
Proof.
```

```
  intros.
  induction S.
  induction A.
  simpl.
  simpl in H. assumption.
  simpl.
  simpl in H. assumption.
  simpl in H. contradiction.
  simpl in H. simpl. assumption.
```

```
Qed.
```

```
Lemma prin_in_atomic_subs2:
```

```
  forall (x : nat)(A : atomic)(S : nat * rval),
    prin_in_atomic x (subs_atomic S A) ->
    prin_in_atomic x A.
```

```
Proof.
```

```
  intros.
  induction S.
  induction A.
  simpl.
  simpl in H. assumption.
  simpl.
  simpl in H. assumption.
  simpl in H. contradiction.
  simpl in H. simpl. assumption.
```

```
Qed.
```

```
Lemma length_map:
```

```
  forall (A B : Set)(L : list A)(f : A -> B),
    length (map f L) = length L.
```

```
Proof.
```

```
  intros.
  induction L.
  simpl. trivial.
  simpl. rewrite IHL. trivial.
```

```
Qed.
```

```
Lemma arity_in_atomic_subs2:
```

```
  forall (x : nat)(A : atomic)(S : nat * rval),
    arity_in_atomic x (subs_atomic S A) ->
    arity_in_atomic x A.
```

```
Proof.
```

```
  intros.
  induction S.
  induction A.
  simpl.
  simpl in H.
  assert (length (map (subs_term (a, b)) l) = length l).
  apply length_map.
```



```

rewrite <- H0. assumption.
simpl.
simpl in H.
assert (length (map (subs_term (a, b)) l) = length l).
apply length_map.
rewrite <- H0. assumption.
simpl in H. contradiction.
simpl in H. contradiction.
Qed.

Lemma take_off_tail:
  forall (n : nat)(s : substitution),
    length s = S n ->
      exists Shd : (list (nat * rval)),
        exists z,
          s = Shd ++ (z :: (nil : (list (nat * rval)))) /\ length Shd = n.
Proof.
  induction n.
  intros.
  induction s.
  simpl in H. discriminate.
  clear IHs.
  induction s.
  exists (nil : (list (nat * rval))).
  exists a.
  simpl. split. trivial. trivial.
  simpl in H. discriminate.
  intros.
  induction s.
  simpl in H. discriminate.
  clear IHs.
  simpl in H.
  injection H; intro H1; clear H.
  assert ( exists Shd : list (nat * rval),
    (exists z : nat * rval, s = Shd ++ z :: nil /\ length Shd = n)).
  apply IHn.
  assumption.
  elim H; intros Shd H2; clear H.
  elim H2; intros z H3; clear H2.
  elim H3; intros H4 H5; clear H3.
  exists (a :: Shd).
  exists z.
  simpl. split.
  rewrite H4.
  trivial.
  rewrite H5. trivial.
Qed.

Lemma multisubs_atomic_invert:
  forall (s : substitution)(z : nat * rval)(A : atomic),
    multisubs_atomic (s ++ z :: nil) A =

```

```

    subs_atomic z (multisubs_atomic s A).
Proof.
  induction s.
  intros.
  simpl.
  assert (multisubs_atomic nil A = A).
  apply multisubs_atomic_nil.
  rewrite H. clear H.
  assert (multisubs_atomic (z :: nil) A = multisubs_atomic nil (subs_atomic z A)
  ).
  apply multisubs_cons.
  rewrite H.
  assert (multisubs_atomic nil (subs_atomic z A) = subs_atomic z A).
  apply multisubs_atomic_nil.
  rewrite H0. trivial.
  intros.
  simpl.
  assert (multisubs_atomic (a :: (s ++ z :: nil)) A =
    multisubs_atomic (s ++ z :: nil) (subs_atomic a A)).
  apply multisubs_cons.
  rewrite H. clear H.
  assert (multisubs_atomic (s ++ z :: nil) (subs_atomic a A) =
    subs_atomic z (multisubs_atomic s (subs_atomic a A))).
  apply IHs.
  rewrite H. clear H. clear IHs.
  assert (multisubs_atomic (a :: s) A = multisubs_atomic s (subs_atomic a A)).
  apply multisubs_cons.
  rewrite H. trivial.
Qed.

```

```

Lemma pred_in_atomic_multisubs_aux:
  forall (n : nat)(x : nat)(A : atomic)(s : substitution),
    length s = n ->
    pred_in_atomic x A ->
    pred_in_atomic x (multisubs_atomic s A).

```

```

Proof.
  induction n.
  intros.
  induction s.
  assert (multisubs_atomic nil A = A).
  apply multisubs_atomic_nil.
  rewrite H1. assumption.
  simpl in H. discriminate.
  intros.
  (* Some funky business because I defined multisubs in the wrong way to do simp
  le induction. *)
  (* Have to do it in reverse *)
  assert (exists Shd : list (nat * rval),
    exists z : nat * rval, s = Shd ++ z :: nil /\ length Shd = n).
  apply take_off_tail.
  assumption.

```

```

elim H1; intros Shd H2; clear H1.
elim H2; intros z H3; clear H2.
elim H3; intros H4 H5; clear H3.
rewrite H4.
assert (multisubs_atomic (Shd ++ z :: nil) A =
  subs_atomic z (multisubs_atomic Shd A)).
apply multisubs_atomic_invert.
rewrite H1.
clear H1.
assert (pred_in_atomic x (multisubs_atomic Shd A)).
apply IHn. assumption.
apply pred_in_atomic_subs. assumption.
Qed.

Lemma prin_in_atomic_multisubs_aux:
  forall (n : nat)(x : nat)(A : atomic)(s : substitution),
    length s = n ->
      prin_in_atomic x A ->
        prin_in_atomic x (multisubs_atomic s A).
Proof.
  induction n.
  intros.
  induction s.
  assert (multisubs_atomic nil A = A).
  apply multisubs_atomic_nil.
  rewrite H1. assumption.
  simpl in H. discriminate.
  intros.
  (* Some funky business because I
defined multisubs in the wrong way to do simple induction. *)
  (* Have to do it in reverse *)
  assert (exists Shd : list (nat * rval),
    exists z : nat * rval, s = Shd ++ z :: nil /\ length Shd = n).
  apply take_off_tail.
  assumption.
  elim H1; intros Shd H2; clear H1.
  elim H2; intros z H3; clear H2.
  elim H3; intros H4 H5; clear H3.
  rewrite H4.
  assert (multisubs_atomic (Shd ++ z :: nil) A =
    subs_atomic z (multisubs_atomic Shd A)).
  apply multisubs_atomic_invert.
  rewrite H1.
  clear H1.
  assert (prin_in_atomic x (multisubs_atomic Shd A)).
  apply IHn. assumption.
  apply prin_in_atomic_subs. assumption.
Qed.

Lemma pred_in_atomic_multisubs:
  forall (x : nat)(A : atomic)(s : substitution),

```

```

    pred_in_atomic x A ->
    pred_in_atomic x (multisubs_atomic s A).
Proof.
  intros.
  apply pred_in_atomic_multisubs_aux with (n := length s).
  trivial. assumption.
Qed.

Lemma prin_in_atomic_multisubs:
  forall (x : nat)(A : atomic)(s : substitution),
    prin_in_atomic x A ->
    prin_in_atomic x (multisubs_atomic s A).
Proof.
  intros.
  apply prin_in_atomic_multisubs_aux with (n := length s).
  trivial. assumption.
Qed.

Lemma pred_in_atomic_multisubs_aux2:
  forall (n : nat)(x : nat)(A : atomic)(s : substitution),
    length s = n ->
    pred_in_atomic x (multisubs_atomic s A) ->
    pred_in_atomic x A.
Proof.
  induction n.
  intros.
  induction s.
  assert (multisubs_atomic nil A = A).
  apply multisubs_atomic_nil.
  rewrite H1 in H0. assumption.
  simpl in H. discriminate.
  intros.
  assert (exists Shd : list (nat * rval),
    exists z : nat * rval, s = Shd ++ z :: nil /\ length Shd = n).
  apply take_off_tail.
  assumption.
  elim H1; intros Shd H2; clear H1.
  elim H2; intros z H3; clear H2.
  elim H3; intros H4 H5; clear H3.
  rewrite H4 in H0.
  assert (multisubs_atomic (Shd ++ z :: nil) A =
    subs_atomic z (multisubs_atomic Shd A)).
  apply multisubs_atomic_invert.
  rewrite H1 in H0.
  clear H1.
  apply IHn with (s := Shd). assumption.
  apply pred_in_atomic_subs2 with (S := z). assumption.
Qed.

Lemma prin_in_atomic_multisubs_aux2:
  forall (n : nat)(x : nat)(A : atomic)(s : substitution),

```

```

    length s = n ->
    prin_in_atomic x (multisubs_atomic s A) ->
    prin_in_atomic x A.
Proof.
  induction n.
  intros.
  induction s.
  assert (multisubs_atomic nil A = A).
  apply multisubs_atomic_nil.
  rewrite H1 in H0. assumption.
  simpl in H. discriminate.
  intros.
  assert (exists Shd : list (nat * rval),
    exists z : nat * rval, s = Shd ++ z :: nil /\ length Shd = n).
  apply take_off_tail.
  assumption.
  elim H1; intros Shd H2; clear H1.
  elim H2; intros z H3; clear H2.
  elim H3; intros H4 H5; clear H3.
  rewrite H4 in H0.
  assert (multisubs_atomic (Shd ++ z :: nil) A =
    subs_atomic z (multisubs_atomic Shd A)).
  apply multisubs_atomic_invert.
  rewrite H1 in H0.
  clear H1.
  apply IHn with (s := Shd). assumption.
  apply prin_in_atomic_subs2 with (S := z). assumption.
Qed.

Lemma pred_in_atomic_multisubs2:
  forall (x : nat)(A : atomic)(s : substitution),
    pred_in_atomic x (multisubs_atomic s A) ->
    pred_in_atomic x A.
Proof.
  intros.
  apply pred_in_atomic_multisubs_aux2 with (A := A)(s := s)(n := length s).
  trivial. assumption.
Qed.

Lemma prin_in_atomic_multisubs2:
  forall (x : nat)(A : atomic)(s : substitution),
    prin_in_atomic x (multisubs_atomic s A) ->
    prin_in_atomic x A.
Proof.
  intros.
  apply prin_in_atomic_multisubs_aux2 with (A := A)(s := s)(n := length s).
  trivial. assumption.
Qed.

Lemma arity_in_atomic_multisubs_aux2:
  forall (n : nat)(x : nat)(A : atomic)(s : substitution),

```

```

length s = n ->
arity_in_atomic x (multisubs_atomic s A) ->
arity_in_atomic x A.
Proof.
induction n.
intros.
induction s.
assert (multisubs_atomic nil A = A).
apply multisubs_atomic_nil.
rewrite H1 in H0. assumption.
simpl in H. discriminate.
intros.
assert (exists Shd : list (nat * rval),
  exists z : nat * rval, s = Shd ++ z :: nil /\ length Shd = n).
apply take_off_tail.
assumption.
elim H1; intros Shd H2; clear H1.
elim H2; intros z H3; clear H2.
elim H3; intros H4 H5; clear H3.
rewrite H4 in H0.
assert (multisubs_atomic (Shd ++ z :: nil) A =
  subs_atomic z (multisubs_atomic Shd A)).
apply multisubs_atomic_invert.
rewrite H1 in H0.
clear H1.
apply IHn with (s := Shd). assumption.
apply arity_in_atomic_subs2 with (S := z). assumption.
Qed.

Lemma arity_in_atomic_multisubs2:
forall (x : nat)(A : atomic)(s : substitution),
  arity_in_atomic x (multisubs_atomic s A) ->
  arity_in_atomic x A.
Proof.
intros.
apply arity_in_atomic_multisubs_aux2 with (A := A)(s := s)(n := length s).
trivial. assumption.
Qed.

Lemma term_in_match_form_atomic_list:
forall (t : rval)(F : form)(LA : list atomic),
  term_in_atomic_list (val_of_rval t) (match_form_atomic_list F LA) ->
  term_in_atomic_list (val_of_rval t) LA \/ term_in_form (val_of_rval t) F.
Proof.
intros.
unfold match_form_atomic_list in H.
unfold term_in_atomic_list in H.
elim H; intros A H2; clear H.
elim H2; intros H3 H4; clear H2.
assert (exists y, In y (match_list_atomic_list (body F) LA) /\ A = (fun s : su
bstitution => multisubs_atomic s (head F)) y).

```

```

apply in_map with (f := (fun s : substitution => multisubs_atomic s (head F))
).
assumption.
elim H; intros y H5; clear H.
elim H5; intros H6 H7; clear H5.
induction F.
simpl in H3.
simpl in H6.
simpl in H7.
simpl.

Lemma term_in_term_multisubs:
  forall (t : rval)(y : substitution)(a : term),
    multisubs_term y a = val_of_rval t ->
      (exists x, In (x, t) y) \ / (a = val_of_rval t).
Proof.
  intros.
  induction a.
  (* ident *)
  induction y.
  simpl in H.
  right; assumption.
  simpl in H.
  induction a.
  simpl in H.
  assert ((exists x, In (x, t) y) \ / ident n = val_of_rval t).
  apply IHy. assumption.
  elim H0; intro H1; clear H0.
  left.
  elim H1; intros x H2; clear H1.
  exists x. simpl. right; assumption.
  right; assumption.
  (* kident *)
  induction y.
  simpl in H.
  right; assumption.
  simpl in H.
  induction a.
  simpl in H.
  assert ((exists x, In (x, t) y) \ / kident k = val_of_rval t).
  apply IHy. assumption.
  elim H0; intro H1; clear H0.
  left.
  elim H1; intros x H2; clear H1.
  exists x. simpl. right; assumption.
  right; assumption.

  (* var *)
  left.
  induction y.
  simpl in H. induction t; discriminate.

```

```

    simpl in H.
    induction a.
    simpl in H.
    induction (eq_nat_dec a n).
    induction b.
    assert (multisubs_term y (ident n0) = ident n0).
    apply multisubs_term_ident.
    rewrite H0 in H.
    induction t; simpl in H.
    injection H; intro H1; clear H.
    exists a. simpl. left. rewrite H1. trivial.
    discriminate.
    assert (multisubs_term y (kident k) = kendit k).
    apply multisubs_term_kident.
    rewrite H0 in H.
    induction t; simpl in H.
    discriminate.
    injection H; intro H1; clear H.
    exists a. simpl. left. rewrite H1. trivial.

    assert (exists x, In (x, t) y).
    apply IHy.
    assumption.
    elim H0; intros x H1; clear H0.
    exists x. simpl. right. assumption.
Qed.

Lemma term_in_term_list_multisubs:
  forall (t : rval)(y : substitution)(a : list term),
    In (val_of_rval t) (multisubs_term_list y a) ->
      (exists x, In (x, t) y) \/ (In (val_of_rval t) a).
Proof.
  intros.
  induction a.
  simpl in H. contradiction.
  simpl in H.
  elim H; intro H1; clear H.
  assert ((exists x, In (x, t) y) \/ (a = val_of_rval t)).
  apply term_in_term_multisubs.
  assumption.
  elim H; intro H2; clear H.
  elim H2; intros x H3; clear H2.
  left. exists x. assumption.
  right. simpl. left; assumption.
  assert ((exists x, In (x, t) y) \/ In (val_of_rval t) a0).
  apply IHa. assumption.
  elim H; intro H2; clear H.
  elim H2; intros x H3; clear H2.
  left. exists x. assumption.
  right. simpl. right. assumption.
Qed.

```



```

Lemma term_in_atomic_multisubs:
  forall (t : rval)(y : substitution)(a : atomic),
    term_in_atomic (val_of_rval t) (multisubs_atomic y a) ->
      (exists x, In (x, t) y) \/\ (term_in_atomic (val_of_rval t) a).
Proof.
  intros.
  induction a.
  simpl in H.
  simpl.
  apply term_in_term_list_multisubs.
  unfold multisubs_term_list. assumption.
  simpl in H.
  simpl.
  apply term_in_term_list_multisubs.
  unfold multisubs_term_list. assumption.
  simpl in H.
  simpl.
  assert ((exists x : nat, In (x, t) y) \/\ t0 = val_of_rval t).
  apply term_in_term_multisubs.
  symmetry. assumption.
  elim H0; intro H1; clear H0.
  left; assumption. right. symmetry. assumption.
  simpl in H.
  simpl.
  assert ((exists x : nat, In (x, t) y) \/\ t0 = val_of_rval t).
  apply term_in_term_multisubs.
  symmetry. assumption.
  elim H0; intro H1; clear H0.
  left; assumption. right. symmetry. assumption.
Qed.

rewrite H7 in H4.
assert ((exists x, In (x, t) y) \/\ (term_in_atomic (val_of_rval t) a)).
apply term_in_atomic_multisubs. assumption.
elim H; intro H8; clear H.
elim H8; intros x H9; clear H8.
left.
apply term_in_match_list_atomic_list with (LA1 := l)(LA2 := LA)(S := y)(x := x
).
assumption. assumption.
right. left. assumption.
Qed.

Lemma pred_in_match_form_atomic_list:
  forall (x : nat)(A : atomic)(F : form)(LA : list atomic),
    In A (match_form_atomic_list F LA) ->
      pred_in_atomic x A ->
      pred_in_form x F.
Proof.
  intros.

```

```

induction F.
unfold pred_in_form.
left.
unfold match_form_atomic_list in H.
simpl in H.
assert (exists z, In z (match_list_atomic_list l LA) /\ A = (fun s => multisub
s_atomic s a) z).
apply in_map with (f := (fun s => multisubs_atomic s a)).
assumption.
elim H1; intros z H2; clear H1.
elim H2; intros H3 H4; clear H2. clear H.
rewrite H4 in H0.
apply pred_in_atomic_multisubs2 with (s := z).
assumption.
Qed.

```

```

Lemma prin_in_match_form_atomic_list:
  forall (x : nat)(A : atomic)(F : form)(LA : list atomic),
    In A (match_form_atomic_list F LA) ->
      prin_in_atomic x A ->
        prin_in_form x F.

```

```

Proof.
  intros.
  induction F.
  unfold prin_in_form.
  left.
  unfold match_form_atomic_list in H.
  simpl in H.
  assert (exists z, In z (match_list_atomic_list l LA) /\ A = (fun s => multisub
s_atomic s a) z).
  apply in_map with (f := (fun s => multisubs_atomic s a)).
  assumption.
  elim H1; intros z H2; clear H1.
  elim H2; intros H3 H4; clear H2. clear H.
  rewrite H4 in H0.
  apply prin_in_atomic_multisubs2 with (s := z).
  assumption.
Qed.

```

```

Lemma arity_in_match_form_atomic_list:
  forall (x : nat)(A : atomic)(F : form)(LA : list atomic),
    In A (match_form_atomic_list F LA) ->
      arity_in_atomic x A ->
        arity_in_form x F.

```

```

Proof.
  intros.
  induction F.
  unfold arity_in_form.
  left.
  unfold match_form_atomic_list in H.
  simpl in H.

```

```

assert (exists z, In z (match_list_atomic_list l LA) /\ A = (fun s => multisub
s_atomic s a) z).
apply in_map with (f := (fun s => multisubs_atomic s a)).
assumption.
elim H1; intros z H2; clear H1.
elim H2; intros H3 H4; clear H2. clear H.
rewrite H4 in H0.
apply arity_in_atomic_multisubs2 with (s := z).
assumption.
Qed.

```

Lemma term_in_match_form_list_atomic_list:

```

forall (t : rval)(LF : list form)(LA : list atomic),
  term_in_atomic_list (val_of_rval t) (match_form_list_atomic_list LF LA) ->
  term_in_atomic_list (val_of_rval t) LA /\ term_in_form_list (val_of_rval t)
  LF.

```

Proof.

```

intros.
induction LF.
unfold match_form_list_atomic_list in H.
simpl in H.
unfold term_in_atomic_list in H.
elim H; intros x H1; clear H.
elim H1; intros H2 H3; clear H1.
simpl in H2. contradiction.
unfold match_form_list_atomic_list in H.
simpl in H.
unfold term_in_atomic_list in H.
elim H; intros x H1; clear H.
elim H1; intros H2 H3; clear H1.
assert (In x (match_form_atomic_list a LA) /\ In x (flatten (map (fun f : form
=> match_form_atomic_list f LA) LF))).
apply in_app_or. assumption.
elim H; intro H4; clear H.
assert (term_in_atomic_list (val_of_rval t) LA /\ term_in_form (val_of_rval t)
a).
apply term_in_match_form_atomic_list.
unfold term_in_atomic_list.
exists x. split. assumption. assumption.
elim H; intro H5; clear H. left; assumption. right.
unfold term_in_form_list.
exists a. split. simpl. left; trivial. assumption.
unfold term_in_atomic_list in IHLF.
assert ((exists x : atomic, In x LA /\ term_in_atomic (val_of_rval t) x) /\
term_in_form_list (val_of_rval t) LF).
apply IHLF.
exists x. split.
unfold match_form_list_atomic_list. assumption. assumption.
elim H; intro H5; clear H.
elim H5; intros xx H6; clear H5.
elim H6; intros H7 H8; clear H6.

```

```

left. unfold term_in_atomic_list.
exists xx. split. assumption. assumption.
right.
unfold term_in_form_list. unfold term_in_form_list in H5.
elim H5; intros xx H6; clear H5.
elim H6; intros H7 H8; clear H6.
exists xx. split. simpl. right; assumption. assumption.
Qed.

```

```

Lemma pred_in_match_form_list_atomic_list:
forall (x : nat)(A : atomic)(LF : list form)(LA : list atomic),
  In A (match_form_list_atomic_list LF LA) ->
    pred_in_atomic x A ->
    pred_in_form_list x LF.

```

```

Proof.
  intros.
  induction LF.
  simpl in H. contradiction.
  unfold match_form_list_atomic_list in H.
  simpl in H.
  assert (In A (match_form_atomic_list a LA) \/ In A (flatten (map (fun f : form
    => match_form_atomic_list f LA) LF))).
  apply in_app_or.
  assumption.
  elim H1; intro H2; clear H1.
  unfold pred_in_form_list.
  exists a.
  split. left; trivial.
  apply pred_in_match_form_atomic_list with (A := A)(LA := LA).
  assumption. assumption.
  unfold match_form_list_atomic_list in IHLF.
  unfold pred_in_form_list.
  assert (pred_in_form_list x LF).
  apply IHLF. assumption.
  clear IHLF H2.
  unfold pred_in_form_list in H1.
  elim H1; intros x0 H2; clear H1.
  elim H2; intros H3 H4; clear H2.
  exists x0. split. simpl. right; assumption. assumption.
Qed.

```

```

Lemma prin_in_match_form_list_atomic_list:
forall (x : nat)(A : atomic)(LF : list form)(LA : list atomic),
  In A (match_form_list_atomic_list LF LA) ->
    prin_in_atomic x A ->
    prin_in_form_list x LF.

```

```

Proof.
  intros.
  induction LF.
  simpl in H. contradiction.
  unfold match_form_list_atomic_list in H.

```

```

    simpl in H.
    assert (In A (match_form_atomic_list a LA) \ / In A (flatten (map (fun f : form
      => match_form_atomic_list f LA) LF))).
    apply in_app_or.
    assumption.
    elim H1; intro H2; clear H1.
    unfold prin_in_form_list.
    exists a.
    split. left; trivial.
    apply prin_in_match_form_atomic_list with (A := A)(LA := LA).
    assumption. assumption.
    unfold match_form_list_atomic_list in IHLF.
    unfold prin_in_form_list.
    assert (prin_in_form_list x LF).
    apply IHLF. assumption.
    clear IHLF H2.
    unfold prin_in_form_list in H1.
    elim H1; intros x0 H2; clear H1.
    elim H2; intros H3 H4; clear H2.
    exists x0. split. simpl. right; assumption. assumption.
Qed.

```

```

Lemma arity_in_match_form_list_atomic_list:
  forall (x : nat)(A : atomic)(LF : list form)(LA : list atomic),
    In A (match_form_list_atomic_list LF LA) ->
      arity_in_atomic x A ->
      arity_in_form_list x LF.

```

Proof.

```

  intros.
  induction LF.
  simpl in H. contradiction.
  unfold match_form_list_atomic_list in H.
  simpl in H.
  assert (In A (match_form_atomic_list a LA) \ / In A (flatten (map (fun f : form
    => match_form_atomic_list f LA) LF))).
  apply in_app_or.
  assumption.
  elim H1; intro H2; clear H1.
  unfold arity_in_form_list.
  exists a.
  split. left; trivial.
  apply arity_in_match_form_atomic_list with (A := A)(LA := LA).
  assumption. assumption.
  unfold match_form_list_atomic_list in IHLF.
  unfold arity_in_form_list.
  assert (arity_in_form_list x LF).
  apply IHLF. assumption.
  clear IHLF H2.
  unfold arity_in_form_list in H1.
  elim H1; intros x0 H2; clear H1.
  elim H2; intros H3 H4; clear H2.

```

exists x0. split. simpl. right; assumption. assumption.
Qed.

Lemma term_in_extend_one:

```
forall (t : rval)(LF : list form)(LA : list atomic),
  term_in_atomic_list (val_of_rval t) (extend_one LF LA) ->
  term_in_atomic_list (val_of_rval t) LA /\ term_in_form_list (val_of_rval t)
  LF.
```

Proof.

```
intros.
unfold extend_one in H.
unfold term_in_atomic_list in H.
elim H; intros x H2; clear H.
elim H2; intros H3 H4; clear H2.
assert (set_In x LA /\ set_In x (match_form_list_atomic_list LF LA)).
apply set_union_elim with (Aeq_dec := eq_atomic_dec).
unfold set_In. assumption.
elim H; intro H5; clear H.
left.
unfold term_in_atomic_list.
exists x. split. assumption. assumption.
apply term_in_match_form_list_atomic_list.
unfold term_in_atomic_list.
exists x. split. assumption. assumption.
```

Qed.

(* HERE *)

Lemma in_filter:

```
forall (A : Type)(f : A -> bool)(L : list A)(x : A),
  In x (filter f L) -> In x L.
```

Proof.

```
intros. induction L. simpl in H. contradiction.
simpl in H.
induction (f a). simpl in H. elim H; intros. subst x. simpl. left; trivial.
simpl. right. apply IHL. assumption.
simpl. right. apply IHL. assumption.
```

Qed.

Lemma in_kthing_set_ext:

```
forall (x : atomic)(LF : list form)(LA : list atomic),
  In x (kthing_set_ext (extset LF) LA) ->
  (exists k, In k (extset LF) /\ x = extbare (kident k)).
```

Proof.

```
intros.
unfold kthing_set_ext in H.
assert (exists y, In y
  (filter
    (fun p : kthing * kthing * kthing =>
      let (p2, k3) := p in
      let (k1, k2) := p2 in
```

```

        if In_dec eq_atomic_dec (extbare (kident k1)) LA
        then if extdot_dec k1 k2 k3 then true else false
        else false)
      (list_prod (list_prod (extset LF) (extset LF)) (extset LF)))
    /\ x = (fun p : kthing * kthing * kthing =>
      let (_, k3) := p in extbare (kident k3)) y).
  (*
in_map
  : forall (A B : Set) (f : A -> B) (L : list A) (x : B),
    In x (map f L) -> exists y : A, In y L /\ x = f y
  *)
  apply in_map.
  assumption. clear H.
  elim H0; intros; clear H0.
  elim H; intros; clear H.
  destruct x0.
  exists k. split.
  assert (In (p, k) (list_prod (list_prod (extset LF) (extset LF)) (extset LF)))
  .
  apply in_filter with (f := (fun p : kthing * kthing * kthing =>
    let (p2, k3) := p in
    let (k1, k2) := p2 in
    if In_dec eq_atomic_dec (extbare (kident k1)) LA
    then if extdot_dec k1 k2 k3 then true else false
    else false)).
  assumption. clear H0.
  assert (In k (extset LF)).
  apply in_prod_r with (LA := (list_prod (extset LF) (extset LF)))(x := p).
  assumption. assumption. clear H0. assumption.
Qed.

```

Lemma term_in_extend_one_k:

```

  forall (t : rval)(LF : list form)(LA : list atomic),
    term_in_atomic_list (val_of_rval t) (extend_one_k LF LA) ->
    term_in_atomic_list (val_of_rval t) LA /\
    (exists k, In k (extset LF) /\ t = rvkident k).

```

Proof.

```

  intros.
  unfold extend_one_k in H.
  unfold term_in_atomic_list in H.
  elim H; intros x H2; clear H.
  elim H2; intros H3 H4; clear H2.
  assert (set_In x LA /\ set_In x (kthing_set_ext (extset LF) LA)).
  apply set_union_elim with (Aeq_dec := eq_atomic_dec).
  unfold set_In. assumption.
  elim H; intro H5; clear H.
  left.
  unfold term_in_atomic_list.
  exists x. split. assumption. assumption.
  right.
  assert (exists k, In k (extset LF) /\ x = extbare (kident k)).

```

```

    apply in_kthing_set_ext with (LA := LA).
    assumption.
    elim H; intros. clear H. elim H0; intros; clear H0.
    exists x0. split. assumption.
    subst x. simpl in H4.
    destruct t. simpl in H4. discriminate.
    simpl in H4. injection H4; intros. subst k. trivial.
Qed.

```

```

Lemma prin_in_extend_one:
  forall (p : nat)(LF : list form)(LA : list atomic),
    prin_in_atomic_list p (extend_one LF LA) ->
    prin_in_atomic_list p LA /\ prin_in_form_list p LF.

```

```

Proof.
  intros.
  unfold extend_one in H.
  unfold prin_in_atomic_list in H.
  elim H; intros x H2; clear H.
  elim H2; intros H3 H4; clear H2.
  assert (set_In x LA /\ set_In x (match_form_list_atomic_list LF LA)).
  apply set_union_elim with (Aeq_dec := eq_atomic_dec).
  unfold set_In. assumption.
  unfold set_In in H.
  elim H; intro H5; clear H.
  left.
  unfold prin_in_atomic_list.
  exists x. split. assumption. assumption.

  right.
  apply prin_in_match_form_list_atomic_list with (A := x)(LA := LA).
  assumption. assumption.
Qed.

```

```

Lemma prin_in_extend_one_k:
  forall (p : nat)(LF : list form)(LA : list atomic),
    prin_in_atomic_list p (extend_one_k LF LA) ->
    prin_in_atomic_list p LA.

```

Proof.

```

  intros.
  unfold extend_one_k in H.
  unfold prin_in_atomic_list in H.
  elim H; intros x H2; clear H.
  elim H2; intros H3 H4; clear H2.
  assert (set_In x LA /\ set_In x (kthing_set_ext (extset LF) LA)).
  apply set_union_elim with (Aeq_dec := eq_atomic_dec).
  unfold set_In. assumption.
  unfold set_In in H.
  elim H; intro H5; clear H.
  unfold prin_in_atomic_list.
  exists x. split. assumption. assumption.
  assert (exists k, In k (extset LF) /\ x = extbare (kident k)).

```



```

    apply in_kthing_set_ext with (LA := LA).
    assumption.
    elim H; intros; clear H.
    elim H0; intros; clear H0. subst x. simpl in H4. contradiction.
Qed.

Lemma pred_in_extend_one:
  forall (p : nat)(LF : list form)(LA : list atomic),
    pred_in_atomic_list p (extend_one LF LA) ->
    pred_in_atomic_list p LA /\ pred_in_form_list p LF.
Proof.
  intros.
  unfold extend_one in H.
  unfold pred_in_atomic_list in H.
  elim H; intros x H2; clear H.
  elim H2; intros H3 H4; clear H2.
  assert (set_In x LA /\ set_In x (match_form_list_atomic_list LF LA)).
  apply set_union_elim with (Aeq_dec := eq_atomic_dec).
  unfold set_In. assumption.
  unfold set_In in H.
  elim H; intro H5; clear H.
  left.
  unfold pred_in_atomic_list.
  exists x. split. assumption. assumption.

  right.
  apply pred_in_match_form_list_atomic_list with (A := x)(LA := LA).
  assumption. assumption.
Qed.

Lemma pred_in_extend_one_k:
  forall (p : nat)(LF : list form)(LA : list atomic),
    pred_in_atomic_list p (extend_one_k LF LA) ->
    pred_in_atomic_list p LA.
Proof.
  intros.
  unfold extend_one_k in H.
  unfold pred_in_atomic_list in H.
  elim H; intros x H2; clear H.
  elim H2; intros H3 H4; clear H2.
  assert (set_In x LA /\ set_In x (kthing_set_ext (extset LF) LA)).
  apply set_union_elim with (Aeq_dec := eq_atomic_dec).
  unfold set_In. assumption.
  unfold set_In in H.
  elim H; intro H5; clear H.
  unfold pred_in_atomic_list.
  exists x. split. assumption. assumption.

  assert (exists k, In k (extset LF) /\ x = extbare (kident k)).
  apply in_kthing_set_ext with (LA := LA).
  assumption.

```

```

elim H; intros; clear H.
elim H0; intros; clear H0.
subst x. simpl in H4. contradiction.
Qed.

```

```

Lemma arity_in_extend_one:
  forall (p : nat)(LF : list form)(LA : list atomic),
    arity_in_atomic_list p (extend_one LF LA) ->
    arity_in_atomic_list p LA /\ arity_in_form_list p LF.

```

```

Proof.
  intros.
  unfold extend_one in H.
  unfold arity_in_atomic_list in H.
  elim H; intros x H2; clear H.
  elim H2; intros H3 H4; clear H2.
  assert (set_In x LA /\ set_In x (match_form_list_atomic_list LF LA)).
  apply set_union_elim with (Aeq_dec := eq_atomic_dec).
  unfold set_In. assumption.
  unfold set_In in H.
  elim H; intro H5; clear H.
  left.
  unfold arity_in_atomic_list.
  exists x. split. assumption. assumption.

  right.
  apply arity_in_match_form_list_atomic_list with (A := x)(LA := LA).
  assumption. assumption.
Qed.

```

```

Lemma arity_in_extend_one_k:
  forall (p : nat)(LF : list form)(LA : list atomic),
    arity_in_atomic_list p (extend_one_k LF LA) ->
    arity_in_atomic_list p LA.

```

```

Proof.
  intros.
  unfold extend_one_k in H.
  unfold arity_in_atomic_list in H.
  elim H; intros x H2; clear H.
  elim H2; intros H3 H4; clear H2.
  assert (set_In x LA /\ set_In x (kthing_set_ext (extset LF) LA)).
  apply set_union_elim with (Aeq_dec := eq_atomic_dec).
  unfold set_In. assumption.
  unfold set_In in H.
  elim H; intro H5; clear H.
  unfold arity_in_atomic_list.
  exists x. split. assumption. assumption.

  assert (exists k, In k (extset LF) /\ x = extbare (kident k)).
  apply in_kthing_set_ext with (LA := LA).
  assumption.
  elim H; intros; clear H.

```

```

elim H0; intros; clear H0.
subst x. simpl in H4. contradiction.
Qed.

```

Lemma term_in_extend_one_n:

```

forall (n : nat)(t : rval)(LF : list form)(LA : list atomic),
  term_in_atomic_list (val_of_rval t) (iter n (extend_one_one_k LF) LA) ->
  term_in_atomic_list (val_of_rval t) LA /\
  term_in_form_list (val_of_rval t) LF /\
  (exists k : kthing, In k (extset LF) /\ t = rvkident k).

```

Proof.

```

induction n.
simpl. intros.
left; assumption.
simpl. intros.
unfold extend_one_one_k in H.
assert (
  term_in_atomic_list (val_of_rval t) (extend_one LF
    (iter n
      (fun LA : list atomic => extend_one_k LF (extend_one LF LA))
      LA)) /\
  (exists k, In k (extset LF) /\ t = rvkident k)).
apply term_in_extend_one_k with (LA := (extend_one LF
  (iter n
    (fun LA : list atomic => extend_one_k LF (extend_one LF LA))
    LA))).
assumption.
clear H.
elim H0; intros; clear H0.
assert (
  term_in_atomic_list (val_of_rval t)
    (iter n (fun LA : list atomic => extend_one_k LF (extend_one LF LA)) LA) \
  /
  term_in_form_list (val_of_rval t) LF).
apply term_in_extend_one. assumption.
elim H0; intros; clear H0.
apply IHn. unfold extend_one_one_k.
assumption.
right. left. assumption.
right. right. assumption.
Qed.

```

Lemma pred_in_extend_one_n:

```

forall (n p : nat)(LF : list form)(LA : list atomic),
  pred_in_atomic_list p (iter n (extend_one_one_k LF) LA) ->
  pred_in_atomic_list p LA /\ pred_in_form_list p LF.

```

Proof.

```

induction n.
simpl. intros. left; assumption.
simpl. intros.
unfold extend_one_one_k in H.

```

```

assert (pred_in_atomic_list p (extend_one LF
  (iter n
    (fun LA : list atomic => extend_one_k LF (extend_one LF LA))
    LA))).
apply pred_in_extend_one_k with (LA := (extend_one LF
  (iter n
    (fun LA : list atomic => extend_one_k LF (extend_one LF LA))
    LA)))(LF := LF).
assumption.
assert (pred_in_atomic_list p (iter n
  (fun LA : list atomic => extend_one_k LF (extend_one LF LA))
  LA) \ / pred_in_form_list p LF).
apply pred_in_extend_one.
assumption.
clear H H0. elim H1; intros; clear H1.
apply IHn. assumption.
right. assumption.
Qed.

```

Lemma prin_in_extend_one_n:

```

forall (n p : nat)(LF : list form)(LA : list atomic),
  prin_in_atomic_list p (iter n (extend_one_one_k LF) LA) ->
  prin_in_atomic_list p LA \ / prin_in_form_list p LF.

```

Proof.

```

induction n.
simpl. intros. left; assumption.
simpl. intros.
unfold extend_one_one_k in H.
assert (prin_in_atomic_list p (extend_one LF
  (iter n
    (fun LA : list atomic => extend_one_k LF (extend_one LF LA))
    LA))).
apply prin_in_extend_one_k with (LA := (extend_one LF
  (iter n
    (fun LA : list atomic => extend_one_k LF (extend_one LF LA))
    LA)))(LF := LF).
assumption.
assert (prin_in_atomic_list p (iter n
  (fun LA : list atomic => extend_one_k LF (extend_one LF LA))
  LA) \ / prin_in_form_list p LF).
apply prin_in_extend_one.
assumption.
clear H H0. elim H1; intros; clear H1.
apply IHn. assumption.
right. assumption.
Qed.

```

Lemma arity_in_extend_one_n:

```

forall (n p : nat)(LF : list form)(LA : list atomic),
  arity_in_atomic_list p (iter n (extend_one_one_k LF) LA) ->
  arity_in_atomic_list p LA \ / arity_in_form_list p LF.

```

Proof.

```

induction n.
simpl. intros. left; assumption.
simpl. intros.
unfold extend_one_one_k in H.
assert (arity_in_atomic_list p (extend_one LF
  (iter n
    (fun LA : list atomic => extend_one_k LF (extend_one LF LA))
    LA))).
apply arity_in_extend_one_k with (LA := (extend_one LF
  (iter n
    (fun LA : list atomic => extend_one_k LF (extend_one LF LA))
    LA)))(LF := LF).
assumption.
assert (arity_in_atomic_list p (iter n
  (fun LA : list atomic => extend_one_k LF (extend_one LF LA))
  LA) \/ arity_in_form_list p LF).
apply arity_in_extend_one.
assumption.
clear H H0. elim H1; intros; clear H1.
apply IHn. assumption.
right. assumption.

```

Qed.

Lemma term_in_alg:

```

forall (n : nat)(t : rval)(LF : list form),
  term_in_atomic_list (val_of_rval t) (iter n (extend_one_one_k LF) (kthing_start_set LF)) ->
  term_in_atomic_list (val_of_rval t) (kthing_start_set LF) \/
  term_in_form_list (val_of_rval t) LF \/
  (exists k : kthing, In k (extset LF) /\ t = rvkident k).

```

Proof.

```

intros.
apply term_in_extend_one_n with (n := n).
assumption.

```

Qed.

Lemma pred_in_alg:

```

forall (n p : nat)(LF : list form),
  pred_in_atomic_list p (iter n (extend_one_one_k LF) (kthing_start_set LF)) -
  >
  pred_in_form_list p LF.

```

Proof.

```

intros.
assert (pred_in_atomic_list p (kthing_start_set LF) \/ pred_in_form_list p LF)
.
apply pred_in_extend_one_n with (n := n).
assumption.
elim H0; intro H1; clear H0.
unfold pred_in_atomic_list in H1.
elim H1; intros x H2; clear H1.

```

```

elim H2; intros H3 H4; clear H2.
unfold kthing_start_set in H3.
assert (set_In x nil \ / set_In x (kthing_start_set_aux (extset LF))).
apply set_union_elim with (Aeq_dec := eq_atomic_dec). assumption.
elim H0; intros; clear H0. simpl in H1; contradiction.
clear H3. unfold set_In in H1.
unfold kthing_start_set_aux in H1.
assert (exists y, In y (filter
  (fun pr : kthing * kthing =>
    let (x, y) := pr in if exttype_dec x y then true else false)
  (list_prod (extset LF) (extset LF))) /\ x = ((fun pr : kthing * k
    thing =>
      let (_, y) := pr in extbare (kident y))) y).
apply in_map. assumption. clear H1.
elim H0; intros; clear H0.
elim H1; intros; clear H1.
subst x. destruct x0. simpl in H4. contradiction.
assumption.
Qed.

```

Lemma prin_in_alg:

```

forall (n p : nat)(LF : list form),
  prin_in_atomic_list p (iter n (extend_one_one_k LF) (kthing_start_set LF)) -
  >
  prin_in_form_list p LF.

```

Proof.

```

intros.
assert (prin_in_atomic_list p (kthing_start_set LF) \ / prin_in_form_list p LF)
.
apply prin_in_extend_one_n with (n := n).
assumption.
elim H0; intro H1; clear H0.
unfold prin_in_atomic_list in H1.
elim H1; intros x H2; clear H1.
elim H2; intros H3 H4; clear H2.
unfold kthing_start_set in H3.
assert (set_In x nil \ / set_In x (kthing_start_set_aux (extset LF))).
apply set_union_elim with (Aeq_dec := eq_atomic_dec). assumption.
elim H0; intros; clear H0. simpl in H1; contradiction.
clear H3. unfold set_In in H1.
unfold kthing_start_set_aux in H1.
assert (exists y, In y (filter
  (fun pr : kthing * kthing =>
    let (x, y) := pr in if exttype_dec x y then true else false)
  (list_prod (extset LF) (extset LF))) /\ x = ((fun pr : kthing * k
    thing =>
      let (_, y) := pr in extbare (kident y))) y).
apply in_map. assumption. clear H1.
elim H0; intros; clear H0.
elim H1; intros; clear H1.
subst x. destruct x0. simpl in H4. contradiction.

```

```

    assumption.
Qed.

Lemma arity_in_alg:
  forall (n p : nat)(LF : list form),
    arity_in_atomic_list p (iter n (extend_one_one_k LF) (kthing_start_set LF))
    ->
    arity_in_form_list p LF.
Proof.
  intros.
  assert (arity_in_atomic_list p (kthing_start_set LF) \/ arity_in_form_list p L
  F).
  apply arity_in_extend_one_n with (n := n).
  assumption.
  elim H0; intro H1; clear H0.
  unfold arity_in_atomic_list in H1.
  elim H1; intros x H2; clear H1.
  elim H2; intros H3 H4; clear H2.
  unfold kthing_start_set in H3.
  assert (set_In x nil \/ set_In x (kthing_start_set_aux (extset LF))).
  apply set_union_elim with (Aeq_dec := eq_atomic_dec). assumption.
  elim H0; intros; clear H0. simpl in H1; contradiction.
  clear H3. unfold set_In in H1.
  unfold kthing_start_set_aux in H1.
  assert (exists y, In y (filter
    (fun pr : kthing * kthing =>
      let (x, y) := pr in if exttype_dec x y then true else false)
    (list_prod (extset LF) (extset LF))) /\ x = ((fun pr : kthing * k
    thing =>
      let (_, y) := pr in extbare (kident y))) y).
  apply in_map. assumption. clear H1.
  elim H0; intros; clear H0.
  elim H1; intros; clear H1.
  subst x. destruct x0. simpl in H4. contradiction.
  assumption.
Qed.

```

```

(* Here is a function that given a set of predicates, arities, and terms constru
cts all ground atoms *)
(* Start with term lists *)
Fixpoint all_term_lists (LT : list term)(n : nat){struct n} : list (list term):=
  match n with
  | 0 => nil :: nil
  | S x =>
    map (fun pr => match pr with (a, b) => a :: b end) (list_prod LT (all_term
_lists LT x))
  end.

```

```

Definition all_bare_atomics_arity (LT : list term)(LP : list nat)(n : nat) : lis
t atomic :=

```

```

    (map (fun pr => match pr with (p, lt) => bare p lt end) (list_prod LP (all_term_lists LT n))).

Definition all_says1_atomics_arity (LT : list term)(LP : list nat)(W : nat)(n : nat) : list atomic :=
  (map (fun pr => match pr with (p, lt) => says W p lt end) (list_prod LP (all_term_lists LT n))).

Fixpoint all_says_atomics_arity (LT : list term)(LP : list nat)(LW : list nat)(n : nat) {struct LW} : list atomic :=
  match LW with
  | nil => nil
  | hd :: tl =>
    (all_says1_atomics_arity LT LP hd n) ++ (all_says_atomics_arity LT LP tl n)
  end.

Definition all_extbare_atomics (LT : list term) : list atomic :=
  (map (fun t => extbare t) LT).

Definition all_extsays_atomics (LT : list term)(LP : list nat) : list atomic :=
  (map (fun pr1 => match pr1 with (p, t) => extsays p t end) (list_prod LP LT)).

Fixpoint all_atomics (LT : list term)(LP : list nat)(LW : list nat)(LA : list nat) {struct LA} : list atomic :=
  match LA with
  | nil => all_extbare_atomics LT ++ all_extsays_atomics LT LW
  | hd :: tl =>
    (all_bare_atomics_arity LT LP hd) ++ (all_says_atomics_arity LT LP LW hd)
    ++ (all_atomics LT LP LW tl)
  end.

(* Extraction functions that take out all terms, predicates and arities from list of formulas *)
(* Don't worry about duplicates *)
Definition extract_term_atomic (A : atomic) : list term :=
  match A with
  | bare p lt => lt
  | says w p lt => lt
  | extbare t => t :: nil
  | extsays w t => t :: nil
  end.

Fixpoint extract_term_list_atomic (LA : list atomic) {struct LA} : list term :=
  match LA with
  | nil => nil
  | hd :: tl =>
    (extract_term_atomic hd) ++ (extract_term_list_atomic tl)
  end.

Definition extract_term_form (F : form) : list term :=

```



```

match F with
| clause H B =>
  (extract_term_atomic H) ++ (extract_term_list_atomic B)
end.

Fixpoint extract_term_list_form (LF : list form){struct LF} : list term :=
match LF with
| nil => nil
| hd :: tl =>
  (extract_term_form hd) ++ (extract_term_list_form tl)
end.

Definition extract_pred_atomic (A : atomic) : list nat :=
match A with
| bare p lt => p :: nil
| says w p lt => p :: nil
| extbare t => nil
| extsays w t => nil
end.

Fixpoint extract_pred_list_atomic (LA : list atomic){struct LA} : list nat :=
match LA with
| nil => nil
| hd :: tl =>
  (extract_pred_atomic hd) ++ (extract_pred_list_atomic tl)
end.

Definition extract_pred_form (F : form) : list nat :=
match F with
| clause H B =>
  (extract_pred_atomic H) ++ (extract_pred_list_atomic B)
end.

Fixpoint extract_pred_list_form (LF : list form){struct LF} : list nat :=
match LF with
| nil => nil
| hd :: tl =>
  (extract_pred_form hd) ++ (extract_pred_list_form tl)
end.

Definition extract_prin_atomic (A : atomic) : list nat :=
match A with
| bare p lt => nil
| says w p lt => w :: nil
| extbare t => nil
| extsays w t => w :: nil
end.

Fixpoint extract_prin_list_atomic (LA : list atomic){struct LA} : list nat :=
match LA with
| nil => nil

```

```

    | hd :: tl =>
      (extract_prin_atomic hd) ++ (extract_prin_list_atomic tl)
end.

Definition extract_prin_form (F : form) : list nat :=
  match F with
  | clause H B =>
    (extract_prin_atomic H) ++ (extract_prin_list_atomic B)
  end.

Fixpoint extract_prin_list_form (LF : list form){struct LF} : list nat :=
  match LF with
  | nil => nil
  | hd :: tl =>
    (extract_prin_form hd) ++ (extract_prin_list_form tl)
  end.

Definition extract_arity_atomic (A : atomic) : list nat :=
  match A with
  | bare p lt => (length lt) :: nil
  | says w p lt => (length lt) :: nil
  | extbare t => nil
  | extsays w t => nil
  end.

Fixpoint extract_arity_list_atomic (LA : list atomic){struct LA} : list nat :=
  match LA with
  | nil => nil
  | hd :: tl =>
    (extract_arity_atomic hd) ++ (extract_arity_list_atomic tl)
  end.

Definition extract_arity_form (F : form) : list nat :=
  match F with
  | clause H B =>
    (extract_arity_atomic H) ++ (extract_arity_list_atomic B)
  end.

Fixpoint extract_arity_list_form (LF : list form){struct LF} : list nat :=
  match LF with
  | nil => nil
  | hd :: tl =>
    (extract_arity_form hd) ++ (extract_arity_list_form tl)
  end.

Lemma term_is_in_atomic:
  forall (t : rval)(A : atomic),
    term_in_atomic (val_of_rval t) A ->
    In (val_of_rval t) (extract_term_atomic A).
Proof.
  intros.

```

```

induction A; simpl; simpl in H; try assumption.
left; symmetry; assumption.
left; symmetry; assumption.
Qed.

Lemma term_is_in_list_atomic:
  forall (t : rval)(LA : list atomic),
    term_in_atomic_list (val_of_rval t) LA ->
      In (val_of_rval t) (extract_term_list_atomic LA).

```

```

Proof.
  intros.
  induction LA.
  unfold term_in_atomic_list in H.
  elim H; intros x H2; clear H.
  elim H2; intros H3 H4; clear H2.
  simpl in H3. contradiction.
  simpl.
  apply in_or_app.
  unfold term_in_atomic_list in H.
  elim H; intros x H2; clear H.
  elim H2; intros H3 H4; clear H2.
  simpl in H3.
  elim H3; intro H5; clear H3.
  assert (In (val_of_rval t) (extract_term_atomic a)).
  apply term_is_in_atomic.
  rewrite H5. assumption.
  left; assumption.
  right.
  apply IHLA.
  unfold term_in_atomic_list.
  exists x. split. assumption. assumption.
Qed.

```

```

Lemma term_is_in_form:
  forall (t : rval)(F : form),
    term_in_form (val_of_rval t) F ->
      In (val_of_rval t) (extract_term_form F).
Proof.
  intros.
  induction F.
  simpl.
  simpl in H.
  elim H; intro H2; clear H.
  assert (In (val_of_rval t) (extract_term_atomic a)).
  apply term_is_in_atomic.
  assumption.
  apply in_or_app.
  left; assumption.
  assert (In (val_of_rval t) (extract_term_list_atomic l)).
  apply term_is_in_list_atomic.
  assumption.

```

```

    apply in_or_app. right; assumption.
Qed.

```

```

Lemma term_is_in_form_list:
  forall (t : rval)(LF : list form),
    term_in_form_list (val_of_rval t) LF ->
      In (val_of_rval t) (extract_term_list_form LF).

```

```

Proof.
  intros.
  induction LF.
  unfold term_in_form_list in H.
  elim H; intros x H2; clear H.
  elim H2; intros H3 H4; clear H2.
  simpl in H3. contradiction.
  simpl.
  apply in_or_app.
  unfold term_in_form_list in H.
  elim H; intros x H2; clear H.
  elim H2; intros H3 H4; clear H2.
  simpl in H3.
  elim H3; intro H5; clear H3.
  left.
  apply term_is_in_form.
  rewrite H5. assumption.
  right.
  apply IHLF.
  unfold term_in_form_list.
  exists x. split; assumption.
Qed.

```

```

Lemma term_is_in_form_list2:
  forall (t : rval)(LF : list form),
    term_in_form_list (val_of_rval t) LF \ /
    term_in_atomic_list (val_of_rval t) (kthing_start_set2 LF) ->
      In (val_of_rval t) (extract_term_list_form LF) \ /
      In (val_of_rval t) (extract_term_list_atomic (kthing_start_set2 LF)).

```

```

Proof.
  intros.
  elim H; intro H1; clear H.
  unfold term_in_form_list in H1.
  elim H1; intros x H2; clear H1.
  elim H2; intros H3 H4; clear H2.
  left.
  apply term_is_in_form_list.
  unfold term_in_form_list.
  exists x. split; assumption.
  right.
  apply term_is_in_list_atomic. assumption.
Qed.

```

```

Lemma pred_is_in_atomic:

```

```

forall (t : nat)(A : atomic),
  pred_in_atomic t A ->
    In t (extract_pred_atomic A).
Proof.
  intros.
  induction A; simpl; simpl in H; try assumption; try (left; assumption).
Qed.

```

```

Lemma pred_is_in_list_atomic:
  forall (t : nat)(LA : list atomic),
    pred_in_atomic_list t LA ->
      In t (extract_pred_list_atomic LA).
Proof.
  intros.
  induction LA.
  unfold pred_in_atomic_list in H.
  elim H; intros x H2; clear H.
  elim H2; intros H3 H4; clear H2.
  simpl in H3. contradiction.
  simpl.
  apply in_or_app.
  unfold pred_in_atomic_list in H.
  elim H; intros x H2; clear H.
  elim H2; intros H3 H4; clear H2.
  simpl in H3.
  elim H3; intro H5; clear H3.
  left.
  apply pred_is_in_atomic.
  rewrite H5. assumption.
  right.
  apply IHLA.
  unfold pred_in_atomic_list.
  exists x. split. assumption. assumption.
Qed.

```

```

Lemma pred_is_in_form:
  forall (t : nat)(F : form),
    pred_in_form t F ->
      In t (extract_pred_form F).
Proof.
  intros.
  induction F.
  simpl.
  simpl in H.
  elim H; intro H2; clear H.
  assert (In t (extract_pred_atomic a)).
  apply pred_is_in_atomic.
  assumption.
  apply in_or_app.
  left; assumption.
  assert (In t (extract_pred_list_atomic l)).

```

```

    apply pred_is_in_list_atomic.
    assumption.
    apply in_or_app. right; assumption.
Qed.

```

```

Lemma pred_is_in_form_list:
  forall (t : nat)(LF : list form),
    pred_in_form_list t LF ->
    In t (extract_pred_list_form LF).

```

```

Proof.
  intros.
  induction LF.
  unfold pred_in_form_list in H.
  elim H; intros x H2; clear H.
  elim H2; intros H3 H4; clear H2.
  simpl in H3. contradiction.
  simpl.
  apply in_or_app.
  unfold pred_in_form_list in H.
  elim H; intros x H2; clear H.
  elim H2; intros H3 H4; clear H2.
  simpl in H3.
  elim H3; intro H5; clear H3.
  left.
  apply pred_is_in_form.
  rewrite H5. assumption.
  right.
  apply IHLF.
  unfold pred_in_form_list.
  exists x. split; assumption.
Qed.

```

```

Lemma prin_is_in_atomic:
  forall (t : nat)(A : atomic),
    prin_in_atomic t A ->
    In t (extract_prin_atomic A).

```

```

Proof.
  intros.
  induction A; simpl; simpl in H; try assumption; try (left; assumption).
Qed.

```

```

Lemma prin_is_in_list_atomic:
  forall (t : nat)(LA : list atomic),
    prin_in_atomic_list t LA ->
    In t (extract_prin_list_atomic LA).

```

```

Proof.
  intros.
  induction LA.
  unfold prin_in_atomic_list in H.
  elim H; intros x H2; clear H.
  elim H2; intros H3 H4; clear H2.

```

```

    simpl in H3. contradiction.
    simpl.
    apply in_or_app.
    unfold prin_in_atomic_list in H.
    elim H; intros x H2; clear H.
    elim H2; intros H3 H4; clear H2.
    simpl in H3.
    elim H3; intro H5; clear H3.
    left.
    apply prin_is_in_atomic.
    rewrite H5. assumption.
    right.
    apply IHLA.
    unfold prin_in_atomic_list.
    exists x. split. assumption. assumption.
Qed.

```

```

Lemma prin_is_in_form:
  forall (t : nat)(F : form),
    prin_in_form t F ->
      In t (extract_prin_form F).

```

```

Proof.
  intros.
  induction F.
  simpl.
  simpl in H.
  elim H; intro H2; clear H.
  assert (In t (extract_prin_atomic a)).
  apply prin_is_in_atomic.
  assumption.
  apply in_or_app.
  left; assumption.
  assert (In t (extract_prin_list_atomic l)).
  apply prin_is_in_list_atomic.
  assumption.
  apply in_or_app. right; assumption.
Qed.

```

```

Lemma prin_is_in_form_list:
  forall (t : nat)(LF : list form),
    prin_in_form_list t LF ->
      In t (extract_prin_list_form LF).

```

```

Proof.
  intros.
  induction LF.
  unfold prin_in_form_list in H.
  elim H; intros x H2; clear H.
  elim H2; intros H3 H4; clear H2.
  simpl in H3. contradiction.
  simpl.
  apply in_or_app.

```

```

  unfold prin_in_form_list in H.
  elim H; intros x H2; clear H.
  elim H2; intros H3 H4; clear H2.
  simpl in H3.
  elim H3; intro H5; clear H3.
  left.
  apply prin_is_in_form.
  rewrite H5. assumption.
  right.
  apply IHLF.
  unfold prin_in_form_list.
  exists x. split; assumption.
Qed.

```

```

Lemma arity_is_in_atomic:
  forall (t : nat)(A : atomic),
    arity_in_atomic t A ->
      In t (extract_arity_atomic A).

```

```

Proof.
  intros.
  induction A; simpl; simpl in H; try assumption; try (left; assumption).
Qed.

```

```

Lemma arity_is_in_list_atomic:
  forall (t : nat)(LA : list atomic),
    arity_in_atomic_list t LA ->
      In t (extract_arity_list_atomic LA).

```

```

Proof.
  intros.
  induction LA.
  unfold arity_in_atomic_list in H.
  elim H; intros x H2; clear H.
  elim H2; intros H3 H4; clear H2.
  simpl in H3. contradiction.
  simpl.
  apply in_or_app.
  unfold arity_in_atomic_list in H.
  elim H; intros x H2; clear H.
  elim H2; intros H3 H4; clear H2.
  simpl in H3.
  elim H3; intro H5; clear H3.
  left.
  apply arity_is_in_atomic.
  rewrite H5. assumption.
  right.
  apply IHLA.
  unfold arity_in_atomic_list.
  exists x. split. assumption. assumption.
Qed.

```

```

Lemma arity_is_in_form:

```



```

forall (t : nat)(F : form),
  arity_in_form t F ->
    In t (extract_arity_form F).
Proof.
  intros.
  induction F.
  simpl.
  simpl in H.
  elim H; intro H2; clear H.
  assert (In t (extract_arity_atomic a)).
  apply arity_is_in_atomic.
  assumption.
  apply in_or_app.
  left; assumption.
  assert (In t (extract_arity_list_atomic l)).
  apply arity_is_in_list_atomic.
  assumption.
  apply in_or_app. right; assumption.
Qed.

```

```

Lemma arity_is_in_form_list:
  forall (t : nat)(LF : list form),
    arity_in_form_list t LF ->
      In t (extract_arity_list_form LF).

```

```

Proof.
  intros.
  induction LF.
  unfold arity_in_form_list in H.
  elim H; intros x H2; clear H.
  elim H2; intros H3 H4; clear H2.
  simpl in H3. contradiction.
  simpl.
  apply in_or_app.
  unfold arity_in_form_list in H.
  elim H; intros x H2; clear H.
  elim H2; intros H3 H4; clear H2.
  simpl in H3.
  elim H3; intro H5; clear H3.
  left.
  apply arity_is_in_form.
  rewrite H5. assumption.
  right.
  apply IHLF.
  unfold arity_in_form_list.
  exists x. split; assumption.
Qed.

```

(* Show that ground of set_union follows from ground of append of the lists *)

```

Lemma ground_atomic_list_app:

```

```

forall (LA1 LA2 : list atomic),
  ground_atomic_list (LA1 ++ LA2) = true ->
    ground_atomic_list LA1 = true /\ ground_atomic_list LA2 = true.
Proof.
  intros.
  unfold ground_atomic_list in H.
  assert (forall x, In x (LA1 ++ LA2) -> ground_atomic x = true).
  intros.
  apply forall_bool_in with (L := LA1 ++ LA2).
  assumption. assumption.
  split.
  unfold ground_atomic_list.
  apply in_forall_bool.
  intros.
  apply H0.
  apply in_or_app. left; assumption.
  unfold ground_atomic_list.
  apply in_forall_bool.
  intros.
  apply H0.
  apply in_or_app. right; assumption.
Qed.

Lemma ground_atomic_list_app2:
  forall (LA1 LA2 : list atomic),
    ground_atomic_list LA1 = true /\ ground_atomic_list LA2 = true ->
      ground_atomic_list (LA1 ++ LA2) = true.
Proof.
  intros.
  elim H; intros H2 H3; clear H.
  unfold ground_atomic_list.
  apply in_forall_bool.
  intros.
  assert (In x LA1 \/ In x LA2).
  apply in_app_or. assumption.
  elim H0; intro H1; clear H0.
  apply ground_atomic_list_in with (LA := LA1). assumption. assumption.
  apply ground_atomic_list_in with (LA := LA2). assumption. assumption.
Qed.

Lemma ground_set_union:
  forall (LA1 LA2 : list atomic),
    ground_atomic_list (LA1 ++ LA2) = true ->
      ground_atomic_list (set_union eq_atomic_dec LA1 LA2) = true.
Proof.
  intros.

  cut (forall x, In x (set_union eq_atomic_dec LA1 LA2) -> ground_atomic x = true).
  intros.
  unfold ground_atomic_list.

```

```

apply in_forall_bool. assumption.
intros.

assert (set_In x LA1 /\ set_In x LA2).
apply set_union_elim with (Aeq_dec := eq_atomic_dec).
unfold set_In. assumption.

assert (ground_atomic_list LA1 = true /\ ground_atomic_list LA2 = true).
apply ground_atomic_list_app.
assumption.
elim H2; intros H3 H4; clear H2.
unfold set_In in H1.

elim H1; intro H2; clear H1.
apply ground_atomic_list_in with (LA := LA1). assumption. assumption.
apply ground_atomic_list_in with (LA := LA2). assumption. assumption.
Qed.

Lemma ground_extend_one :
  forall (LF : list form)(Base : list atomic),
    no_head_vars_form_list LF ->
    ground_atomic_list Base = true ->
    ground_atomic_list (extend_one LF Base) = true.
Proof.
  intros.
  unfold extend_one.
  apply ground_set_union.
  apply ground_atomic_list_app2.
  split. assumption.
  apply ground_extend_match_form_list_atomic_list.
  assumption. assumption.
Qed.

Lemma ground_extend_one_k :
  forall (LF : list form)(Base : list atomic),
    no_head_vars_form_list LF ->
    ground_atomic_list Base = true ->
    ground_atomic_list (extend_one_k LF Base) = true.
Proof.
  intros.
  unfold extend_one_k.
  apply ground_set_union.
  apply ground_atomic_list_app2.
  split. assumption.
  unfold ground_atomic_list.
  apply in_forall_bool.
  intros.
  unfold kthing_set_ext in H1.
  assert (exists y, In y (filter
    (fun p : kthing * kthing * kthing =>
      let (p2, k3) := p in

```

```

      let (k1, k2) := p2 in
      if In_dec eq_atomic_dec (extbare (kident k1)) Base
      then if extdot_dec k1 k2 k3 then true else false
      else false)
    (list_prod (list_prod (extset LF) (extset LF)) (extset LF))) /\
x = ((fun p : kthing * kthing * kthing => let (_, k3) := p in extbare (kident
k3))) y).
apply in_map. assumption. clear H1.
elim H2; intros; clear H2.
elim H1; intros; clear H1.
destruct x0. subst x. simpl. trivial.
Qed.

```

```

Lemma ground_extend_one_n:
  forall (n : nat)(LF : list form)(LA : list atomic),
    no_head_vars_form_list LF ->
    ground_atomic_list LA = true ->
    ground_atomic_list (iter n (extend_one_one_k LF) LA) = true.

```

Proof.

```

  unfold extend_one_one_k.
  induction n.
  intros.
  simpl. assumption.
  intros.
  simpl.
  apply ground_extend_one_k with (Base :=
    extend_one LF (iter n (fun LA0 : list atomic => extend_one_k LF (extend_one
    LF LA0)) LA)).
  assumption.
  apply ground_extend_one. assumption.
  apply IHn. assumption. assumption.
Qed.

```

```

Lemma ground_kthing_start_set:
  forall (LF : list form),
    ground_atomic_list (kthing_start_set LF) = true.

```

Proof.

```

  intros.
  unfold ground_atomic_list.
  apply in_forall_bool.
  intros.
  unfold kthing_start_set in H.
  assert (set_In x nil \/ set_In x (kthing_start_set_aux (extset LF))).
  apply set_union_elim with (Aeq_dec := eq_atomic_dec). assumption.
  elim H0; intros; clear H0. simpl in H1; contradiction.
  clear H. unfold set_In in H1.
  unfold kthing_start_set_aux in H1.
  assert (exists y, In y (filter
    (fun pr : kthing * kthing =>
      let (x, y) := pr in if exttype_dec x y then true else false)
    (list_prod (extset LF) (extset LF)))) /\ x = ((fun pr : kthing * k

```

```

      thing =>
      let (_, y) := pr in extbare (kident y))) y).
    apply in_map. assumption. clear H1.
    elim H; intros; clear H.
    elim H0; intros; clear H0.
    subst x. destruct x0. simpl. trivial.
  Qed.

```

```

Lemma ground_alg:
  forall (n : nat)(LF : list form),
    no_head_vars_form_list LF ->
      ground_atomic_list (iter n (extend_one_one_k LF) (kthing_start_set LF)) = true.

```

```

Proof.
  intros.
  apply ground_extend_one_n.
  assumption.
  apply ground_kthing_start_set.
Qed.

```

```

Lemma fits_in_all_term_lists:
  forall (a : nat)(l : list term)(LF : list form),
    (forall x : term, In x l -> In x (extract_term_list_form LF)) ->
      a = length l ->
      In l (all_term_lists (extract_term_list_form LF) a).

```

```

Proof.
  induction a.
  induction l; intros.
  simpl. left; trivial.
  simpl in H0. discriminate.
  induction l; intros.
  simpl in H0. discriminate.
  clear IHl.
  simpl in H0. injection H0; intro H1; clear H0.
  simpl.
  apply in_map2.
  exists ((a0, l)).
  split.
  apply in_prod.
  apply H. simpl. left; trivial.
  apply IHa.
  intros x H2.
  apply H. simpl. right; assumption.
  assumption. trivial.
Qed.

```

```

Lemma map_append_distribute:
  forall (A B : Set)(L1 L2 : list A)(f : A -> B),
    (map f L1 ++ map f L2) = map f (L1 ++ L2).
Proof.
  intros.

```

```

induction L2.
simpl.
assert (map f L1 ++ nil = map f L1).
apply app_nil.
rewrite H. clear H.
assert (L1 ++ nil = L1).
apply app_nil.
rewrite H. trivial.
simpl.
induction L1.
simpl. trivial.
simpl.
assert (map f L1 ++ f a :: map f L2 = map f (L1 ++ a :: L2)).
apply IHL1.
simpl in IHL2.
injection IHL2. trivial.
rewrite H. trivial.
Qed.

Lemma fits_in_all_term_lists2:
  forall (a : nat)(l : list term)(LF : list form),
    (forall x : term, In x l -> In x (extract_term_list_form LF) \ / In x (extract_term_list_atomic (kthing_start_set2 LF))) ->
    a = length l ->
    In l (all_term_lists ((extract_term_list_form LF) ++ (extract_term_list_atomic (kthing_start_set2 LF))) a).
Proof.
  induction a.
  induction l; intros.
  simpl. left; trivial.
  simpl in H0. discriminate.
  induction l; intros.
  simpl in H0. discriminate.
  clear IHL.
  simpl in H0. injection H0; intro H1; clear H0.
  simpl.
  assert (
    In a0 (extract_term_list_form LF) \ /
    In a0 (extract_term_list_atomic (kthing_start_set2 LF))).
  apply H. simpl. left; trivial.
  elim H0; intro H2; clear H0.
  apply in_map2.
  exists ((a0, l)).
  split; try trivial.
  apply in_prod.
  apply in_or_app. left. assumption.
  apply IHa.
  intros x H3.
  apply H. simpl. right; assumption.
  assumption.
  apply in_map2.

```

```

exists (a0, l).
split; try trivial.
apply in_prod.
apply in_or_app. right. assumption.
apply IHa.
intros x H3.
apply H. simpl. right; assumption.
assumption.
Qed.

```

```

Lemma nil_app:
  forall (A : Set)(L : list A),
    nil ++ L = L.

```

```

Proof.
  intros. induction L.
  simpl. trivial.
  simpl. trivial.
Qed.

```

```

Lemma in_all_term_lists:
  forall (T T2 : list term)(n : nat)(x : list term),
    In x (all_term_lists T2 n) ->
    In x (all_term_lists (T ++ T2) n).

```

```

Proof.
  induction n; intros.
  simpl in H. simpl. assumption.
  simpl. simpl in H.
  assert (exists y, In y (list_prod T2 (all_term_lists T2 n)) /\ x = ((fun pr :
    term * list term => let (a, b) := pr in a :: b)) y).
  apply in_map with (f:=(fun pr : term * list term => let (a, b) := pr in a :: b
  )).
  assumption.
  clear H.
  elim H0; intros y H1; clear H0.
  elim H1; intros H2 H3; clear H1.
  induction y.
  assert (In a T2). apply in_prod_l with (x := a)(y := b)(LA := T2)(LB := (all_t
  erm_lists T2 n)).
  assumption.
  assert (In b (all_term_lists T2 n)). apply in_prod_r with (x := a)(y := b)(LA
  := T2)(LB := (all_term_lists T2 n)).
  assumption.
  clear H2.
  apply in_map2.
  assert (In b (all_term_lists (T ++ T2) n)).
  apply IHn. assumption.
  exists (a, b).
  split.
  apply in_prod.
  apply in_or_app. right. assumption.
  assumption.

```

```

    assumption.
Qed.

Lemma fits_in_all_bare:
  forall (LF : list form)(n0 : nat)(l : list term),
    In n0 (extract_pred_list_form LF) ->
    In (length l) (extract_arity_list_form LF) ->
    (forall x : term, In x l -> In x (extract_term_list_form LF) \ / In x (extract_term_list_atomic (kthing_start_set2 LF))) ->
    In (bare n0 l)
    (all_atomics
      ((extract_term_list_form LF) ++ (extract_term_list_atomic (kthing_start_set2 LF)))
      (extract_pred_list_form LF)
      (extract_prin_list_form LF)
      (extract_arity_list_form LF)).
Proof.
  intros.
  induction (extract_arity_list_form LF).
  simpl in H0. contradiction.
  simpl.
  apply in_or_app.
  simpl in H0.
  elim H0; intro H3; clear H0.
  left.
  clear IHl0.
  unfold all_bare_atomics_arity.
  apply in_map2.
  exists ((n0, l)).
  split.
  apply in_prod.
  assumption.
  clear H.
  assert (In l
    (all_term_lists ((extract_term_list_form LF) ++ (extract_term_list_atomic (kthing_start_set2 LF))) a)).
  apply fits_in_all_term_lists2.
  intros x H.
  apply H1.
  assumption. assumption.
  assumption. trivial.

  right.
  apply in_or_app.
  right.
  apply IHl0. assumption.
Qed.

Lemma fits_in_all_says:
  forall (LF : list form)(w n0 : nat)(l : list term),
    In n0 (extract_pred_list_form LF) ->

```



```

In w (extract_prin_list_form LF) ->
In (length l) (extract_arity_list_form LF) ->
(forall x : term, In x l -> In x (extract_term_list_form LF) \/\ In x (extract_term_list_atomic (kthing_start_set2 LF))) ->
In (says w n0 l)
(all_atomics
  ((extract_term_list_form LF) ++ (extract_term_list_atomic (kthing_start_set2 LF)))
  (extract_pred_list_form LF)
  (extract_prin_list_form LF)
  (extract_arity_list_form LF)).
Proof.
  intros.
  induction (extract_arity_list_form LF).
  simpl in H1. contradiction.
  simpl.
  apply in_or_app.
  right.
  simpl in H1.
  apply in_or_app.
  elim H1; intro H3; clear H1.
  left.
  clear IH10.
  induction (extract_prin_list_form LF).
  simpl in H0; contradiction.
  simpl.
  apply in_or_app.
  simpl in H0.
  elim H0; intro H4; clear H0.
  left.
  unfold all_says1_atomics_arity.
  apply in_map2.
  exists ((n0, l)).
  split; try (rewrite H4; trivial).
  apply in_prod.
  assumption.
  clear H.
  apply fits_in_all_term_lists2.
  apply H2.
  assumption.
  right.
  apply IH11. assumption.

  right.
  apply IH10.
  assumption.
Qed.

Lemma fits_in_all_extbare:
  forall (LF : list form)(x : term),
    In x (extract_term_list_form LF) \/\ In x (extract_term_list_atomic (kthing_s

```

```

tart_set2 LF))->
In (extbare x)
(all_atomics
  ((extract_term_list_form LF) ++ (extract_term_list_atomic (kthing_start_se
    t2 LF)))
  (extract_pred_list_form LF)
  (extract_prin_list_form LF)
  (extract_arity_list_form LF)).
Proof.
  intros.
  induction (extract_arity_list_form LF).
  simpl.
  apply in_or_app.
  left.
  unfold all_extbare_atomics.
  apply in_map2.
  exists x. split.
  apply in_or_app. assumption.
  trivial.
  simpl.
  apply in_or_app.
  right.
  apply in_or_app.
  right.
  assumption.
Qed.

```

```

Lemma fits_in_all_extsays:
forall (LF : list form)(w : nat)(x : term),
  In w (extract_prin_list_form LF) ->
  In x (extract_term_list_form LF) \ / In x (extract_term_list_atomic (kthing_s
    tart_set2 LF))->
  In (extsays w x)
  (all_atomics
    ((extract_term_list_form LF) ++ (extract_term_list_atomic (kthing_start_se
      t2 LF)))
    (extract_pred_list_form LF)
    (extract_prin_list_form LF)
    (extract_arity_list_form LF)).

```

```

Proof.
  intros.
  induction (extract_arity_list_form LF).
  simpl.
  apply in_or_app.
  right.
  unfold all_extsays_atomics.
  apply in_map2.
  exists (w, x). split.
  apply in_prod. assumption.
  apply in_or_app. assumption. trivial.
  simpl.

```

```

    apply in_or_app.
    right.
    apply in_or_app.
    right.
    assumption.
Qed.

```

```

Lemma or_symmetric:
  forall (P Q : Prop),
    P  $\vee$  Q -> Q  $\vee$  P.
Proof.
  intros.
  elim H; intro H1; clear H.
  right; assumption. left; assumption.
Qed.

```

```

Lemma in_extset_in_kthing_start_set2:
  forall k LF,
    In k (extset LF) ->
    In (extbare (kident k)) (kthing_start_set2 LF).
Proof.
  intros.
  unfold kthing_start_set2.
  apply in_map2. exists k.
  split. assumption. trivial.
Qed.

```

```

Lemma upperbound_goodness_bare:
  forall (n : nat)(n0 : nat)(l : list term)(LF : list form),
    no_head_vars_form_list LF ->
    In (bare n0 l) (iter n (extend_one_one_k LF) (kthing_start_set LF)) ->
    In (bare n0 l) (all_atomics
      ((extract_term_list_form LF) ++ (extract_term_list_atomic (kthing_start_set2 LF)))
      (extract_pred_list_form LF)
      (extract_prin_list_form LF)
      (extract_arity_list_form LF)).
Proof.
  intros.
  assert (pred_in_form_list n0 LF).
  apply pred_in_alg with (n := n).
  unfold pred_in_atomic_list.
  exists (bare n0 l).
  split. assumption.
  simpl. trivial.
  assert (arity_in_form_list (length l) LF).
  apply arity_in_alg with (n := n).
  unfold arity_in_atomic_list.
  exists (bare n0 l).
  split. assumption.
  simpl. trivial.

```

```

assert (ground_term_list l = true).
assert (ground_atomic_list (iter n (extend_one_one_k LF) (kthing_start_set LF)
) = true).
apply ground_alg; try assumption.
unfold ground_atomic_list in H3.
assert (ground_atomic (bare n0 l) = true).
apply forall_bool_in with (L := iter n (extend_one_one_k LF) (kthing_start_set
LF)).
assumption. assumption.
simpl in H4. assumption.
assert (forall x, In x l -> exists i, x = val_of_rval i).
intros x H4.
unfold ground_term_list in H3.
assert (ground_term x = true).
apply forall_bool_in with (L := l).
assumption. assumption.
induction x.
exists (rvident n1). simpl. trivial.
exists (rvkident k). simpl. trivial.
simpl in H5. discriminate.
(*
term_in_alg
: forall (n : nat) (t : rval) (LF : list form),
term_in_atomic_list (val_of_rval t)
(iter n (extend_one_one_k LF) (kthing_start_set LF)) ->
term_in_atomic_list (val_of_rval t) (kthing_start_set LF) /\
term_in_form_list (val_of_rval t) LF /\
(exists k : kthing, In k (extset LF) /\ t = rvkident k)
*)
assert (forall t, In (val_of_rval t) l -> term_in_atomic_list (val_of_rval t)
(kthing_start_set LF) /\
term_in_form_list (val_of_rval t) LF /\ (exists k : kthing, In k (extset LF) /\
t = rvkident k)).
intros t H5.
apply term_in_alg with (n := n).
unfold term_in_atomic_list.
exists (bare n0 l).
split. assumption.
simpl. assumption.

assert (In n0 (extract_pred_list_form LF)).
apply pred_is_in_form_list.
assumption.
assert (In (length l) (extract_arity_list_form LF)).
apply arity_is_in_form_list.
assumption.
(*
fits_in_all_bare
: forall (LF : list form) (n0 : nat) (l : list term),
In n0 (extract_pred_list_form LF) ->
In (length l) (extract_arity_list_form LF) ->

```

```

(forall x : term,
  In x l ->
  In x (extract_term_list_form LF) \/  

  In x (extract_term_list_atomic (kthing_start_set2 LF))) ->
In (bare n0 l)
  (all_atomics
    (extract_term_list_form LF ++
      extract_term_list_atomic (kthing_start_set2 LF))
    (extract_pred_list_form LF) (extract_prin_list_form LF)
    (extract_arity_list_form LF))
*)
apply fits_in_all_bare.
assumption. assumption.
intros.
assert (exists i, x = val_of_rval i).
apply H4. assumption. clear H4.
elim H9; intros; clear H9. subst x.
assert (term_in_atomic_list (val_of_rval x0) (kthing_start_set LF) \/  

  term_in_form_list (val_of_rval x0) LF \/  

  (exists k : kthing, In k (extset LF) /\ x0 = rvkident k)).
apply H5. assumption.
assert (In (val_of_rval x0) (extract_term_list_form LF) \/  

  In (val_of_rval x0) (extract_term_list_atomic (kthing_start_set2 LF))).
(*
term_is_in_form_list2
: forall (t : rval) (LF : list form),
  term_in_form_list (val_of_rval t) LF \/  

  term_in_atomic_list (val_of_rval t) (kthing_start_set2 LF) ->
  In (val_of_rval t) (extract_term_list_form LF) \/  

  In (val_of_rval t) (extract_term_list_atomic (kthing_start_set2 LF))
*)
apply term_is_in_form_list2.
elim H4; intros; clear H4.
right. unfold term_in_atomic_list in H9.
elim H9; intros; clear H9. elim H4; intros; clear H4.
assert (In x (kthing_start_set2 LF)).
apply in_kthing_start_set_in_kthing_start_set2. assumption.
unfold term_in_atomic_list.
exists x. split. assumption. assumption.
elim H9; intros; clear H9. left. assumption.
elim H4; intros; clear H4. right.
elim H9; intros; clear H9.
assert (In (extbare (kident x)) (kthing_start_set2 LF)).
apply in_extset_in_kthing_start_set2. assumption.
subst x0. simpl.
unfold term_in_atomic_list. exists (extbare (kident x)).
split. assumption. simpl. trivial.
assumption.
Qed.

```

Lemma upperbound_goodness_says:

```

forall (n : nat)(n0 n1 : nat)(l : list term)(LF : list form),
  no_head_vars_form_list LF ->
  In (says n1 n0 l) (iter n (extend_one_one_k LF) (kthing_start_set LF)) ->
  In (says n1 n0 l) (all_atomics
    ((extract_term_list_form LF) ++ (extract_term_list_atomic (kthing_start_set LF)))
    (extract_pred_list_form LF)
    (extract_prin_list_form LF)
    (extract_arity_list_form LF)).
Proof.
  intros.
  assert (pred_in_form_list n0 LF).
  apply pred_in_alg with (n := n).
  unfold pred_in_atomic_list.
  exists (says n1 n0 l).
  split. assumption.
  simpl. trivial.
  assert (prin_in_form_list n1 LF).
  apply prin_in_alg with (n := n).
  unfold prin_in_atomic_list.
  exists (says n1 n0 l).
  split. assumption.
  simpl. trivial.
  assert (arity_in_form_list (length l) LF).
  apply arity_in_alg with (n := n).
  unfold arity_in_atomic_list.
  exists (says n1 n0 l).
  split. assumption.
  simpl. trivial.
  assert (ground_term_list l = true).
  assert (ground_atomic_list (iter n (extend_one_one_k LF) (kthing_start_set LF)) = true).
  apply ground_alg; try assumption.
  unfold ground_atomic_list in H3.
  assert (ground_atomic (says n1 n0 l) = true).
  apply forall_bool_in with (L := iter n (extend_one_one_k LF) (kthing_start_set LF)).
  assumption. assumption.
  simpl in H4. assumption.
  assert (forall x, In x l -> exists i, x = val_of_rval i).
  intros x H41.
  unfold ground_term_list in H3.
  assert (ground_term x = true).
  apply forall_bool_in with (L := l).
  assumption. assumption.
  induction x.
  exists (rvident n2). simpl. trivial.
  exists (rvkident k). simpl. trivial.
  simpl in H5. discriminate.
  (*
term_in_alg

```

```

      : forall (n : nat) (t : rval) (LF : list form),
      term_in_atomic_list (val_of_rval t)
        (iter n (extend_one_one_k LF) (kthing_start_set LF)) ->
      term_in_atomic_list (val_of_rval t) (kthing_start_set LF) \ /
      term_in_form_list (val_of_rval t) LF \ /
      (exists k : kthing, In k (extset LF) /\ t = rvkident k)
*)
  assert (forall t, In (val_of_rval t) l -> term_in_atomic_list (val_of_rval t)
    (kthing_start_set LF) \ /
    term_in_form_list (val_of_rval t) LF \ / (exists k : kthing, In k (extset LF) /\
    t = rvkident k)).
  intros t H51.
  apply term_in_alg with (n := n).
  unfold term_in_atomic_list.
  exists (says n1 n0 l).
  split. assumption.
  simpl. assumption.

  assert (In n0 (extract_pred_list_form LF)).
  apply pred_is_in_form_list.
  assumption.
  assert (In n1 (extract_prin_list_form LF)).
  apply prin_is_in_form_list.
  assumption.
  assert (In (length l) (extract_arity_list_form LF)).
  apply arity_is_in_form_list.
  assumption.
  apply fits_in_all_says.
  assumption. assumption. assumption.
  intros.
  assert (exists i, x = val_of_rval i).
  apply H5. assumption. clear H5.
  elim H11; intros; clear H11. subst x.
  assert (term_in_atomic_list (val_of_rval x0) (kthing_start_set LF) \ /
    term_in_form_list (val_of_rval x0) LF \ /
    (exists k : kthing, In k (extset LF) /\ x0 = rvkident k)).
  apply H6. assumption.
  assert (In (val_of_rval x0) (extract_term_list_form LF) \ /
    In (val_of_rval x0) (extract_term_list_atomic (kthing_start_set2 LF))).
  (*
term_is_in_form_list2
  : forall (t : rval) (LF : list form),
  term_in_form_list (val_of_rval t) LF \ /
  term_in_atomic_list (val_of_rval t) (kthing_start_set2 LF) ->
  In (val_of_rval t) (extract_term_list_form LF) \ /
  In (val_of_rval t) (extract_term_list_atomic (kthing_start_set2 LF))
*)
  apply term_is_in_form_list2.
  elim H5; intros; clear H5.
  right. unfold term_in_atomic_list in H11.
  elim H11; intros; clear H11. elim H5; intros; clear H5.

```

```

assert (In x (kthing_start_set2 LF)).
apply in_kthing_start_set_in_kthing_start_set2. assumption.
unfold term_in_atomic_list.
exists x. split. assumption. assumption.
elim H11; intros; clear H11. left. assumption.
elim H5; intros; clear H5. right.
elim H11; intros; clear H11.
assert (In (extbare (kident x)) (kthing_start_set2 LF)).
apply in_extset_in_kthing_start_set2. assumption.
subst x0. simpl.
unfold term_in_atomic_list. exists (extbare (kident x)).
split. assumption. simpl. trivial.
assumption.
Qed.

Lemma upperbound_goodness_extbare:
  forall (n : nat)(t : term)(LF : list form),
    no_head_vars_form_list LF ->
    In (extbare t) (iter n (extend_one_one_k LF) (kthing_start_set LF)) ->
    In (extbare t) (all_atomics
      ((extract_term_list_form LF) ++ (extract_term_list_atomic (kthing_start_set2 LF)))
      (extract_pred_list_form LF)
      (extract_prin_list_form LF)
      (extract_arity_list_form LF)).
Proof.
  intros.
  assert (ground_atomic_list (iter n (extend_one_one_k LF) (kthing_start_set LF))
    ) = true).
  apply ground_alg; try assumption.
  assert (ground_atomic (extbare t) = true).
  apply forall_bool_in with (L := iter n (extend_one_one_k LF) (kthing_start_set LF)).
  unfold ground_atomic_list in H1.
  assumption. assumption.
  assert (ground_term t = true).
  simpl in H2. assumption.
  clear H1 H2.
  assert (exists i, t = val_of_rval i).
  induction t.
  exists (rvident n0). simpl. trivial.
  exists (rvkident k). simpl. trivial.
  simpl in H3. discriminate.
  elim H1; intros i H2; clear H1.
  (*
term_in_alg
  : forall (n : nat) (t : rval) (LF : list form),
    term_in_atomic_list (val_of_rval t)
      (iter n (extend_one_one_k LF) (kthing_start_set LF)) ->
    term_in_atomic_list (val_of_rval t) (kthing_start_set LF) \ /
    term_in_form_list (val_of_rval t) LF \ /

```



```

      (exists k : kthing, In k (extset LF) /\ t = rvkident k)
*)
  assert (term_in_atomic_list (val_of_rval i) (kthing_start_set LF) \/
term_in_form_list (val_of_rval i) LF \/ (exists k : kthing, In k (extset LF) /\
i = rvkident k)).
  apply term_in_alg with (n := n).
  unfold term_in_atomic_list.
  exists (extbare t).
  split. assumption.
  simpl. subst t. trivial.

  apply fits_in_all_extbare; try assumption.
  subst t.
  apply term_is_in_form_list2.
  elim H1; intros; clear H1.
  unfold term_in_atomic_list in H2.
  elim H2; intros; clear H2.
  elim H1; intros; clear H1.
  assert (In x (kthing_start_set2 LF)).
  apply in_kthing_start_set_in_kthing_start_set2. assumption.
  unfold term_in_atomic_list. right.
  exists x. split. assumption. assumption.

  elim H2; intros; clear H2. left. assumption.
  elim H1; intros; clear H1.
  elim H2; intros; clear H2. right.

  assert (In (extbare (kident x)) (kthing_start_set2 LF)).
  apply in_extset_in_kthing_start_set2. assumption.
  subst i. simpl.
  unfold term_in_atomic_list. exists (extbare (kident x)).
  split. assumption. simpl. trivial.
Qed.

Lemma upperbound_goodness_extsays:
  forall (n : nat)(t : term)(w : nat)(LF : list form),
    no_head_vars_form_list LF ->
    In (extsays w t) (iter n (extend_one_one_k LF) (kthing_start_set LF)) ->
    In (extsays w t) (all_atomics
      ((extract_term_list_form LF) ++ (extract_term_list_atomic (kthing_start_se
t2 LF)))
      (extract_pred_list_form LF)
      (extract_prin_list_form LF)
      (extract_arity_list_form LF)).
Proof.
  intros.
  assert (ground_atomic_list (iter n (extend_one_one_k LF) (kthing_start_set LF)
) = true).
  apply ground_alg; try assumption.
  assert (ground_atomic (extsays w t) = true).

```

```

apply forall_bool_in with (L := iter n (extend_one_one_k LF) (kthing_start_set
  LF)).
unfold ground_atomic_list in H1.
assumption. assumption.
assert (ground_term t = true).
simpl in H2. assumption.
clear H1 H2.
assert (exists i, t = val_of_rval i).
induction t.
exists (rvident n0). simpl. trivial.
exists (rvkident k). simpl. trivial.
simpl in H3. discriminate.
elim H1; intros i H2; clear H1.
assert (term_in_atomic_list (val_of_rval i) (kthing_start_set LF) \
term_in_form_list (val_of_rval i) LF \ (exists k : kthing, In k (extset LF) /\
i = rvkident k)).
apply term_in_alg with (n := n).
unfold term_in_atomic_list.
exists (extsays w t).
split. assumption.
simpl. subst t. trivial.
assert (prin_in_form_list w LF).
apply prin_in_alg with (n := n).
unfold prin_in_atomic_list.
exists (extsays w t).
split. assumption.
simpl. trivial.

assert (In w (extract_prin_list_form LF)).
apply prin_is_in_form_list.
assumption.

apply fits_in_all_extsays; try assumption.
subst t.
apply term_is_in_form_list2.
elim H1; intros; clear H1.
unfold term_in_atomic_list in H2.
elim H2; intros; clear H2.
elim H1; intros; clear H1.
assert (In x (kthing_start_set2 LF)).
apply in_kthing_start_set_in_kthing_start_set2. assumption.
unfold term_in_atomic_list. right.
exists x. split. assumption. assumption.

elim H2; intros; clear H2. left. assumption.
elim H1; intros; clear H1.
elim H2; intros; clear H2. right.

assert (In (extbare (kident x)) (kthing_start_set2 LF)).
apply in_extset_in_kthing_start_set2. assumption.
subst i. simpl.

```

```

    unfold term_in_atomic_list. exists (extbare (kident x)).
    split. assumption. simpl. trivial.
Qed.

```

```

Lemma upperbound_goodness:
  forall (n : nat)(A : atomic)(LF : list form),
    no_head_vars_form_list LF ->
    In A (iter n (extend_one_one_k LF) (kthing_start_set LF)) ->
    In A (all_atomics
      ((extract_term_list_form LF) ++ (extract_term_list_atomic (kthing_start_set LF)
        t2 LF)))
      (extract_pred_list_form LF)
      (extract_prin_list_form LF)
      (extract_arity_list_form LF)).

```

```

Proof.
  intros.
  induction A.
  apply upperbound_goodness_bare with (n := n); assumption.
  apply upperbound_goodness_says with (n := n); assumption.
  apply upperbound_goodness_extbare with (n := n); assumption.
  apply upperbound_goodness_extsays with (n := n); assumption.
Qed.

```

```

Lemma once_the_same_always_the_same :
  forall (LF : list form)(LA : list atomic),
    extend_one_one_k LF LA = LA ->
    forall (n : nat),
      (iter n (extend_one_one_k LF) LA) = LA.

```

```

Proof.
  intros.
  induction n.
  simpl. trivial.
  simpl.
  rewrite IHn.
  assumption.
Qed.

```

```

Lemma monotonic_increasing:
  forall (LF : list form)(LA : list atomic),
    forall (x : atomic), In x LA -> In x (extend_one_one_k LF LA).

```

```

Proof.
  intros.
  unfold extend_one_one_k.
  unfold extend_one_k.
  apply set_union_intro1.
  unfold set_In.
  unfold extend_one.
  apply set_union_intro1.
  unfold set_In.
  assumption.
Qed.

```

```

Lemma dup_exists:
  forall (l : nat)(f : nat -> nat)(x0 : nat),
    (forall (n : nat), iter n f x0 <= iter (S n) f x0) ->
      iter l f x0 < l ->
        iter l f x0 = iter (S l) f x0.
Proof.
  induction l.
  intros.
  simpl in H0.
  absurd (x0 < 0).
  apply lt_n_0. assumption.
  intros.
  assert ({l <= iter l f x0}+{iter l f x0 < l}).
  apply le_lt_dec.
  elim H1; intro H2; clear H1.
  assert (iter l f x0 <= iter (S l) f x0).
  apply H.
  assert (l <= iter (S l) f x0).
  apply le_trans with (m := iter l f x0).
  assumption. assumption.
  assert (iter (S l) f x0 <= l).
  apply lt_n_Sm_le. assumption.
  assert (l = iter (S l) f x0).
  apply le_antisym. assumption. assumption.
  rewrite <- H5 in H1.
  assert (l = iter l f x0).
  apply le_antisym; assumption.
(* exists l. *)
  simpl in H5.
  simpl.
  rewrite <- H5.
  rewrite <- H6 in H5.
  assumption.
  cut (iter l f x0 = iter (S l) f x0).
  intro.
  simpl in H1.
  simpl.
  pattern (iter l f x0) at 1.
  rewrite H1.
  trivial.
  apply IHl.
  assumption. assumption.
Qed.

Fixpoint is_set (A : Set)(L : list A) {struct L} : Prop :=
  match L with
  | hd :: tl =>
    (~ In hd tl) /\ is_set A tl
  | nil => True
  end.

```

```

Lemma is_set_not_in:
  forall (A : Set)(x : A)(L : list A),
    is_set A (x :: L) ->
      ~ In x L.
Proof.
  intros.
  unfold is_set in H.
  elim H; intros H2 H3; clear H.
  assumption.
Qed.

Lemma length_incl_n:
  forall (n : nat)(A : Set)(Aeq : forall (a b : A), {a=b}+{a<>b})(L2 L1 : list A
  ),
    length L1 = n ->
      is_set A L1 ->
        (forall x, In x L1 -> In x L2) ->
          length L1 <= length L2.
Proof.
  induction n.
  intros.
  induction L1.
  simpl.
  apply le_0_n.
  simpl in H. discriminate.
  intros.
  induction L1.
  simpl in H. discriminate.
  clear IHL1.
  simpl in H0.
  elim H0; intros H2 H3; clear H0.
  simpl.
  simpl in H.
  injection H; intro H4; clear H.

  assert (length L1 <= length (set_remove Aeq a L2)).
  apply IHn.
  apply Aeq. assumption. assumption.
  intros.
  assert (In x L2).
  apply H1. simpl. right; assumption.
  assert ({x = a}+{x <> a}).
  apply Aeq.
  elim H5; intro H6; clear H5.
  rewrite H6 in H. contradiction.
  clear IHn H1 H2 H3 H4 H n L1.

  induction L2.
  simpl in H0. contradiction.
  simpl in H0.

```

```

    simpl.
    induction (Aeq a a0).
    elim H0; intro H1; clear H0.
    rewrite H1 in a1. symmetry in a1.
    contradiction. assumption.
    simpl.
    elim H0; intro H1; clear H0.
    left; assumption.
    right. apply IHL2.
    assumption.

    assert (S (length (set_remove Aeq a L2)) <= length L2).
    assert (In a L2).
    apply H1. simpl. left; trivial.
    clear n IHn L1 H1 H2 H3 H4 H.

    induction L2.
    simpl in H0. contradiction.
    simpl in H0.
    elim H0; intro H1; clear H0.
    rewrite H1.
    simpl.
    induction (Aeq a a).
    trivial. absurd (a = a). assumption. trivial.
    simpl.
    induction (Aeq a a0).
    trivial.
    simpl.
    apply le_n_S.
    apply IHL2.
    assumption.

    assert (S (length L1) <= S (length (set_remove Aeq a L2))).
    apply le_n_S.
    assumption.
    apply le_trans with (m := S (length (set_remove Aeq a L2))).
    assumption.
    assumption.
Qed.

Lemma length_incl:
  forall (A : Set)(Aeq : forall (a b : A), {a=b}+{a<>b})(L2 L1 : list A),
    is_set A L1 ->
    (forall x, In x L1 -> In x L2) ->
    length L1 <= length L2.
Proof.
  intros.
  apply length_incl_n with (n := length L1).
  assumption. trivial.
  assumption. assumption.
Qed.

```

```

Lemma set_add_extends:
  forall (A : Set)(L : list A)(E : A)(EQ : forall (a b : A), {a = b} + {a <> b})
  ,
    exists t1, set_add EQ E L = L ++ t1.

```

Proof.

```

  induction L.
  intros.
  simpl. exists (E :: nil). trivial.
  intros.
  simpl.
  induction (EQ E a).
  exists (nil : list A).
  symmetry.
  apply app_nil with (L := a :: L).
  assert (exists t10, set_add EQ E L = L ++ t10).
  apply IHL.
  elim H; intros t10 H2; clear H.
  rewrite H2.
  exists t10. trivial.

```

Qed.

```

Lemma set_union_extends:
  forall (A : Set)(L1 L2 : list A)(EQ : forall (a b : A), {a = b} + {a <> b}),
    exists t1, set_union EQ L1 L2 = L1 ++ t1.

```

Proof.

```

  intros.
  induction L2.
  simpl. exists (nil : list A).
  symmetry.
  apply app_nil with (L := L1).
  simpl.
  assert (exists t10, set_add EQ a (set_union EQ L1 L2) = (set_union EQ L1 L2) +
    + t10).
  apply set_add_extends.
  elim H; intros t10 H2; clear H.
  rewrite H2.
  elim IHL2; intros t1 H3; clear IHL2.
  rewrite H3.
  exists (t1 ++ t10).
  (* symmetry. *)
  apply app_ass with (l := L1)(m := t1)(n := t10).

```

Qed.

```

Lemma length_app_nil:
  forall (A : Set)(t1 L : list A),
    length L = length (L ++ t1) ->
    t1 = nil.

```

Proof.

```

  induction t1. trivial.
  intros.

```

```

induction L.
  simpl in H. discriminate.
  apply IHL.
  simpl in H. injection H. trivial.
Qed.

Lemma same_length:
  forall (LF : list form)(LA : list atomic),
    length LA = length (extend_one_one_k LF LA) ->
    LA = extend_one_one_k LF LA.
Proof.
  intros.
  unfold extend_one_one_k in H.
  unfold extend_one_k in H.
  unfold extend_one_one_k.
  unfold extend_one_k.

  assert (exists t1, set_union eq_atomic_dec (extend_one LF LA) (kthing_set_ext
    (extset LF) (extend_one LF LA)) = (extend_one LF LA) ++ t1).
  apply set_union_extends.
  elim H0; intros t1 H1; clear H0.
  rewrite H1.
  rewrite H1 in H.

  unfold extend_one in H1.
  unfold extend_one in H.
  unfold extend_one.
  assert (exists t12, set_union eq_atomic_dec LA (match_form_list_atomic_list LF
    LA) = LA ++ t12).
  apply set_union_extends.
  elim H0; intros t12 H2; clear H0.

  rewrite H2 in H1.
  rewrite H2.
  rewrite H2 in H.

  assert ((LA ++ t12) ++ t1 = LA ++ (t12 ++ t1)).
  apply app_ass.
  rewrite H0 in H. rewrite H0.

  assert ((t12 ++ t1) = nil).
  apply length_app_nil with (L := LA).
  assumption.
  rewrite H3.
  symmetry.
  apply app_nil.
Qed.

Lemma is_set_set_add :
  forall (A : Set)(Aeq : forall (a b : A), {a=b}+{a<>b})(E : A)(L : list A),
    is_set A L ->

```



```

    is_set A (set_add Aeq E L).
Proof.
  induction L; simpl; intros.
  auto.
  elim H; intros H1 H2; clear H.
  induction (Aeq E a).
  simpl.
  split; assumption.
  simpl.
  split.
  clear IHL H2.
  induction L.
  simpl. unfold not. intro H2.
  elim H2; intro H3; clear H2.
  contradiction. contradiction.
  simpl.
  unfold not.
  intro H2.
  induction (Aeq E a0).
  contradiction.
  simpl in H2.
  elim H2; intro H3; clear H2.
  rewrite H3 in H1.
  simpl in H1.
  unfold not in H1. apply H1.
  left; trivial.
  assert (~ In a (set_add Aeq E L)).
  apply IHL.
  unfold not.
  intro H4.
  unfold not in H1.
  apply H1.
  simpl. right; assumption.
  contradiction.
  apply IHL.
  assumption.
Qed.

```

```

Lemma is_set_set_union :
  forall (A : Set)(Aeq : forall (a b : A), {a=b}+{a<>b})(L R : list A),
    is_set A L ->
    is_set A (set_union Aeq L R).
Proof.
  induction R; intros.
  simpl. assumption.
  simpl.
  apply is_set_set_add.
  apply IHR.
  assumption.
Qed.

```

```

Lemma is_set_iter_extend:
  forall (LF : list form)(n : nat),
    is_set atomic (iter n (extend_one_one_k LF) (kthing_start_set LF)).
Proof.
  intros.
  induction n.
  simpl.
  unfold kthing_start_set.
  apply is_set_set_union.
  simpl. trivial.
  simpl.
  set (Base := iter n (extend_one_one_k LF) (kthing_start_set LF)).
  unfold extend_one_one_k.
  unfold extend_one_k.
  unfold extend_one.
  apply is_set_set_union.
  apply is_set_set_union.
  unfold Base.
  assumption.
Qed.

Lemma dup_exists_extend_one:
  forall (l : nat)(LF : list form),
    length (iter l (extend_one_one_k LF) (kthing_start_set LF)) < l ->
    iter l (extend_one_one_k LF) (kthing_start_set LF) = iter (S l) (extend_one_
    one_k LF) (kthing_start_set LF).
Proof.
  induction l.
  intros.
  simpl in H.
  absurd (length (kthing_start_set LF) < 0).
  apply lt_n_0. assumption.
  intro. intro.
  assert (
    forall (n : nat)(x : atomic),
      In x (iter n (extend_one_one_k LF) (kthing_start_set LF)) ->
      In x (iter (S n) (extend_one_one_k LF) (kthing_start_set LF))).
  intros. simpl.
  apply monotonic_increasing. assumption.
  intros.
  assert ({l <= length (iter l (extend_one_one_k LF) (kthing_start_set LF))}+{le
  ngth (iter l (extend_one_one_k LF) (kthing_start_set LF)) < l}).
  apply le_lt_dec.
  elim H1; intro H2; clear H1.
  assert (length (iter l (extend_one_one_k LF) (kthing_start_set LF)) <= length
  (iter (S l) (extend_one_one_k LF) (kthing_start_set LF))).
  apply length_incl. apply eq_atomic_dec.
  apply is_set_iter_extend.
  intros.
  apply H0. assumption.
  assert (l <= length (iter (S l) (extend_one_one_k LF) (kthing_start_set LF))).

```

```

apply le_trans with (m := length (iter 1 (extend_one_one_k LF) (kthing_start_set LF)))
et LF))).
assumption.
assert (length (iter (S 1) (extend_one_one_k LF) (kthing_start_set LF)) <= 1).
apply lt_n_Sm_le. assumption.
assert (1 = length (iter (S 1) (extend_one_one_k LF) (kthing_start_set LF))).
apply le_antisym. assumption.
rewrite <- H5 in H1.
assert (1 = length (iter 1 (extend_one_one_k LF) (kthing_start_set LF))).
apply le_antisym; assumption.
simpl in H5.
simpl.
assert (length (iter 1 (extend_one_one_k LF) (kthing_start_set LF)) = length (
extend_one_one_k LF (iter 1 (extend_one_one_k LF) (kthing_start_set LF)))).
rewrite <- H5.
rewrite <- H6. trivial.
assert (iter 1 (extend_one_one_k LF) (kthing_start_set LF) = extend_one_one_k
LF (iter 1 (extend_one_one_k LF) (kthing_start_set LF))).
apply same_length.
assumption.
pattern (iter 1 (extend_one_one_k LF) (kthing_start_set LF)) at 1.
rewrite <- H8.
trivial.
assert (iter 1 (extend_one_one_k LF) (kthing_start_set LF) = iter (S 1) (extend
d_one_one_k LF) (kthing_start_set LF)).
apply IHL.
assumption.
simpl.
simpl in H1.
pattern (iter 1 (extend_one_one_k LF) (kthing_start_set LF)) at 1.
rewrite H1. trivial.
Qed.

(*
Lemma kthing_start_set_aux_sound:
  forall (LK : list kthing)(y : kthing),
    In y (kthing_start_set_aux LK) ->
      extder y.
Proof.
  intros.

  induction LK.
  simpl in H. contradiction.
  induction (eq_kthing_dec y a).
  rewrite a0 in H.
  rewrite a0.
  simpl in H.
  induction (P a n (or_introl (In a LK) (refl_equal a))).
  assumption.
  simpl in H.
  rewrite a0 in IHLK.

```

```

apply IHLK with (P := (fun (b : kthing)(n0 : nat) (H1 : In b LK) => P b n0 (or
_intror (a = b) H1))).
assumption.

simpl in H.
induction (P a n (or_introl (In a LK) (refl_equal a))).
simpl in H.
elim H; intro H1; clear H.
symmetry in H1.
contradiction.
apply IHLK with (P := (fun (b : kthing) (n0 : nat)(H1 : In b LK) => P b n0 (or
_intror (a = b) H1))).
assumption.

simpl in H.
(*
induction (P b n0 (or_intror (a = b) H1)).
contradiction.
simpl in H.
*)
*)
apply IHLK with (P := (fun (b : kthing) (n0 : nat) (H1 : In b LK) => P b n0 (o
r_intror (a = b) H1))).
assumption.

Qed.

Lemma kthing_start_set_aux_complete:
  forall (LK : list kthing)(n : nat)(P : _)(y : kthing),
    In y LK ->
      extder n y ->
        In y (kthing_start_set_aux LK n P).
Proof.
  intros.

  induction LK.
  simpl in H. contradiction.
  simpl in H. elim H; intro H1; clear H.
  rewrite <- H1.
  rewrite <- H1 in H0.
  simpl.
  induction (P a n (or_introl (In a LK) (refl_equal a))).
  simpl. left; trivial. contradiction.
  assert (forall x, In x LK -> {extder n x}+{~extder n x}).
  intros x H2.
  apply P. simpl. right; assumption.

  simpl.
  set (F1 := (fun (b : kthing) (n0 : nat) (H3 : In b LK) =>
    P b n0 (or_intror (a = b) H3))).
  set (F2 := (P a n (or_introl (In a LK) (refl_equal a)))).
  assert (In y (kthing_start_set_aux LK n F1)).

```

```

    apply IHLK. assumption.
    induction F2.
    simpl. right. assumption.
    simpl. assumption.

Qed.

Lemma kthing_in_extset:
  forall (y : kthing)(LK : list kthing)(n : nat)(D : _),
    In y
      (kthing_start_set_aux LK n D) ->
    In y LK.
Proof.
  intros.
  induction LK.
  simpl in H. contradiction.
  simpl in H.
  induction (D a n (or_introl (In a LK) (refl_equal a))).
  simpl in H.
  elim H; intro H1; clear H.
  simpl. left; assumption.
  simpl. right. apply IHLK with (D:= (fun (b : kthing) (n0 : nat)(H1 : In b LK)
=> D b n0 (or_intror (a = b) H1))).
  assumption.
  simpl in H.
  simpl. right.
  apply IHLK with (D:= (fun (b : kthing) (n0 : nat)(H1 : In b LK) => D b n0 (or_
intror (a = b) H1))).
  assumption.
Qed.

Lemma kthing_start_set_aux_n_der:
  forall LF A LN,
    (forall x, In x LN -> In x (extract_extn_list_form LF)) ->
    In A
      (kthing_start_set_aux_n (extset LF) LN
        (extset_der LF)) -> der LF A.
Proof.
  intros.
  induction LN.
  simpl in H0. contradiction.
  simpl in H0.
  assert (
    (In A (map (fun k : kthing => extbare a (kident k)) (kthing_start_set_aux
      (extset LF) a (extset_der LF))))
    /\ (In A (kthing_start_set_aux_n (extset LF) LN (extset_der LF)))
  ).
  apply in_app_or.
  assumption. clear H0.
  elim H1; intro H2; clear H1.

```

```

    assert (exists y, In y (kthing_start_set_aux (extset LF) a (extset_der LF))
    /\ A = (fun k : kthing => extbare a (kident k)) y).
    apply in_map with (f:=(fun k : kthing => extbare a (kident k))).
    assumption. clear H2.
    elim H0; intros y H3; clear H0.
    elim H3; intros H5 H6; clear H3.
    rewrite H6.
    apply extension_der.
    apply kthing_in_extset with (LK := extset LF)(D := extset_der LF)(n := a).
    assumption.
    apply H. simpl. left; trivial.
    apply kthing_start_set_aux_sound with (LK := extset LF)(n := a)(P := extset_
    der LF).
    assumption.
    apply IHLN.
    intros x H3.
    apply H. simpl. right; assumption.
    assumption.
  Qed.

*)

Lemma in_filter_in2:
  forall (A : Type)(f : A -> bool)(L : list A)(x : A),
    In x (filter f L) -> In x L /\ f x = true.
Proof.
  intros.
  assert (In x (filter f L) <-> In x L /\ f x = true).
  apply filter_In.
  unfold iff in H0. elim H0; intros.
  apply H1. assumption.
Qed.

Lemma alg_sound :
  forall (n : nat)(LF : list form)(A : atomic),
    no_head_vars_form_list LF ->
    In A (iter n (extend_one_one_k LF) (kthing_start_set LF)) ->
    der LF A.
Proof.
  induction n.
  intros.

  (* base cases *)
  simpl in H0.

  (* kthing ground case *)

  unfold kthing_start_set in H0.
  assert (In A (kthing_start_set_aux (extset LF))).
  assert (set_In A nil \/ set_In A (kthing_start_set_aux (extset LF))).

```

```

apply set_union_elim with (Aeq_dec := eq_atomic_dec).
unfold set_In. assumption.
elim H1; intro H2; clear H1.
simpl in H2. contradiction.
unfold set_In in H2.
assumption. clear H0.

unfold kthing_start_set_aux in H1.
assert (exists y, In y (filter
  (fun pr : kthing * kthing =>
    let (x, y) := pr in if exttype_dec x y then true else false)
  (list_prod (extset LF) (extset LF)))) /\ A = ((fun pr : kthing * k
    thing =>
      let (_, y) := pr in extbare (kident y))) y).
apply in_map. assumption. clear H1.
elim H0; intros; clear H0.
elim H1; intros; clear H1.
assert (In x (filter
  (fun pr : kthing * kthing => let (xl, yl) := pr in if exttype_dec xl yl then
    true else false)
  (list_prod (extset LF) (extset LF)))) <->
  In x (list_prod (extset LF) (extset LF)) /\
  (fun pr : kthing * kthing => let (xl, yl) := pr in if exttype_dec xl yl then
    true else false) x = true).
apply filter_In with (f := (fun pr : kthing * kthing =>
  let (xl, yl) := pr in if exttype_dec xl yl then true else false)).
elim H1; intros; clear H1.
clear H4.
assert (In x (list_prod (extset LF) (extset LF)) /\
  (let (xl, yl) := x in if exttype_dec xl yl then true else false) = true).
apply H3. clear H3. assumption. clear H3.
elim H1; intros; clear H1. clear H0.
destruct x.
induction (exttype_dec k k0).
subst A.
apply extension_der1 with (K1 := k)(K2 := k0).
apply in_prod_l with (y := k0)(LB := extset LF). assumption.
apply in_prod_r with (x := k)(LA := extset LF). assumption.
assumption. discriminate.

(* inductive cases *)

intros.
simpl in H0.
set (Base := iter n (extend_one_one_k LF) (kthing_start_set LF)) in H0.
unfold extend_one_one_k in H0.
unfold extend_one_k in H0.

assert (set_In A (extend_one LF Base) \ / set_In A (kthing_set_ext (extset LF)
  (extend_one LF Base))).
apply set_union_elim with (Aeq_dec:=eq_atomic_dec).

```

```

assumption.

elim H1; intro H2; clear H1.

(* in extend_one, same as in Binder *)
unfold extend_one in H2.
assert (set_In A Base \ / set_In A (match_form_list_atomic_list LF Base)).
apply set_union_elim with (Aeq_dec:=eq_atomic_dec).
assumption.
elim H1; intro H3; clear H1.
apply IHn. assumption. assumption.
apply match_form_list_atomic_list_sound with (LA:=Base).
unfold Base.
apply ground_alg.
assumption.
unfold set_In in H3.
assumption.
unfold forallelts.
intros.
apply IHn.
assumption.
assumption.

(* kthing_set_ext case *)

clear H0.
unfold set_In in H2.
unfold kthing_set_ext in H2.
assert (exists y, In y (filter
  (fun p : kthing * kthing * kthing =>
    let (p2, k3) := p in
    let (k1, k2) := p2 in
    if In_dec eq_atomic_dec (extbare (kident k1))
      (extend_one LF Base)
    then if extdot_dec k1 k2 k3 then true else false
    else false)
  (list_prod (list_prod (extset LF) (extset LF)) (extset LF)))) /\
A = ((fun p : kthing * kthing * kthing =>
  let (_, k3) := p in extbare (kident k3))) y).
apply in_map. assumption. clear H2.
elim H0; intros; clear H0.
elim H1; intros; clear H1. subst A. destruct x.

assert (In (p, k) (list_prod (list_prod (extset LF) (extset LF)) (extset LF))
/\
(fun p : kthing * kthing * kthing =>
  let (p2, k3) := p in
  let (k1, k2) := p2 in
  if In_dec eq_atomic_dec (extbare (kident k1))
    (extend_one LF Base)
  then if extdot_dec k1 k2 k3 then true else false

```



```

        else false) (p, k) = true).
apply in_filter_in2 with (x := (p, k))(f := (fun p : kthing * kthing * kthing
=>
    let (p2, k3) := p in
    let (k1, k2) := p2 in
    if In_dec eq_atomic_dec (extbare (kident k1))
      (extend_one LF Base)
    then if extdot_dec k1 k2 k3 then true else false
    else false)).
assumption.
clear H0.
elim H1; intros; clear H1.
destruct p.
(*
extension_der2
  : forall LF : list form,
    nat ->
    forall K1 K2 K3 : kthing,
    In K1 (extset LF) ->
    In K2 (extset LF) ->
    In K3 (extset LF) ->
    extdot K1 K2 K3 ->
    der LF (extbare (kident K1)) -> der LF (extbare (kident K3))
*)
apply extension_der2 with (K1 := k0)(K2 := k1)(K3 := k).
assert (In (k0, k1) (list_prod (extset LF) (extset LF))).
apply in_prod_l with (LA := (list_prod (extset LF) (extset LF)))(y := k)(LB :=
  extset LF).
assumption.
apply in_prod_l with (y := k1)(LB := extset LF).
assumption.
assert (In (k0, k1) (list_prod (extset LF) (extset LF))).
apply in_prod_l with (LA := (list_prod (extset LF) (extset LF)))(y := k)(LB :=
  extset LF).
assumption.
apply in_prod_r with (x := k0)(LA := (extset LF)).
assumption.
apply in_prod_r with (x := (k0, k1))(LA := list_prod (extset LF) (extset LF)).
assumption.
induction (extdot_dec k0 k1 k).
assumption.
induction (In_dec eq_atomic_dec (extbare (kident k0)) (extend_one LF Base)).
discriminate. discriminate.
assert (In (extbare (kident k0)) (extend_one LF Base)).
induction (In_dec eq_atomic_dec (extbare (kident k0)) (extend_one LF Base)).
assumption. discriminate.
unfold extend_one in H1.
assert (set_In (extbare (kident k0)) Base \ /
  set_In (extbare (kident k0)) (match_form_list_atomic_list LF Base)).
apply set_union_elim with (Aeq_dec:=eq_atomic_dec).
unfold set_In. assumption.

```

```

elim H3; intros; clear H3.
unfold set_In in H4.
clear H1.
apply IHn.
assumption.
fold Base.
assumption.
unfold set_In in H4.
clear H1.
apply match_form_list_atomic_list_sound with (LA:=Base).
unfold Base.
apply ground_alg.
assumption.
assumption.
unfold forallelts.
intros.
apply IHn.
assumption.
assumption.
Qed.

```

```

Lemma no_vars_occurs_term:
  forall (a : term),
    (forall v : nat, var_occurs_term v a = false) ->
      ground_term a = true.

```

```

Proof.
  intros.
  induction a.
  simpl. trivial.
  simpl. trivial.
  assert (var_occurs_term n (var n) = false).
  apply H.
  simpl in H0.
  induction (eq_nat_dec n n).
  discriminate.
  contradiction b. trivial.
Qed.

```

```

Lemma no_vars_occurs_term_list:
  forall (a : list term),
    (forall v : nat, var_occurs_term_list v a = false) ->
      ground_term_list a = true.

```

```

Proof.
  unfold ground_term_list.
  unfold var_occurs_term_list.
  intros.
  induction a.
  simpl. trivial.
  simpl.
  simpl in H.
  assert (ground_term a = true).

```

```

    apply no_vars_occurs_term.
    intros.
    assert (var_occurs_term v a || exists_bool (var_occurs_term v) a0 = false).
    apply H.
    induction (var_occurs_term v a).
    simpl in H0. discriminate. trivial.
    rewrite H0. simpl.
    apply IHa.
    intros.
    assert (var_occurs_term v a || exists_bool (var_occurs_term v) a0 = false).
    apply H.
    induction (var_occurs_term v a).
    simpl in H1. discriminate.
    simpl in H1. assumption.
Qed.

```

```

Lemma no_vars_occurs_atomic:
  forall (a : atomic),
    (forall v : nat, var_occurs_atomic v a = false) ->
      ground_atomic a = true.

```

```

Proof.
  induction a.
  unfold ground_atomic.
  unfold var_occurs_atomic.
  intros.
  apply no_vars_occurs_term_list.
  assumption.
  unfold ground_atomic.
  unfold var_occurs_atomic.
  intros.
  apply no_vars_occurs_term_list.
  assumption.
  unfold ground_atomic.
  unfold var_occurs_atomic.
  intros.
  apply no_vars_occurs_term.
  assumption.
  unfold ground_atomic.
  unfold var_occurs_atomic.
  intros.
  apply no_vars_occurs_term.
  assumption.
Qed.

```

```

Lemma monotonic_increasing2:
  forall (n n0 : nat)(LF : list form)(LA : list atomic)(A : atomic),
    In A (iter n (extend_one_one_k LF) LA) ->
      In A (iter n0 (extend_one_one_k LF) (iter n (extend_one_one_k LF) LA)).
Proof.
  induction n0.
  intros.

```

```

    simpl. assumption.
    intros.
    simpl.
    apply monotonic_increasing.
    apply IHn0. assumption.
Qed.

```

```

Lemma iter_plus:
  forall (A : Set)(f : A -> A)(x0 : A)(a b : nat),
    iter (a + b) f x0 = iter b f (iter a f x0).

```

```

Proof.
  intros.
  induction a.
  simpl. trivial.
  simpl. rewrite IHa.
  clear IHa.
  assert (iter b f (f (iter a f x0)) = iter (S b) f (iter a f x0)).
  simpl.
  induction b.
  simpl. trivial.
  simpl. rewrite IHb. trivial.
  rewrite H.
  clear H.
  simpl. trivial.
Qed.

```

```

Lemma ge_in_iter:
  forall (n n0 : nat)(LF : list form)(LA : list atomic)(A : atomic),
    n <= n0 ->
      In A (iter n (extend_one_one_k LF) LA) ->
      In A (iter n0 (extend_one_one_k LF) LA).

```

```

Proof.
  intros.
  assert (n0 = n + (n0 - n)).
  apply le_plus_minus.
  assumption.
  assert (iter (n + (n0 - n)) (extend_one_one_k LF) LA = iter (n0 - n) (extend_o
ne_one_k LF) (iter n (extend_one_one_k LF) LA)).
  apply iter_plus.
  rewrite H1.
  rewrite H2.
  clear H1 H2.
  apply monotonic_increasing2. assumption.
Qed.

```

```

Lemma in_filter_in3:
  forall (A : Type)(f : A -> bool) x l, (In x l /\ f x = true) -> In x (filter f
l).

```

```

Proof.
  intros.
  assert (In x (filter f l) <-> In x l /\ f x = true).

```

```

    apply filter_In.
    unfold iff in H0. elim H0; intros.
    apply H2. assumption.
Qed.

```

```

Lemma kthing_start_set_complete:
  forall LF K1 K2,
    In K1 (extset LF) ->
    In K2 (extset LF) ->
    extttype K1 K2 ->
    In (extbare (kident K2)) (kthing_start_set LF).

```

```

Proof.
  intros.
  unfold kthing_start_set.
  apply set_union_intro2.
  unfold set_In.
  unfold kthing_start_set_aux.
  apply in_map2.
  exists (K1, K2).
  split; try trivial.
  apply in_filter_in3.
  split. apply in_prod.
  assumption. assumption.
  induction (extttype_dec K1 K2). trivial. contradiction.
Qed.

```

```

Lemma extend_one_k_complete:
  forall LF K1 K2 K3,
    In K1 (extset LF) ->
    In K2 (extset LF) ->
    In K3 (extset LF) ->
    extdot K1 K2 K3 ->
    der LF (extbare (kident K1)) ->
    (exists n : nat,
      In (extbare (kident K1))
      (iter n (extend_one_one_k LF) (kthing_start_set LF))) ->
    (exists n : nat,
      In (extbare (kident K3))
      (iter n (extend_one_one_k LF) (kthing_start_set LF))).

```

```

Proof.
  intros.
  elim H4; intros n Hder; clear H4.
  exists (S n).
  simpl.
  set (Base := (iter n (extend_one_one_k LF) (kthing_start_set LF))).
  fold Base in Hder.
  unfold extend_one_one_k.
  assert (In (extbare (kident K1)) (extend_one LF Base)).
  unfold extend_one.
  apply set_union_intro. left. assumption.

```

```

unfold extend_one_k.
apply set_union_intro. right.
unfold set_In.
set (Base2 := extend_one LF Base).
fold Base2 in H4.
unfold kthing_set_ext.
apply in_map2.
exists (K1, K2, K3). split; try trivial.
apply in_filter_in3.
split.
apply in_prod.
apply in_prod. assumption. assumption. assumption.
induction (In_dec eq_atomic_dec (extbare (kident K1)) Base2).
induction (extdot_dec K1 K2 K3). trivial.
contradiction.
contradiction.
Qed.

```

```

Lemma alg_complete :
  forall (LF : list form)(A : atomic),
    no_head_vars_form_list LF ->
      der LF A ->
        exists n,
          In A (iter n (extend_one_one_k LF) (kthing_start_set LF)).

```

Proof.

```

intros.
induction H0.
induction F.
simpl. simpl in H2.
(* We know that there is a number n that works for every atomic formula in the
   body of F,
   we need to get the max of these and add one to get an n that works for F it
   self. *)
(* We don't actually need the fact that the body atomic formulas are derivable
   (H1) *)
clear H1.
(* Clean up H2 a bit *)
assert (forall x, In x l -> exists n, In (multisubs_atomic S x) (iter n (extend_one_one_k LF) (kthing_start_set LF))).
intros.
apply H2; assumption.
clear H2.

(* We can't just induct on l, since we need clause a l to be in LF, but that screws up the induction hypothesis *)
(* Solution is to assert weaker fact, clear H0 *)
assert (exists n, (forall x, In x l -> In (multisubs_atomic S x) (iter n (extend_one_one_k LF) (kthing_start_set LF)))).
clear H0.
induction l.

```

```

(* Base case is empty body, easy *)
exists 0.
intros.
simpl in H0. contradiction.

(* This gets confusing *)
(* First simplify IH1 *)
assert (exists n : nat,
  (forall x : atomic,
    In x l -> In (multisubs_atomic S x) (iter n (extend_one_one_k LF) (kthing_
      start_set LF)))).
apply IH1.
intros.
apply H1. simpl. right; trivial.
clear IH1.
(* Now extract assertion about a0, the added element to l *)
assert (exists n0, In (multisubs_atomic S a0) (iter n0 (extend_one_one_k LF) (
  kthing_start_set LF))).
apply H1. simpl. left; trivial.
(* No more info in H1, clear it to save mental energy *)
clear H1.
(* Get concrete values for n and n0 *)
elim H0; intros n H3; clear H0.
elim H2; intros n0 H4; clear H2.
(* Which one is bigger? *)
assert ({n <= n0} + {n0 < n}).
apply le_lt_dec.
elim H0; intro H1; clear H0.
(* n <= n0 means that we can use n0 for n1 *)
exists n0.
intros.
simpl in H0.
elim H0; intro H5; clear H0.
rewrite <- H5.
assumption.
assert (In (multisubs_atomic S x) (iter n (extend_one_one_k LF) (kthing_start_
  set LF))).
apply H3. assumption.
apply ge_in_iter with (n := n).
assumption. assumption.
(* n0 < n means we use n for n1 *)
exists n.
intros.
simpl in H0.
elim H0; intro H5; clear H0.
rewrite <- H5.
apply ge_in_iter with (n := n0)(n0 := n).
apply lt_le_weak.
assumption. assumption.
apply H3. assumption.

```

```

(* Now we're ready to provide an n that works *)
clear H1.
elim H2; intros n H3; clear H2.
(* Add one to the max of the bodies *)
exists (Datatypes.S n).
simpl.
assert (In (multisubs_atomic S (head (clause a 1))) (match_form_list_atomic_list LF (iter n (extend_one_one_k LF) (kthing_start_set LF)))).
(* Here we use the big result about completeness of matching *)
apply match_form_list_atomic_list_complete with (LF := LF)(LA := iter n (extend_one_one_k LF) (kthing_start_set LF)).
assumption.
(* Here we need that big result about ground extension *)
apply ground_alg.
assumption.
assumption.
simpl.
unfold forallelts.
intros.
apply H3. assumption.
simpl in H1.
set (LA := iter n (extend_one_one_k LF) (kthing_start_set LF)).
unfold extend_one_one_k.
unfold extend_one_k.
apply set_union_intro. left.
unfold set_In.
unfold LA.
unfold extend_one.
apply set_union_intro2.
unfold set_In.
assumption.

exists 0. simpl.
apply kthing_start_set_complete with (K1 := K1)(K2 := K2); assumption.

apply extend_one_k_complete with (K1 := K1)(K2 := K2)(K3 := K3); try assumption.
n.
apply IHder. assumption.
Qed.

```

Theorem termination :

```

forall (LF : list form),
  no_head_vars_form_list LF ->
  exists n : nat,
    (iter n (extend_one_one_k LF) (kthing_start_set LF)) =
    (iter (S n) (extend_one_one_k LF) (kthing_start_set LF)).
Proof.

```



```

intros.
(* Basic idea :
   from upperbound_goodness we know that everything that appears in iter is fr
   om finite set
   from dup_exists_extend_one we know that if the length of iter is less than
   l, then there is a fixpoint
   Taken together we know the algorithm terminates.
*)
set (UB := all_atomics
    ((extract_term_list_form LF) ++ (extract_term_list_atomic (kthing_start_set2
    LF)))
    (extract_pred_list_form LF) (extract_prin_list_form LF) (extract_arity_list_
    form LF)).
set (lenUB := length UB).
exists (S (length UB)).
apply dup_exists_extend_one with (LF := LF).
assert (forall x, In x (iter (S (length UB)) (extend_one_one_k LF) (kthing_sta
rt_set LF)) -> In x UB).
intros.
unfold UB.
apply upperbound_goodness with (n := S (length UB)).
assumption. assumption.
assert (length (iter (S (length UB)) (extend_one_one_k LF) (kthing_start_set L
F)) <= length UB).
apply length_incl.
apply eq_atomic_dec.
apply is_set_iter_extend.
assumption.
apply le_lt_n_Sm.
assumption.
Qed.

```

```

Theorem termination2 :
forall (LF : list form),
no_head_vars_form_list LF ->
(iter (S (length (all_atomics
    ((extract_term_list_form LF) ++ (extract_term_list_atomic (kthing_start_se
t2 LF)))
    (extract_pred_list_form LF) (extract_prin_list_form LF) (extract_arity_lis
t_form LF))))
    (extend_one_one_k LF) (kthing_start_set LF)) =
(iter (S (S (length (all_atomics
    ((extract_term_list_form LF) ++ (extract_term_list_atomic (kthing_start_se
t2 LF)))
    (extract_pred_list_form LF) (extract_prin_list_form LF) (extract_arity_lis
t_form LF))))
    (extend_one_one_k LF) (kthing_start_set LF))).

```

```

Proof.
intros.
(* Basic idea :
   from upperbound_goodness we know that everything that appears in iter is fr

```

```

om finite set
from dup_exists_extend_one we know that if the length of iter is less than
l, then there is a fixpoint
Taken together we know the algorithm terminates.
*)
set (UB := all_atomics
  ((extract_term_list_form LF) ++ (extract_term_list_atomic (kthing_start_set2
    LF)))
  (extract_pred_list_form LF) (extract_prin_list_form LF) (extract_arity_list_
    form LF)).
set (lenUB := length UB).
apply dup_exists_extend_one with (LF := LF).
assert (forall x, In x (iter (S (length UB)) (extend_one_one_k LF) (kthing_sta
rt_set LF)) -> In x UB).
intros.
unfold UB.
apply upperbound_goodness with (n := S (length UB)).
assumption. assumption.
assert (length (iter (S (length UB)) (extend_one_one_k LF) (kthing_start_set L
F)) <= length UB).
apply length_incl.
apply eq_atomic_dec.
apply is_set_iter_extend.
assumption.
apply le_lt_n_Sm.
assumption.
Qed.

```

Lemma fixpoint_init:

```

forall (LF : list form)(LA : list atomic)(A : atomic)(n m : nat),
  iter n (extend_one_one_k LF) LA = iter (S n) (extend_one_one_k LF) LA ->
  In A (iter m (extend_one_one_k LF) LA) ->
  In A (iter n (extend_one_one_k LF) LA).

```

Proof.

```

intros.
assert ({m <= n} + {n < m}).
apply le_lt_dec.
elim H1; intro H2; clear H1.
apply ge_in_iter with (n := m). assumption. assumption.
assert (m = n + (m - n)).
apply le_plus_minus.
apply lt_le_weak. assumption.
rewrite H1 in H0.
assert (iter (n + (m - n)) (extend_one_one_k LF) LA = iter (m - n) (extend_one
_one_k LF) (iter n (extend_one_one_k LF) LA)).
apply iter_plus.
rewrite H3 in H0.
clear H1 H3.
simpl in H.
assert (iter (m - n) (extend_one_one_k LF) (iter n (extend_one_one_k LF) LA) =
  iter n (extend_one_one_k LF) LA).

```

```

    apply once_the_same_always_the_same.
    rewrite <- H. trivial.
    rewrite H1 in H0. assumption.
Qed.

Theorem der_dec:
  forall (LF : list form)(A : atomic),
    no_head_vars_form_list LF ->
      {der LF A} + {~ der LF A}.
Proof.
  intros.
  set (UB := all_atomics
    ((extract_term_list_form LF) ++ (extract_term_list_atomic (kthing_start_set2
      LF)))
    (extract_pred_list_form LF) (extract_prin_list_form LF) (extract_arity_list_
      form LF)).
  set (lenUB := length UB).
  assert (
    (iter (S lenUB) (extend_one_one_k LF) (kthing_start_set LF)) =
    (iter (S (S lenUB)) (extend_one_one_k LF) (kthing_start_set LF))).
  unfold lenUB.
  unfold UB.
  apply termination2.
  assumption.
  set (DB := iter (S lenUB) (extend_one_one_k LF) (kthing_start_set LF)).
  assert ({In A DB}+{~In A DB}).
  apply In_dec.
  intros.
  apply eq_atomic_dec.
  elim H1; intro H2; clear H1.
  left.
  apply alg_sound with (n := S lenUB).
  assumption.
  unfold DB in H2.
  assumption.
  right.
  unfold not.
  intro.
  assert (exists n,
    In A (iter n (extend_one_one_k LF) (kthing_start_set LF))).
  apply alg_complete.
  assumption. assumption.
  elim H3; intros n H4; clear H3.
  assert (In A DB).
  unfold DB.
  apply fixpoint_init with (n := S lenUB)(m := n).
  assumption. assumption.
  contradiction.
Qed.

End ExtendedBinder.

```

Recursive Extraction der_dec.

A.2 Extracted BCIC code

In this section we present the automatically extracted code from the proof in Section A.1. The extracted code is generated from the proof using the command:

```
Recursive Extraction der_dec.
```

The extracted code is OCaml code; it compiles and runs without any modifications. The first 50 lines are redundant definition of types already defined in OCaml (though with different constructor names). The entire extraction is 678 lines versus the 9932 lines in the proof script. The difference is due to the fact that extraction only generates code for types of kind `Set`, not of kind `Prop`. For example, data structures are declared to be of kind `Set` so functions such as substitution over data structures will be extracted to working code. Proofs about substitution will usually be of kind `Prop`, so they will disappear during extraction. Because of the properties of extraction, the theorems and lemmas will remain true during execution of the extracted program even though the explicit proofs do not appear in the runtime.

```
type bool =
  | True
  | False

type nat =
  | 0
  | S of nat

type 'a option =
  | Some of 'a
  | None

type ('a, 'b) prod =
  | Pair of 'a * 'b

type sumbool =
  | Left
  | Right

type 'a list =
  | Nil
  | Cons of 'a * 'a list

(** val length : 'a list -> nat **)
```

```

let rec length = function
  | Nil -> 0
  | Cons (a, m) -> S (length m)

(** val app : 'a1 list -> 'a1 list -> 'a1 list **)

let rec app l m =
  match l with
  | Nil -> m
  | Cons (a, l1) -> Cons (a, (app l1 m))

(** val fold_left : ('a1 -> 'a2 -> 'a1) -> 'a2 list -> 'a1 -> 'a1 **)

let rec fold_left f l a0 =
  match l with
  | Nil -> a0
  | Cons (b, t) -> fold_left f t (f a0 b)

(** val list_prod : 'a1 list -> 'a2 list -> ('a1, 'a2) prod list **)

let rec list_prod l l' =
  match l with
  | Nil -> Nil
  | Cons (x, t) ->
    app
      (let rec map = function
        | Nil -> Nil
        | Cons (a, t0) -> Cons ((Pair (x, a)), (map t0))
        in map l') (list_prod t l')

type 'a set = 'a list

(** val set_add : ('a1 -> 'a1 -> sumbool) -> 'a1 -> 'a1 set -> 'a1 set **)

let rec set_add aeq_dec a = function
  | Nil -> Cons (a, Nil)
  | Cons (a1, x1) ->
    (match aeq_dec a a1 with
     | Left -> Cons (a1, x1)
     | Right -> Cons (a1, (set_add aeq_dec a x1)))

(** val set_union : ('a1 -> 'a1 -> sumbool) -> 'a1 set -> 'a1 set -> 'a1 set **)

let rec set_union aeq_dec x = function
  | Nil -> x
  | Cons (a1, y1) -> set_add aeq_dec a1 (set_union aeq_dec x y1)

(** val eq_nat_dec : nat -> nat -> sumbool **)

let rec eq_nat_dec n y1 =

```

```

match n with
| 0 -> (match y1 with
        | 0 -> Left
        | S n0 -> Right)
| S n0 -> (match y1 with
           | 0 -> Right
           | S n1 -> eq_nat_dec n0 n1)

type 'kthing term =
| Ident of nat
| Kident of 'kthing
| Var of nat

type 'kthing atomic =
| Bare of nat * 'kthing term list
| Says of nat * nat * 'kthing term list
| Extbare of 'kthing term
| Extsays of nat * 'kthing term

type 'kthing form =
| Clause of 'kthing atomic * 'kthing atomic list

(** val head : 'a1 form -> 'a1 atomic **)

let head = function
| Clause (h, b) -> h

(** val body : 'a1 form -> 'a1 atomic list **)

let body = function
| Clause (h, b) -> b

(** val lift_term : ('a1 term -> 'a1 term) -> 'a1 atomic -> 'a1 atomic **)

let lift_term f = function
| Bare (p, lt) -> Bare (p,
    (let rec map = function
      | Nil -> Nil
      | Cons (a0, t) -> Cons ((f a0), (map t))
    in map lt))
| Says (w, p, lt) -> Says (w, p,
    (let rec map = function
      | Nil -> Nil
      | Cons (a0, t) -> Cons ((f a0), (map t))
    in map lt))
| Extbare k -> Extbare (f k)
| Extsays (w, k) -> Extsays (w, (f k))

type 'kthing rval =
| Rident of nat
| Rvkident of 'kthing

```

```

(** val subs_term : (nat, 'a1 rval) prod -> 'a1 term -> 'a1 term **)

let subs_term sb t1 =
  let Pair (x, ri) = sb in
  (match t1 with
  | Ident y -> t1
  | Kident k -> t1
  | Var y ->
    (match eq_nat_dec x y with
    | Left ->
      (match ri with
      | Rident i -> Ident i
      | Rvkident k -> Kident k)
    | Right -> t1))

type 'kthing substitution = (nat, 'kthing rval) prod list

(** val multisubs_term : 'a1 substitution -> 'a1 term -> 'a1 term **)

let multisubs_term sbs t =
  fold_left (fun t0 sb -> subs_term sb t0) sbs t

(** val multisubs_atomic : 'a1 substitution -> 'a1 atomic -> 'a1 atomic **)

let multisubs_atomic sbs a =
  lift_term (fun x -> multisubs_term sbs x) a

(** val eq_term_dec : ('a1 -> 'a1 -> sumbool) -> 'a1 term -> 'a1 term ->
    sumbool **)

let eq_term_dec eq_kthing_dec a1 a2 =
  match a1 with
  | Ident x ->
    (match a2 with
    | Ident n0 -> eq_nat_dec x n0
    | Kident k -> Right
    | Var n0 -> Right)
  | Kident x ->
    (match a2 with
    | Ident n -> Right
    | Kident k0 -> eq_kthing_dec x k0
    | Var n -> Right)
  | Var x ->
    (match a2 with
    | Ident n0 -> Right
    | Kident k -> Right
    | Var n0 -> eq_nat_dec x n0)

(** val eq_atomic_dec : ('a1 -> 'a1 -> sumbool) -> 'a1 atomic -> 'a1 atomic
    -> sumbool **)

```



```

let eq_atomic_dec eq_kthing_dec a1 a2 =
  match a1 with
  | Bare (x, x0) ->
    (match a2 with
    | Bare (n0, l0) ->
      (match eq_nat_dec x n0 with
      | Left ->
        let rec f l y1 =
          match l with
          | Nil ->
            (match y1 with
            | Nil -> Left
            | Cons (t, l1) -> Right)
          | Cons (a, l1) ->
            (match y1 with
            | Nil -> Right
            | Cons (t, l2) ->
              (match eq_term_dec eq_kthing_dec a t with
              | Left -> f l1 l2
              | Right -> Right))
        in f x0 l0
      | Right -> Right)
    | Says (n0, n1, l0) -> Right
    | Extbare t -> Right
    | Extsays (n0, t) -> Right)
  | Says (x, x0, x1) ->
    (match a2 with
    | Bare (n1, l0) -> Right
    | Says (n1, n2, l0) ->
      (match eq_nat_dec x n1 with
      | Left ->
        (match eq_nat_dec x0 n2 with
        | Left ->
          let rec f l y1 =
            match l with
            | Nil ->
              (match y1 with
              | Nil -> Left
              | Cons (t, l1) -> Right)
            | Cons (a, l1) ->
              (match y1 with
              | Nil -> Right
              | Cons (t, l2) ->
                (match
                eq_term_dec eq_kthing_dec a t with
                | Left -> f l1 l2
                | Right -> Right))
          in f x1 l0
        | Right -> Right)
      | Right -> Right)

```

```

        | Extbare t -> Right
        | Extsays (n1, t) -> Right)
| Extbare x ->
  (match a2 with
  | Bare (n, 1) -> Right
  | Says (n, n0, 1) -> Right
  | Extbare t0 -> eq_term_dec eq_kthing_dec x t0
  | Extsays (n, t0) -> Right)
| Extsays (x, x0) ->
  (match a2 with
  | Bare (n0, 1) -> Right
  | Says (n0, n1, 1) -> Right
  | Extbare t0 -> Right
  | Extsays (n0, t0) ->
    (match eq_nat_dec x n0 with
    | Left -> eq_term_dec eq_kthing_dec x0 t0
    | Right -> Right))

(** val flatten : 'a1 list list -> 'a1 list **)

let rec flatten = function
| Nil -> Nil
| Cons (hd, tl) -> app hd (flatten tl)

(** val iter : nat -> ('a1 -> 'a1) -> 'a1 -> 'a1 **)

let rec iter n f x =
  match n with
  | 0 -> x
  | S p -> f (iter p f x)

(** val match_term : ('a1 -> 'a1 -> sumbool) -> 'a1 term -> 'a1 term -> 'a1
    substitution option **)

let match_term eq_kthing_dec t1 t2 =
  match t1 with
  | Ident i ->
    (match t2 with
    | Ident j ->
      (match eq_nat_dec i j with
      | Left -> Some Nil
      | Right -> None)
    | Kident j -> None
    | Var j -> None)
  | Kident i ->
    (match t2 with
    | Ident j -> None
    | Kident j ->
      (match eq_kthing_dec i j with
      | Left -> Some Nil
      | Right -> None)

```

```

        | Var j -> None)
    | Var i ->
        (match t2 with
        | Ident j -> Some (Cons ((Pair (i, (Rvident j))), Nil))
        | Kident j -> Some (Cons ((Pair (i, (Rvkident j))), Nil))
        | Var j -> None)

(** val match_term_list : ('a1 -> 'a1 -> sumbool) -> 'a1 term list -> 'a1
    term list -> 'a1 substitution option **)

let rec match_term_list eq_kthing_dec lT1 lT2 =
  match lT1 with
  | Nil -> (match lT2 with
            | Nil -> Some Nil
            | Cons (t, l) -> None)
  | Cons (hd1, tl1) ->
      (match lT2 with
      | Nil -> None
      | Cons (hd2, tl2) ->
          (match match_term eq_kthing_dec hd1 hd2 with
          | Some s ->
              (match match_term_list eq_kthing_dec
                (let rec map = function
                  | Nil -> Nil
                  | Cons (a, t) -> Cons
                      ((multisubs_term s a),
                      (map t))
                in map tl1) tl2 with
              | Some s2 -> Some (app s s2)
              | None -> None)
          | None -> None))

(** val match_atomic : ('a1 -> 'a1 -> sumbool) -> 'a1 atomic -> 'a1 atomic ->
    'a1 substitution option **)

let match_atomic eq_kthing_dec a1 a2 =
  match a1 with
  | Bare (p1, lT1) ->
      (match a2 with
      | Bare (p2, lT2) ->
          (match eq_nat_dec p1 p2 with
          | Left -> match_term_list eq_kthing_dec lT1 lT2
          | Right -> None)
      | Says (w2, p2, lT2) -> None
      | Extbare k -> None
      | Extsays (w, k) -> None)
  | Says (w1, p1, lT1) ->
      (match a2 with
      | Bare (p2, lT2) -> None
      | Says (w2, p2, lT2) ->
          (match eq_nat_dec p1 p2 with

```

```

        | Left ->
            (match eq_nat_dec w1 w2 with
             | Left -> match_term_list eq_kthing_dec lT1 lT2
             | Right -> None)
        | Right -> None)
    | Extbare k -> None
    | Extsays (w, k) -> None)
| Extbare k ->
    (match a2 with
     | Bare (p2, lT2) -> None
     | Says (w2, p2, lT2) -> None
     | Extbare k2 -> match_term eq_kthing_dec k k2
     | Extsays (w2, k2) -> None)
| Extsays (w, k) ->
    (match a2 with
     | Bare (p2, lT2) -> None
     | Says (w2, p2, lT2) -> None
     | Extbare k2 -> None
     | Extsays (w2, k2) ->
        (match eq_nat_dec w w2 with
         | Left -> match_term eq_kthing_dec k k2
         | Right -> None))

(** val match_atomic_list : ('a1 -> 'a1 -> sumbool) -> 'a1 atomic -> 'a1
    atomic list -> 'a1 substitution list **)

let rec match_atomic_list eq_kthing_dec a = function
| Nil -> Nil
| Cons (lAhd, lAtl) ->
    (match match_atomic eq_kthing_dec a lAhd with
     | Some s -> Cons (s, (match_atomic_list eq_kthing_dec a lAtl))
     | None -> match_atomic_list eq_kthing_dec a lAtl)

(** val match_list_atomic_list_aux : ('a1 -> 'a1 -> sumbool) -> 'a1 atomic
    list -> 'a1 atomic list -> 'a1
    substitution -> 'a1 substitution list **)

let rec match_list_atomic_list_aux eq_kthing_dec lA1 lA2 sbs =
    match lA1 with
    | Nil -> Cons (Nil, Nil)
    | Cons (hd, tl) ->
        flatten
        (let rec map = function
         | Nil -> Nil
         | Cons (a, t) -> Cons
             ((let rec map0 = function
              | Nil -> Nil
              | Cons (a0, t0) -> Cons ((app a a0), (map0 t0))
              in map0
              (match_list_atomic_list_aux eq_kthing_dec tl lA2
               (app sbs a))), (map t))

```

```

        in map
          (match_atomic_list eq_kthing_dec (multisubs_atomic sbs hd)
            lA2))

(** val match_list_atomic_list : ('a1 -> 'a1 -> sumbool) -> 'a1 atomic list
    -> 'a1 atomic list -> 'a1 substitution list **)

let match_list_atomic_list eq_kthing_dec lA1 lA2 =
  match_list_atomic_list_aux eq_kthing_dec lA1 lA2 Nil

(** val match_form_atomic_list : ('a1 -> 'a1 -> sumbool) -> 'a1 form -> 'a1
    atomic list -> 'a1 atomic list **)

let match_form_atomic_list eq_kthing_dec f lA =
  let rec map = function
    | Nil -> Nil
    | Cons (a, t) -> Cons ((multisubs_atomic a (head f)), (map t))
  in map (match_list_atomic_list eq_kthing_dec (body f) lA)

(** val kthing_start_set_aux : ('a1 -> 'a1 -> sumbool) -> 'a1 list -> 'a1
    atomic list **)

let kthing_start_set_aux exttype_dec lK =
  let rec map = function
    | Nil -> Nil
    | Cons (a, t) -> Cons ((let Pair (x, y) = a in Extbare (Kident y)),
      (map t))
  in map
    (let rec filter = function
      | Nil -> Nil
      | Cons (x, l0) ->
        let Pair (x0, y) = x in
        (match exttype_dec x0 y with
          | Left -> Cons (x, (filter l0))
          | Right -> filter l0)
    in filter (list_prod lK lK))

(** val kthing_set_ext : ('a1 -> 'a1 -> sumbool) -> ('a1 -> 'a1 -> 'a1 ->
    sumbool) -> 'a1 list -> 'a1 atomic list -> 'a1
    atomic list **)

let kthing_set_ext eq_kthing_dec extdot_dec lK dB =
  let rec map = function
    | Nil -> Nil
    | Cons (a, t) -> Cons ((let Pair (p2, k3) = a in Extbare (Kident k3)),
      (map t))
  in map
    (let rec filter = function
      | Nil -> Nil
      | Cons (x, l0) ->
        let Pair (p2, k3) = x in

```

```

    let Pair (k1, k2) = p2 in
    (match let rec f = function
      | Nil -> Right
      | Cons (a, l2) ->
        (match eq_atomic_dec eq_kthing_dec a (Extbare
          (Kident k1)) with
          | Left -> Left
          | Right -> f l2)
    in f dB with
    | Left ->
      (match extdot_dec k1 k2 k3 with
      | Left -> Cons (x, (filter 10))
      | Right -> filter 10)
    | Right -> filter 10)
  in filter (list_prod (list_prod lK lK) lK))

(** val kthing_start_set : ('a1 -> 'a1 -> sumbool) -> ('a1 form list -> 'a1
    list) -> ('a1 -> 'a1 -> sumbool) -> 'a1 form list
    -> 'a1 atomic list **)

let kthing_start_set eq_kthing_dec extset exttype_dec lF =
  set_union (fun x x0 -> eq_atomic_dec eq_kthing_dec x x0) Nil
  (kthing_start_set_aux exttype_dec (extset lF))

(** val extend_one : ('a1 -> 'a1 -> sumbool) -> 'a1 form list -> 'a1 atomic
    list -> 'a1 atomic list **)

let extend_one eq_kthing_dec lF lA =
  set_union (fun x x0 -> eq_atomic_dec eq_kthing_dec x x0) lA
  (flatten
   (let rec map = function
     | Nil -> Nil
     | Cons (a, t) -> Cons ((match_form_atomic_list eq_kthing_dec a lA),
       (map t))
   in map lF))

(** val extend_one_k : ('a1 -> 'a1 -> sumbool) -> ('a1 form list -> 'a1 list)
    -> ('a1 -> 'a1 -> 'a1 -> sumbool) -> 'a1 form list ->
    'a1 atomic list -> 'a1 atomic list **)

let extend_one_k eq_kthing_dec extset extdot_dec lF lA =
  set_union (fun x x0 -> eq_atomic_dec eq_kthing_dec x x0) lA
  (kthing_set_ext eq_kthing_dec extdot_dec (extset lF) lA)

(** val extend_one_one_k : ('a1 -> 'a1 -> sumbool) -> ('a1 form list -> 'a1
    list) -> ('a1 -> 'a1 -> 'a1 -> sumbool) -> 'a1
    form list -> 'a1 atomic list -> 'a1 atomic list **)

let extend_one_one_k eq_kthing_dec extset extdot_dec lF lA =
  extend_one_k eq_kthing_dec extset extdot_dec lF
  (extend_one eq_kthing_dec lF lA)

```

```

(** val all_term_lists : 'a1 term list -> nat -> 'a1 term list list **)

let rec all_term_lists lT = function
| 0 -> Cons (Nil, Nil)
| S x ->
    let rec map = function
    | Nil -> Nil
    | Cons (a, t) -> Cons ((let Pair (a0, b) = a in Cons (a0, b)),
        (map t))
    in map (list_prod lT (all_term_lists lT x))

(** val all_bare_atomics_arity : 'a1 term list -> nat list -> nat -> 'a1
    atomic list **)

let all_bare_atomics_arity lT lP n =
    let rec map = function
    | Nil -> Nil
    | Cons (a, t) -> Cons ((let Pair (p, lt) = a in Bare (p, lt)), (map t))
    in map (list_prod lP (all_term_lists lT n))

(** val all_says1_atomics_arity : 'a1 term list -> nat list -> nat -> nat ->
    'a1 atomic list **)

let all_says1_atomics_arity lT lP w n =
    let rec map = function
    | Nil -> Nil
    | Cons (a, t) -> Cons ((let Pair (p, lt) = a in Says (w, p, lt)),
        (map t))
    in map (list_prod lP (all_term_lists lT n))

(** val all_says_atomics_arity : 'a1 term list -> nat list -> nat list -> nat
    -> 'a1 atomic list **)

let rec all_says_atomics_arity lT lP lW n =
    match lW with
    | Nil -> Nil
    | Cons (hd, tl) ->
        app (all_says1_atomics_arity lT lP hd n)
            (all_says_atomics_arity lT lP tl n)

(** val all_extbare_atomics : 'a1 term list -> 'a1 atomic list **)

let rec all_extbare_atomics = function
| Nil -> Nil
| Cons (a, t) -> Cons ((Extbare a), (all_extbare_atomics t))

(** val all_extsays_atomics : 'a1 term list -> nat list -> 'a1 atomic list **)

let all_extsays_atomics lT lP =
    let rec map = function

```

```

    | Nil -> Nil
    | Cons (a, t) -> Cons ((let Pair (p, t0) = a in Extsays (p, t0)),
        (map t))
in map (list_prod lP lT)

(** val all_atomics : 'a1 term list -> nat list -> nat list ->
    'a1 atomic list **)

let rec all_atomics lT lP lW = function
| Nil -> app (all_extbare_atomics lT) (all_extsays_atomics lT lW)
| Cons (hd, tl) ->
    app (all_bare_atomics_arity lT lP hd)
        (app (all_says_atomics_arity lT lP lW hd) (all_atomics lT lP lW tl))

(** val extract_term_atomic : 'a1 atomic -> 'a1 term list **)

let extract_term_atomic = function
| Bare (p, lt) -> lt
| Says (w, p, lt) -> lt
| Extbare t -> Cons (t, Nil)
| Extsays (w, t) -> Cons (t, Nil)

(** val extract_term_list_atomic : 'a1 atomic list -> 'a1 term list **)

let rec extract_term_list_atomic = function
| Nil -> Nil
| Cons (hd, tl) ->
    app (extract_term_atomic hd) (extract_term_list_atomic tl)

(** val extract_term_form : 'a1 form -> 'a1 term list **)

let extract_term_form = function
| Clause (h, b) -> app (extract_term_atomic h) (extract_term_list_atomic b)

(** val extract_term_list_form : 'a1 form list -> 'a1 term list **)

let rec extract_term_list_form = function
| Nil -> Nil
| Cons (hd, tl) -> app (extract_term_form hd) (extract_term_list_form tl)

(** val extract_pred_atomic : 'a1 atomic -> nat list **)

let extract_pred_atomic = function
| Bare (p, lt) -> Cons (p, Nil)
| Says (w, p, lt) -> Cons (p, Nil)
| Extbare t -> Nil
| Extsays (w, t) -> Nil

(** val extract_pred_list_atomic : 'a1 atomic list -> nat list **)

let rec extract_pred_list_atomic = function

```



```

| Nil -> Nil
| Cons (hd, tl) ->
    app (extract_pred_atomic hd) (extract_pred_list_atomic tl)

(** val extract_pred_form : 'a1 form -> nat list **)

let extract_pred_form = function
| Clause (h, b) -> app (extract_pred_atomic h) (extract_pred_list_atomic b)

(** val extract_pred_list_form : 'a1 form list -> nat list **)

let rec extract_pred_list_form = function
| Nil -> Nil
| Cons (hd, tl) -> app (extract_pred_form hd) (extract_pred_list_form tl)

(** val extract_prin_atomic : 'a1 atomic -> nat list **)

let extract_prin_atomic = function
| Bare (p, lt) -> Nil
| Says (w, p, lt) -> Cons (w, Nil)
| Extbare t -> Nil
| Extsays (w, t) -> Cons (w, Nil)

(** val extract_prin_list_atomic : 'a1 atomic list -> nat list **)

let rec extract_prin_list_atomic = function
| Nil -> Nil
| Cons (hd, tl) ->
    app (extract_prin_atomic hd) (extract_prin_list_atomic tl)

(** val extract_prin_form : 'a1 form -> nat list **)

let extract_prin_form = function
| Clause (h, b) -> app (extract_prin_atomic h) (extract_prin_list_atomic b)

(** val extract_prin_list_form : 'a1 form list -> nat list **)

let rec extract_prin_list_form = function
| Nil -> Nil
| Cons (hd, tl) -> app (extract_prin_form hd) (extract_prin_list_form tl)

(** val extract_arity_atomic : 'a1 atomic -> nat list **)

let extract_arity_atomic = function
| Bare (p, lt) -> Cons ((length lt), Nil)
| Says (w, p, lt) -> Cons ((length lt), Nil)
| Extbare t -> Nil
| Extsays (w, t) -> Nil

(** val extract_arity_list_atomic : 'a1 atomic list -> nat list **)

```

```

let rec extract_arity_list_atomic = function
| Nil -> Nil
| Cons (hd, tl) ->
    app (extract_arity_atomic hd) (extract_arity_list_atomic tl)

(** val extract_arity_form : 'a1 form -> nat list **)

let extract_arity_form = function
| Clause (h, b) ->
    app (extract_arity_atomic h) (extract_arity_list_atomic b)

(** val extract_arity_list_form : 'a1 form list -> nat list **)

let rec extract_arity_list_form = function
| Nil -> Nil
| Cons (hd, tl) -> app (extract_arity_form hd) (extract_arity_list_form tl)

(** val der_dec : ('a1 -> 'a1 -> sumbool) -> ('a1 form list -> 'a1 list) ->
    ('a1 -> 'a1 -> sumbool) -> ('a1 -> 'a1 -> 'a1 -> sumbool)
    -> 'a1 form list -> 'a1 atomic -> sumbool **)

let der_dec eq_kthing_dec extset exttype_dec extdot_dec lF a =
  let rec f = function
  | Nil -> Right
  | Cons (a0, l0) ->
      (match eq_atomic_dec eq_kthing_dec a0 a with
      | Left -> Left
      | Right -> f l0)
  in f
    (iter (S
      (length
        (all_atomics
          (app (extract_term_list_form lF)
              (extract_term_list_atomic
                (let rec map = function
                | Nil -> Nil
                | Cons (a0, t) -> Cons ((Extbare (Kident a0)), (map t))
                in map (extset lF)))) (extract_pred_list_form lF)
              (extract_prin_list_form lF) (extract_arity_list_form lF))))
          (fun x -> extend_one_one_k eq_kthing_dec extset extdot_dec lF x)
          (kthing_start_set eq_kthing_dec extset exttype_dec lF))

```

A.3 Auditing function calls

This section includes the Coq proof scripts from the example of Section 3.1. The proof script is organized in the following way. First the syntax of expressions and a notion of values are defined. Substitution is next, followed by a single-step reduction to define an operational semantics, `ssred`. The inductive predicate `audited_calls` is then defined. Next a new single-step reduction is defined, `ssred2`, that keeps track of all the names of functions that have been called. The transitive closure of this reduction is defined as `ssred3`. Next is a sequence of technical lemmas relating to where function calls appear, and where function calls can appear under the annotated reduction. The final result corresponds to Theorem 3.1.

```
Require Import List.
Require Import Arith.
Require Import Eqdep_dec.
Require Import Le.
```

Section Example.

```
Definition funcname : Set := nat.
Definition constrname : Set := nat.
```

```
Inductive exp : Set :=
| var : nat -> exp
| abs : exp -> exp
| app : exp -> exp -> exp
| call : funcname -> exp -> exp
| constr0 : constrname -> exp
| constr1 : constrname -> exp -> exp
| constr2 : constrname -> exp -> exp -> exp.
```

```
Fixpoint is_val (e : exp) : Prop :=
  match e with
  | var n => False
  | abs e2 => True
  | app e1 e2 => False
  | call f e2 => False
  | constr0 c => True
  | constr1 c v => is_val v
  | constr2 c v1 v2 => is_val v1 /\ is_val v2
  end.
```

```
Fixpoint is_limited_val (e : exp) : Prop :=
  match e with
```

```

| var n => False
| abs e2 => False
| app e1 e2 => False
| call f e2 => False
| constr0 c => True
| constr1 c v => is_limited_val v
| constr2 c v1 v2 => is_limited_val v1 /\ is_limited_val v2
end.

```

Parameter function_semantics : funcname -> exp -> exp.

```

Fixpoint shift (c : nat)(e : exp){struct e} : exp :=
  match e with
  | var n => if le_lt_dec c n then var (S n) else var n
  | abs e2 => abs (shift (S c) e2)
  | app e1 e2 => app (shift c e1) (shift c e2)
  | call f e2 => call f (shift c e2)
  | constr0 cn => constr0 cn
  | constr1 cn e1 => constr1 cn (shift c e1)
  | constr2 cn e1 e2 => constr2 cn (shift c e1) (shift c e2)
  end.

```

```

Fixpoint subs (j : nat)(v : exp)(e : exp){struct e} : exp :=
  match e with
  | var n => if eq_nat_dec n j then v else var n
  | abs e2 => abs (subs (S j) (shift 0 v) e2)
  | app e1 e2 => app (subs j v e1)(subs j v e2)
  | call f e2 => call f (subs j v e2)
  | constr0 cn => constr0 cn
  | constr1 cn e1 => constr1 cn (subs j v e1)
  | constr2 cn e1 e2 => constr2 cn (subs j v e1) (subs j v e2)
  end.

```

```

Inductive ssred : exp -> exp -> Prop :=
| ssred_app1 :
  forall t1 t2 t1p,
    ssred t1 t1p ->
    ssred (app t1 t2) (app t1p t2)
| ssred_app2 :
  forall t1 t2 t2p,
    ssred t2 t2p ->
    is_val t1 ->
    ssred (app t1 t2) (app t1 t2p)
| ssred_appabs :
  forall t12 t2,
    is_val t2 ->
    ssred (app (abs t12) t2) (subs 0 t2 t12)
| ssred_constr1 :
  forall c t1 t2,
    ssred t1 t2 ->
    ssred (constr1 c t1) (constr1 c t2)

```

```

| ssred_constr21 :
  forall c t1 t1p t2,
    ssred t1 t1p ->
      ssred (constr2 c t1 t2) (constr2 c t1p t2)
| ssred_constr22 :
  forall c t1 t2 t2p,
    ssred t2 t2p ->
      ssred (constr2 c t1 t2) (constr2 c t1 t2p)
| ssred_call :
  forall f t1,
    is_val t1 ->
      ssred (call f t1) (function_semantics f t1).

```

Parameter audit_maycall : exp -> funcname -> Prop.

Inductive audited_calls : exp -> exp -> Prop :=

```

| audited_calls_var :
  forall e n,
    audited_calls e (var n)
| audited_calls_app :
  forall e e1 e2,
    audited_calls e e1 ->
    audited_calls e e2 ->
    audited_calls e (app e1 e2)
| audited_calls_abs :
  forall e e1,
    audited_calls e e1 ->
    audited_calls e (abs e1)
| audited_calls_call :
  forall f e e1,
    audited_calls e e1 ->
    audit_maycall e f ->
    audited_calls e (call f e1)
| audited_calls_constr0 :
  forall e cn,
    audited_calls e (constr0 cn)
| audited_calls_constr1 :
  forall e cn e1,
    audited_calls e e1 ->
    audited_calls e (constr1 cn e1)
| audited_calls_constr2 :
  forall e cn e1 e2,
    audited_calls e e1 ->
    audited_calls e e2 ->
    audited_calls e (constr2 cn e1 e2).

```

Inductive ssred2 : (list funcname * exp) -> (list funcname * exp) -> Prop :=

```

| ssred2_app1 :
  forall t1 l1 t2 l1p t1p,
    ssred2 (l1, t1) (l1p, t1p) ->
    ssred2 (l1, (app t1 t2)) (l1p, (app t1p t2))

```

```

| ssred2_app2 :
  forall t1 l2 t2 l2p t2p,
    ssred2 (l2, t2) (l2p, t2p) ->
      is_val t1 ->
        ssred2 (l2, (app t1 t2)) (l2p, (app t1 t2p))
| ssred2_appabs :
  forall t12 t2 l,
    is_val t2 ->
      ssred2 (l, (app (abs t12) t2)) (l, (subs 0 t2 t12))
| ssred2_constr1 :
  forall c t1 t2 l l2,
    ssred2 (l, t1) (l2, t2) ->
      ssred2 (l, constr1 c t1) (l2, constr1 c t2)
| ssred2_constr21 :
  forall c t1 t1p t2 l1 l2,
    ssred2 (l1, t1) (l2, t1p) ->
      ssred2 (l1, constr2 c t1 t2) (l2, constr2 c t1p t2)
| ssred2_constr22 :
  forall c t1 t2 t2p l1 l2,
    ssred2 (l1, t2) (l2, t2p) ->
      ssred2 (l1, constr2 c t1 t2) (l2, constr2 c t1 t2p)
| ssred2_call :
  forall f t1 l,
    is_val t1 ->
      ssred2 (l, (call f t1)) (f :: l, (function_semantics f t1)).

Inductive ssred3 : (list funcname * exp) -> (list funcname * exp) -> Prop :=
| ssred3_close :
  forall l1 l2 e1 e2,
    ssred2 (l1, e1) (l2, e2) ->
      ssred3 (l1, e1) (l2, e2)
| ssred3_add :
  forall l1 l2 l3 e1 e2 e3,
    ssred3 (l1, e1) (l2, e2) ->
    ssred2 (l2, e2) (l3, e3) ->
      ssred3 (l1, e1) (l3, e3).

Inductive callin : funcname -> exp -> Prop :=
| callin_app1 :
  forall f e1 e2,
    callin f e1 ->
      callin f (app e1 e2)
| callin_app2 :
  forall f e1 e2,
    callin f e2 ->
      callin f (app e1 e2)
| callin_abs :
  forall f e1,
    callin f e1 ->
      callin f (abs e1)

```

```

| callin_call :
  forall f e1,
    callin f (call f e1)
| callin_call2 :
  forall f f0 e,
    callin f e ->
      callin f (call f0 e)
| callin_constr1 :
  forall f cn e,
    callin f e ->
      callin f (constr1 cn e)
| callin_constr21 :
  forall f cn e1 e2,
    callin f e1 ->
      callin f (constr2 cn e1 e2)
| callin_constr22 :
  forall f cn e1 e2,
    callin f e2 ->
      callin f (constr2 cn e1 e2).

```

Axiom nocallin_function_semantics:

```
forall f f0 v, is_val v -> ~ callin f0 (function_semantics f v).
```

Lemma ssred2callin :

```
forall f l1 e1 l2 e2,
  ssred2 (l1, e1) (l2, e2) ->
    ~In f l1 ->
      In f l2 ->
        callin f e1.

```

Proof.

intros.

```

Definition PR := fun (f : funcname)(p : list funcname * exp)
  (p0 : list funcname * exp) =>
  match p with (l1, e1) =>
    match p0 with (l2, e2) =>
      ~ In f l1 -> In f l2 -> callin f e1
    end
  end.

```

Check PR.

Check (ssred2_ind (PR f)).

```

cut (forall p p0 : list funcname * exp, ssred2 p p0 -> PR f p p0).
unfold PR.

```

intros. assert (~ In f l1 -> In f l2 -> callin f e1).

apply H2 with (p := (l1, e1))(p0 := (l2, e2)).

assumption. apply H3. assumption. assumption.

apply (ssred2_ind (PR f)).

unfold PR; intros.

apply callin_app1.

```

apply H3. assumption. assumption.

unfold PR; intros.
apply callin_app2. apply H3. assumption. assumption.

unfold PR; intros. contradiction.
unfold PR; intros.
apply callin_constr1. apply H3. assumption. assumption.
unfold PR; intros.
apply callin_constr21. apply H3. assumption. assumption.
unfold PR; intros.
apply callin_constr22. apply H3; assumption.
unfold PR; intros.
simpl in H4. elim H4; intro H5; clear H4.
subst f0.
apply callin_call. contradiction.
Qed.

Lemma callin_shift:
  forall t f n,
    callin f (shift n t) -> callin f t.
Proof.
  induction t; intros; simpl in H.
  induction (le_lt_dec n0 n).
  inversion H. inversion H.
  inversion H. subst f0 e1.
  assert (callin f t). apply IHt with (n := (S n)). assumption.
  apply callin_abs; assumption.
  inversion H. subst f0 e1 e2.
  assert (callin f t1). apply IHt1 with (n := n). assumption.
  apply callin_app1; assumption.
  subst f0 e1 e2.
  assert (callin f t2). apply IHt2 with (n := n). assumption.
  apply callin_app2; assumption.
  assert ({f0 = f} + {f0 <> f}).
  apply eq_nat_dec.
  elim H0; intro Heq; clear H0.
  subst f0. apply callin_call.
  inversion H. contradiction.
  subst f1 f2 e.
  assert (callin f0 t).
  apply IHt with (n := n). assumption.
  apply callin_call2. assumption.
  assumption.
  inversion H. subst f0 cn e.
  assert (callin f t).
  apply IHt with (n := n). assumption.
  apply callin_constr1. assumption.
  inversion H.
  subst f0 cn e1 e2.
  assert (callin f t1). apply IHt1 with (n := n). assumption.

```



```

    apply callin_constr21. assumption.
    subst f0 cn e1 e2.
    assert (callin f t2). apply IHt2 with (n := n). assumption.
    apply callin_constr22. assumption.
Qed.

```

Lemma callin_subs:

```

  forall t1 f n t2,
    callin f (subs n t2 t1) ->
    (callin f t1 /\ callin f t2).

```

Proof.

```

  induction t1; intros; simpl in H.
  (* var *)
  induction (eq_nat_dec n n0).
  right; assumption.
  left; assumption.
  (* abs *)
  inversion H. subst f0 e1.
  assert (callin f t1 /\ callin f (shift 0 t2)).
  apply IHt1 with (n := (S n)). assumption.
  clear H H2.
  elim H0; intro H1; clear H0.
  left. apply callin_abs. assumption.
  right.
  apply callin_shift with (n := 0). assumption.
  (* app *)
  inversion H.
  subst f0 e1 e2.
  assert (callin f t1_1 /\ callin f t2).
  apply IHt1_1 with (n := n). assumption.
  elim H0; intro H1; clear H0.
  left. apply callin_app1. assumption.
  right; assumption.
  subst f0 e1 e2.
  assert (callin f t1_2 /\ callin f t2).
  apply IHt1_2 with (n := n). assumption.
  elim H0; intro H1; clear H0.
  left. apply callin_app2. assumption.
  right; assumption.
  (* call *)
  inversion H.
  subst f1 f0 e1.
  left. apply callin_call.
  subst f1 f2 e.
  assert (callin f0 t1 /\ callin f0 t2).
  apply IHt1 with (n := n). assumption.
  elim H0; intro. left. apply callin_call2. assumption.
  right. assumption.
  (* constr0 *)
  inversion H.
  (* constr1 *)

```

```

inversion H. subst f0 cn e.
assert (callin f t1 \ / callin f t2).
apply IHt1 with (n := n). assumption.
elim H0; intro. left. apply callin_constr1. assumption.
right; assumption.
(* constr2 *)
inversion H.
subst f0 cn e1 e2.
assert (callin f t1_1 \ / callin f t2).
apply IHt1_1 with (n := n). assumption.
elim H0; intro. left. apply callin_constr21. assumption.
right; assumption.
subst f0 cn e1 e2.
assert (callin f t1_2 \ / callin f t2).
apply IHt1_2 with (n := n). assumption.
elim H0; intro. left. apply callin_constr22. assumption.
right; assumption.
Qed.

```

Lemma ssred2callincallin :

```

forall f l1 l2 e1 e2,
  ssred2 (l1, e1) (l2, e2) ->
    callin f e2 ->
    callin f e1.

```

Proof.

```

intros.
Definition PR3 := fun (f : funcname)(p : list funcname * exp)
  (p0 : list funcname * exp) =>
  match p with (l1, e1) =>
    match p0 with (l2, e2) =>
      callin f e2 -> callin f e1
    end
  end.
end.

```

Check PR3.

Check (ssred2_ind (PR3 f)).

```

cut (forall p p0 : list funcname * exp, ssred2 p p0 -> PR3 f p p0).
unfold PR3.
intros. assert (callin f e2 -> callin f e1).
apply H1 with (p := (l1, e1))(p0 := (l2, e2)).
assumption. apply H2. assumption.
apply (ssred2_ind (PR3 f)); unfold PR3; intros.

```

(* Case app left *)

inversion H3.

(* f is in t1p *)

subst f0 e0 e3. apply callin_app1. apply H2. assumption.

(* f is in t2 *)

subst f0 e0 e3. apply callin_app2. assumption.

```

(* Case app right *)
inversion H4.
  (* f is in t1 *)
  subst f0 e0 e3. apply callin_app1. assumption.
  (* f is in t2p *)
  subst f0 e0 e3. apply callin_app2. apply H2. assumption.
(* Case app-abs *)
assert (callin f t12 \ / callin f t2).
apply callin_subs with (n := 0). assumption.
elim H3; intro. apply callin_app1. apply callin_abs. assumption.
apply callin_app2. assumption.
(* Case constr1 *)
inversion H3. subst f0 cn e. assert (callin f t1).
apply H2; assumption. apply callin_constr1. assumption.
(* Case constr2 left *)
inversion H3.
subst f0 cn e0 e3.
assert (callin f t1). apply H2; assumption.
apply callin_constr21; assumption.
subst f0 cn e0 e3.
apply callin_constr22; assumption.
(* Case constr2 right *)
inversion H3.
subst f0 cn e0 e3.
apply callin_constr21; assumption.
subst f0 cn e0 e3.
assert (callin f t2). apply H2; assumption.
apply callin_constr22; assumption.
(* Case call *)
assert (~callin f (function_semantics f0 t1)).
apply nocallin_function_semantics. assumption. contradiction.
Qed.

```

```

Lemma ssred3callincallin :
  forall f l1 l2 e1 e2,
    ssred3 (l1, e1) (l2, e2) ->
      callin f e2 ->
        callin f e1.

```

Proof.

```

(* set up induction on derivations of ssred2 *)
intros.
cut (forall p p0 : list funcname * exp, ssred3 p p0 -> PR3 f p p0).
unfold PR3.
intros.
exact (H1 (l1, e1) (l2, e2) H H0).
apply (ssred3_ind (PR3 f)).
unfold PR3.
(* base case of just one ssred2 *)
intros.
apply ssred2callincallin with (l1 := l0)(l2 := l3)(e1 := e0)(e2 := e3).
assumption. assumption.

```

```

(* inductive case of extending by one *)
unfold PR3.
intros.
assert (callin f e3).
apply ssred2callincallin with (l1 := l3)(l2 := l4)(e1 := e3)(e2 := e4).
assumption. assumption.
apply H2. assumption.
Qed.

```

```

Lemma ssred3callin :
  forall f l1 e1 l2 e2,
    ssred3 (l1, e1) (l2, e2) ->
      ~In f l1 ->
      In f l2 ->
      callin f e1.

```

```

Proof.
  intros.
  cut (forall p p0 : list funcname * exp, ssred3 p p0 -> PR f p p0).
  unfold PR.
  intros.
  exact (H2 (l1, e1) (l2, e2) H H0 H1).
  apply (ssred3_ind (PR f)).
  unfold PR.
  intros.
  apply ssred2callin with (l1 := l0)(l2 := l3)(e1 := e0)(e2 := e3);
    assumption.
  unfold PR.
  intros.
  assert (~ In f l3 -> In f l4 -> callin f e3).
  apply ssred2callin with (l1 := l3)(l2 := l4)(e1 := e3)(e2 := e4).
  assumption.
  assert (In f l3 -> callin f e0).
  apply H3. assumption.
  clear H3.
  assert ({In f l3} + {~ In f l3}).
  apply In_dec.
  intros x y. decide equality x y.
  elim H3; intro H10. apply H8. assumption.
  assert (callin f e3).
  apply H7. assumption. assumption.
  apply ssred3callincallin with (l1 := l0)(l2 := l3)(e1 := e0)(e2 := e3).
  assumption. assumption.
Qed.

```

```

Lemma audited_calls_callin_maycall:
  forall f e2 e,
    audited_calls e e2 -> callin f e2 -> audit_maycall e f.

```

```

Proof.
  induction e2; intros.
  inversion H0.
  inversion H0. subst f0 e1. apply IHe2. inversion H. assumption. assumption.

```

```

inversion H0. subst f0 e1 e2. apply IHe2_1. inversion H. assumption.
assumption.
subst f0 e1 e2. apply IHe2_2. inversion H; assumption. assumption.
assert ({f = f0}+{f <> f0}).
decide equality f f0.
elim H1; intro Heq; clear H1. subst f.
inversion H. assumption.
inversion H0. absurd (f = f0); assumption.
inversion H. subst f1 f2 e0 e1 f3 e3. apply IHe2. assumption. assumption.
inversion H0. apply IHe2. inversion H. subst e0 cn e1. assumption.
inversion H0. subst f0 cn e0. assumption.
inversion H0. subst f0 cn e1 e2.
inversion H. subst e0 cn e1 e2.
apply IHe2_1. assumption. assumption.
subst f0 cn e1 e2. apply IHe2_2.
inversion H. subst e0 cn e1 e2.
assumption. assumption.
Qed.

```

Theorem onlycallsthrm:

```

forall e e2 l f,
  audited_calls e e ->
  ssred3 (nil, e) (l, e2) ->
  In f l ->
  audit_maycall e f.

```

Proof.

```

intros.
assert (callin f e).
apply ssred3callin with (l1 := (nil : list funcname))(l2 := l)(e2 := e2).
assumption.
simpl. auto. assumption.
apply audited_calls_callin_maycall with (e2 := e).
assumption. assumption.

```

Qed.

End Example.

A.4 Trust in the λ -calculus

This section includes the Coq proof scripts from the example of Section 3.2 (with additional description in Section 5.7.1). The proof script includes definitions of the types used in the system and the syntax of the example language. Next proofs of decidability relating to the type system are built up in stages by first showing decidability of equality between trust annotations, then decidability of equality between bare types and annotated types, then of the decidability of the relations between types. The type system itself is then defined as the predicate `has_type`. Next is a definition of `trusts_audited` which determines when all the `trust` statements have been audited. The proof script then shows that the type system is decidable. A key lemma is that if an expression has two types, the types must be the same. Finally the proof script shows the correctness of `trusts_audited` by defining a single step operational semantics that keeps track of which trust locations have been encountered, similar to Section A.3. A series of lemmas about where trusts may appear culminates in the proof that if an expression passes `trusts_audited`, then when interpreted by the operational semantics every trust that is encountered did indeed have an audit (Theorem 3.2).

```
Require Import List.
Require Import Arith.
Require Import Eqdep_dec.
Require Import Le.
Require Import JMeq.

Section Example.

(* Types *)

Inductive trust_type : Set :=
| tr : trust_type
| dis : trust_type.

Lemma eq_trust_type_dec:
  forall (a b : trust_type),
    {a=b}+{a<>b}.
Proof.
  intros a b.
  decide equality a b.
Qed.
```

```

Inductive bare_type : Set :=
| usertype : nat -> bare_type
| arrow : annotated_type -> annotated_type -> bare_type

with annotated_type : Set :=
| annotate : bare_type -> trust_type -> annotated_type.

Scheme bare_type_ind2 := Induction for bare_type Sort Set
with annotated_type_ind2 := Induction for annotated_type Sort Set.

Lemma eq_bare_type_dec:
  forall (a b : bare_type),
    {a=b}+{a<>b}.
Proof.
  intros. decide equality a b.
Qed.

Lemma eq_annotated_type_dec:
  forall (a b : annotated_type),
    {a=b}+{a<>b}.
Proof.
  intros. decide equality a b.
Qed.

(* Subtyping, <= predicates *)

Inductive trust_lte : trust_type -> trust_type -> Prop :=
| trust_lte_refl : forall (T : trust_type), trust_lte T T
| trust_lte_trdis : trust_lte tr dis.

Lemma trust_lte_dec :
  forall (T1 T2 : trust_type),
    {trust_lte T1 T2}+{~trust_lte T1 T2}.
Proof.
  intros.
  induction T1; induction T2.
  left. apply trust_lte_refl.
  left. apply trust_lte_trdis.
  right. unfold not. intro H.
  inversion H.
  left. apply trust_lte_refl.
Qed.

Inductive bare_lte : bare_type -> bare_type -> Prop :=
| bare_lte_refl :
  forall (T : bare_type), bare_lte T T
| bare_lte_arrow :
  forall (X Y A B : annotated_type),
    ann_lte Y B -> ann_lte A X ->
    bare_lte (arrow X Y) (arrow A B)

```

```

with ann_lte : annotated_type -> annotated_type -> Prop :=
| ann_lte_refl :
  forall (T : annotated_type), ann_lte T T
| ann_lte_annotate :
  forall (A B : bare_type)(U V : trust_type),
    bare_lte A B -> trust_lte U V ->
    ann_lte (annotate A U) (annotate B V).

(* This proof is very tedious, can't get it to go through *)
Axiom ann_lte_transitive :
  forall T1 T2 T3,
    ann_lte T1 T2 -> ann_lte T2 T3 -> ann_lte T1 T3.

(*
Subtle part of this proof is P and P0 we use for generated induction principle
for mutually recursive bare_lte and ann_lte. The problem is that if we just use
the properties:
  bare_type_ind2 with
    (P := fun B1 => forall (B2 : bare_type),
      ({bare_lte B1 B2}+{~bare_lte B1 B2}))
    (P0 := fun A1 => forall (A2 : annotated_type),
      {ann_lte A1 A2}+{~ann_lte A1 A2})
then it doesn't work because of the bare_lte_arrow rule. That rule needs
ann_lte both ways around, the hypotheses that way gives us the not enough
to work with. So we strengthen the conclusion of what we are proving,
which strengthens the hypotheses we get in the induction step.
*)
Lemma ann_lte_dec_aux:
  forall (A1 A2 : annotated_type),
    forall (B1 B2 : bare_type),
      prod ({ann_lte A1 A2}+{~ann_lte A1 A2})
        ({bare_lte B1 B2}+{~bare_lte B1 B2}).
Proof.
  assert(forall (B1 B2 : bare_type),
    ({bare_lte B1 B2}+{~bare_lte B1 B2})*({bare_lte B2 B1}+{~bare_lte B2 B1})).
  intros B2.
  Check bare_type_ind2.
  apply bare_type_ind2 with
    (P := fun B1 =>
      forall (B2 : bare_type), (({bare_lte B1 B2}+{~bare_lte B1 B2}) *
        ({bare_lte B2 B1}+{~bare_lte B2 B1})))
    (P0 := fun A1 =>
      (forall (A2 : annotated_type), (({ann_lte A1 A2}+{~ann_lte A1 A2}) *
        ({ann_lte A2 A1}+{~ann_lte A2 A1}))))); intros.

(* Usertype *)
split.

induction B0.
induction (eq_nat_dec n n0).

```



```

rewrite a. left. apply bare_lte_refl.
right. unfold not. intro H.
inversion H. contradiction.
right. unfold not. intro H.
inversion H.

induction B0.
induction (eq_nat_dec n0 n).
rewrite a. left. apply bare_lte_refl.
right. unfold not. intro H.
inversion H. contradiction.
right. unfold not. intro H.
inversion H.

(* Arrow *)
split.

induction B0.
right. unfold not. intro H1.
inversion H1.
  (* arrow arrow *)
  (* could be same arrow, rule that out first *)
  assert ({(a=a1 /\ a0=a2)}+{~(a=a1 /\ a0=a2)}).
  assert ({a=a1}+{a<>a1}).
  apply eq_annotated_type_dec.
  assert ({a0=a2}+{a0<>a2}).
  apply eq_annotated_type_dec.
  elim H1; intro Haa1; clear H1; elim H2; intro Ha0a2; clear H2; try (left;
    rewrite Haa1; rewrite Ha0a2; split; trivial);
    try (right; unfold not; intro H1; elim H1; intros H2 H3; contradiction).
  elim H1; intro Heq; clear H1.
  elim Heq; intros Heq1 Heq2; clear Heq.
  rewrite Heq1; rewrite Heq2. left. apply bare_lte_refl.
  (* Ok, start checking ann_lte of args *)
  assert (prod ({ann_lte a0 a2}+{~ann_lte a0 a2}) ({ann_lte a2 a0}+
    {~ann_lte a2 a0})).
  apply H0.
  elim H1; intros H2 H3; clear H1. clear H3.
  elim H2; intro H4; clear H2.
  assert (prod ({ann_lte a a1}+{~ann_lte a a1}) ({ann_lte a1 a}+{~ann_lte a1 a})).
  apply H.
  elim H1; intros H5 H6; clear H1. clear H5.
  elim H6; intro H7; clear H6.
  left. apply bare_lte_arrow. assumption. assumption.
  right. unfold not; intro H8. inversion H8.
  unfold not in Heq. apply Heq. split; assumption.
  contradiction.
  right. unfold not; intro H8. inversion H8.
  unfold not in Heq. apply Heq. split; assumption.
  contradiction.

```

```

induction B0.
right. unfold not. intro H1.
inversion H1.
  (* arrow arrow *)
  (* could be same arrow, rule that out first *)
  assert ({(a=a1 /\ a0=a2)}+{~(a=a1 /\ a0=a2)}).
  assert ({a=a1}+{a<>a1}).
  apply eq_annotated_type_dec.
  assert ({a0=a2}+{a0<>a2}).
  apply eq_annotated_type_dec.
  elim H1; intro Haa1; clear H1; elim H2; intro Ha0a2; clear H2; try
    (left; rewrite Haa1; rewrite Ha0a2; split; trivial);
    try (right; unfold not; intro H1; elim H1; intros H2 H3; contradiction).
  elim H1; intro Heq; clear H1.
  elim Heq; intros Heq1 Heq2; clear Heq.
  rewrite Heq1; rewrite Heq2. left. apply bare_lte_refl.
  (* Ok, start checking ann_lte of args *)
  assert (prod ({ann_lte a0 a2}+{~ann_lte a0 a2}) ({ann_lte a2 a0}+
    {~ann_lte a2 a0})).
  apply H0.
  elim H1; intros H2 H3; clear H1. clear H2.
  elim H3; intro H4; clear H3.
  assert (prod ({ann_lte a a1}+{~ann_lte a a1}) ({ann_lte a1 a}+
    {~ann_lte a1 a})).
  apply H.
  elim H1; intros H5 H6; clear H1. clear H6.
  elim H5; intro H7; clear H5.
  left. apply bare_lte_arrow. assumption. assumption.
  right. unfold not; intro H8. inversion H8.
  unfold not in Heq. apply Heq. split; symmetry; assumption.
  contradiction.
  right. unfold not; intro H8. inversion H8.
  unfold not in Heq. apply Heq. split; symmetry; assumption.
  contradiction.

  (* ann_lte annotate *)
  split.

induction A2.
assert ({b=b0 /\ t=t0}+{~(b=b0 /\ t=t0)}).
induction (eq_bare_type_dec b b0); induction (eq_trust_type_dec t t0);
  try (left; split; assumption);
  try (right; unfold not; intro Hq; elim Hq; intros H3 H4; contradiction).
elim H0; intro H1; clear H0.
elim H1; intros H2 H3; clear H1.
left. rewrite H2; rewrite H3. apply ann_lte_refl.
unfold not in H1.
assert (prod ({bare_lte b b0}+{~bare_lte b b0}) ({bare_lte b0 b}+
  {~bare_lte b0 b})).
apply H.
assert ({bare_lte b b0}+{~bare_lte b b0}).

```

```

elim H0; auto.
elim H2; intro Hb; clear H2.
assert ({trust_lte t t0}+{~trust_lte t t0}).
apply trust_lte_dec.
elim H2; intro H3; clear H2.
left. apply ann_lte_annotate; assumption.
right; unfold not; intro H4. inversion H4.
apply H1; split; assumption.
contradiction.
right; unfold not; intro H4. inversion H4.
apply H1; split; assumption.
contradiction.

induction A2.
assert ({b=b0 /\ t=t0}+{~(b=b0 /\ t=t0)}).
induction (eq_bare_type_dec b b0); induction (eq_trust_type_dec t t0);
  try (left; split; assumption);
  try (right; unfold not; intro Hq; elim Hq; intros H3 H4; contradiction).
elim H0; intro H1; clear H0.
elim H1; intros H2 H3; clear H1.
left. rewrite H2; rewrite H3. apply ann_lte_refl.
unfold not in H1.
assert (prod ({bare_lte b b0}+{~bare_lte b b0}) ({bare_lte b0 b}+
  {~bare_lte b0 b})).
apply H.
assert ({bare_lte b0 b}+{~bare_lte b0 b}).
elim H0; auto.
elim H2; intro Hb; clear H2.
assert ({trust_lte t0 t}+{~trust_lte t0 t}).
apply trust_lte_dec.
elim H2; intro H3; clear H2.
left. apply ann_lte_annotate; assumption.
right; unfold not; intro H4. inversion H4.
apply H1; split; symmetry; assumption.
contradiction.
right; unfold not; intro H4. inversion H4.
apply H1; split; symmetry; assumption.
contradiction.

(* ann_lte given bare_lte *)
intros A2 A3.
split.
induction A2; induction A3.
assert ( prod ({bare_lte b b0} + {~ bare_lte b b0}) ({bare_lte b0 b} +
  {~ bare_lte b0 b})).
apply H. clear H.
elim H0; intros H1 H2; clear H0. clear H2.
assert ({trust_lte t t0}+{~trust_lte t t0}).
apply trust_lte_dec.
assert ({b=b0 /\ t=t0}+{~(b=b0 /\ t=t0)}).
induction (eq_bare_type_dec b b0); induction (eq_trust_type_dec t t0);

```

```

    try (left; split; assumption);
    try (right; unfold not; intro H2; elim H2; intros; contradiction).
elim H1; intro H2; clear H1; elim H; intro H3; clear H; elim H0; intro H4;
clear H0.
left; apply ann_lte_annotate; assumption.
left; apply ann_lte_annotate; assumption.
left; elim H4; intros H5 H6; rewrite H5; rewrite H6; apply ann_lte_refl.
right; unfold not; intro H5; inversion H5. unfold not in H4; apply H4; split;
assumption.
contradiction.
left; elim H4; intros H5 H6; rewrite H5; rewrite H6; apply ann_lte_refl.
right; unfold not; intro H5; inversion H5. unfold not in H4; apply H4; split;
assumption.
contradiction.
left; elim H4; intros H5 H6; rewrite H5; rewrite H6; apply ann_lte_refl.
right; unfold not; intro H5; inversion H5. unfold not in H4; apply H4; split;
assumption.
contradiction.

assert ( prod ({bare_lte B1 B2} + {~ bare_lte B1 B2}) ({bare_lte B2 B1} +
  {~ bare_lte B2 B1})).
apply H.
elim H0; intros H1 H2; clear H0.
assumption.
Qed.

```

```

Lemma ann_lte_dec:
  forall (A1 A2 : annotated_type),
    {ann_lte A1 A2}+{~ann_lte A1 A2}.
Proof.
  intros.
  set (B1 := usertype 0).
  set (B2 := usertype 0).
  assert (prod ({ann_lte A1 A2}+{~ann_lte A1 A2})
    ({bare_lte B1 B2}+{~bare_lte B1 B2})).
  apply ann_lte_dec_aux.
  elim H; intros.
  assumption.
Qed.

```

```

Lemma bare_lte_dec:
  forall (B1 B2 : bare_type),
    {bare_lte B1 B2}+{~bare_lte B1 B2}.
Proof.
  intros.
  set (A1 := annote (usertype 0) tr).
  set (A2 := annote (usertype 0) tr).
  assert (prod ({ann_lte A1 A2}+{~ann_lte A1 A2})
    ({bare_lte B1 B2}+{~bare_lte B1 B2})).
  apply ann_lte_dec_aux.
  elim H; intros.

```

```

    assumption.
Qed.

```

```

Definition trustid : Set := nat.

```

```

Inductive expr : Set :=
| const_int : nat -> expr
| const_bool : bool -> expr
| var : nat -> expr
| abs : annotated_type -> expr -> expr
| app : expr -> expr -> expr
| trust : trustid -> expr -> expr
| distrust : expr -> expr
| check : expr -> expr
| cast : expr -> annotated_type -> annotated_type -> expr.

```

```

Lemma eq_bool_dec:
  forall (a b : bool),
    {a=b}+{a<>b}.

```

```

Proof.
  intros a b.
  decide equality a b.
Qed.

```

```

Lemma eq_expr_dec:
  forall (a b : expr),
    {a=b}+{a<>b}.

```

```

Proof.
  intros a b.
  decide equality a b.
  apply eq_nat_dec.
  apply eq_bool_dec.
  apply eq_nat_dec.
  apply eq_annotated_type_dec.
  apply eq_nat_dec.
  apply eq_annotated_type_dec.
  apply eq_annotated_type_dec.
Qed.

```

```

(* Join function *)

```

```

Definition join (U V : trust_type) : trust_type :=
  match U with
  | tr => V
  | dis => dis
  end.

```

```

Eval compute in (join dis dis).

```

```

Eval compute in (join tr dis).
Eval compute in (join tr tr).

(* Typing contexts *)

(* contexts are lists, references are by number in list *)
Inductive context : Set :=
| nilctx : context
| cons : annotated_type -> context -> context.

(* Audit is a proposition that is defined externally *)

Parameter audit : expr -> trustid -> Prop.

Axiom audit_dec:
  forall (E : expr)(id : trustid),
    {audit E id}+{~audit E id}.

(* Typing judgment *)
(* type_trust has bug... needs to know location it is processing, match
   that with loc in audit *)

Inductive has_type : context -> expr -> annotated_type -> Prop :=
| type_int : forall n C,
  has_type C (const_int n) (annotate (usertype 0) tr)
| type_bool : forall p C,
  has_type C (const_bool p) (annotate (usertype 1) tr)
| type_varz : forall T C,
  has_type (cons T C) (var 0) T
| type_varn : forall n T T2 C,
  has_type C (var n) T ->
  has_type (cons T2 C) (var (S n)) T
| type_abs : forall C E T T2,
  has_type (cons T C) E T2 ->
  has_type C (abs T E) (annotate (arrow T T2) tr)
| type_app : forall C E1 E2 T1 T2 U V,
  has_type C E1 (annotate (arrow T1 (annotate T2 U)) V) ->
  has_type C E2 T1 ->
  has_type C (app E1 E2) (annotate T2 (join U V))
| type_trust : forall C E T U id,
  has_type C E (annotate T U) ->
  has_type C (trust id E) (annotate T tr)
| type_distrust : forall C E T U,
  has_type C E (annotate T U) ->
  has_type C (distrust E) (annotate T dis)
| type_check : forall C E T,
  has_type C E (annotate T tr) ->
  has_type C (check E) (annotate T tr)
| type_cast : forall C E T T2,
  has_type C E T ->

```

```

ann_lte T T2 ->
has_type C (cast E T T2) T2.

```

Inductive trusts_audited : expr -> expr -> Prop :=

```

| audited_int :
  forall P n,
    trusts_audited P (const_int n)
| audited_bool :
  forall P b,
    trusts_audited P (const_bool b)
| audited_var :
  forall P n,
    trusts_audited P (var n)
| audited_abs :
  forall P T E,
    trusts_audited P E ->
    trusts_audited P (abs T E)
| audited_app :
  forall P E1 E2,
    trusts_audited P E1 ->
    trusts_audited P E2 ->
    trusts_audited P (app E1 E2)
| audited_trust :
  forall P E id,
    audit P id ->
    trusts_audited P E ->
    trusts_audited P (trust id E)
| audited_distrust :
  forall P E,
    trusts_audited P E ->
    trusts_audited P (distrust E)
| audited_check :
  forall P E,
    trusts_audited P E ->
    trusts_audited P (check E)
| audited_cast :
  forall P E T1 T2,
    trusts_audited P E ->
    trusts_audited P (cast E T1 T2).

```

Definition trusts_audited_top (E : expr) : Prop := trusts_audited E E.

Parameter audittest : audit (app (trust 0 (var 0)) (var 1)) 0.

Fixpoint nth (C : context)(n : nat){struct C} : option annotated_type :=

```

match C with
| nilctx => None
| cons H T => match n with
  | 0 => Some H
  | S np => nth T np

```

```

end
end.

Lemma has_type_varn2 :
  forall n Tc C,
    has_type C (var n) Tc ->
      nth C n = Some Tc.
Proof.
  induction n.
  intros.
  inversion H.
  simpl. trivial.
  intros.
  inversion H.
  assert (nth C0 n = Some Tc).
  apply IHn.
  assumption.
  simpl. assumption.
Qed.

Lemma has_type_varn :
  forall Tc C n,
    nth C n = Some Tc ->
      has_type C (var n) Tc.
Proof.
  induction C. intros. simpl in H. discriminate.
  intros.
  induction n.
  simpl in H. injection H; intros.
  rewrite H0.
  apply type_varz.
  apply type_varn.
  apply IHC.
  simpl in H. assumption.
Qed.

Theorem single_type:
  forall E C A1 A2,
    has_type C E A1 ->
    has_type C E A2 ->
    A1 = A2.
Proof.
  induction E; intros.

  (* const_int *)
  inversion H; inversion H0. trivial.

  (* const_bool *)
  inversion H; inversion H0. trivial.

```



```

(* var *)
assert (nth C n = Some A1).
apply has_type_varn2. assumption.
assert (nth C n = Some A2).
apply has_type_varn2. assumption.
rewrite H1 in H2. injection H2. trivial.

(* abs *)
inversion H; inversion H0.
assert (T1 = T2).
apply IHE with (C := cons a C); assumption.
subst T1. subst A1. trivial.

(* app *)
inversion H; inversion H0.
assert (annotate (arrow T1 (annotate T2 U)) V = annotate (arrow T0 (annotate T3 U0))
  V0).
apply IHE1 with (C := C); assumption.
injection H13; intros.
subst T2. subst U. subst V. trivial.

(* trust *)
inversion H; inversion H0.
assert (annotate T U = annotate T0 U0).
apply IHE with (C := C); assumption.
injection H11; intros.
subst T. trivial.

(* distrust *)
inversion H; inversion H0.
assert (annotate T U = annotate T0 U0).
apply IHE with (C := C); assumption.
injection H9; intros.
subst T. trivial.

(* check *)
inversion H; inversion H0.
assert (annotate T tr = annotate T0 tr).
apply IHE with (C := C); assumption.
injection H9; intros.
subst T. trivial.

(* cast *)
inversion H; inversion H0.
subst A1. subst A2. trivial.
Qed.

Lemma is_app_type:
  forall (a : annotated_type),

```

```

{ x : annotated_type*bare_type*trust_type*trust_type | match x with
  (T1, T2, U, V) => a = annotate (arrow T1 (annotate T2 U)) V end} +
{ ~ exists x, match x with (T1, T2, U, V) => a = annotate (arrow T1
  (annotate T2 U)) V end}.

```

Proof.

```

intros.
induction a.
induction b.
right.
unfold not; intro.
elim H; intros.
induction x.
induction a.
induction a.
discriminate.

induction a0.
left.
exists (a, b, t0, t).
trivial.

```

Qed.

Theorem has_type_dec:

```

forall (E : expr)(C : context),
  { A : annotated_type | has_type C E A } + { ~ exists A, has_type C E A }.

```

Proof.

```

induction E; intros.

(* const_int *)
left. exists (annotate (usertype 0) tr).
apply type_int.

(* const_bool *)
left. exists (annotate (usertype 1) tr).
apply type_bool.

(* var n *)
set (nthres := nth C n).
assert (nth C n = nthres). trivial.
induction nthres.
left. exists a.
apply has_type_varn. assumption.
right. unfold not; intro.
elim H0; intros.
assert (nth C n = Some x).
apply has_type_varn2.
assumption.
rewrite H in H2; discriminate.

(* abs *)

```

```

assert ( {A : annotated_type | has_type (cons a C) E A} +
        {~ (exists A : annotated_type, has_type (cons a C) E A)}).
apply IHE.
elim H; intro H1; clear H.
elim H1; intros.
left.
exists (annotate (arrow a x) tr).
apply type_abs. assumption.
right.
unfold not; intro.
inversion H.
inversion H0.
unfold not in H1. apply H1.
exists T2. assumption.

(* app *)
assert ({A : annotated_type | has_type C E1 A} +
        {~ (exists A : annotated_type, has_type C E1 A)}).
apply IHE1.
assert ({A : annotated_type | has_type C E2 A} +
        {~ (exists A : annotated_type, has_type C E2 A)}).
apply IHE2.
clear IHE1 IHE2.
elim H0; intro H2; clear H0.
elim H; intro H1; clear H.
elim H1; intros A1 H3; clear H1.
elim H2; intros A2 H4; clear H2.
assert (
  { x : annotated_type*bare_type*trust_type*trust_type | match x with
    (T1, T2, U, V) => A1 = annotate (arrow T1 (annotate T2 U)) V end} +
  { ~ exists x, match x with (T1, T2, U, V) => A1 = annotate (arrow T1
    (annotate T2 U)) V end}).
apply is_app_type.
elim H; intro H5; clear H.
elim H5; intros; clear H5.
induction x. induction a. induction a.
assert ({a = A2}+{a <> A2}).
apply eq_annotated_type_dec.
elim H; intros; clear H.

left.
exists (annotate b1 (join b0 b)).
apply type_app with (T1 := a).
rewrite <- p. assumption.
rewrite a0. assumption.

right.
unfold not; intros.
elim H; intros.
inversion H0.
assert (T1 = A2).

```

```

apply single_type with (C := C)(E := E2); assumption.
assert (annotate (arrow T1 (annotate T2 U)) V = A1).
apply single_type with (C := C)(E := E1); assumption.
subst A1.
injection H10; intros.
subst a. contradiction.

right.
unfold not; intros.
elim H; intros.
inversion H0.
unfold not in H5.
apply H5.
exists (T1, T2, U, V).
apply single_type with (C := C)(E := E1); assumption.

right.
unfold not; intros.
elim H; intros.
inversion H0.
unfold not in H1. apply H1.
exists (annotate (arrow T1 (annotate T2 U)) V).
assumption.

right.
unfold not; intros.
elim H0; intros.
inversion H1.
unfold not in H2; apply H2.
exists T1. assumption.

(* trust *)
assert ( {A : annotated_type | has_type C E A} +
        {~ (exists A : annotated_type, has_type C E A)} ).
apply IHE.

elim H; intros.
elim a; intros.
induction x.

left.
exists (annotate b tr).

apply type_trust with (C := C)(U := t0)(id := t).
assumption.

right.
unfold not; intros.
elim H0; intros.
inversion H1.
unfold not in b.

```

```

apply b.
exists (annotate T U).
assumption.

(* distrust *)
assert ( {A : annotated_type | has_type C E A} +
        {~ (exists A : annotated_type, has_type C E A)} ).
apply IHE.
elim H; intros.
elim a; intros.
induction x.

left.
exists (annotate b dis).
apply type_distrust with (C := C)(U := t).
assumption.

right.
unfold not; intros.
elim H0; intros.
inversion H1.
unfold not in b.
apply b.
exists (annotate T U).
assumption.

(* check *)
assert ( {A : annotated_type | has_type C E A} +
        {~ (exists A : annotated_type, has_type C E A)} ).
apply IHE.
elim H; intros.
elim a; intros.
induction x.
induction t.

left.
exists (annotate b tr).
apply type_check with (C := C).
assumption.

right.
unfold not; intros.
elim H0; intros.
inversion H1.
assert (annotate b dis = annotate T tr).
apply single_type with (C := C)(E := E); assumption.
discriminate.

right.
unfold not; intros.

```

```

elim H0; intros.
inversion H1.
unfold not in b.
apply b.
exists (annotate T tr).
assumption.

(* cast *)
assert ( {A : annotated_type | has_type C E A} +
        {~ (exists A : annotated_type, has_type C E A)} ).
apply IHE.
elim H; intros.
elim a1; intros.
assert ({x=a}+{x<>a}).
apply eq_annotated_type_dec.
elim H0; intros.
subst x.
clear IHE H H0 a1.
assert ({ann_lte a a0}+{~ann_lte a a0}).
apply ann_lte_dec.
elim H; intros.
left.
exists a0.
apply type_cast.
assumption. assumption.
right.
unfold not; intros.
elim H0; intros.
inversion H1.
subst x. contradiction.
right.
unfold not; intros.
elim H1; intros.
inversion H2.
assert (x=a).
apply single_type with (C := C)(E := E); assumption.
contradiction.
right.
unfold not; intros.
elim H0; intros.
inversion H1.
unfold not in b.
apply b.
exists a.
assumption.
Qed.

(* Now we prove that if trusts are audited, in operational
   semantics every trust we come across will have been audited. *)
Fixpoint shift (c : nat)(e : expr){struct e} : expr :=
  match e with

```

```

| var n => if le_lt_dec c n then var (S n) else var n
| abs t e2 => abs t (shift (S c) e2)
| app e1 e2 => app (shift c e1) (shift c e2)
| const_int i => const_int i
| const_bool b => const_bool b
| trust id e1 => trust id (shift c e1)
| distrust e1 => distrust (shift c e1)
| check e1 => check (shift c e1)
| cast e1 t1 t2 => cast (shift c e1) t1 t2
end.

Fixpoint subs (j : nat)(v : expr)(e : expr){struct e} : expr :=
  match e with
  | var n => if eq_nat_dec n j then v else var n
  | abs t e2 => abs t (subs (S j) (shift 0 v) e2)
  | app e1 e2 => app (subs j v e1)(subs j v e2)
  | const_int i => const_int i
  | const_bool b => const_bool b
  | trust id e1 => trust id (subs j v e1)
  | distrust e1 => distrust (subs j v e1)
  | check e1 => check (subs j v e1)
  | cast e1 t1 t2 => cast (subs j v e1) t1 t2
  end.

Inductive ssred : expr -> expr -> Prop :=
| ssred_app1 :
  forall e1 e2 e1p,
    ssred e1 e1p ->
    ssred (app e1 e2) (app e1p e2)
| ssred_app2 :
  forall e1 e2 e2p,
    ssred e2 e2p ->
  (*      is_val e1 -> *)
    ssred (app e1 e2) (app e1 e2p)
| ssred_appabs :
  forall e12 e2 t,
  (*      is_val t2 -> *)
    ssred (app (abs t e12) e2) (subs 0 e2 e12)
| ssred_trust :
  forall e1 id,
    ssred (trust id e1) e1
| ssred_distrust :
  forall e1,
    ssred (distrust e1) e1
| ssred_check :
  forall e1,
    ssred (check e1) e1
| ssred_cast :
  forall e1 t1 t2,
    ssred (cast e1 t1 t2) e1.

```

```

Inductive ssred2 : (list trustid * expr) -> (list trustid * expr) -> Prop :=
| ssred2_app1 :
  forall l1 l2 e1 e2 e1p,
    ssred2 (l1, e1) (l2, e1p) ->
    ssred2 (l1, (app e1 e2)) (l2, (app e1p e2))
| ssred2_app2 :
  forall l1 l2 e1 e2 e2p,
    ssred2 (l1, e2) (l2, e2p) ->
  (* is_val e1 -> *)
    ssred2 (l1, (app e1 e2)) (l2, (app e1 e2p))
| ssred2_appabs :
  forall l1 e12 e2 t,
  (* is_val t2 -> *)
    ssred2 (l1, (app (abs t e12) e2)) (l1, (subs 0 e2 e12))
| ssred2_trust :
  forall l1 e1 id,
    ssred2 (l1, (trust id e1)) (id :: l1, e1)
| ssred2_distrust :
  forall l1 e1,
    ssred2 (l1, (distrust e1)) (l1, e1)
| ssred2_check :
  forall l1 e1,
    ssred2 (l1, (check e1)) (l1, e1)
| ssred2_cast :
  forall l1 e1 t1 t2,
    ssred2 (l1, (cast e1 t1 t2)) (l1, e1).

Inductive ssred3 : (list trustid * expr) -> (list trustid * expr) -> Prop :=
| ssred3_close :
  forall l1 l2 e1 e2,
    ssred2 (l1, e1) (l2, e2) ->
    ssred3 (l1, e1) (l2, e2)
| ssred3_add :
  forall l1 l2 l3 e1 e2 e3,
    ssred3 (l1, e1) (l2, e2) ->
    ssred2 (l2, e2) (l3, e3) ->
    ssred3 (l1, e1) (l3, e3).

Inductive callin : trustid -> expr -> Prop :=
| callin_app1 :
  forall f e1 e2,
    callin f e1 ->
    callin f (app e1 e2)
| callin_app2 :
  forall f e1 e2,
    callin f e2 ->
    callin f (app e1 e2)
| callin_abs :
  forall f e1 T,

```



```

    callin f e1 ->
    callin f (abs T e1)
| callin_trust1 :
  forall f e1,
    callin f (trust f e1)
| callin_trust2 :
  forall f e1 id,
    callin f e1 ->
    callin f (trust id e1)
| callin_distrust :
  forall f e1,
    callin f e1 ->
    callin f (distrust e1)
| callin_check :
  forall f e1,
    callin f e1 ->
    callin f (check e1)
| callin_cast :
  forall f e1 T1 T2,
    callin f e1 ->
    callin f (cast e1 T1 T2).

Lemma ssred2callin :
  forall f l1 e1 l2 e2,
    ssred2 (l1, e1) (l2, e2) ->
    ~In f l1 ->
    In f l2 ->
    callin f e1.
Proof.
  intros.
  Definition PR := fun (f : trustid)(p : list trustid * expr)
    (p0 : list trustid * expr) =>
    match p with (l1, e1) =>
      match p0 with (l2, e2) =>
        ~ In f l1 -> In f l2 -> callin f e1
      end
    end.
  Check PR.
  Check (ssred2_ind (PR f)).

cut (forall p p0 : list trustid * expr, ssred2 p p0 -> PR f p p0).
unfold PR.
intros. assert (~ In f l1 -> In f l2 -> callin f e1).
apply H2 with (p := (l1, e1))(p0 := (l2, e2)).
assumption. apply H3. assumption. assumption.
apply (ssred2_ind (PR f)).
unfold PR; intros.
apply callin_app1.
apply H3. assumption. assumption.

```

```

unfold PR; intros.
apply callin_app2. apply H3. assumption. assumption.

unfold PR; intros. contradiction.
unfold PR; intros.
simpl in H3. elim H3; intro H4; clear H3. subst id.
apply callin_trust1. contradiction.

unfold PR; intros. contradiction.

unfold PR; intros. contradiction.

unfold PR; intros. contradiction.
Qed.

Lemma callin_shift:
  forall t f n,
    callin f (shift n t) -> callin f t.
Proof.
  induction t; intros; simpl in H.

  assumption. assumption.

  induction (le_lt_dec n0 n).
  inversion H. inversion H.

  inversion H. subst f0 e1.
  assert (callin f t). apply IHt with (n := (S n)). assumption.
  apply callin_abs; assumption.

  inversion H. subst f0 e1 e2.
  assert (callin f t1). apply IHt1 with (n := n). assumption.
  apply callin_app1; assumption.
  subst f0 e1 e2.
  assert (callin f t2). apply IHt2 with (n := n). assumption.
  apply callin_app2; assumption.

  assert ({f = t} + {f <> t}).
  apply eq_nat_dec.
  elim H0; intro Heq; clear H0.
  subst f. apply callin_trust1.
  inversion H. contradiction.
  subst f0 id e1.
  assert (callin f t0).
  apply IHt with (n := n). assumption.
  apply callin_trust2. assumption.

  inversion H. subst f0 e1.
  assert (callin f t).
  apply IHt with (n := n). assumption.
  apply callin_distrust. assumption.

```

```

inversion H. subst f0 e1.
assert (callin f t).
apply IHt with (n := n). assumption.
apply callin_check. assumption.

inversion H. subst f0 e1 T1 T2.
assert (callin f t).
apply IHt with (n := n). assumption.
apply callin_cast. assumption.
Qed.

Lemma callin_subs:
  forall t1 f n t2,
    callin f (subs n t2 t1) ->
      (callin f t1 \ / callin f t2).
Proof.
  induction t1; intros; simpl in H.
  (* consts *)
  left; assumption. left; assumption.
  (* var *)
  induction (eq_nat_dec n n0).
  right; assumption.
  left; assumption.
  (* abs *)
  inversion H. subst f0 T e1.
  assert (callin f t1 \ / callin f (shift 0 t2)).
  apply IHt1 with (n := (S n)). assumption.
  clear H H2.
  elim H0; intro H1; clear H0.
  left. apply callin_abs. assumption.
  right.
  apply callin_shift with (n := 0). assumption.
  (* app *)
  inversion H.
  subst f0 e1 e2.
  assert (callin f t1_1 \ / callin f t2).
  apply IHt1_1 with (n := n). assumption.
  elim H0; intro H1; clear H0.
  left. apply callin_app1. assumption.
  right; assumption.
  subst f0 e1 e2.
  assert (callin f t1_2 \ / callin f t2).
  apply IHt1_2 with (n := n). assumption.
  elim H0; intro H1; clear H0.
  left. apply callin_app2. assumption.
  right; assumption.
  (* trust *)
  inversion H.
  subst f0 f e1.
  left. apply callin_trust1.

```

```

subst f0 id e1.
assert (callin f t1 \ / callin f t2).
apply IHt1 with (n := n). assumption.
elim H0; intro. left. apply callin_trust2. assumption.
right. assumption.
(* distrust *)
inversion H. subst f0 e1.
assert (callin f t1 \ / callin f t2).
apply IHt1 with (n := n). assumption.
elim H0; intros. left. apply callin_distrust. assumption.
right. assumption.
(* check *)
inversion H. subst f0 e1.
assert (callin f t1 \ / callin f t2).
apply IHt1 with (n := n). assumption.
elim H0; intros. left. apply callin_check. assumption.
right. assumption.
(* cast *)
inversion H. subst f0 e1 T1 T2.
assert (callin f t1 \ / callin f t2).
apply IHt1 with (n := n). assumption.
elim H0; intros. left. apply callin_cast. assumption.
right. assumption.
Qed.

Lemma ssred2callincallin :
  forall f l1 l2 e1 e2,
    ssred2 (l1, e1) (l2, e2) ->
      callin f e2 ->
        callin f e1.
Proof.
  intros.
  Definition PR3 := fun (f : trustid)(p : list trustid * expr)
    (p0 : list trustid * expr) =>
    match p with (l1, e1) =>
      match p0 with (l2, e2) =>
        callin f e2 -> callin f e1
      end
    end.
  Check PR3.
  Check (ssred2_ind (PR3 f)).

  cut (forall p p0 : list trustid * expr, ssred2 p p0 -> PR3 f p p0).
  unfold PR3.
  intros. assert (callin f e2 -> callin f e1).
  apply H1 with (p := (l1, e1))(p0 := (l2, e2)).
  assumption. apply H2. assumption.
  apply (ssred2_ind (PR3 f)); unfold PR3; intros.

  (* Case app left *)
  inversion H3.

```

```

    (* f is in t1p *)
    subst f0 e4 e5. apply callin_app1. apply H2. assumption.
    (* f is in t2 *)
    subst f0 e4 e5. apply callin_app2. assumption.
    (* Case app right *)
    inversion H3.
    (* f is in t1 *)
    subst f0 e4 e5. apply callin_app1. assumption.
    (* f is in t2p *)
    subst f0 e4 e5. apply callin_app2. apply H2. assumption.
    (* Case app-abs *)
    assert (callin f e12 \ / callin f e0).
    apply callin_subs with (n := 0). assumption.
    elim H2; intro. apply callin_app1. apply callin_abs. assumption.
    apply callin_app2. assumption.
    (* trust *)
    apply callin_trust2. assumption.
    apply callin_distrust. assumption.
    apply callin_check. assumption.
    apply callin_cast. assumption.
Qed.

Lemma ssred3callincallin :
  forall f l1 l2 e1 e2,
    ssred3 (l1, e1) (l2, e2) ->
      callin f e2 ->
        callin f e1.
Proof.
  (* set up induction on derivations of ssred2 *)
  intros.
  cut (forall p p0 : list trustid * expr, ssred3 p p0 -> PR3 f p p0).
  unfold PR3.
  intros.
  exact (H1 (l1, e1) (l2, e2) H H0).
  apply (ssred3_ind (PR3 f)).
  unfold PR3.
  (* base case of just one ssred2 *)
  intros.
  apply ssred2callincallin with (l1 := l0)(l2 := l3)(e1 := e0)(e2 := e3).
  assumption. assumption.
  (* inductive case of extending by one *)
  unfold PR3.
  intros.
  assert (callin f e3).
  apply ssred2callincallin with (l1 := l3)(l2 := l4)(e1 := e3)(e2 := e4).
  assumption. assumption.
  apply H2. assumption.
Qed.

Lemma ssred3callin :
  forall f l1 e1 l2 e2,

```

```

    ssred3 (l1, e1) (l2, e2) ->
      ~In f l1 ->
      In f l2 ->
      callin f e1.
Proof.
  intros.
  cut (forall p p0 : list trustid * expr, ssred3 p p0 -> PR f p p0).
  unfold PR.
  intros.
  exact (H2 (l1, e1) (l2, e2) H H0 H1).
  apply (ssred3_ind (PR f)).
  unfold PR.
  intros.
  apply ssred2callin with (l1 := l0)(l2 := l3)(e1 := e0)(e2 := e3); assumption.
  unfold PR.
  intros.
  assert (~ In f l3 -> In f l4 -> callin f e3).
  apply ssred2callin with (l1 := l3)(l2 := l4)(e1 := e3)(e2 := e4). assumption.
  assert (In f l3 -> callin f e0).
  apply H3. assumption.
  clear H3.
  assert ({In f l3} + {~ In f l3}).
  apply In_dec.
  intros x y. decide equality x y.
  elim H3; intro H10. apply H8. assumption.
  assert (callin f e3).
  apply H7. assumption. assumption.
  apply ssred3callincallin with (l1 := l0)(l2 := l3)(e1 := e0)(e2 := e3).
  assumption. assumption.
Qed.

Lemma audited_calls_callin_maycall:
  forall f e2 e,
    trusts_audited e e2 -> callin f e2 -> audit e f.
Proof.
  induction e2; intros; inversion H0.
  subst f0 e1. apply IHe2. inversion H. assumption. assumption.
  subst f0 e1 e2. apply IHe2_1. inversion H. assumption. assumption.
  subst f0 e1 e2. apply IHe2_2. inversion H; assumption. assumption.
  subst f0 f e1. inversion H. assumption.
  subst f0 id e1. apply IHe2. inversion H. assumption. assumption.
  subst f0 e1. apply IHe2. inversion H. assumption. assumption.
  subst f0 e1. apply IHe2. inversion H. assumption. assumption.
  subst f0 e1 T1 T2. apply IHe2. inversion H. assumption. assumption.
Qed.

Theorem auditedtrusts:
  forall e e2 l id,
    trusts_audited e e ->
    ssred3 (nil, e) (l, e2) ->
    In id l ->

```

```

    audit e id.
Proof.
  intros.
  assert (callin id e).
  apply ssred3callin with (l1 := (nil : list trustid))(l2 := 1)(e2 := e2).
  assumption.
  simpl. auto. assumption.
  apply audited_calls_callin_maycall with (e2 := e).
  assumption. assumption.
Qed.

End Example.

```

A.5 Proof by computation

In this section we present the proof script that shows some results related to proof by computation (Chapter 5). In particular, the following proof script defines the `decide` function and related functions (Section 5.2.1). The proof script then shows that decision procedures always give correct results (Theorem 5.1), that every decision procedure gives the same answer (Theorem 5.2), that every proof that a decision procedure gives a particular answer is the same, and that every proof that a decision procedure says “yes” is precisely `refl_equal true` (used for Theorem 5.3). These results together show that terms using `decide` will be automatically and correctly decided in BCIC.

We include this proof script because it is relatively simple, it proves interesting and valuable properties about BCIC, and it uses somewhat unorthodox proof methods for some theorems.

```
Require Import List.
Require Import Arith.
Require Import Eqdep_dec.
Require Import Le.
Require Import JMeq.

(* decide goes from prop to prop, requires decidability *)
Definition decide (P : Prop)(Pdec : {P}+{~P}) : Prop :=
  (match Pdec with | left Prf => true | right Prf => false end)
  = true.

(* decide_not is the reverse of decide *)
Definition decide_not (P : Prop)(Pdec : {P}+{~P}) : Prop :=
  (match Pdec with | left Prf => false | right Prf => true end)
  = true.

(* decide_partial goes from prop to prop, requires partial decidability *)
Definition decide_part (P : Prop)(Pdec : {P}+{True}) : Prop :=
  (match Pdec with | left Prf => true | right Prf => false end)
  = true.

(* Exactness of decide *)
Theorem exactly:
  forall (P : Prop)(Pdec : {P} + {~ P}),
    decide P Pdec <=> P.
Proof.
  intros.
  unfold decide.
```



```

    unfold iff.
    induction Pdec; split; intros.
    assumption. trivial. discriminate. contradiction.
Qed.

(* Partial decision procedures are not exact, just implication *)
Theorem exactly_partial:
  forall (P : Prop)(Pdec : {P} + {True}),
    decide_part P Pdec -> P.
Proof.
  intros.
  unfold decide_part in H.
  induction Pdec.
  assumption. discriminate.
Qed.

(* All decision procedures give same answer for all props *)
Theorem one_decision_procedure:
  forall (P : Prop)(Pdec1 : {P}+{~ P})(Pdec2 : {P}+{~P}),
    decide P Pdec1 <-> decide P Pdec2.
Proof.
  intros.
  assert (decide P Pdec1 <-> P). apply exactly.
  assert (decide P Pdec2 <-> P). apply exactly.
  assert (P <-> decide P Pdec2). apply iff_sym. assumption.
  apply iff_trans with (A := decide P Pdec1)(B := P)(C := decide P Pdec2);
    assumption.
Qed.

Lemma eq_bool_dec:
  forall (x y : bool), {x=y}+{x<>y}.
Proof.
  intros; decide equality x y.
Qed.

(* All proofs of decide are the same *)
Lemma one_proof :
  forall (P : Prop)(Pdec : {P}+{~ P})(prf1 prf2 : decide P Pdec),
    prf1 = prf2.
Proof.
  unfold decide.
  intros.
  induction Pdec; try discriminate.
  apply eq_proofs_unicity.
  assert (forall (x y : bool), {x=y}+{x<>y}).
  apply eq_bool_dec.
  intros. assert ({x=y}+{x<>y}).
  apply H. elim H0; intros; auto.
Qed.

(* All proofs of decide_part are the same *)

```

```

Lemma one_proof_part :
  forall (P : Prop)(Pdec : {P}+{True})(prf1 prf2 : decide_part P Pdec),
    prf1 = prf2.
Proof.
  unfold decide_part.
  intros.
  induction Pdec; try discriminate.
  apply eq_proofs_unicity.
  assert (forall (x y : bool), {x=y}+{x<>y}).
  apply eq_bool_dec.
  intros. assert ({x=y}+{x<>y}).
  apply H. elim H0; intros; auto.
Qed.

(* All proofs of true = true are the same *)
Lemma one_proof_true_eq_true:
  forall (prf1 prf2 : true = true),
    prf1 = prf2.
Proof.
  apply eq_proofs_unicity.
  assert (forall (x y : bool), {x=y}+{x<>y}).
  apply eq_bool_dec.
  intros. assert ({x=y}+{x<>y}).
  apply H. elim H0; intros; auto.
Qed.

(*
Want to do it this way, but doesn't typecheck

Lemma refl_equal_proof2:
  forall (P : Prop)(Pdec : {P}+{~ P})(prf : convert P Pdec = true),
    prf = refl_equal true.

Solution is JM equality.
*)

Lemma jmeq_proofs:
  forall (P : Prop)(Pdec : {P}+{~P})(prf1 : decide P Pdec)(prf2 : true = true),
    JMeq prf1 prf2.
Proof.
  intros.
  unfold decide in prf1.
  destruct Pdec.
  assert (prf1 = prf2).
  apply one_proof_true_eq_true.
  rewrite H.
  apply JMeq_refl.
  discriminate.
Qed.

```

```

Lemma jmeq_proofs_part:
  forall (P : Prop)(Pdec : {P}+{True})(prf1 : decide_part P Pdec)(prf2 : true = true),
    JMeq prf1 prf2.
Proof.
  intros.
  unfold decide_part in prf1.
  destruct Pdec.
  assert (prf1 = prf2).
  apply one_proof_true_eq_true.
  rewrite H.
  apply JMeq_refl.
  discriminate.
Qed.

Theorem refl_equal_proof:
  forall (P : Prop)(Pdec : {P}+{~ P})(prf : decide P Pdec),
    JMeq prf (refl_equal true).
Proof.
  intros.
  apply jmeq_proofs.
Qed.

Theorem refl_equal_proof_part:
  forall (P : Prop)(Pdec : {P}+{True})(prf : decide_part P Pdec),
    JMeq prf (refl_equal true).
Proof.
  intros.
  apply jmeq_proofs_part.
Qed.

```


Bibliography

- [ABEL08] Martín Abadi, Mihai Budiu, Úlfar Erlingsson, and Jay Ligatti. Control-flow integrity: Principles, implementations, and applications. *ACM Transactions on Information and System Security*, 2008. To appear.
- [ABLP93] Martín Abadi, Michael Burrows, Butler Lampson, and Gordon Plotkin. A calculus for access control in distributed systems. *ACM Transactions on Programming Languages and Systems*, 15(4):706–734, October 1993.
- [AF99] Andrew W. Appel and Edward W. Felten. Proof-carrying authentication. In *Proceedings of the 5th ACM Conference on Computer and Communications Security*, pages 52–62, November 1999.
- [AF00] Andrew W. Appel and Amy P. Felty. A semantic model of types and machine instructions for proof-carrying code. In *Proceedings of the 27th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, pages 243–253, January 2000.
- [And72] James P. Anderson. Computer security technology planning study. Technical Report ESD-TR-73-51, Electronic Systems Division, Hanscom Air Force Base, October 1972. Available online at: <http://csrc.nist.gov/publications/history/ande72.pdf>.
- [And01] Ross Anderson. *Security Engineering: A Guide to Building Dependable Distributed Systems*, chapter 7. Wiley, 2001.
- [App00] Andrew W. Appel. Hints on proving theorems in Twelf, February 2000. Available online at: <http://www.cs.princeton.edu/~appel/twelf-tutorial/>.
- [App01] Andrew W. Appel. Foundational proof-carrying code. In *Proceedings of the 16th Annual Symposium on Logic in Computer Science*, pages 247–258, June 2001.
- [Bar] Bruno Barras. Coq.logic.eqdep_dec. Adapted from Thomas Kleymann’s LEGO proof. At http://coq.inria.fr/V8.1/stdlib/Coq.Logic.Eqdep_dec.html.

- [Bar97] Bruno Barras. A formalization of the calculus of construction, October 1997. Available online at: <http://coq.inria.fr/contribs/coq-in-coq.tar.gz>.
- [BC04a] Yves Bertot and Pierre Castéran. *Interactive Theorem Proving and Program Development*. Springer, 2004.
- [BC04b] Yves Bertot and Pierre Castéran. *Interactive Theorem Proving and Program Development*, chapter 16. Springer, 2004.
- [BDL96] Clark W. Barrett, David L. Dill, and Jeremy R. Levitt. Validity checking for combinations of theories with equality. In *FMCAD'96*, November 1996.
- [BGF06] Moritz Y. Becker, Andrew D. Gordon, and Cédric Fournet. SecPAL: Design and semantics of a decentralized authorization language. Technical Report MSR-TR-2006-120, Microsoft Research, September 2006.
- [BSF02] Lujo Bauer, Michael A. Schneider, and Edward W. Felten. A general and flexible access-control system for the Web. In *Proceedings of the 11th USENIX Security Symposium 2002*, pages 93–108, 2002.
- [BW97] Bruno Barras and Benjamin Werner. Coq in Coq. Technical report, INRIA-Rocquencourt, 1997.
- [CCNS05] Bor-Yuh Evan Chang, Adam Chlipala, George C. Necula, and Robert R. Schneck. The Open Verifier framework for foundational verifiers. In *Proceedings of the 2005 ACM SIGPLAN International Workshop on Types in Language Design and Implementation (TLDI)*, pages 1–12, 2005.
- [CDM04] Ajay Chander, Drew Dean, and John Mitchell. A distributed high assurance reference monitor. In *7th International Security Conference (ISC'04), LNCS 3225*, pages 231–244, September 2004.
- [CG02] Solange Coupet-Grimal. Linear temporal logic, July 2002. Available online at: <http://coq.inria.fr/contribs/LTL.html>.
- [CH88] Thierry Coquand and Gérard Huet. The calculus of constructions. *Information and Computation*, 76(2/3):95–120, 1988.
- [Chl06] Adam Chlipala. Modular development of certified program verifiers with a proof assistant. In *Proceedings of the 11th ACM SIGPLAN International Conference on Functional Programming (ICFP'06)*, pages 160–171, 2006.
- [Chl07a] Adam Chlipala. A certified type-preserving compiler from lambda calculus to assembly language. In *Proceedings of ACM SIGPLAN 2007 Conference on Programming Language Design and Implementation (PLDI'07)*, June 2007.
- [Chl07b] Adam James Chlipala. *Implementing Certified Programming Language Tools*

- in *Dependent Type Theory*. PhD thesis, University of California, Berkeley, dec 2007.
- [Coq86] Thierry Coquand. An analysis of Girard’s paradox. In *First IEEE Symposium on Logic in Computer Science*, pages 227–236, 1986.
- [CPM90] Thierry Coquand and Christine Paulin-Mohring. Inductively defined types. In *Proceedings of Colog’88*, 1990. Volume 417 of Lecture Notes in Computer Science, Springer-Verlag.
- [CW96] W. Chen and D. S. Warren. Tabled evaluation with delaying for general logic programs. *Journal of the ACM*, 43(1):20–74, January 1996.
- [DeT02] John DeTreville. Binder, a logic-based security language. In *Proceedings of the 2002 IEEE Symposium on Security and Privacy*, pages 105–113, 2002.
- [DLS02] Manuvir Das, Sorin Lerner, and Mark Seigle. ESP: Path-sensitive program verification in polynomial time. In *Proceedings of the 2002 Conference on Programming Language Design and Implementation (PLDI’02)*, June 2002.
- [ECM07] ECMA. C# and common language infrastructure standards, 2007. Online at <http://msdn2.microsoft.com/en-us/netframework/aa569283.aspx>.
- [Ers58] A. P. Ershov. On programming of arithmetic operations. *Communications of the ACM*, 1(8), August 1958.
- [ES00] U. Erlingsson and F. B. Schneider. IRM enforcement of Java stack inspection. In *Proceedings of the 2000 IEEE Symposium on Security and Privacy*, pages 246–255, 2000.
- [FJOS03] Cormac Flanagan, Rajeev Joshi, Xinming Ou, and James B. Saxe. Theorem proving using lazy proof explication. In *Proceedings of the 15th International Conference on Computer Aided Verification (CAV 2003)*, July 2003.
- [FLL⁺02] C. Flanagan, K. Leino, M. Lillibridge, C. Nelson, J. Saxe, and R. Stata. Extended static checking for java. In *Proceedings of the 2002 Conference on Programming Language Design and Implementation (PLDI)*, 2002.
- [Fou] Perl Foundation. Perl 5.8.8 documentation: perlsec - Perl security. Online at <http://perldoc.perl.org/perlsec.html>.
- [Gon] Georges Gonthier. A computer-checked proof of the four colour theorem. <http://research.microsoft.com/~gonthier/4colproof.pdf>.
- [Gut04] Peter Gutmann. *Cryptographic Security Architecture: Design and Verification*. Springer Verlag, 2004.
- [Hen03] Dimitri Hendriks. *Metamathematics in Coq*. PhD thesis, Utrecht University, 2003.

- [HHP93] Robert Harper, Furio Honsell, and Gordon Plotkin. A framework for defining logics. *Journal of the ACM*, 40(1):143–184, 1993.
- [Jim01] Trevor Jim. SD3: A trust management system with certified evaluation. In *Proceedings of the 2001 IEEE Symposium on Security and Privacy*, pages 106–115, May 2001.
- [JM94] Joxan Jaffar and Michael J. Maher. Constraint logic programming: A survey. *Journal of Logic Programming*, 19/20:503–581, 1994.
- [Joh77] Stephen Johnson. Lint, a C program checker. Computer Science Technical Report 65, Bell Laboratories, December 1977.
- [LA03] Eunyong Lee and Andrew W. Appel. Policy-enforced linking of untrusted components. In *Proceedings of the 11th ACM SIGSOFT Symposium on Foundations of Software Engineering*, pages 371–374, September 2003.
- [Lam71] B. Lampson. Protection. In *5th Princeton Symposium on Information Sciences and Systems*, March 1971.
- [Ler06] Xavier Leroy. Formal certification of a compiler back-end. In *33rd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL’06)*, 2006.
- [Let04] P. Letouzey. *Programmation fonctionnelle certifiée – L’extraction de programmes dans l’assistant Coq*. PhD thesis, Université Paris-Sud, July 2004. English translation available online at: <http://www.pps.jussieu.fr/~letouzey>.
- [LGF03] Ninghui Li, Benjamin N. Grosof, and Joan Feigenbaum. Delegation logic: A logic-based approach to distributed authorization. *ACM Transactions on Information and System Security*, 6(1):128–171, February 2003.
- [LLFS⁺07] Chris Lesniewski-Laas, Bryan Ford, Jacob Strauss, Robert Morris, and M. Frans Kaashoek. Alpaca: extensible authorization for distributed services. In *Proceedings of the 14th ACM Conference on Computer and Communications Security (CCS 2007)*, pages 432–444, 2007.
- [LLL⁺02] Sebastian Lange, Brian LaMacchia, Matthew Lyons, Rudi Martin, Brian Pratt, and Greg Singleton. *.NET Framework Security*. Addison Wesley, 2002.
- [LM03] Ninghui Li and John C. Mitchell. Datalog with constraints: A foundation for trust management languages. In *Proceedings of the Fifth International Symposium on Practical Aspects of Declarative Languages*, January 2003.
- [LMW02] Ninghui Li, John C. Mitchell, and William H. Winsborough. Design of a role-based trust-management framework. In *Proceedings of the 2002 IEEE Symposium on Security and Privacy*, pages 114–130, May 2002.

- [LY97] T. Lindholm and F. Yellin. *The JavaTM Virtual Machine Specification*. Addison Wesley, 1997.
- [McB02] Conor McBride. Elimination with a motive. In *TYPES 2000*, pages 197–216, 2002.
- [McC06] Stephen McCamant. A machine-checked safety proof for a CISC-compatible SFI technique. MIT Computer Science and Artificial Intelligence Laboratory technical report 2006-035, MIT, May 2006.
- [MM06] Stephen McCamant and Greg Morrisett. Evaluating SFI for a CISC architecture. In *Proceedings of the 15th USENIX Security Symposium*, pages 209–224, 2006.
- [MMZ⁺01] Matthew W. Moskewicz, Conor F. Madigan, Ying Zhao, Lintao Zhang, and Sharad Malik. Chaff: Engineering an Efficient SAT Solver. In *Proceedings of the 38th Design Automation Conference (DAC'01)*, 2001.
- [MWCG98] Greg Morrisett, David Walker, Karl Crary, and Neal Glew. From System F to Typed Assembly Language. In *Proceedings of the 25th ACM Symposium on Principles of Programming Languages*, pages 85–97, 1998.
- [MWCG99] Greg Morrisett, David Walker, Karl Crary, and Neal Glew. From System F to typed assembly language. *ACM Transactions on Programming Languages and Systems*, 21(3):528–569, May 1999.
- [Mye99] Andrew C. Myers. *Mostly-Static Decentralized Information Flow Control*. PhD thesis, Massachusetts Institute of Technology, Cambridge, MA, January 1999.
- [Nec97] George C. Necula. Proof-carrying code. In *Proceedings of the 24th ACM SIGPLAN-SIGACT Symposium on the Principles of Programming Languages (POPL'97)*, pages 106–119, 1997.
- [Net07] Microsoft Developer Network. About ActiveX controls, 2007. Online at <http://msdn2.microsoft.com/en-us/library/Aa751971.aspx>.
- [NO79] G. Nelson and D. Oppen. Simplification by cooperating decision procedures. *Transactions on Programming Languages and Systems*, 1(2):245–257, 1979.
- [NPW02] Tobias Nipkow, Lawrence C. Paulson, and Markus Wenzel. *Isabelle:HOL*, volume 2283 of *LNCS*. Springer-Verlag, 2002.
- [NRL⁺75] Peter G. Neumann, L. Robinson, Karl N. Levitt, R. S. Boyer, and A. R. Saxena. A provably secure operating system. Technical Report M79-225, Stanford Research Institute, June 1975.
- [NS03] George C. Necula and Robert R. Schneck. A sound framework for untrusted

- p verification-condition generators. In
- Proceedings of the 18th Annual IEEE Symposium on Logic in Computer Science*
- , pages 248–260, 2003.
- [OG02] Martijn Oostdijk and Herman Geuvers. Proof by computation in the Coq system. *Theoretical Computer Science*, 272(1-2):293–314, 2002.
 - [ØP97] P. Ørbæk and J. Palsberg. Trust in the λ -calculus. *Journal of Functional Programming*, 7(6):557–591, 1997.
 - [ORS92] S. Owre, J. Rushby, and N. Shankar. PVS: A prototype verification system. In *11th International Conference on Automated Deduction*, pages 748–752, 1992.
 - [PM89] Christine Paulin-Mohring. Extracting F_ω ’s programs from proofs in the calculus of constructions. In *Principles of Programming Languages (POPL’89)*, pages 89–104, 1989.
 - [PS99] Frank Pfenning and Carsten Schürmann. System description: Twelf — A meta-logical framework for deductive systems. In *Proceedings of the 16th International Conference on Automated Deduction*, pages 202–206. Springer-Verlag LNAI 1632, July 1999.
 - [RSS⁺97] P. Rao, K. Sagonas, T. Swift, D. S. Warren, and J. Freire. XSB: A system for efficiently computing well-founded semantics. In *Proceedings of the 4th International Conference on Logic Programming and Nonmonotonic Reasoning*, volume 1265 of *Lecture Notes in Artificial Intelligence*, pages 430–440, Berlin, July 1997. Springer.
 - [RV01] Alan Robinson and Andrei Voronkov, editors. *Handbook of Automated Reasoning*. MIT Press, 2001.
 - [SBD02] Aaron Stump, Clark W. Barrett, and David L. Dill. CVC: a cooperating validity checker. In *CAV 2002*, July 2002.
 - [SM03] Andrei Sabelfeld and Andrew C. Myers. Language-based information-flow security. *IEEE Journal on Selected Areas in Communications*, 21(1):5–19, 2003.
 - [SN02] Robert R. Schneck and George C. Necula. A gradual approach to a more trustworthy, yet scalable, proof-carrying code. In *Proceedings of the Conference on Automated Deduction (CADE’02)*, pages 47–62, 2002.
 - [SS94] Leon Sterling and Ehud Shapiro. *The Art of Prolog*. The MIT Press, 2nd edition, 1994.
 - [SS05] Andrei Sabelfeld and David Sands. Declassification: Dimensions and principles. In *Proceedings of the 18th IEEE Workshop on Computer Security Foundations (CSFW’05)*, pages 255–269, 2005.
 - [Tea] The Coq Development Team. The Coq proof assistant. <http://coq.inria.fr/>.

- [Ull88] J. D. Ullman. *Principles of Database and Knowledge-base Systems*, volume 2. Computer Science Press, 1988.
- [WA05] Nathan Whitehead and Martín Abadi. BCiC: A system for code authentication and verification. In *Proceedings of the 11th International Conference on Logic for Programming Artificial Intelligence and Reasoning (LPAR'04)*, pages 110–124, 2005.
- [WABL93] Edward Wobber, Martín Abadi, Michael Burrows, and Butler Lampson. Authentication in the Taos operating system. In *Proceedings of the 14th ACM Symposium on Operating System Principles*, pages 256–269, 1993.
- [WAN04] Nathan Whitehead, Martín Abadi, and George Necula. By reason and authority: A system for authorization of proof-carrying code. In *Proceedings of the 17th IEEE Computer Security Foundations Workshop (CSFW'04)*, pages 236–250, 2004.
- [Wei98] Klaus Weich. Decision procedures for intuitionistic propositional logic by program extraction. In *TABLEAUX 1998, LNCS 1397*, pages 292–306, 1998.
- [Whi07] Nathan Whitehead. A certified distributed security logic for authorizing code. In *International Workshop, TYPES 2006, Revised Selected Papers, LNCS 4502*, pages 253–268, 2007.
- [WJA07] Nathan Whitehead, Jordan Johnson, and Martín Abadi. Policies and proofs for code auditing. In *Proceedings of the 5th International Symposium on Automated Technology for Verification and Analysis*, pages 1–14, October 2007.
- [WLAG93] Robert Wahbe, Steven Lucco, Thomas E. Anderson, and Susan L. Graham. Efficient software-based fault isolation. *ACM SIGOPS Operating Systems Review*, 27(5):203–216, December 1993.